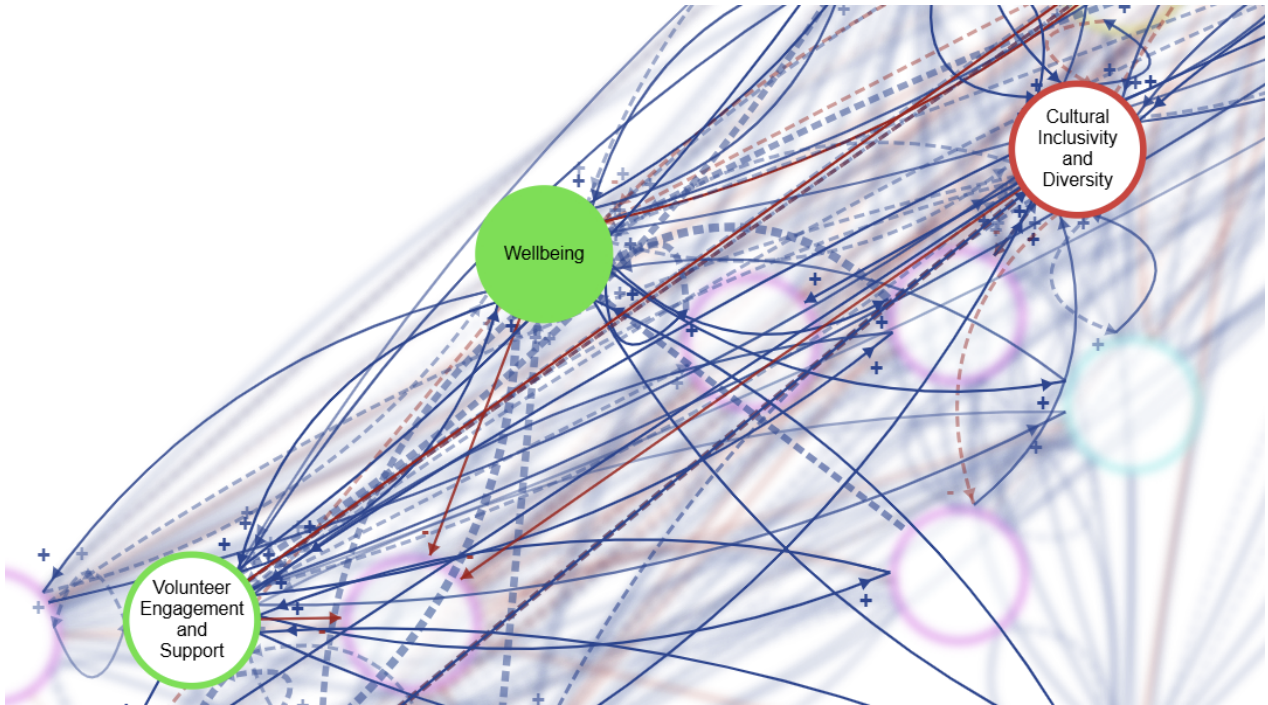


UNIVERSITY OF AMSTERDAM

MASTERS THESIS

Detecting and Mitigating Hallucinations in LLM-Generated Causal Loop Diagrams



Author:
Nitai Nijholt

Examiner:
Dr. Debraj Roy
Supervisor:
Dr. Rick Quax
Assessor:
Dr. Caspar van Lissa

*A thesis submitted in partial fulfillment of the requirements
for the degree of Master of Science in Computational Science
in the*

Computational Science Lab
Informatics Institute

January 2026



Declaration of Authorship

I, Nitai Nijholt, declare that this thesis, entitled ‘Detecting and Mitigating Hallucinations in LLM-Generated Causal Loop Diagrams’ and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at the University of Amsterdam.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

The research for this thesis was conducted within a software environment which is property of Causalix, actively using parts of the code and intellectual property developed by others, therefore, a clear definition of scope (what was developed as part of the thesis, what was not) is in order. The main CLD generator algorithm was conceived by Rick Quax with feedback from Cillian Hourican and implemented by Andrew Crossley. Contributions from the author of this thesis (Nitai Nijholt) on the generation algorithm limited to adding a mediation check and conceiving a number of these prompts, in addition to other prompts being contributed by Rick, Andrew and Cillian.

The gathering and transcribing of validation datasets into computational format of the CLD's Depressive Disorder, Social Norms, and Emergency Response datasets—was done by Cillian Hourican.

The underlying software environment contains substantially more features and potential analysis outputs than are required for the research presented in this thesis; accordingly, only a subset of the available functionality is used. Where any algorithm or script executed within this environment produced additional intermediate artefacts or auxiliary outputs, only the artefacts explicitly referenced in the thesis were used in the reported analyses and to support the thesis conclusions.

The contributions presented in this thesis, developed by the author, encompass: the LLM-as-a-Judge framework, the LLM-as-a-Corrector, the Deep Research multi-agent system uncertainty quantification metrics for the generator component, and the simulation framework. Additionally, the author developed the comprehensive evaluation frameworks for the Generator, Deep Research, and Judge components, the generate-then-search methodology, as well as the parallelization architecture, inference time benchmarking, and cost tracking instrumentation. Any other part of the thesis is developed by Nitai Nijholt unless otherwise stated.

Signed:



Date: 3 January 2026

“All models are wrong, but some are useful.”

George E. P. Box

UNIVERSITY OF AMSTERDAM

Abstract

Faculty of Science
Informatics Institute

Master of Science in Computational Science

Detecting and Mitigating Hallucinations in LLM-Generated Causal Loop Diagrams

by Nitai Nijholt

Causal loop diagrams (CLDs) can map cause and effect in complex systems but are costly for experts to produce, as cost scales with the square of the number of variables. LLMs can generate initial CLDs, reducing costs, but suffer from hallucinations. LLM-powered text-to-CLD extraction is the gold standard for grounding but is constrained by source text availability. The alternative, LLM-based CLD generation, loses grounding, increasing the risk of hallucinations.

Applying established hallucination mitigation methods at test time—through LLM-as-a-judge, LLM-as-a-corrector, and UQ metrics (cosine similarity and logprob-derived measures)—previously untested in CLD generation, we ask: (1) Can LLM-as-a-judge and LLM-as-a-corrector improve CLD F1 versus literature ground truth? (2) Can UQ metrics detect hallucinations? (3) Can adaptive test-time compute, guided by UQ metrics, improve the cost–accuracy trade-off? Findings show, in our dataset of three health CLDs, LLM-based CLD generation performance exceeds a random baseline. In a controlled synthetic hallucination setting (GT Synth), Correctness Judge performance is high, but it drops in the literature ground-truth setting (GT Lit), with only our causal mechanistic prompt beating random baselines. The Citation Judge, which retrieves supporting articles post hoc, fails to exceed random baselines despite reaching 80% alignment with human validators. LLM-as-a-corrector improves judge scores and slightly improves CLD quality vs GT Synth, but not vs GT Lit. UQ metrics (perplexity, min probability, max window entropy, cosine similarity) do not effectively flag hallucinations independently but do as ensemble input to supervised classifiers in-distribution; however, fail to generalise out-of-CLD. As UQ and corrector were ineffective, we pilot a multi-agent deep research (DR) system that finds direct causal evidence for 62.4% of false positives and 42.9% of false negatives. Single-rater human validation ($n=50$) suggests that at applicability threshold $t = 0.7$, 66% of DR-evidenced edges pass the threshold and, among passing edges, 90.9% are correct-causal; however, a preliminary second-rater overlap ($n=13$) indicates only fair agreement on categorical verdicts and poor agreement on applicability scores, so calibration-based extrapolations should be interpreted as exploratory. Under this extrapolation, ground-truth enrichment suggests large gains in aggregate edge F1: $0.267 \rightarrow 0.527$. Using a mechanistic Correctness Judge to flag hallucinations for adaptive deployment of DR yields an estimated 15% accuracy-per-dollar increase. Results suggest that LLM-based CLD generation and multi-agent validation-driven research can assist experts in CLD creation; future work includes validating the system across additional CLD domains and conducting component-wise ablation studies of DR to find more efficient compute–accuracy trade-offs.

Keywords: causal loop diagrams, large language models, hallucination detection, LLM-as-a-Judge, LLM-as-a-Corrector, uncertainty quantification, log probability, test-time compute, multi-agent literature search, causal graph generation, health systems modeling

Acknowledgements

I dedicate this work to my mother, Rati Manjari van Sijll, who always stood behind me and gave me a start in life that I remain thankful for every day. Also to my late father, who started and nurtured my love for computers and engineering, and to my grandmother, who through her stroke was thankful for the care I provided her, while being understanding when this thesis required more of my attention to finish.

I also want to thank my supervisor, Dr. Rick Quax, who provided me with the resources and guidance to conduct this research, and taught me many key principles of modelling, simulation, and scientific data analysis which proved invaluable. I also want to thank my examiner, Dr. Roy, who provided clear, actionable feedback which helped in improving the work. I also want to thank all involved including my second assessor Dr. van Lissa, for their understanding of the thesis timeline revision as a result of caring for my grandmother and their accommodation in its assessment.

Lastly, I want to thank Andrew Crossley especially, for building the software framework within which this research was conducted, Cillian Hourican for his advice and help in providing the ground truth datasets, and Mick Scheide (Data Manager, Antoni van Leeuwenhoek) for providing human validation for RQ1.

Contents

Declaration of Authorship	i
Abstract	iv
Acknowledgements	v
Abbreviations	xix
1 Introduction	1
2 Background	8
2.1 Modelling, simulation, and why conceptual validity matters	8
2.1.1 The role of simulations in science.	8
2.1.2 Model abstraction and computational trade-offs.	8
2.1.3 GMB as a foundation for computational models.	9
2.2 Causal Loop Diagrams (CLDs)	10
2.2.1 Definition.	10
2.3 Group Model Building (GMB) and the CLD-to-Computational Model Pipeline	10
2.4 Large language model (LLM) background	12
2.4.0.1 Architectural details of large language models	12
2.5 Hallucinations in LLMs (general)	18
2.5.1 Taxonomy of Hallucinations	19
2.5.2 Hallucination Mitigation Methods	19
2.5.2.1 Black-box methods.	19
2.5.2.2 White-box methods.	20
3 Literature Review	21
3.1 Standard CLD construction paradigms	21
3.1.1 Participatory group model building (GMB)	21
3.1.2 Literature-driven expert synthesis	21
3.1.3 Hybrid workflows and progression to quantitative models	22
3.2 Automated CLD construction paradigms (text-to-CLD vs. generation)	22
3.3 LLM-Based Causal Loop Diagram Generation	22
3.3.1 From LLM-Powered Causal Pattern Matching to Causal Loop Diagrams	23
3.3.1.1 Positioning of this thesis	24
3.3.2 How Hallucinations Manifest in Causal Structure Generation	25
3.3.2.1 Formal CLD Hallucination Taxonomies	26

3.3.3	CLD Hallucination Mitigation Approaches	26
3.3.3.1	RAG and the constraints of corpus dependence	27
3.3.3.2	Arguing for corpus independence from a practicality standpoint	27
3.3.4	Comparison of LLM-Based CLD Approaches	27
3.4	Prompting strategies for LLM-based causal reasoning	29
3.4.1	Dominant Prompting Paradigms.	29
3.4.2	Few-Shot Prompting.	29
3.4.3	Chain-of-Thought (CoT) Prompting.	30
3.4.4	Role-Based Prompting.	30
3.4.5	Prompting in Causal Discovery Contexts.	30
3.4.6	Prompting for Hallucination Detection and Correction.	31
3.4.7	Bradford Hill Criteria (BHC) as a Prompting Framework.	31
3.4.8	Application to Judge and Corrector Roles.	32
3.4.9	LLM-as-a-Judge.	32
3.4.10	LLM-as-a-Corrector.	32
3.5	LLM-as-a-Judge: Automated Evaluation of LLM Outputs	33
3.5.1	Foundations and Empirical Validation	33
3.5.2	Systematic Biases in LLM Judges	33
3.5.3	Evaluation Modes: Pointwise vs. Pairwise	34
3.5.4	LLM-as-a-Corrector: Beyond Detection to Remediation	34
3.5.5	Implications for CLD Hallucination Detection	35
3.5.6	Limitations	36
3.6	Logprob Uncertainty Quantification (LUQ) Metrics for Hallucination Detection	36
3.6.1	The Uncertainty–Hallucination Link.	37
3.6.2	LUQ Metrics: From Sequence to Token Granularity	37
3.6.2.1	Perplexity (Sequence-Level).	37
3.6.2.2	Maximum Window Entropy (Subsequence-Level).	38
3.6.2.3	Minimum Probability (Token-Level).	38
3.6.3	Worked Example: Applying LUQ Metrics	38
3.6.4	Types of Uncertainty in the Logprob Signal	39
3.6.4.1	Uncertainty sources in autoregressive generation.	39
3.6.4.2	Aleatoric Uncertainty (Data Uncertainty).	39
3.6.4.3	Epistemic Uncertainty (Model Uncertainty).	40
3.6.5	Limitations of LUQ Metrics	40
3.7	Embedding-Based Grounding Metrics	41
3.7.1	Design choices: reference text and input/query text.	41
3.7.2	Retrieval-Augmented Generation	41
3.7.3	Limitations of Retrieval-Constrained Generation	42
3.7.4	A Hybrid Approach: Generate-then-Search	42
3.7.5	Limitations of the Hybrid Approach	42
3.8	Test-time compute and multi-agent validation	43
3.8.1	Serial Compute: Iterative Refinement	43
3.8.1.1	ReAct: Reasoning and Acting.	43
3.8.1.2	Multi-Agent Debate.	43
3.8.1.3	Chain-of-Verification (CoVe).	44
3.8.1.4	Misconception Refutation (MR).	44
3.8.2	Parallel Compute: Sampling and Selection	44
3.8.2.1	Self-Consistency.	44
3.8.2.2	Best-of-N Sampling.	44
3.8.3	Adaptive Compute Allocation	45

3.9	Synthesis and bridge to methods	45
4	Problem Setup	46
4.1	Task: Hallucination Detection and Mitigation for CLD Edges	46
4.2	Hallucination Definition	46
5	System Architecture	49
5.1	Pipeline Overview	49
5.2	Generator	50
5.2.1	Design rationale: Generator	50
5.2.2	Generator implementation	50
5.2.3	Design rationale: UQ metrics	51
5.2.4	UQ metrics implementation	52
5.3	Corruptor (Synthetic corruption for GT Synth)	53
5.4	Judges	53
5.4.1	Design rationale: LLM-as-a-Judge	53
5.4.2	Correctness Judge implementation	54
5.4.3	Citation Judge implementation	54
5.5	Citation Search (Brave)	56
5.6	Corrector	56
5.6.1	Design rationale: LLM-as-a-Corrector	56
5.6.2	Corrector implementation	57
5.7	Deep Research System	57
5.7.1	Design rationale: Deep Research	57
5.7.2	Deep Research implementation	58
6	Experimental Data	61
6.1	Evaluation Datasets (GT Lit, LLM Simulations, GT Synth)	61
6.2	Validation and Auxiliary Datasets	61
6.3	CLD Domains	62
6.4	Context Parameters for LLM Generation	63
6.5	CLD Selection Rationale	63
6.6	Edge Structure	64
7	Experimental Design	65
7.1	Experimental Design Framework	67
7.2	Metrics	69
7.3	Statistical Analysis	74
7.4	Master Experimental Overview	77
7.5	Experimental Details	78
7.6	Preliminary: Generator Selection	78
7.6.0.1	Generator Model Selection	78
7.6.0.2	Generator Prompt Selection (Nitai_C)	79
7.6.0.3	Generator Temperature Selection	80
7.7	Preliminaries: Generator Performance	81
7.8	Preliminary: TruthfulQA Verification	81
7.9	RQ1a: LLM-as-a-Judge Hallucination Detection	82
7.9.0.1	RQ1a: Correctness Judge (Lit)	83
7.9.0.2	RQ1a: Correctness Judge (Synth)	83
7.9.0.3	RQ1a: Citation Judge (Lit)	83
7.9.0.4	RQ1a: Citation Judge (Synth)	84

7.10	Sensitivity: Prompt Sensitivity Analysis	84
7.11	Validation: Uleman’s Provider Study	85
7.11.1	Search Provider Comparison (Uleman’s CLD)	86
7.11.2	Human–LLM Agreement	86
7.11.3	Physics Sanity Check	87
7.11.4	Retrieval Success Analysis	88
7.12	RQ1b: LLM-as-a-Corrector Evaluation	88
7.12.0.1	RQ1b: Correctness Corrector (Synth)	89
7.12.0.2	RQ1b: Correctness Corrector (Lit)	89
7.13	RQ2: Context-Insensitive Hallucination Detection	90
7.13.0.1	RQ2: UQ (Logprobs)	90
7.13.0.2	RQ2: CosSim (Embeddings)	91
7.13.0.3	RQ2: Analysis Pipeline (6 Phases)	91
7.14	RQ3: Deep Research Literature Validation	92
7.14.0.1	Deep Research	92
7.14.0.2	Computational Cost	93
7.14.0.3	Limitations	93
7.14.0.4	Exploratory Analysis: Ground Truth Enrichment	93
7.14.0.5	Human Validation	93
7.14.0.6	Threshold Selection and Extrapolation	94
7.14.0.7	Smart Trigger	95
7.15	Reproducibility	95
7.16	Environment Setup	95
7.17	Unified Analysis Entry Points	95
7.18	Component-Specific Reproduction	96
7.19	Threats to Validity	96
7.19.1	Internal Validity (leakage, pseudo-replication, and dependence)	96
7.19.2	Construct Validity (what “hallucination” measures)	96
7.19.3	External Validity (generalisation across domains, models, and time)	97
7.19.4	Conclusion Validity (inference limits, power, and multiple comparisons)	97
7.19.5	Reliability and Instrumentation (judge and retrieval behaviour)	98
8	Experiments and Results	99
8.1	Generator Model Selection	99
8.2	Generator Performance Baseline	100
8.2.1	Random Baseline Comparison	100
8.2.2	Temperature Sensitivity Analysis	101
8.3	Correctness Judge Verification	102
8.4	RQ1a: Corruption Detection - Correctness	103
8.4.1	Performance Metrics Overview	103
8.4.2	Detailed Analysis	104
8.5	RQ1a: Ground Truth - Correctness	105
8.5.1	Performance Metrics Overview	106
8.5.2	Detailed Analysis	106
8.6	RQ1a: Corruption Detection - Citation	107
8.6.1	Performance Metrics Overview	107
8.6.2	Detailed Analysis	108
8.7	RQ1a: Ground Truth - Citation	109
8.7.1	Performance Metrics Overview	109
8.7.2	Detailed Analysis	110

8.8	RQ1a: External Validation - Uleman's Expert CLD	111
8.8.1	Search Provider Comparison	111
8.9	RQ1a: Human Validation of LLM Citation Judges	112
8.9.1	Per-Dataset and Combined Agreement	112
8.9.2	Disagreement Analysis	113
8.9.3	Physics CLD Validation	113
8.9.4	Disentangling Retrieval Quality from Domain Complexity	114
8.10	RQ1a: Main Findings (Summary)	115
8.11	RQ1b: Corrector Experiments	116
8.11.1	Synthetic Corruption Correctness Results	116
8.11.2	Corrector Prompt Ablation Study (Synthetic Corruption)	117
8.11.3	Ground Truth Correctness Baseline prompt Results	118
8.11.4	Corrector Prompt Ablation Study (Ground Truth)	118
8.11.5	RQ1b: Main Findings (Summary)	119
8.12	RQ2: Hallucination Detection Using UQ Metrics	120
8.12.1	Single UQ Metric Performance	120
8.12.2	Feature Distributions: Correct vs. Hallucinations	122
8.12.3	Recursive Feature Elimination Analysis	124
8.12.4	Ensemble Classification Results	124
8.12.5	RQ2: Main Findings (Summary)	125
8.13	RQ3: Deep Research Validation (Exploratory Pilot)	126
8.13.1	Computational Cost	127
8.13.2	Per-CLD Detection Rate Analysis	128
8.13.3	Exploratory: Ground Truth Enrichment Analysis	129
8.13.4	Smart-Trigger DR Deployment Estimate	132
8.14	RQ3: Main Findings (Summary)	133
8.15	Effect Size Summary	134
8.16	Computational Scaling Analysis	136
8.16.1	Time and Cost Scaling	136
8.16.2	Token Cost Analysis	137
8.16.3	Parallelization Speedup	137
9	Discussion	139
9.1	Direct Answers to the Research Questions	139
9.2	Roadmap for the Detailed Discussion	140
9.3	Preliminary Results: Generator Selection and Baseline Performance	140
9.3.1	Model Selection Rationale	140
9.3.2	Random Baseline Comparison	141
9.3.3	Temperature Hyperparameter Robustness	141
9.3.4	TruthfulQA Verification	142
9.3.5	Definitions of Ground Truth, Hallucination, and Transfer Validity	142
9.4	RQ1a (Correctness Judge): Judge Performance	143
9.4.1	Synthetic Corruption Detection	143
9.4.2	Literature Ground Truth Performance	143
9.4.3	GT Synth vs. GT Lit Performance Gap	143
9.4.4	Corruption Type Detection Patterns	144
9.4.5	Point-Biserial Correlations	144
9.4.6	Social Norms CLD Anomaly	144
9.4.7	Judge Score Distributions	144
9.5	RQ1a (Citation Judge): Judge Performance	145

9.5.1	Retrieval Implementation Considerations	145
9.5.1.1	Limitations of generate-then-search in this thesis.	145
9.5.1.2	Majority voting per citation limitations and future work.	146
9.5.2	Synthetic Corruption Setting	146
9.5.3	The “Evidence of Absence” Paradox	147
9.5.4	Below-Random AUC Performance	147
9.5.5	High Recall Masking Poor Discrimination	147
9.5.6	Uleman’s External Validation	147
9.5.7	Physics CLD Validation and Retrieval Limitations	148
9.6	Mechanisms of Judge Failure	149
9.6.1	The “Evidence of Absence” Paradox (Citation Judge)	149
9.6.2	Plausibility Bias (Correctness Judge)	149
9.6.3	Partial Credit and Non-Scientific Evidence Acceptance (Citation Judge)	150
9.7	Empirical Validation: LLM–Human Agreement in CLD Evaluation	151
9.7.1	Results.	151
9.7.2	Systematic leniency bias.	151
9.7.3	Domain and model variability.	151
9.8	RQ1b: Corrector Experiments	151
9.8.1	Corrector Action Mix and Repair Limitations	151
9.8.2	Goodhart’s Law Manifestation	152
9.8.3	Per-CLD Variation in Corrector Effectiveness	152
9.8.4	Implications for Production Pipelines	153
9.9	RQ2: UQ Metrics	153
9.9.1	Single Metric Performance	153
9.9.2	Counterintuitive Cosine Similarity Direction	153
9.9.3	Non-Normality and Statistical Approach	154
9.9.4	Ensemble Classifier Performance	154
9.9.5	Cross-Domain Generalisation Failure	154
9.9.6	Implications for Original RQ3 Design	155
9.10	RQ3: Deep Research Validation	155
9.10.1	Methodological Pivot and Rationale	155
9.10.1.1	Original Design and Its Prerequisites.	155
9.10.1.2	The Pivot.	155
9.10.2	Design Rationale	156
9.10.3	Detection Rate Findings	156
9.10.4	Per-CLD Detection Rate Variation	156
9.10.5	Confidence Score Uninformativeness	157
9.10.6	Human Validation and Applicability Analysis	157
9.10.7	Ground Truth Enrichment Analysis	157
9.10.8	Adaptive-Trigger Deployment Analysis	157
9.10.9	Limitations of the Pivot	157
9.10.10	Framing: Exploratory Pilot Study	158
9.11	Synthesis Across Research Questions	158
9.12	Practical Recommendations	159
9.13	Limitations	159
9.13.1	Single Model Dependency	159
9.13.2	Health Domain Focus	160
9.13.3	Ground Truth Construct Validity	160
9.13.4	Sensitivity Analysis Constraints	160
9.13.5	Human Validation Sample Size	161

9.13.6	Statistical Power Asymmetry & Sample Size Considerations	161
9.13.7	Limited Stochasticity Control	161
9.13.8	Confirmation Bias in Search	161
9.13.9	Budget Constraints	162
9.13.10	Synthetic Corruption Rate Fixed at 30%	162
9.13.11	Local Embedding Model Limitations	162
9.13.12	Search Provider Heterogeneity	162
9.13.13	Fetching Efficiency and Cost–Performance Tradeoffs	162
9.13.14	Token-Level Uncertainty Quantification	163
9.13.15	Sample Size Considerations	163
9.14	Future Work	163
9.14.1	Completing RQ3: Deep Research Validation	163
9.14.2	Revisiting Adaptive Test-Time Compute	164
9.14.3	Addressing Corrector Limitations	164
9.14.4	General Methodological Extensions	164
9.14.5	Active Learning Integration	165
9.14.6	Addressing False Negatives	165
9.14.7	Causal Context Integration in Judging and Deep Research	165
10	Conclusion	166
10.1	Key Contributions	167
10.2	Concluding Remarks	167
11	Ethics and Data Management	169
11.1	Ethical Considerations	169
11.2	Data Management	169
A	Function-to-Algorithm Mapping	171
B	Supplementary Sensitivity Analysis	172
B.1	Judge Prompt Sensitivity	172
B.1.1	Derivation: First-Order Sobol Index for a Categorical Factor	172
B.2	Corrector Prompt Sensitivity	175
C	Reproducibility	177
C.1	Data Availability	177
C.1.1	Ground Truth CLD Datasets	177
C.1.2	Validation and Auxiliary Datasets	177
C.1.2.1	Physics CLD Details.	178
C.1.3	Experiment Result Datasets by Research Question	179
C.1.4	Final Experiment Results	179
C.2	Code Availability	180
C.2.1	Experiment Execution Scripts	180
C.2.2	Analysis Scripts	181
C.2.3	Complete <code>final_runs/</code> Script and Command Inventory	181
C.2.4	Prompt Configuration Files	182
C.3	Software Environment	183
C.3.1	Environment Installation	183
C.3.2	Key Software Dependencies	184
C.4	Execution Commands	184
C.4.1	Unified Analysis Entry Points (Single-command pipelines)	184

C.4.2	RQ1 Judge Verification (TruthfulQA)	185
C.4.3	RQ1a Ground Truth Experiments	185
C.4.4	RQ1a Corruption Detection Experiments	186
C.4.5	RQ1b Corrector Experiments	187
C.4.6	RQ1a Validation Studies	188
C.4.7	RQ2 Hallucination Detection Analysis	191
C.4.8	RQ3 Deep Research Experiments	191
C.4.9	Generator Temperature Sensitivity Analysis	195
C.4.10	Generator Model Comparison	196
C.4.11	Random Baseline Generator	197
C.4.12	Prompt Sensitivity Analysis	198
C.4.13	Parallelization Benchmark	198
C.4.14	Time and Cost Scaling Analysis	199
C.4.15	Physics CLD Validation	201
C.4.16	Cost Analysis	202
C.4.17	Power Analysis	203
C.4.18	Embedding Model Benchmark	203
C.5	Output Artefacts	204
C.5.1	Metadata Files	204
C.5.2	Result Files	204
C.5.3	Naming Convention	204
C.6	Output Token Verification	204
D	Statistical Procedures Reference	206
D.0.0.1	Omnibus and Post-Hoc Tests.	206
D.0.0.2	Parametric Sensitivity Analysis (Appendix B).	206
D.0.0.3	Multiple-Comparison and Interval Methods.	207
D.0.0.4	RQ2 Classifier Algorithms (Ensemble Models, Hyperparameters).	207
E	Experimental Prompts	208
E.1	Prompt Engineering Ablation Study	208
E.1.1	Experimental Design	208
E.1.2	Prompt Engineering Techniques	208
E.1.3	Results	210
E.1.4	Key Findings	210
E.1.5	Interpretation	211
E.1.6	Limitations	211
E.2	Generator Prompt (Nitai_C)	211
E.3	Correctness Judge Prompts	215
E.3.1	Baseline	215
E.3.2	Chain-of-Thought (CoT)	215
E.3.3	Mechanistic (Bradford Hill)	216
E.4	Citation Judge Prompts	217
E.4.1	Baseline	217
E.4.2	Chain-of-Thought (CoT)	218
E.4.3	Mechanistic (Bradford Hill)	218
E.4.4	Mechanistic-Lit (Literature-Grounded)	219
E.5	Corrector Prompts	220
E.5.1	Baseline	220
E.5.2	Chain-of-Thought (CoT)	221

E.5.3	Mechanistic (Bradford Hill)	223
F	Node Generation Prompt Ablation	226
G	Embedding Model Benchmark	227
H	Algorithmic Descriptions	229
H.1	Algorithm 0: CLD Generation	229
H.2	Algorithm 1: Judge-Correctness	230
H.3	Algorithm 2: Judge-Citation	230
H.4	Algorithm 3: Corruption	231
H.5	Algorithm 4: Corrector	232
H.6	Algorithm 5: UQ Metrics Computation	233
H.7	Algorithm 6: Hallucination Classification	233
H.8	Algorithm 7: Citation Search (Brave)	234
H.9	Algorithm 8: Citation Fetcher	235
H.10	Deep Research Multi-Agent Components	236
H.11	Algorithm 9: Deep Research Multi-Agent Pipeline	237
H.12	Algorithm 10: Node Concept Similarity (Similarity-Weighted Node F1)	239
I	ROC Curves	241
I.1	Corruption Detection Correctness ROC Curves	241
I.2	Corruption Detection Citation ROC Curves	242
I.3	Ground Truth Correctness ROC Curves	242
I.4	Ground Truth Citation ROC Curves	243
J	Judge Score Heatmaps	244
J.1	Ground Truth Correctness Heatmap	244
J.2	Ground Truth Citation Heatmap	245
J.3	Corruption Detection Correctness Heatmap	246
J.4	Corruption Detection Citation Heatmap	247
	Bibliography	249

List of Figures

Figure 1.1	Expert review burden vs. CLD size	3
Figure 2.1	Tokenization and embedding pipeline	12
Figure 2.2	Attention mechanism	13
Figure 2.3	Multi-head attention	14
Figure 2.4	Encoder and decoder building blocks	15
Figure 2.5	Encoder-decoder translation example	15
Figure 2.6	Transformer architecture	16
Figure 2.7	Decoder-only LLM architecture	17

Figure 2.8	Temperature scaling	17
Figure 3.1	Token probabilities during LLM generation.	39
Figure 5.1	CLD pipeline overview	49
Figure 5.2	Deep Research multi-agent pipeline	59
Figure 7.1	End-to-end experimental pipeline overview.	66
Figure 8.1	Generator vs. random baseline (edge F1)	101
Figure 8.2	Corruption detection metrics (Correctness Judge)	104
Figure 8.3	Corruption detection analysis (Correctness Judge)	105
Figure 8.4	GT validation metrics (Correctness Judge)	106
Figure 8.5	GT validation analysis (Correctness Judge)	107
Figure 8.6	Corruption detection metrics (Citation Judge)	108
Figure 8.7	Corruption detection analysis (Citation Judge)	109
Figure 8.8	GT validation metrics (Citation Judge)	110
Figure 8.9	GT validation analysis (Citation Judge)	110
Figure 8.10	UQ metric distributions by CLD	122
Figure 8.11	RFE analysis for hallucination detection	124
Figure 8.12	Deep Research results	128
Figure 8.13	Human validation tradeoff curve	131
Figure 8.14	Deep Research cost-accuracy tradeoff	133
Figure 8.15	Generation vs. judging scaling	136
Figure 8.16	Parallelization benchmark	138
Figure B.1	Prompt sensitivity analysis (RQ1a)	173
Figure B.2	Corrector prompt sensitivity (RQ1b)	175
Figure C.1	CLD judging time scaling	200
Figure C.2	Judging efficiency by configuration	201
Figure E.1	Average F1 by prompt	210
Figure I.1	ROC: corruption (correctness)	241
Figure I.2	ROC: corruption (citation)	242
Figure I.3	ROC: GT (correctness)	242
Figure I.4	ROC: GT (citation)	243
Figure J.1	GT correctness: judge score heatmap	245
Figure J.2	GT citation: judge score heatmap	246
Figure J.3	Corruption (correctness): judge score heatmap	247
Figure J.4	Corruption (citation): judge score heatmap	248

List of Tables

Table 3.1	LLM-based CLD generation approaches	28
-----------	---	----

Table 4.1	Two uses of TP/FP/FN in this thesis (generator scoring vs. judge scoring).	47
Table 5.1	UQ metrics captured during edge generation. Conceptual formulas shown; implementation uses capping for numerical stability (see Section 7.2).	52
Table 6.1	LLM context parameters by CLD	63
Table 6.2	Example edge row	64
Table 6.3	Example corrector columns	64
Table 6.4	Example UQ metric columns	64
Table 7.1	Master experimental configuration	77
Table 8.1	Generator Model Comparison: Edge Recovery Performance. Bold indicates highest F1 per CLD; in aggregate row, bold indicates highest Precision, Recall, and F1 separately. Edges processed refers to the number of ordered (source, target) edge slots evaluated (including NONE).	100
Table 8.2	Random Baseline Comparison for Generator Edge Recovery	101
Table 8.3	Generator Temperature Sensitivity: Edge F1 Score by CLD	102
Table 8.4	TruthfulQA Judge Verification (n=1,634)	103
Table 8.5	Search Provider Comparison on Uleman’s Expert CLD (150 edges)	111
Table 8.6	Human Validation: LLM Judge vs. Expert Inter-Rater Agreement	112
Table 8.7	Confusion Matrix: LLM Judge vs. Human Expert (Combined, n=72)	113
Table 8.8	Physics CLD Validation: Correctness vs Citation-Based Judging. “Citation (All)” shows approval across all edges; “Citations Retrieved” shows the percentage of edges where citation content was successfully fetched; “Citation (Retr.)” shows approval among edges with successfully retrieved citations.	113
Table 8.9	Retrieval Success Comparison. Expert-provided references are curated by domain experts; automated fetch uses web search. Retrieval success indicates percentage of edges where citation content was successfully fetched.	114
Table 8.10	Corrector Performance on Synthetically Corrupted Data (Baseline Prompt)	116
Table 8.11	Corrector Prompt Ablation on Synthetic Corruptions: Performance Comparison	117
Table 8.12	Corrector Performance on Ground Truth Data Using Correctness-Based Validation (Baseline Prompt)	118
Table 8.13	Corrector Prompt Ablation on Ground Truth: Performance Comparison	119
Table 8.14	Single UQ Metric Performance for Hallucination Detection (Meta-Analysis)	121
Table 8.15	Shapiro-Wilk Normality Tests for UQ Metric Distributions	123
Table 8.16	Ensemble Classifier Performance Across Validation Phases (Block-Level Cross-Validation)	125
Table 8.17	Deep Research Detection Rate and Discrimination Analysis (n=285 edges)	127
Table 8.18	Deep Research Computational Cost by CLD	127
Table 8.19	Per-CLD Pairwise Discrimination Tests (Fisher’s Exact, Bonferroni $m = 3$)	129
Table 8.20	LLM Generation Metrics: Two Enrichment Scenarios (Aggregate and Per-CLD)	130
Table 8.21	Human Validation: Evidence Verdict Distribution by Edge Classification ($n = 50$)	131
Table 8.22	Applicability Score Rubric for Human Validation	131
Table 8.23	Human validation tradeoff curve (pooled): coverage vs. correct-causal rate	131
Table 8.24	LLM Generation Metrics with Human-Validated Ground Truth Enrichment (Aggregate, extrapolated)	132
Table 8.25	Smart-trigger Deep Research enrichment estimates. Trigger: Mechanistic GT correctness score ≤ 0.5 . Calibration: human-validated applicability ≥ 0.7	132
Table 8.26	Effect Size Summary: Hypothesis Operationalisation, Statistical Tests, and Conclusions	135
Table 8.27	Computational Efficiency Summary by Experiment	137
Table 8.28	Parallelization Benchmark: Wall-Clock Time and Speedup by Judge Type	138

Table 11.1	Data and code availability for thesis reproducibility.	170
Table A.1	Complete Function-to-Algorithm Mapping	171
Table B.1	Judge Prompt Sensitivity Analysis	174
Table B.2	RM-ANOVA Assumption Checks for Judge Prompt Sensitivity (Appendix K)	174
Table B.3	Corrector Prompt Sensitivity Analysis	175
Table B.4	RM-ANOVA Assumption Checks for Corrector Prompt Sensitivity (Appendix K)	176
Table C.1	Ground Truth CLD Dataset Files	177
Table C.2	Validation and Auxiliary Datasets	177
Table C.3	Physics CLD Validation Dataset	178
Table C.4	Experiment Result Datasets by Research Question	179
Table C.5	Primary Experiment Execution Scripts	180
Table C.6	Analysis and Evaluation Scripts	181
Table C.7	Complete <code>final_runs/</code> Script and Command Inventory	182
Table C.8	Prompt Configuration Files (13 prompts: 2 judge tasks $\times$ 3 variants + 1 mech-lit + 1 corrector $\times$ 3 variants + 3 corruption)	183
Table C.9	System and Runtime Environment	183
Table C.10	Key Software Dependencies and Citations	184
Table C.11	TruthfulQA Verification Configuration	185
Table C.12	GT Synth Citation Judge Configuration (GPT-5-mini)	186
Table C.13	RQ1a GT Lit Correctness Judge Configuration	186
Table C.14	RQ1a GT Synth Correctness Judge Configuration	187
Table C.15	RQ1a GT Lit Citation Judge Configuration	187
Table C.16	RQ1b Corrector (GT Synth) Configuration	188
Table C.17	RQ1b Corrector (GT Lit) Configuration	188
Table C.18	Uleman’s Citation Provider Comparison Configuration	189
Table C.19	Human-LLM Agreement Validation Configuration	189
Table C.20	Human validation rubric for RQ1	190
Table C.21	RQ2 UQ Logprobs Configuration	191
Table C.22	RQ2 Cosine Similarity (Embeddings) Configuration	191
Table C.23	RQ3 Deep Research Configuration	192
Table C.24	GT Enrichment Experiment Configuration	193
Table C.25	Human validation rubric for RQ3	194
Table C.26	RQ3 Human Validation: Evidence Applicability Score Distribution by Edge Classification ($n = 50$)	195
Table C.27	Temperature Sensitivity Experiment Configuration	196
Table C.28	Generator Model Comparison Configuration	197
Table C.29	Random Baseline Configuration	197
Table C.30	Prompt Sensitivity Analysis Configuration	198
Table C.31	Parallelization Benchmark Configuration	199
Table C.32	Time and Cost Scaling Analysis Configuration	199
Table C.33	Physics CLD Validation Configuration	202
Table C.34	Cost Analysis Configuration	202
Table C.35	Embedding Model Benchmark Configuration	203
Table C.36	Experiment Output File Types	204
Table C.37	Output Token and API Call Analysis by Prompt Variant	204
Table E.1	Prompt Engineering Techniques Classification Matrix	209
Table E.2	F1 Scores by Prompt Variant and Causal Loop Diagram	210

Table F.1	Node Generation Ablation: Hybrid F1 Scores by Prompt Variant	226
Table G.1	Embedding Model Benchmark ($n = 30$ causal narratives from CLD)	227
Table G.2	MTEB Scores: MPNet vs. Qwen3-Embedding-8B	227
Table H.1	CLD Generation (<code>generate_variables + discover_relationships</code>)	229
Table H.2	Judge-Correctness (<code>judge_all_edges_serial</code> , approach=“correctness”)	230
Table H.3	Judge-Citation (<code>judge_all_edges_with_citations_serial</code>)	231
Table H.4	Corruption (<code>_corrupt_relationships</code>)	232
Table H.5	Corrector (<code>correct_edges_serial</code>)	232
Table H.6	UQ Metrics (<code>logit_metrics.py</code>)	233
Table H.7	Hallucination Classification (<code>train_classifiers</code>)	234
Table H.8	Citation Search (<code>_search_with_brave + SearchEngine.search</code>)	235
Table H.9	Citation Fetcher (<code>fetch_citations_with_jina</code>)	236
Table H.10	Deep Research Multi-Agent Pipeline	237
Table H.11	Deep Research Pipeline (<code>pydantic_ai_deepresearch_orchestration.py</code>)	238
Table H.12	Node Concept Similarity (<code>compare_session_graph_to_validation</code> , similarity-weighted node metrics)	240

Abbreviations

CLD	C ausal L oop D iagram
LLM	L arge L anguage M odel
Judge	LLM -as-a- J udge
Corrector	LLM -as-a- C orrector
UQ	U ncertainty Q uantification
RAG	R etrieval- A ugmented G eneration
DR	D eep R esearch
GT Lit	G round T ruth from L iterature
GT Synth	G round T ruth S ynthetic
FP	F alse P ositive
FN	F alse N egative
TP	T rue P ositive
CoT	C hain-of- T hought
BHC	B radford H ill C riteria
GMB	G roup M odel B uilding
LUQ	L ogprob U ncertainty Q uantification
AUC	A rea U nder the C urve
ROC	R eciever O perating C haracteristic
CI	C onfidence I nterval
ANOVA	A nalysis of V ariance
API	A pplication P rogramming I nterface
RFE	R ecursive F eature E limination
NLP	N atural L anguage P rocessing

Chapter 1

Introduction

Causal Loop Diagrams (CLDs) are qualitative representations of hypothesized causal structure—directed graphs where nodes represent variables and signed edges represent positive or negative causal influences—that serve as precursors to computational models, enabling intervention simulations and informing new theories and experiments. They make feedback structure explicit (reinforcing and balancing loops), support rapid hypothesis generation about plausible mechanisms, and provide a shared language for interdisciplinary synthesis [1, 2]. CLDs can also be translated into quantitative system dynamics models (e.g., stock–flow and differential-equation simulations), enabling simulation of trajectories over time, counterfactual intervention testing, and analysis of sensitivity and unintended consequences in complex feedback systems [1].

In practice, CLDs are typically produced through a labour-intensive, iterative workflow: practitioners combine literature review with expert interviews and facilitated Group Model Building (GMB) workshops to elicit variables, surface feedback loops, negotiate scope, and converge on a shared causal story that is “good enough” for the model’s purpose [1, 2]. Over the last decades, system dynamicists have experimented with approaches to increase stakeholder involvement and learning in the model-building process [3]. Yet a substantial portion of the effort remains qualitative and deliberative: teams must decide not only *whether* a proposed influence exists, but whether it is meaningfully *direct* or better represented via intermediary concepts, and what evidence or rationale supports the claim [4, 5]. This makes CLD construction sensitive to practical constraints (time, coordination costs, and cognitive biases), even before any formal simulation model is built [3].

The information aggregation challenge. The rapid increase in research output brings opportunities for knowledge synthesis and subsequent new knowledge creation, but also significant challenges in efficiently aggregating this information. This aggregation step is crucial to presenting a complete perspective in which interactions between variables are accounted for—a key factor in understanding complex biological systems in the clinic, in research, and for constructing computational models. However, individual researchers cannot keep up with new research given the exponentially increasing volume of publications and the fundamental limits of knowledge accumulation feasible for a single person.

Assuming directed pairwise relationship discussion, CLD construction cost scales as $O(|V|^2)$ for a system with $|V|$ variables, making large-variable systems prohibitively expensive to create—especially through group model-building sessions where expert time is scarce and coordination is challenging (Figure 1.1). Figure 1.1 illustrates this theoretical expert review burden: candidate relationships scale as $O(|V|^2)$; baseline assumes 5 min/pair across 3 experts, while an LLM-assisted workflow assumes 50% valid generation requiring only light validation (1 min/pair) and the remaining 50% requiring full review, yielding the potential time savings indicated by the shaded area. This thesis focuses on health-domain CLDs—specifically models of depressive symptoms, social norms & obesity, and older persons’ emergency department visits—where causal modelling directly informs clinical decision-making and intervention design.

To address this limitation and aggregate siloed knowledge, domain expert sessions can be used to exchange and compile specialized knowledge into a single causal loop diagram, which can then be converted into a computational model for use in research, diagnosis, and intervention simulation. However, since expert time is scarce, these sessions are difficult to organise. Expert time is too valuable to be spent reiterating the basics, yet establishing consensus there is often precisely what a large part of these modelling sessions is spent on, when it could be better utilised for adding specialized contributions to the computational model.

If an initial starting model could be provided that aligns with the consensus in scientific literature, experts could spend their limited time contributing specialized knowledge rather than reiterating established findings. This would lead to more comprehensive models built more efficiently.

LLMs as literature synthesis engines. This framing reveals a key analogy and a potential solution. In traditional GMB, domain experts synthesize their knowledge—drawn from years of reading scientific literature, clinical experience, and theoretical training—into a shared causal model. Large language models, trained on vast corpora of scientific text, perform an analogous synthesis: they encode statistical patterns over how causal relationships are described in the literature. When an LLM generates the claim “stress increases cortisol,” it reflects not a discovery of causality but a reproduction of how scientists have written about this relationship across thousands of papers.

Given large language models trained on trillions of tokens, a portion of which consists of scientific literature—the primary source and record of most expert knowledge—LLMs have the advantage of effectively compressing orders of magnitude more information than a single expert can process in a lifetime. They also allow for the retrieval of relevant information in negligible time, as well as enabling synthesis across techniques applied in separate, often siloed, branches of science, thereby facilitating knowledge synthesis, which supports and accelerates new knowledge generation.

This analogy suggests a practical value proposition: if LLMs can reliably reproduce expert-hypothesized causal structures at inference time, they can provide *literature consensus as a starting point* for GMB. Rather than replacing expert deliberation, LLM-generated CLDs could serve as initial drafts that experts refine, accelerating the costly elicitation process.

The hallucination problem. However, this paradigm shift is contingent on one critical requirement: the LLM must faithfully reproduce relevant information from the literature. LLMs are known to hallucinate—generating plausible-sounding but factually incorrect content—a challenge that persists despite increases in pretraining compute and data scale [6, 7]. If the starting CLD produces too many spurious connections instead of valid representations of literature findings, the generated CLD will not serve as a useful starting point; it may actively mislead expert deliberation.

Current literature on LLM-based CLD construction focuses primarily on *causal translation*—extracting causal relationships from provided text—which can reduce hallucination through Retrieval-Augmented Generation (RAG). However, this approach assumes availability of a relevant corpus and is fundamentally capped by retrieval quality. An alternative paradigm is *LLM-based CLD generation*, where the model synthesizes causal relationships from its parametric knowledge without requiring an external corpus. This approach is relatively inexpensive and corpus-independent, but suffers from hallucinations that must be detected and mitigated.

Corpus-independent hallucination reduction methods—such as multi-agent LLM-as-a-Judge configurations and low-cost Logprob Uncertainty Quantification (LUQ)—have shown promise for reducing hallucinations in general LLM applications, but remain untested in the CLD generation context. The critical question—which this thesis addresses—is whether we can detect when LLMs diverge from expert consensus (hallucinate) and whether such divergences can be corrected or validated.

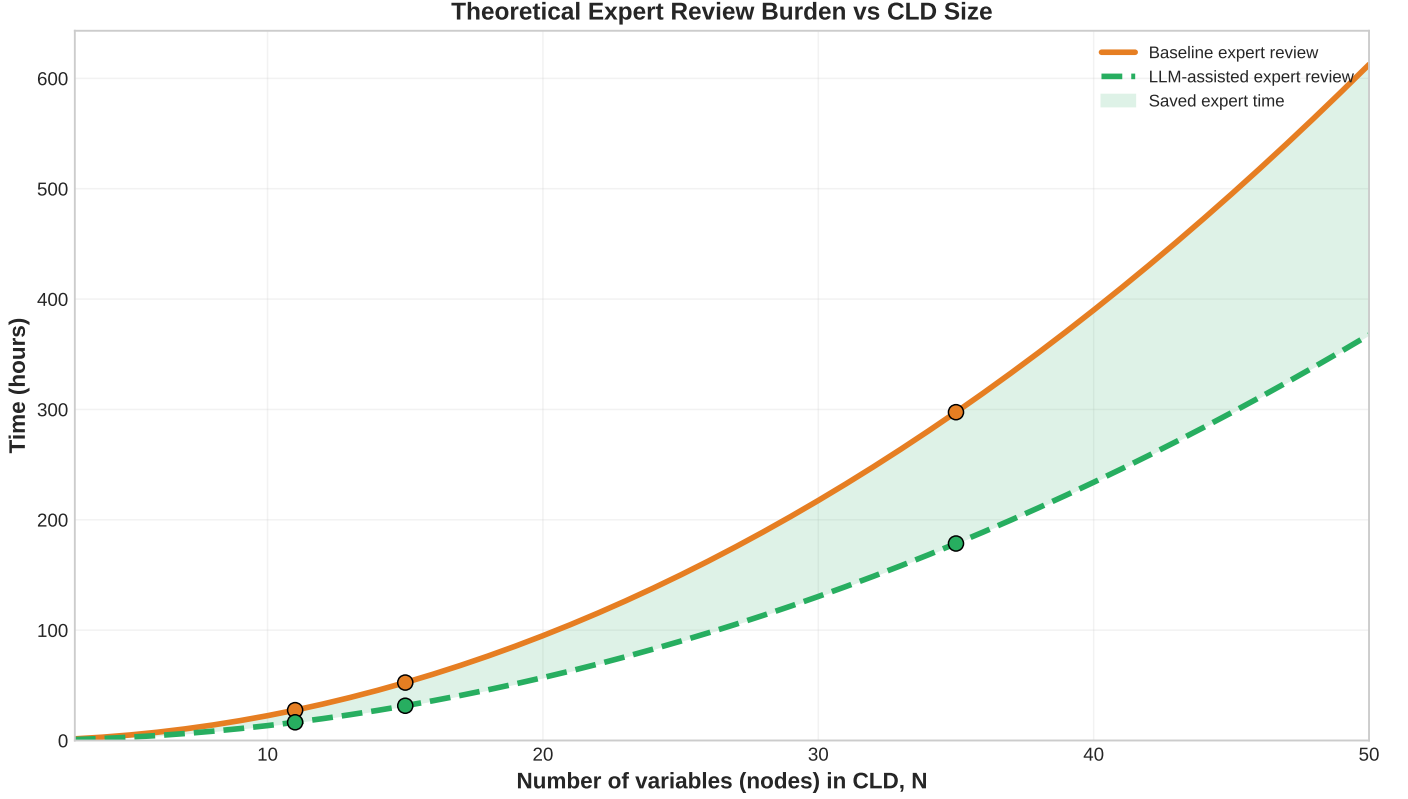


Figure 1.1: Theoretical expert review burden vs CLD size. Candidate relationships scale as $O(|V|^2)$. Baseline assumes 5 min/pair across 3 experts. LLM-assisted assumes 50% valid generation requiring only light validation (1 min/pair); remaining 50% require full review. Shaded area shows potential time savings.

Operationalizing hallucinations and ground truth. Hallucinations in LLMs manifest as confabulations, inconsistencies, or failures to follow instructions [8, 9]. In the CLD context, we operationalise *hallucinations* as incorrectness in causal links: the erroneous presence (false positive) or absence (false negative) of edges, wrong polarity, or invalid causal reasoning. To evaluate LLM-generated CLDs, we require ground truth—CLDs against which generated outputs can be compared.

This thesis uses two complementary ground truth types. *Literature-derived ground truth* (GT Lit) consists of CLDs published in peer-reviewed literature with explicitly defined causal structures (named variables, directed edges, polarities). A generated edge is classified as a *true positive* if it matches an edge in the literature CLD, a *false positive* if it appears in the generated CLD but not the literature CLD, and a *false negative* if it appears in the literature CLD but was not generated. Both false positives and false negatives are considered hallucinations: FPs represent spurious causal claims, while FNs represent failures to capture established causal relationships. *Synthetic ground truth* (GT Synth) provides controlled hallucinations by programmatically corrupting valid LLM-generated edges—introducing spurious edges, flipping polarities, or replacing valid causal reasoning with plausible-but-flawed explanations. Here, hallucination status is known by construction (corrupted = hallucination), enabling precise measurement of detection methods under controlled conditions.

This dual operationalisation enables quantitative evaluation using standard classification metrics (precision, recall, F1). GT Synth provides controlled conditions for method development, while GT Lit tests generalisation to realistic expert-derived ground truth—acknowledging that literature CLDs represent expert-hypothesized relationships rather than empirically validated causal ground truth, a common limitation in complex systems where controlled experiments are infeasible.

Generation: reproduction, synthesis, or new knowledge creation? Can LLMs perform causal inference? This is a key question, and the answer is not apparent from the properties of the autoregressive language modelling process and the next-token prediction objective [10]. The goal of an LLM is to produce the most likely token in a sequence based on its training data. Given this, when prompting an LLM for causal relationships between variables A and B, the model produces an aggregate best guess—the most likely continuation of the sequence given its training data, reflecting patterns learned from related sequences. For reproducing literature consensus or performing synthesis, this may be sufficient, as the LLM has been trained on scientific literature. Although, it may of course not reproduce perfectly, or contain the same biases as the literature, in addition to LLM-architectural biases or biases in its training set. Also, for causal inference in novel settings (for which no relevant data is in its training set), LLMs may not be reliable, as they are known to perform worse out-of-distribution [11, 12], and estimates of out-of-distribution performance may be overstated as benchmarks enter training datasets through retraining processes. This serves as a note of caution against assuming performance transfers to out-of-distribution settings—a concern amplified by the fact that for many commercial LLMs, the training distribution is unknown, making this impossible to verify.

Furthermore, whether, when, and to what extent LLMs can effectively compress and translate the information reflected in their training data to token space such that it leads to correct answers remains a difficult question. State-of-the-art models exhibit unpredictable failure modes and jagged capability frontiers [13]. Established practices exist to identify and mitigate these hallucinations, and this thesis draws from and applies these methods to the CLD generation context.

Beyond reproduction: expert bias mitigation and the possibility of surfacing overlooked relationships. Crucially, it is possible that not all LLM-generated edges that diverge from expert models are errors. Expert groups operate under time constraints, cognitive biases, and disciplinary blind spots; they may omit relationships due to scope decisions, lack of awareness of adjacent literatures, or simple oversight. This problem intensifies in large-variable systems: for a CLD with $|V|$ variables, considering all pairwise relationships scales as $O(|V|^2)$, making exhaustive deliberation impractical (cf. Figure 1.1). An expert group modelling a 50-variable system faces 2,450 potential edges—far more than can be carefully discussed in a typical workshop.

In such settings, LLM-generated edges not found in expert CLDs may represent plausible causal relationships supported by scientific literature that experts simply did not consider. If validated against the literature, these edges could *augment* expert models rather than corrupt them. If evidence supports LLMs’ effectiveness in this manner, it could reframe the LLM’s role: not merely as a reproducer of expert consensus, but also as a tool for surfacing relationships the literature supports but experts overlooked. The challenge then becomes distinguishing genuine hallucinations (unsupported fabrications) from overlooked-but-valid relationships—a distinction that motivates the Deep Research validation approach explored in this thesis.

The false negative problem. Conversely, *false negatives*—relationships that experts include but LLMs omit—represent a distinct challenge. These gaps may arise from several sources: (1) tacit clinical or practical knowledge that experts hold but is underrepresented in published literature; (2) recent findings not yet incorporated into LLM training data; (3) domain-specific terminology that differs from how relationships are described in text; (4) relationships established through unpublished pilot studies, clinical intuition, or cross-disciplinary insights that experts bring to GMB sessions; or (5) LLM reproduction failure—cases where the relationship *is* well-documented in literature but the LLM fails to retrieve or generate it, representing a form of omission-hallucination distinct from fabrication. False negatives are particularly problematic because they represent genuine expert knowledge that LLM-generated drafts would miss, potentially leading to incomplete models if experts do not carefully review and augment the LLM output.

Mitigating hallucinations. LLM hallucinations—fabricated or inconsistent outputs—undermine reliability for scientific applications [14]. A growing body of work has explored both black-box and white-box mitigation strategies. Black-box approaches include prompting techniques such as chain-of-thought reasoning [15, 16],

retrieval-augmented generation [17, 18], multi-agent orchestration and debate [19, 20], LLM-as-a-Judge evaluation [21], and test-time compute scaling [19, 22, 23]. White-box approaches—which require model weight access—include fine-tuning [24], reinforcement learning from human feedback [25], and activation monitoring or steering [26]. These methods have shown promise for reducing hallucinations across various NLP tasks [27].

LLM-as-a-Judge (Judge) approaches offer the advantage of context-awareness: they can evaluate outputs against nuanced domain requirements without requiring explicit ground truth, based on the parametric knowledge of the Judging LLM. However, Judges remain vulnerable to biases inherent to transformer architectures and can be susceptible to adversarial manipulation [28, 29]. Conversely, more naive hallucination detection UQ metrics based on generator logprobs or retrieval based approaches—such as embedding-based similarity [17, 18], perplexity [27, 30], and minimum generation probability [14]—provide computationally efficient signals that are robust to contextual manipulation but lack semantic understanding of task requirements. The combination of LLM-as-a-Judge with naive UQ metrics remains under-explored in CLD generation, yet represents a promising approach: UQ metrics could flag high-uncertainty generations for judge evaluation, potentially reducing the biases of judge-only systems while enabling more efficient allocation of test-time compute for self-correction. This motivates the research questions of this thesis.

Research Questions

This thesis investigates how LLM-as-a-Judge and UQ metrics can detect and mitigate hallucinations in LLM-generated CLDs, structured around the following research questions:

RQ1: Can LLM-as-a-Judge methods detect hallucinations in LLM-generated CLDs?

RQ1a: How effectively can correctness-based and citation-based judges distinguish valid from hallucinated causal relationships?

RQ1b: Can LLM-as-a-Corrector improve CLD accuracy when guided by judge feedback?

RQ2: Can uncertainty quantification (UQ) metrics predict hallucinations in generated CLDs?

Specifically, can logprob-derived uncertainty metrics (perplexity, minimum probability, window entropy) and embedding-based similarity detect hallucinated edges?

RQ3: Can smart test-time compute allocation improve CLD accuracy efficiency?

Can adaptive deployment of LLM-as-a-Judge & Corrector—triggered by generator uncertainty metrics—provide a more efficient accuracy-per-dollar tradeoff than always-on LLM-as-a-Judge & Corrector?

Significance and Contributions

This research addresses reliability challenges in LLM-based scientific knowledge generation by systematically evaluating hallucination detection methods in the CLD domain. The work is timely: as LLM-based systems are increasingly deployed for knowledge work assistance across institutional and global scales, the accuracy of AI-generated outputs carries significant real-world consequences—particularly in health domains where CLDs inform clinical decision-making and intervention design.

Integrating hallucination detection techniques within a CLD-building system is novel and addresses a critical gap: while LLM-as-a-Judge and LUQ methods have been studied separately in general NLP contexts, their combined application to structured scientific knowledge generation—specifically causal diagrams—remains unexplored. This thesis makes the following contributions:

1. **Systematic evaluation of LLM-as-a-Judge for CLD validation.** We compare correctness-based and citation-based judges across multiple prompt variants and validation settings. We establish that judges achieve substantial agreement with human experts, but exhibit systematic leniency bias that makes them better suited for screening and triage than definitive validation. Performance transfers only partially from synthetic corruptions to expert-derived ground truth, with effectiveness varying by domain and prompt design.
2. **Evaluation of LLM-as-a-Corrector for CLD refinement.** We assess whether LLM correctors, conditioned on judge feedback, can improve CLD quality. We find that correctors yield modest gains on synthetic corruptions but fail to improve accuracy against literature-derived expert ground truth. A key finding is that correctors optimise for judge satisfaction rather than expert consensus—judge scores improve while ground truth alignment worsens—suggesting a limitation of judge-guided correction in our experimental setup.
3. **Analysis of generator uncertainty metrics for hallucination detection.** We evaluate whether generation-time signals (logprob-derived uncertainty and embedding similarity) can predict hallucinated edges without external retrieval. We find that individual metrics provide weak discriminative signal, and while ensemble classifiers achieve strong in-distribution performance, they exhibit poor cross-domain generalisation, limiting their utility as standalone detectors.
4. **Deep Research validation paradigm.** We pilot automated multi-agent literature search as an alternative to correction-based approaches, shifting from *correction* to *validation*. We find that Deep Research retrieves direct causal evidence for a substantial fraction of edges classified as false positives *and false negatives* against expert ground truth. Under strong assumptions about evidence applicability and edge-level construct alignment, this suggests that expert CLDs may systematically underestimate LLM generation quality and that some apparent “hallucinations” (edges not found in literature ground truth CLD’s) could represent overlooked-but-valid causal relationships in isolation. However, during second-rater validation, we find low inter-rater reliability, meaning more research is required.
5. **Ground truth enrichment framework.** We develop and evaluate a methodology for enriching expert CLDs with literature-validated LLM-generated edges. Through human validation, we identify causal evidence the Deep Research system finds to be strong and largely applicable to the evaluated causal relationships, suggesting that the ground truth may be incomplete and that the LLM-generated CLDs may be missing causal relationships under strong assumptions (relationship isolation; extrapolation of evidence find rate/quality in the human sample). Including these relationships in the ground truth leads to a substantial improvement in the F1 score of the LLM-generated CLDs. If the effect proves robust after further validation and relaxing of assumptions (a direction for future work), it reframes the evaluation of LLM-generated CLDs: rather than viewing divergence from expert models as error, validated divergences may represent opportunities for model augmentation. However, low inter-rater reliability on human validation of system results strongly cautions against such an optimistic interpretation and warrants further investigation before more validation is done.

The practical implication is that LLM-generated CLDs, when validated through the methods evaluated here, can provide literature-grounded starting points for expert model-building sessions. This allows domain experts to focus on contributing specialized knowledge rather than reiterating established consensus, improving the efficiency of the modelling process and resulting in more comprehensive CLDs for translation into computational models for research, clinical applications, and intervention simulations.

Thesis Structure

- Chapter 2 – Background:** Formalizes CLDs as signed directed graphs, situates them within the modelling-simulation cycle, gives an overview of LLM architecture, used prompting techniques, hallucination issues, and established solutions from other fields.
- Chapter 3 – Literature Review:** Contrasts text-to-CLD versus corpus-independent generation paradigms; surveys LLM-as-a-Judge, LUQ metrics, and test-time compute strategies in more detail.
- Chapter 4 – Problem Setup:** Defines the CLD generation task, formalizes hallucination types (FP/FN, polarity, direction, mechanistic validity), and specifies the GT Lit and GT Synth ground-truth settings used throughout RQ1–RQ3.
- Chapter 5 – System Architecture:** Describes the end-to-end system components: generator, correctness and citation judges, the corrector, and the Deep Research (DR) multi-agent literature validation pipeline.
- Chapter 6 – Experimental Data:** Details evaluation datasets (GT Lit, LLM simulations, GT Synth), validation and auxiliary datasets, CLD domains, context parameters, and edge structure conventions.
- Chapter 7 – Experimental Design:** Specifies the design framework, evaluation metrics, baselines, statistical analysis plan, and master experimental overview across RQ1–RQ3.
- Chapter 8 – Experiments and Results:** Reports generator baseline comparisons; judge performance on synthetic versus literature ground truth; corrector effects on CLD F1 and judge scores; cross-domain UQ metric generalisation failure; ensemble classifier performance; and DR found causal evidence for 62% of “hallucinations.”
- Chapter 9 – Discussion:** Interprets the GT Synth–GT Lit performance gap; identifies judge and corrector failure mechanisms; discusses UQ metric results and the pivot to DR that reframes many false positives as overlooked-but-valid edges; discusses limitations and offers directions for future work and practitioner recommendations.
- Chapter 10 – Conclusion:** Positions LLM-generated CLDs as expert-refinable drafts rather than final outputs; highlights literature-supported “hallucinations” as enrichment opportunities.
- Chapter 11 – Ethics and Data Management:** Addresses ethical considerations, data handling practices, and responsible use of LLM-generated causal models.

Chapter 2

Background

2.1 Modelling, simulation, and why conceptual validity matters

2.1.1 The role of simulations in science.

A simulation can be used to study a model of a system by manipulating its inputs to obtain knowledge about its properties, which may have correlates in the original system or inform new theory development. Simulations enable studying certain phenomena at reduced cost, such as in robot training; at temporal or spatial scales which are difficult or impossible to observe, as in computational astrophysics; or in cases where the consequences of mistakes may be severe, such as in surgery planning. Simulation studies also enable testing what-if scenarios and interventions—for example, examining the effects of vaccines in a pandemic model to assess intervention strategies. Simulations have become so integral to many branches of science that they are often called the third pillar of science, alongside theory and experiment. Nonetheless, properties observed in a model may transfer only partially, or not at all, to the system under study. It is precisely this consideration—determining the extent to which model behaviours generalise—that yields valuable insights.

2.1.2 Model abstraction and computational trade-offs.

A model of a system by definition abstracts away parts of the system, and this information loss inherently limits the transferability of conclusions to the system itself. Thus, all models are inherently flawed representations; however, studying these models can still be highly informative. Making informed choices about which features of the system to include in the model is crucial to reproducing the most important dynamics relevant to the study’s context. To reduce information loss and increase the transferability of conclusions, scientists strive to create higher-resolution representations of systems. This often leads to more complex models through increased temporal and spatial resolutions, additional variables, or the relaxation of simplifying assumptions—which may introduce non-linearities and interaction effects. This increase in complexity comes at a computational cost: greater model fidelity typically requires more computational resources, creating a fundamental trade-off between model accuracy and computational tractability. Both computational scientists and domain experts must collaborate to navigate this trade-off efficiently. Domain experts understand which phenomena are most important to capture, while computational scientists possess knowledge of algorithmic optimisations, numerical methods, parallel computing strategies, and complexity analysis. This interdisciplinary collaboration is essential for making intelligent modelling decisions that balance scientific fidelity with computational feasibility. Key considerations include algorithm selection, data structure optimisation, verification and validation procedures, and uncertainty quantification methods.

2.1.3 GMB as a foundation for computational models.

Bringing domain experts together to assimilate their knowledge into a model is a critical preliminary step, especially for complex systems such as those in biology, where subject matter experts may be specialized in only part of the system they wish to model. Convening such groups may present time, cost, and coordination challenges, and the resulting models will inevitably be limited by knowledge gaps and cognitive biases. Nonetheless, these collaborative sessions serve as a valuable basis for model building, and established methods exist to make these sessions more efficient and productive. Within system dynamics—a branch of science concerned with understanding the behaviour of complex systems over time through feedback loops, stocks, and flows—a key type of model is the Causal Loop Diagram (CLD).

The Modelling and Simulation Cycle. The established modelling and simulation (M&S) cycle [31, 32] proceeds through distinct stages: a real system or phenomenon is first mapped to a *conceptual model* through qualitative modelling, then transformed via quantitative modelling into a *computational model*, which is discretized and implemented as a simulation. Results feed back through validation, prediction, and hypothesis testing to refine understanding of the original system. Crucially, validation is required at each stage transition—including the earliest qualitative step. A flawed conceptual model propagates errors downstream: incorrect causal structure at the conceptual stage undermines all subsequent quantitative modelling and simulation efforts, regardless of their numerical sophistication.

Why model and simulate? Ziegler characterises modelling as the systematic organisation of knowledge about a given system [31]. Cellier extends this view, positioning modelling as a unifying activity across scientific and engineering disciplines and arguing that simulation serves both analytical purposes (direct problems) and design applications (inverse problems) [32]. For complex health and social systems—such as those captured by the CLDs in this thesis—simulation is often the *only* feasible approach for several compelling reasons:

- **The physical system is not available for experimentation:** Randomized controlled trials (RCTs) on complex biopsychosocial interventions are often ethically infeasible, prohibitively expensive, or logistically impossible. One cannot experimentally manipulate social norms, community-level obesity prevalence, or healthcare system structures at will, and even if possible, this may raise ethical concerns.
- **Time constants are incompatible with research timelines:** Causal processes in chronic disease and social dynamics unfold over years to decades—far exceeding typical research project durations. Simulation enables exploration of long-term system trajectories within practical timeframes.
- **Control variables may be inaccessible:** In observational health data, key confounders (genetics, lifetime exposures, unmeasured social determinants) cannot be experimentally controlled. Simulation allows systematic exploration of what-if scenarios.
- **To develop deeper understanding and generate hypotheses:** Before investing in expensive empirical studies, simulation-based exploration can identify which causal pathways merit investigation, prioritise intervention targets, and reveal unexpected system behaviours (e.g., counterintuitive feedback effects).
- **What-if scenario simulation:** Simulation enables systematic exploration of hypothetical interventions and policy scenarios that would be impossible or unethical to test in reality. Decision-makers can evaluate the potential consequences of alternative strategies—such as resource allocation changes, prevention programmes, or policy modifications—before committing to costly real-world implementation.

These considerations motivate the value of accurate CLD construction: the qualitative causal structure encoded in a CLD forms the foundation upon which all subsequent quantitative modelling and simulation rests.

2.2 Causal Loop Diagrams (CLDs)

2.2.1 Definition.

A causal loop diagram (CLD) is a qualitative system-dynamics representation of hypothesized causal structure within a bounded system. It is a signed, directed graph whose nodes denote system variables and whose arcs encode *ceteris paribus* monotonic influences: a “+” arc indicates that, relative to a reference level, an increase (decrease) in the source tends to produce an increase (decrease) in the target; a “−” arc indicates the opposite tendency. Salient delays may be marked on arcs to indicate that the dominant effect is time-lagged. Unlike directed acyclic graphs (DAGs), CLDs admit cycles and explicitly emphasise feedback as a first-class structural feature.

Formalization. A CLD can be defined as $G = (V, E, \sigma, \delta)$, where V is a finite set of variables; $E \subseteq V \times V$ is a set of directed arcs; $\sigma : E \rightarrow \{+1, -1\}$ assigns each arc a polarity; and $\delta : E \rightarrow \{0, 1\}$ flags salient delays. The intended semantics are that for each arc $i \rightarrow j \in E$ there exists a (possibly nonlinear) structural relation

$$x_j = f_j(x_{\text{pa}(j)}, u_j, t)$$

such that, over the domain of interest and holding other parents approximately constant, $\sigma(i \rightarrow j) = +1$ implies $\partial f_j / \partial x_i \geq 0$ and $\sigma(i \rightarrow j) = -1$ implies $\partial f_j / \partial x_i \leq 0$.

Feedback loops. A *feedback loop* is any simple directed cycle. Its polarity is the product of its arc signs: $\prod_{e \in C} \sigma(e) = +1$ denotes a *reinforcing* loop (R), which tends to amplify deviations; $\prod_{e \in C} \sigma(e) = -1$ denotes a *balancing* loop (B), which tends toward goal-seeking or stabilization. (Equivalently, reinforcing loops contain an even number of negative arcs, balancing loops an odd number.) Loop polarity characterises qualitative tendencies and does not presume linearity, stationarity, or constant effect sizes.

Scope and use. CLDs deliberately abstract from magnitudes, functional forms, stock-and-flow accounting, and statistical identification of causal effects. They encode theory- or evidence-informed hypotheses about directional dependencies and dominant feedbacks to support elicitation, communication, and high-level reasoning about system behaviour, and they are commonly used as a precursor or complement to quantified stock-and-flow models, experiments, and statistical/causal analyses.

Epistemological status. CLDs encode *hypothesized*—not experimentally validated—causal relationships. In complex biopsychosocial systems, isolating causal effects through controlled experiments is often infeasible or unethical; the systems involve too many interacting variables, long time horizons, and ethical constraints on manipulation. Published CLDs therefore represent expert consensus synthesized from scientific literature, domain knowledge, and group deliberation—not ground-truth causality established through intervention. When we ask whether an LLM can “generate” a CLD, we ask whether it can reproduce expert-hypothesized causal structures, and when we detect “hallucinations,” we identify divergence from expert models rather than departure from proven causal facts.

2.3 Group Model Building (GMB) and the CLD-to-Computational Model Pipeline

The systematic construction of CLDs typically occurs through *Group Model Building* (GMB)—a participatory method where domain experts collaboratively develop shared mental models of complex systems [5]. This

process transforms tacit expert knowledge into explicit causal structures that can subsequently inform computational models.

The CLD-to-SDM pipeline. Crielaard et al. [5] formalize the transition from conceptual to computational models through an *annotated CLD* (aCLD) that captures additional information required for quantification. The key steps are:

1. **Research question and context of validity:** Define the specific research question and establish the spatial/temporal scales over which the model should be valid. The context determines which variables are stocks (accumulating over time), constants (fixed within the temporal scale), or auxiliaries (instantaneously computed).
2. **Variable selection via scale separation map:** Identify all relevant variables and organise them by temporal and spatial scale. Variables acting on smaller temporal scales than the variable of interest can be averaged out; variables on larger scales can be treated as constants.
3. **Causal link identification:** For each pair of variables, determine whether a causal link exists. Links are classified as:
 - **Present (with consensus):** Experts agree the relationship exists
 - **Uncertain:** Experts disagree or lack evidence
 - **Known-to-be-absent:** Explicitly ruled out (critically different from simply unrecorded)
4. **Intermediary variable documentation:** Record mediating variables underlying each causal link. If two proposed links share an intermediary variable, it should be added to the model to avoid spurious independence assumptions.
5. **Functional form specification:** For each link, determine the mathematical relationship (linear, sigmoidal, polynomial, etc.). Sources include expert elicitation, literature review, or computational fitting.
6. **Interaction term identification:** Identify many-to-one relationships where multiple variables jointly affect an outcome (AND gates, OR gates, conditional effects).
7. **Evidence annotation:** Document the source and quality of evidence for each link using frameworks like GRADE [33] (high/moderate/low/very low).

Uncertainty quantification. A key contribution of the Crielaard et al. framework is explicit treatment of uncertainty at each stage:

- **Structural uncertainty:** Uncertainty about which links exist and their functional forms, leading to a “model space” of possible model topologies.
- **Parameter uncertainty:** Uncertainty about parameter values within a given model structure.
- **Propagation:** Both sources of uncertainty should be propagated through simulation to obtain confidence intervals on predictions.

Implications for LLM-based CLD generation. Crielaard’s systematic framework [5] highlights what LLMs would need to replicate to construct comprehensive aCLDs: not merely variable-relationship pairs, but also consensus levels, intermediary variables, functional forms, and evidence quality. However, a simpler starting

point—accurately providing variables, relationships reflecting literature consensus, and their references—would already reduce expert time requirements. This is especially valuable if achievable in a source-text-corpus-independent manner, as corpus dependence constrains the currently most prevalent text-to-CLD methods in the literature.

Current LLM-based approaches focus on core causal structure (variables, links, polarity) without capturing these richer annotations. Some approaches like Theoraizer [34] are more closely aligned with the method in this thesis, as they also perform CLD generation while capturing uncertainty quantification through logprobs and references to CLDs—but provide no clear measurement indicating the hallucination reduction potential of these methods.

This thesis aims to validate multi-agent test-time compute methods such as LLM-as-a-Judge and Corrector, alongside Logprob Uncertainty Quantification (LUQ) and embedding-similarity-based hallucination reduction methods. These approaches are well-established outside the field of CLD generation but have not been systematically validated within it.

The gap between LLM-generated CLDs and fully annotated aCLDs—which also include functional forms and rigorous consensus assessment—represents a direction for future work, and underscores why expert validation remains essential even if LLMs provide useful initial CLD drafts. The intended contribution of this thesis is that, by establishing where LLMs can and cannot be useful in constructing initial CLDs, expert time can be spent more efficiently in creating aCLDs. These aCLDs, in turn, offer scientific value by informing theory, can be translated into computational models in which interventions can be simulated, and through this, help clarify the role of LLMs in contributing to the scientific process.

2.4 Large language model (LLM) background

2.4.0.1 Architectural details of large language models

Tokenization and embeddings. Modern large language models (LLMs) operate on *tokens*: discrete units (characters, subwords, or words) produced by a tokenizer from raw text. Each token index is mapped to a continuous *embedding* vector, yielding a sequence of vectors $x_{1:L} \in \mathbb{R}^{L \times d}$. In this representation space, semantic relatedness is often operationalised via cosine similarity,

$$\text{sim}(u, v) = \frac{u^\top v}{\|u\| \|v\|} \in [-1, 1],$$

which is also the basis of embedding-based retrieval: by retrieving vectors nearest to a query embedding and injecting the associated text as context, Retrieval-Augmented Generation (RAG) can ground generation in external evidence [35].

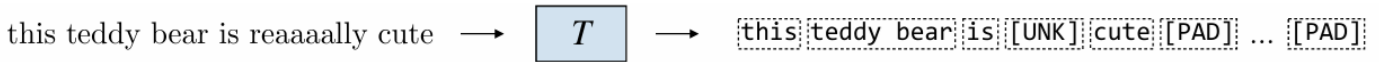


Figure 2.1: Tokenization maps raw text into a sequence of discrete tokens (including special tokens such as [UNK] and padding), which are then embedded as vectors. Reproduced (cropped) from Amidi and Amidi [36], used under the MIT License.

The attention mechanism. The dominant architecture underlying LLMs is the Transformer [37]. Its central mechanism is (scaled dot-product) attention, which computes content-dependent mixing weights over

the input:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V,$$

where $X \in \mathbb{R}^{L \times d_{\text{model}}}$ is the matrix of input token representations (one row per token) and the query, key, and value matrices are computed by learned linear projections $Q = XW_Q$, $K = XW_K$, and $V = XW_V$. Here $W_Q, W_K \in \mathbb{R}^{d_{\text{model}} \times d_k}$ and $W_V \in \mathbb{R}^{d_{\text{model}} \times d_v}$ are trainable parameter matrices, so $Q, K \in \mathbb{R}^{L \times d_k}$ and $V \in \mathbb{R}^{L \times d_v}$; the scaling factor d_k is the key/query dimensionality. Because attention computes all pairwise query–key interactions, *dense* self-attention scales quadratically in sequence length L (forming an $L \times L$ attention pattern). In common (non-IO-aware) implementations that materialize the attention matrix, this leads to $\mathcal{O}(L^2)$ memory usage and $\mathcal{O}(L^2)$ work (up to factors of the embedding dimension), and the attention distribution can become increasingly diffuse for long contexts (each position receiving weight on the order of $1/L$ in the extreme). To preserve order information in a permutation-invariant attention mechanism, Transformers incorporate positional information (e.g., learned or fixed positional embeddings) added to token embeddings prior to attention.

This quadratic cost motivates two common optimisation directions. *Sparse attention* restricts each token to attend only to a subset of positions (e.g., local windows or structured patterns), reducing query–key interactions and yielding sub-quadratic time and memory growth in L (with scaling determined by the sparsity factorization) [38]. In contrast, *FlashAttention* preserves *exact dense* attention but makes it IO-aware, improving wall-clock performance without changing the $\mathcal{O}(L^2)$ scaling of dense attention [39].

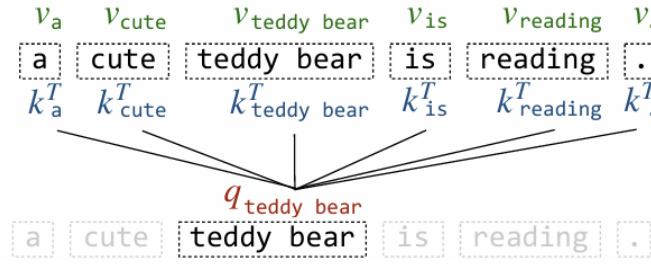


Figure 2.2: Attention computes a query-dependent weighted combination over keys/values, selecting which tokens should influence the representation at a given position. Reproduced (cropped) from Amidi and Amidi [36], used under the MIT License.

Multi-head attention. In practice, Transformers use *multi-head attention*, computing several attention operations in parallel and combining them with a learned output projection. This allows different heads to specialize in different relational patterns in the sequence.

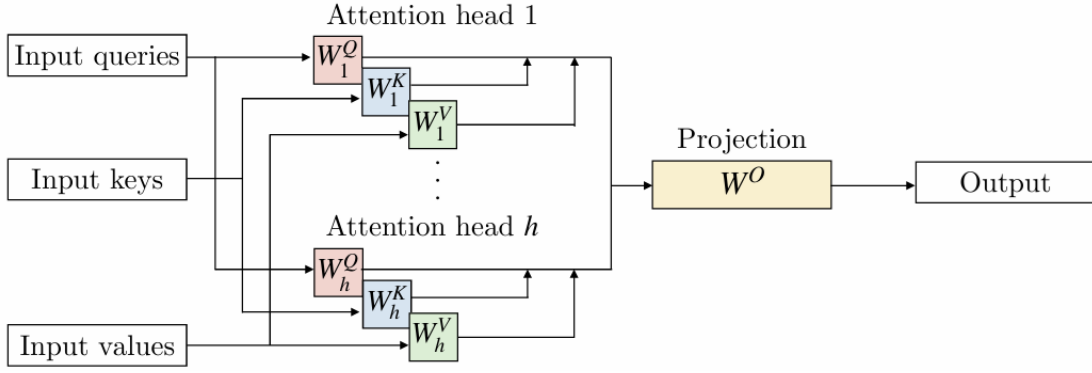


Figure 2.3: Multi-head attention: multiple sets of W_Q, W_K, W_V are applied in parallel, then concatenated and projected to the model dimension. Reproduced (cropped) from Amidi and Amidi [36], used under the MIT License.

Feed-forward networks. Each Transformer layer includes a *position-wise feed-forward network* (FFN) applied independently to each token representation. The FFN typically consists of two linear transformations with a non-linear activation (commonly ReLU or GELU) in between:

$$\text{FFN}(x) = W_2 \cdot \sigma(W_1 x + b_1) + b_2,$$

where $W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ projects to a larger hidden dimension d_{ff} (typically $4 \times d_{\text{model}}$), and $W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$ projects back. This expansion-contraction pattern allows the model to learn complex, non-linear transformations of individual token representations after the attention layer has mixed information across positions.

Residual connections and layer normalization. Transformer layers employ *residual connections* (also called skip connections) around both the attention and feed-forward sublayers, followed by *layer normalization*. For a sublayer function f (either attention or FFN), the output is computed as:

$$\text{LayerNorm}(x + f(x)),$$

where the residual connection $x + f(x)$ adds the sublayer’s input directly to its output. This design serves two purposes: (1) residual connections facilitate gradient flow during training, enabling effective optimisation of deep networks; and (2) layer normalization stabilizes activations by normalizing across the feature dimension, reducing sensitivity to the scale of inputs. These “Add & Norm” blocks, visible in Figure 2.6, are applied after every attention and feed-forward sublayer throughout the Transformer stack.

Encoder–decoder architecture. The original Transformer was introduced in an encoder–decoder setting [37]: an encoder produces contextual representations of an input sequence, and a decoder generates an output sequence while attending to the encoder via cross-attention.

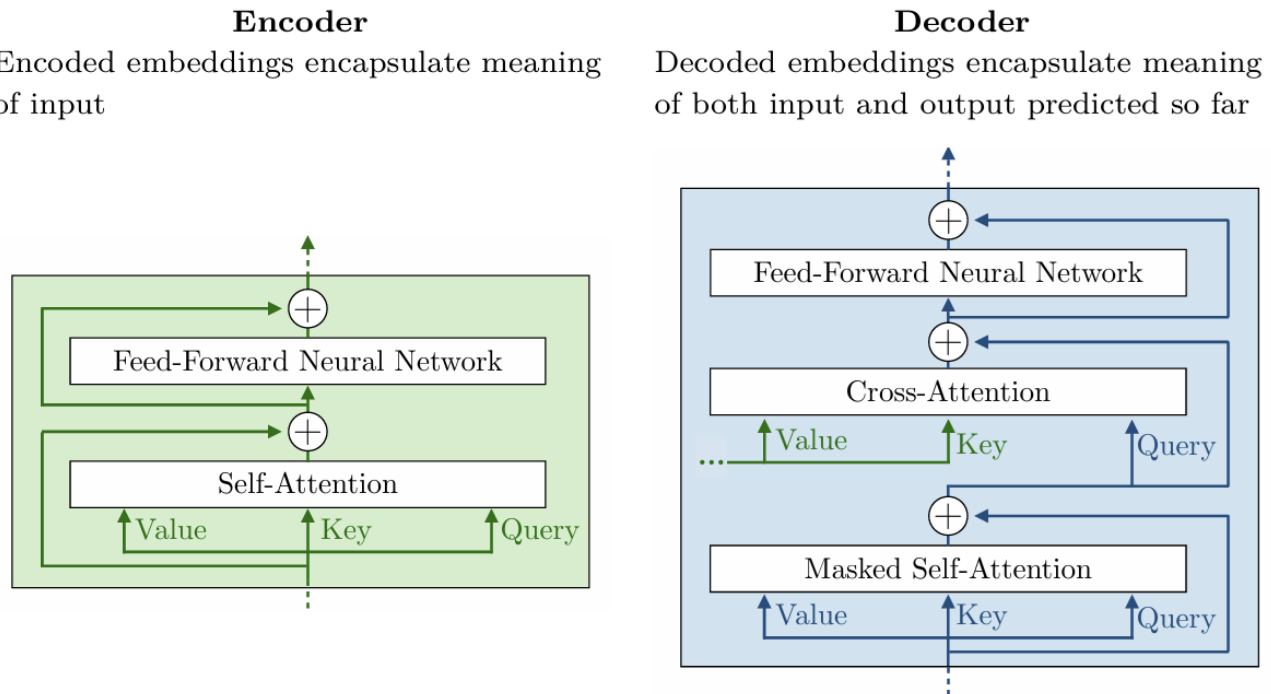


Figure 2.4: Encoder and decoder building blocks. The decoder includes masked self-attention (for causality) and cross-attention (to condition on encoder outputs). Reproduced (cropped) from Amidi and Amidi [36], used under the MIT License.

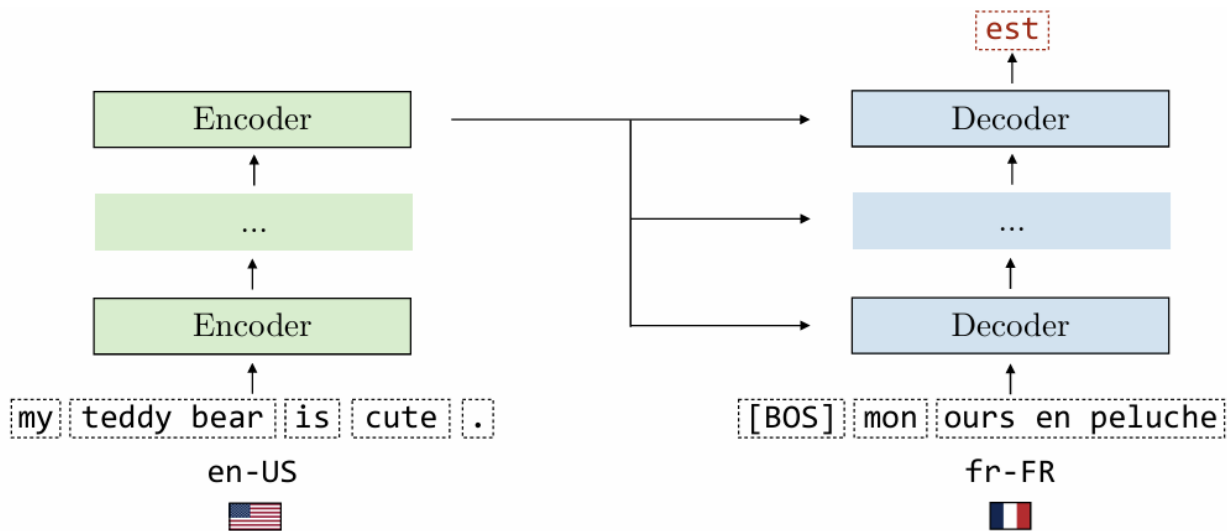


Figure 2.5: Encoder–decoder example (e.g., translation): the encoder reads the input sequence, and the decoder generates the output sequence autoregressively. Reproduced (cropped) from Amidi and Amidi [36], used under the MIT License.

Decoder-only models and autoregressive generation. Many widely used LLMs are *decoder-only* Transformers (the GPT family) trained with a next-token prediction objective [16]. Figure 2.6 shows the original encoder–decoder Transformer architecture from which decoder-only models are derived.

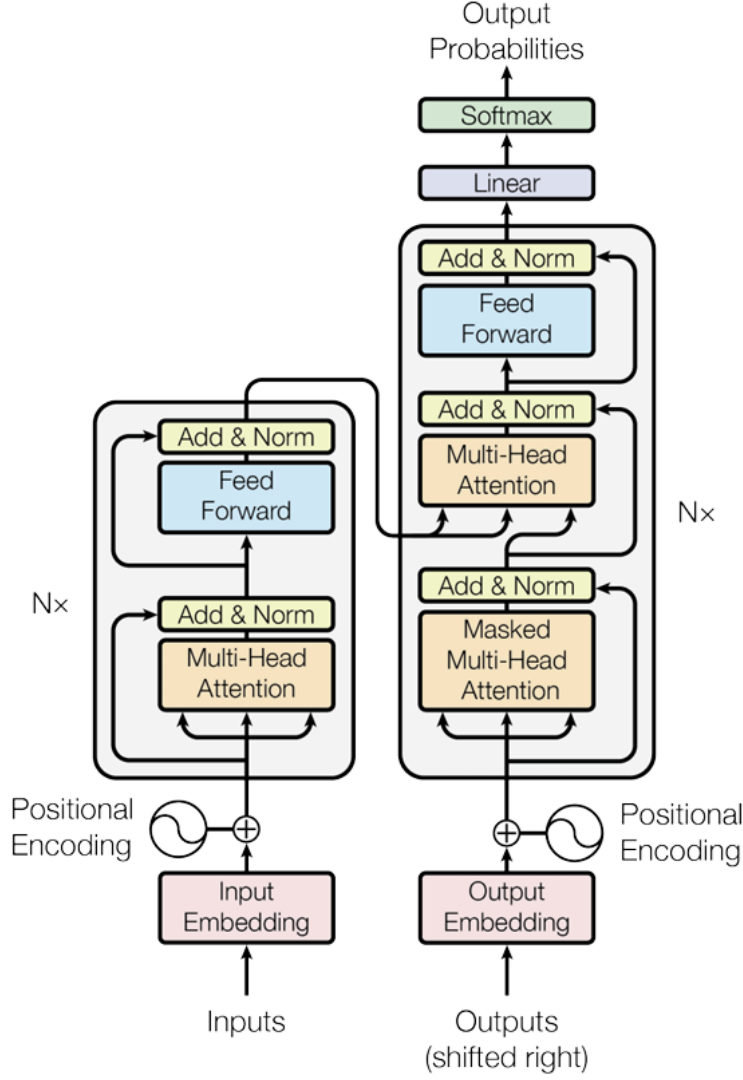


Figure 2.6: The Transformer architecture, consisting of stacked encoder (left) and decoder (right) blocks. Each encoder layer comprises multi-head self-attention and a feed-forward network with residual connections and layer normalization. The decoder additionally includes masked self-attention to enforce causality and cross-attention over encoder outputs. Adapted from Vaswani et al. [37].

Training typically uses *teacher forcing*: at position t , the model is given the ground-truth prefix $x_{<t}$ and is optimised to assign high probability to the next token x_t . Concretely, parameters are learned by minimizing the negative log-likelihood (equivalently, cross-entropy) over a corpus,

$$\mathcal{L} = - \sum_{t=1}^T \log P(x_t | x_{<t}).$$

The architectural mechanism that enforces this conditional structure inside the Transformer is *masked* self-attention: a causal (typically lower-triangular) attention mask prevents each position from attending to any future tokens $x_{>t}$, ensuring that information flows only from past to present.

This yields an autoregressive factorization of the sequence likelihood:

$$P(x_{1:T}) = \prod_{t=1}^T P(x_t | x_{<t}),$$

and generation proceeds one token at a time, conditioning strictly on previously produced tokens; once a token is emitted it becomes part of the immutable context for subsequent predictions.

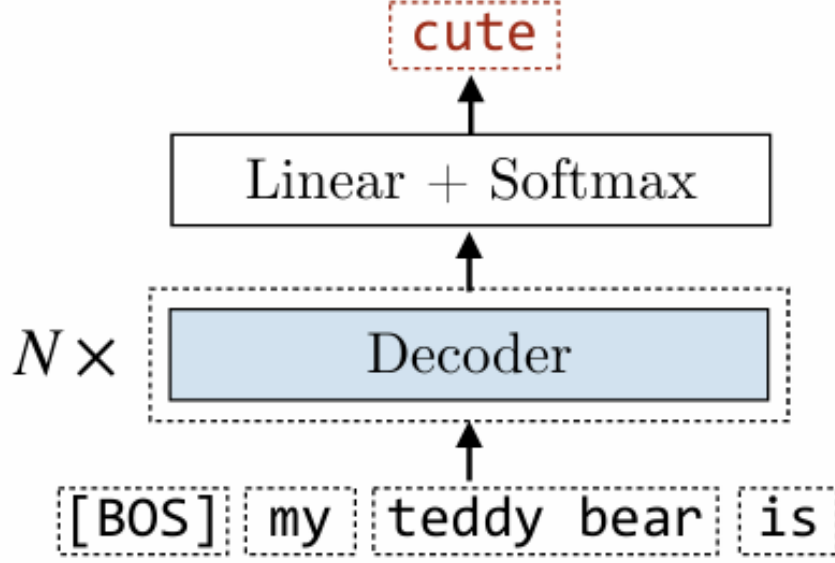


Figure 2.7: Decoder-only LLMs stack masked self-attention blocks and predict the next token via a linear layer and softmax. Reproduced (cropped) from Amidi and Amidi [36], used under the MIT License.

Decoding strategies. For fixed parameters and fixed context, the forward pass produces a deterministic next-token distribution via a softmax over logits. Apparent stochasticity in model outputs arises primarily from *decoding*: sampling from this distribution using choices such as temperature scaling, top- k truncation, or nucleus (top- p) sampling [40]. Consequently, repeated generations with identical prompts can vary, and controlling the sampling strategy is a key lever for trading off diversity versus determinism.

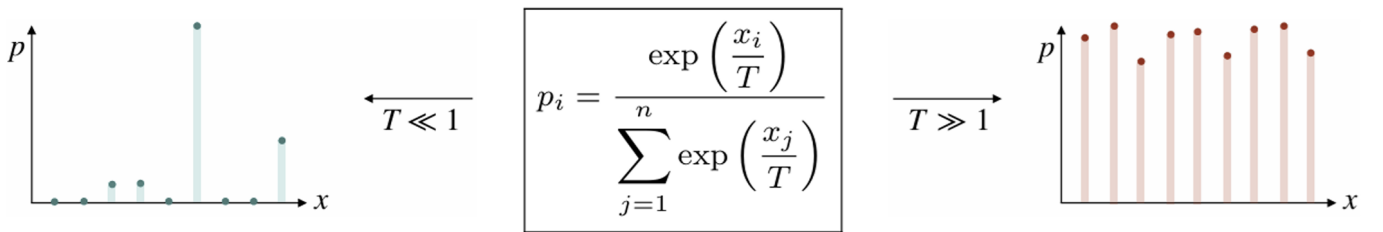


Figure 2.8: Temperature scaling reshapes the next-token distribution: low T concentrates probability mass (more deterministic), while high T flattens it (more diverse). Reproduced (cropped) from Amidi and Amidi [36], used under the MIT License.

Autoregressive generation and hallucination mechanisms. Modern LLMs generate text through an *autoregressive* language modelling process: tokens are generated sequentially, conditioned on a context c (e.g., the prompt and any provided documents), such that the probability of a token sequence $x_{1:T}$ factorizes as:

$$P(x_{1:T} | c) = \prod_{t=1}^T P(x_t | x_{<t}, c) \quad (2.1)$$

At each generation step, the model produces a probability distribution over its vocabulary $\mathcal{V}$, and a next token is sampled from this distribution. For vocabulary item $v \in \mathcal{V}$ with logit s_v , the corresponding probability is given by the softmax:

$$P(v \mid x_{<t}, c) = \frac{\exp(s_v)}{\sum_{j \in \mathcal{V}} \exp(s_j)} \quad (2.2)$$

where $\{s_j\}_{j \in \mathcal{V}}$ are the model’s logits for the next-token distribution at step t .

This sequential, generation process has structural properties that contribute to hallucination [8]:

Hallucination snowballing. A form of error propagation that occurs once an incorrect token is generated, the model conditions subsequent generation on this error, compounding inaccuracy. Zhang et al. [41] demonstrated that LLM hallucinations can “snowball” - an initial minor error leads the model to generate increasingly confident but fabricated content to maintain coherence with its erroneous premise. This creates a failure mode where the model appears fluent and confident while being factually wrong.

Decoding-induced degeneration. The choice of decoding strategy affects hallucination rates. Holtzman et al. [40] showed that maximization-based decoding (beam search) can lead to repetitive, degenerate text, while sampling from the full distribution introduces randomness that may select low-probability but plausible-sounding tokens - potentially hallucinations. This tension between coherence and diversity is inherent to autoregressive generation.

Three key parameters control the sampling process:

- **Temperature (T):** Scales logits before softmax, with $P(v) \propto \exp(s_v/T)$. Lower temperatures ($T < 1$) sharpen the distribution toward high-probability tokens, increasing determinism but risking repetition. Higher temperatures ($T > 1$) flatten the distribution, increasing diversity but also the probability of sampling unlikely (potentially hallucinated) tokens.
- **Top- k sampling:** Restricts sampling to the k highest-probability tokens, redistributing probability mass among them. This prevents sampling from the long tail of implausible tokens, but a fixed k may be too restrictive for peaked distributions or too permissive for flat ones. Therefore, nucleus sampling is often used over top- k for its dynamic properties.
- **Top- p (nucleus) sampling:** Dynamically selects the smallest set of tokens whose cumulative probability exceeds threshold p (e.g., $p = 0.9$). This adapts to the shape of each distribution - sampling from few tokens when the model is confident, more when uncertain [40].

Each parameter represents a trade-off: more restrictive settings reduce hallucination risk but may produce less natural text, while more permissive settings increase fluency at the cost of factual reliability.

2.5 Hallucinations in LLMs (general)

Recent work from OpenAI supports LLM hallucinations arise naturally from the pretraining objective—by reducing generation to an “Is-It-Valid” binary classification problem, cross-entropy-trained base models will still produce errors (notably on arbitrary facts or under model misspecification), even with error-free data [6]. Moreover, prevalent 0–1 graded benchmarks penalize abstentions and uncertainty, so models are effectively rewarded for confident guesses, which sustains overconfident hallucinations. Under the Open World Assumption, hallucination is a direct consequence of generalisation in an unbounded environment: finite training data can never guarantee correctness in all future situations [7].

2.5.1 Taxonomy of Hallucinations

A comprehensive survey on LLM hallucinations [9] establishes a principled taxonomy distinguishing two main types: *factuality hallucinations* (contradictions with real-world facts, fabrication, overclaims, and unverifiable content) and *faithfulness hallucinations* (instruction inconsistency, context inconsistency, and logical inconsistency). In the context of CLD generation, most errors map to *relation-error factuality hallucinations* (incorrect causal relationships) and *logical inconsistency faithfulness hallucinations* (invalid feedback loop structures or polarity assignments).

The survey strongly supports the view that LLM hallucinations arise when the model operates outside its *knowledge boundary*—which may be incomplete (long-tail knowledge), outdated (temporal cutoff), or legally restricted (copyright). CLDs inherently involve niche domain variables, implicit causal relations, and theory-level abstractions—exactly the type of long-tail and underspecified knowledge where hallucinations are most likely.

These considerations motivate the exploration of *corpus-independent* hallucination mitigation methods that do not rely solely on retrieval quality, as detailed in Section 3.6.

2.5.2 Hallucination Mitigation Methods

Methods for combating hallucinations can be grouped into *black-box* approaches (no access to model weights required) and *white-box* approaches (modifying or inspecting model internals).

2.5.2.1 Black-box methods.

Prompt-based. Prompt engineering specifies the model’s role and constraints via system prompts, uses Chain-of-Thought (CoT) prompting to scaffold reasoning [15], and provides one or more input–output exemplars (few-/multi-shot learning) to anchor the desired behaviour [16]; see Section 3.4 for a detailed review of prompting strategies.

Retrieval-Augmented Generation. RAG conditions generation on external knowledge by embedding the query and a corpus into a shared vector space and retrieving the most relevant context using distance metrics such as cosine similarity or L_2 distance; retrieved sources can come from vector databases or structured stores such as knowledge graphs [17]. In *reverse retrieval* settings, the model output and corpus are embedded in a vector space and the documents most similar to the output are retrieved, identifying which sources best support a given response.

System orchestration. Verification agents assign LLMs explicit checking roles, e.g., LLM-as-a-judge to assess another model’s output and LLM-as-a-corrector to reflect and repair detected errors. More generally, multi-agent ensembles can be arranged serially or in parallel with distinct instruction sets to “divide and conquer” the reasoning and verification process [19]. This principle is supported by research on decomposition-based prompting, which demonstrates that breaking complex tasks into smaller subproblems—solved sequentially with earlier solutions informing later ones—significantly improves LLM performance on multi-step reasoning tasks [42]. Relatedly, *test-time compute* increases inference-time computation—often via multi-step reasoning, agentic reflection, and/or selection among candidates via verifiers/reward models—to improve answer quality and reduce hallucinations [22, 23, 43].

Corpus-independent metrics. *Corpus-independent* metrics do not require a researcher-provided text corpus to retrieve from or compare against at inference time. Hallucinations can be flagged using statistical signals that do not require deep semantic interpretation: (i) low cosine similarity between generated text and retrieved/cited

sources suggests unsupported content, (ii) perplexity or log-likelihood anomalies can indicate low-confidence generation [14], (iii) elevated semantic entropy (uncertainty over meaning) correlates with hallucination risk [44], and (iv) instability in multi-hop reasoning trajectories—e.g., analysed via Hamiltonian interpretations of reasoning paths—can signal unreliable chains [45].

2.5.2.2 White-box methods.

Fine-tuning and RLHF. Weight updates using supervised fine-tuning and reinforcement learning from human feedback (RLHF) improve task adherence and can reduce hallucinations [25]; parameter-efficient methods such as LoRA reduce training cost and preserve generality by adapting low-rank components and/or fine-tuning only subsets of layers [24], with fine-tuning improving performance on hallucination detection tasks [46].

Test-time compute. Beyond orchestration as a black-box strategy, test-time compute can also be viewed as a model-level paradigm shift: when pre-training gains diminish with scale, allocating compute at inference (e.g., multiple rollouts, reflection, RL-guided selection) becomes a primary driver of improved reasoning and reduced errors [47].

Activation monitoring and steering. Monitoring internal activations can associate hallucinations with specific neurons or layers and mitigate them by steering activations; related work suggests that suppressing undesirable neuron behaviours (e.g., overly high sensitivity) via interventions such as dropout during fine-tuning can reduce hallucination propensity [14, 26].

Taken together, these mitigation strategies reveal different approaches along different axes of hallucination reduction exist, and that not a single technique but a design space spanning prompt design, retrieval and external grounding, agentic verification pipelines, inference-time scaling, and weight- or activation-level interventions is available. With a broad overview of different types of hallucination detection and mitigation methods given as well as background CLD’s and LLMs the literature review will focus specifically around the related work to our questions, focussed on use of LLMs in constructing CLDs, how hallucinations manifest there, and mitigation methods as well as adaptive test-time compute.

Chapter 3

Literature Review

Most CLD work in system dynamics remains grounded in manual and participatory practice. Practitioners construct CLDs through interviews, literature synthesis, and facilitated workshops because diagram quality depends on careful construct definition, loop reasoning, and—critically—judgments about directness, mediation, and evidentiary support [1, 2, 4, 5]. The effectiveness of such participatory approaches has been systematically reviewed: Rouwette et al. [3] synthesise assessment studies and conclude that GMB can be valuable for involvement, communication, and shared understanding, but remains resource-intensive and context-dependent.

3.1 Standard CLD construction paradigms

In practice, CLDs are most commonly constructed through expert- and stakeholder-driven workflows rather than fully automated pipelines. These paradigms vary in the balance between participatory elicitation (capturing tacit knowledge, scope assumptions, and stakeholder mental models) and literature synthesis (capturing published evidence and consensus), but share a broadly iterative structure: variable elicitation and refinement, causal link elicitation (direction and polarity), feedback loop identification, and repeated revision to improve construct clarity and model usefulness for the stated purpose [1, 2, 4].

3.1.1 Participatory group model building (GMB)

GMB workshops combine facilitated sessions with domain experts and stakeholders to elicit variables, propose and negotiate causal links, and converge on a shared causal narrative that is “good enough” for downstream use (e.g., communication, policy discussion, or subsequent formalisation into a simulation model) [1, 2]. Empirical reviews indicate that GMB can improve involvement, communication, and shared understanding, but outcomes are context-dependent and the process remains resource-intensive and difficult to scale to large variable sets [3].

3.1.2 Literature-driven expert synthesis

An alternative (or complement) to workshops is expert-led construction grounded in literature review: practitioners define constructs and link rationales based on prior studies, then iteratively refine the diagram as new evidence and scope constraints are incorporated [4, 5]. While this can strengthen evidentiary support, it still requires substantial manual effort to assess directness vs. mediation, reconcile conflicting findings, and document assumptions consistently across many candidate links.

3.1.3 Hybrid workflows and progression to quantitative models

Many applied projects combine these approaches: a literature-informed draft is used to structure workshop discussion, or workshop outputs are subsequently validated and extended through targeted evidence gathering. In system dynamics practice, CLDs are often treated as intermediate artefacts that can be translated into more formal representations (e.g., stock–flow structures and parameterised simulation models) once scope, feedback structure, and key mechanisms are sufficiently specified [1].

Scaling of manual synthesis work. A core reason the workflow is costly is combinatorial: for a CLD with $|V|$ variables there are $O(|V|^2)$ ordered candidate links to consider, even before accounting for polarity, delays, and scope decisions. If the modelling goal requires assessing *directness*—i.e., for each candidate $X \rightarrow Y$, asking whether the influence is plausibly direct or better represented via one (or more) mediators among the remaining $|V| - 2$ concepts—the deliberative workload can increase substantially. In the worst case, considering mediation pathways for each ordered pair introduces an additional factor that scales with $|V|$, pushing the upper bound of the review task toward $O(|V|^3)$ checks, though in practice modellers use heuristics and domain knowledge to prune the search space [4, 5].

3.2 Automated CLD construction paradigms (text-to-CLD vs. generation)

Current literature on LLM-based CLD construction can be organised into two paradigms. First, *text-to-CLD* approaches (also described as *causal translation*) extract variables and causal links from provided documents, and can reduce hallucination risk through retrieval grounding, but assume availability of a relevant corpus and are capped by retrieval quality. Second, *corpus-independent CLD generation* synthesizes candidate causal structures from the model’s parametric knowledge without requiring an external corpus (i.e., we directly prompt the LLM to create a CLD or set of causal relationships and take its output to be the CLD); this is inexpensive and broadly applicable, but lacks a grounding corpus to reduce risk of hallucinations.

Large language models show promise for automating Causal Loop Diagram construction but exhibit systematic failure modes that demand specialized uncertainty quantification approaches. This review synthesizes 2022–2025 literature connecting general hallucination detection metrics to the specific challenge of generating reliable causal structures from text, distinguishing between benchmarks that evaluate (a) causal extraction from text, (b) causal reasoning over given graphs, and (c) causal structure discovery from statistical data.

The remainder of this chapter reviews related work and evidence for each paradigm, then synthesizes a CLD-specific taxonomy of failure modes and verification signals, motivating the hybrid approach evaluated in this thesis (RQ1–RQ3).

3.3 LLM-Based Causal Loop Diagram Generation

Notably, generating causal loop diagrams directly is relatively under-researched in the literature found in this review, with most approaches focusing on giving the LLM text, or a causal graph, from which the causal structure is to be extracted or reasoned over, with hallucination mitigation methods largely coming down to different forms of retrieval-augmented generation. To understand the capabilities and limitations of LLMs for CLD-related tasks, we first survey benchmark studies that evaluate different forms of causal reasoning, before examining specific CLD generation and extraction approaches.

3.3.1 From LLM-Powered Causal Pattern Matching to Causal Loop Diagrams

Benchmark studies reveal highly varying performance across causal task types using LLMs, though each evaluates a distinct form of causal reasoning:

COPA (Choice of Plausible Alternatives) tests commonsense causal judgement: given a premise (e.g., “The man broke his toe”) and two alternatives, models select which is more plausibly the cause or effect. This multiple-choice format with everyday scenarios yields near-perfect GPT-4 accuracy but requires no graph construction or variable extraction—it tests recognition of familiar causal patterns rather than systematic causal inference.

CLadder [11] tests causal query answering over provided graphs: models receive a causal graph with binary variables and must answer queries at three levels of Pearl’s ladder—associational (“What is $P(Y|X)$?”), interventional (“What happens if we set $X=1$?”), and retrospective what-if (“Would Y have occurred if X had been different?”). GPT-4 achieves only $\sim 62\%$ accuracy, with performance degrading sharply on interventional and retrospective what-if queries. *Relation to CLD generation:* CLadder evaluates reasoning *with* a given causal structure, whereas CLD generation requires synthesizing a plausible causal structure from a task prompt or domain specification without being given the graph. However, polarity errors in CLD generation may reflect similar failures in interventional reasoning.

The **CaLM benchmark** [48] provides the most comprehensive evaluation, testing 28 models across 92 causal targets organised into four modules. Tasks range from causal attribution (identifying causes of events) to effect prediction and what-if reasoning. The key finding—“as complexity of causal reasoning increases, accuracy progressively deteriorates, eventually falling almost to zero”—suggests that CLD generation involving multi-step causal chains will face compounding errors. This is especially relevant for CLDs, where feedback loops and longer cause–effect chains mean that errors in individual edges can propagate through downstream reasoning about the system. It also motivates edge-level error correction to prevent compounding failures, for example via LLM-as-a-judge and LLM-as-a-corrector pipelines (if these components are empirically shown to work in this setting).

CausalBench evaluates three subtasks: correlation identification, causal skeleton identification (undirected graph structure), and causality identification (directed edges). LLMs significantly lag behind traditional algorithms on networks exceeding 50 nodes and struggle with collider structures while excelling at chains.

Darvariu et al. [49] frame LLMs as low-cost sources of background knowledge for causal graph discovery: by eliciting structural priors from an LLM, the hypothesis space over candidate graphs can be reduced before running downstream discovery algorithms. They propose metrics to assess LLM causal judgements independently of any specific discovery method and show that LLM-derived priors are most useful for edge directionality (e.g., 84% correctly oriented edges for LLaMA2-70B when combined with observational data).

Corr2Cause [50] tests inferring causal graph structure from correlation statements: given statements like “A is correlated with B” and “B is correlated with C,” models must determine whether “A causes B” or identify the correct DAG structure. Testing 17 models on 200K+ samples, all achieved near-random performance ($\sim 29\%$ F1). This benchmark isolates pure causal structure inference without natural language complexity. The near-random performance suggests low success for text-to-CLD systems in settings where causal structure must be inferred from limited signals. In our CLD generation task, we similarly prompt the model to answer “Does A cause B?” (surrounded by prompt scaffolding) at the edge level; based on Corr2Cause, this edge-level decision may be challenging, although this remains to be seen since Corr2Cause conditions on correlation statements rather than the domain-grounded prompts used in this thesis.

However, evidence closer related to our use case shows that, for causal graph discovery from domain knowledge, LLMs show complementary strengths and weaknesses. Jiralerspong et al. [51] demonstrate that querying LLMs

about pairwise causal relationships using breadth-first search can achieve state-of-the-art results on real-world causal graphs (typically DAG-oriented discovery). This is adjacent to CLD generation in that it elicits edge-level causal hypotheses from domain knowledge, but it does not, by itself, target signed feedback-loop structures as CLDs do.

In the text-to-CLD causal extraction task, an example is the System Dynamics Bot [52], which extracts causal variables and directed causal links from natural language text to construct feedback diagrams and outputs structured CLDs. Evaluated against human-coded ground truth from system dynamics literature and participant responses, the bot recovers only 59% of causal links and 66% of feedback loops using GPT-4.

Liu and Keith [53] use curated prompting techniques extracting CLDs from dynamic hypothesis texts, comparing four prompt designs: zero-shot, few-shot, guided instructions, and a two-stage prompt that first extracts variables then maps signed causal links. On 44 textbook DH-CLD pairs (expert CLDs hand-encoded in DOT), curated prompting—especially the two-stage setup, which the generator algorithm employed in this thesis also uses—produces CLDs closest to expert diagrams for simple structures, but errors persist (missed loops, wrong polarities, and weak handling of exogenous/implicit relationships) as complexity grows.

Complementing these point evaluations, the SD-AI benchmark [54] proposes a standardised test suite and public leaderboard for CLD *causal translation* (i.e., text-to-CLD extraction) and conformance, showing substantial variation across LLMs and motivating continued benchmarking of instruction-following and technical correctness in AI-assisted system dynamics tooling.

An approach closer to home exploring CLD generation comes from **Theoraizer** [34], which is closest to the approach we explore in this thesis. Theoraizer operationalises CLD construction as a stepwise, edge-level elicitation pipeline in which an LLM functions as a “digital extension” of an expert group rather than a replacement. Users provide (or optionally generate) a variable list, and the system then queries the LLM over all ordered variable pairs to (i) classify whether a causal relation exists, (ii) infer directionality, and (iii) assign polarity (+/−). A key design choice is to force single-token outputs (e.g., “C/N”, “yes/no”) and use the model’s log probabilities as “model-implied” confidence scores for each edge, enabling thresholding to trade off sparsity vs. coverage. To reduce prompt sensitivity, Theoraizer uses prompt ensembling (multiple prompt variants aggregated), and keeps humans in the loop by exposing intermediate tables/plots and encouraging iterative editing of variables and edges.

Methodologically, Theoraizer is closely aligned with CLD generation workflows that decompose structure construction into repeated pairwise decisions (e.g., prompting “Does A cause B?”), and it explicitly targets scalability: evaluating $|V|(|V| - 1)$ candidate edges becomes computational rather than purely cognitive labour. In validation, Theoraizer reports that LLM–human agreement on causal judgements is comparable to inter-expert agreement, suggesting LLMs can act as plausible “stand-in” raters for initial candidate CLDs, and supporting this by finding comparable LLM–human and human–human inter-rater agreement. At the same time, the paper emphasises limitations that mirror hallucination risks in causal structure generation: judgements are made pairwise and in isolation (encouraging dense graphs and “short-circuit” links that may actually be mediated), outputs reflect linguistic priors rather than epistemic truth, and biases/hallucinations necessitate expert review.

3.3.1.1 Positioning of this thesis

Contrasting their approach with this thesis, we add a mediating-variable check to address this limitation of reviewing links in isolation, and we apply hallucination detection and correction methods established in the broader field of LLM hallucination detection to CLD generation (e.g., LLM-as-a-judge, LLM-as-a-corrector, and uncertainty-quantification signals). In terms of using the logprobs-based uncertainty quantification, our approach slightly differs, as our generating model also produces a causal narrative through which variables are

related (and which can also be used for post-hoc validation). Each token carries possible top-k alternatives over which the probabilities are distributed. This allows uncertainty quantification over this whole narrative, which is what this thesis will explore. All of this of course relies on the premise that these uncertainties reflect a model’s tendency to hallucinate; this along with the hallucination reduction techniques used will be discussed later in this review.

However, firstly we still want to address a broader question about what such edge-level pipelines recover from LLMs: are they primarily reproducing patterns that are well represented in training data, or do LLM outputs constitute causal reasoning? Work from the “causal parrots” hypothesis [10] offers a theoretical framework unifying these findings: LLMs encode causal facts within “meta-SCMs” (structural causal models describing relationships between causal facts in training data) but cannot perform genuine interventional or what-if reasoning. When LLMs succeed at causal tasks, they retrieve correlations between causal statements seen during training.

For our purpose, namely LLMs providing initial CLDs that reflect the literature, simply reproducing an accurate reflection of consensus on the causal relationships found in their literature may be enough. For genuine causal reasoning this model may break down and implies that models may accurately reproduce causal structures from well-documented domains that are part of training data while fabricating plausible-sounding but incorrect links in novel domains. The difficulty with this then becomes knowing what is part of the training data and what is not, which is often unclear, and also separate from the equally important questions of whether the model can faithfully reproduce that training data, or if the claims found in that data are true. All of these issues directly motivate hallucination mitigation approaches accompanying LLM-based CLD construction pipelines.

3.3.2 How Hallucinations Manifest in Causal Structure Generation

Hallucinations in CLD generation take distinct forms compared to free-text generation, and related work treats these failures through different benchmarks, annotations, and mitigation strategies. To synthesise this literature, we organise findings per hallucination type. Across the surveyed work, we summarise five recurring edge/graph-level error categories reported across benchmarks and expert analyses:

Fabricated causal links represent the most direct hallucination type—generating relationships between variables without textual grounding. The ReCAST benchmark [55] directly evaluates multi-node causal graph construction from academic text—a task closely aligned with text-to-CLD causal translation. Unlike synthetic benchmarks, ReCAST uses unstructured, naturally written academic texts of varying length and complexity. Models must identify causal variables and construct directed graphs capturing the causal relationships described. The best-performing model achieves $F_1 = 0.477$, with common errors including fabricated links (e.g., “loan usage flexibility $\rightarrow$ cash crop cultivation” when no such relationship exists), difficulty with implicit causal information, and failure to connect causally relevant information spread across lengthy texts.

Polarity errors involve incorrect $+/ -$ assignment to causal links. This manifests when LLMs state A increases B when it actually decreases B—a failure to apply “*ceteris paribus*” reasoning correctly. Unlike free-text hallucinations, CLDs require internally consistent sign patterns around feedback loops.

Spurious variable introduction creates variables not grounded in source text through over-generalisation. Reinholtz et al. [56] document examples like “Macroeconomic and finance conditions” with unknown polarity, or “Global technology learning”—concepts too vague to support directional interpretation. Their study uses Pipeline Algebra to construct CLDs from text, with expert refinement identifying systematic LLM errors as hallucination mitigation.

Missing causal links represent faithfulness failures where explicit source relationships are omitted. The System Dynamics Bot study found that missing links often occur when “authors assumed readers would know”

implicit information—highlighting the challenge of implicit versus explicit causal knowledge in text-to-CLD extraction.

Direction reversal and **DAG violations** constitute structural hallucinations unique to graph outputs. LLMs may reverse cause and effect or create cycles violating directed acyclic graph constraints. Jiralerspong et al. [51], in their pairwise causal query approach to graph discovery, document that LLMs sometimes predict bidirectional causation or inconsistent edge directions when queried about the same variable pair in different orderings. Also in CLD construction, where feedback loops (or lack of them) have a crucial effect on the system’s dynamics, this failure mode may be problematic.

3.3.2.1 Formal CLD Hallucination Taxonomies

The **SD-AI benchmark** [54] provides a systematic hallucination taxonomy for text-to-CLD tasks, maintaining a public leaderboard comparing model capabilities.¹ The taxonomy distinguishes: (1) *lack of conformance* (structural/formatting failures), (2) *variable extraction failures*, (3) *spurious edges* (false positives), (4) *missing edges* (false negatives), (5) *polarity errors*, and (6) *wrong causal narratives*. This thesis adopts categories 3–6 as the primary focus, with conformance addressed through structured outputs (JSON schema constraints) with the Judge being a qualitative check on conformance, and exploratory variable-level analysis relegated to Appendix D, noting that main experiments use a ground truth given node set to isolate performance on edges to make the direct mapping to ground truths feasible by removing node-level differences from the comparison. The operational definitions and ground truth specifications used in our experiments appear in Section 4.2.

Expert evaluation at MIT [56] produced a complementary practical taxonomy with five categories: (A) ambiguous/overly broad variables, (B) duplicate/syntactically odd labels, (C) mis-specified linkage or polarity, (D) redundant elements, and (E) process-level limitations. These taxonomies provide actionable categories for automated detection systems in text-to-CLD pipelines, and although no comparable analysis for CLD generation approaches has been found (with closest work being the aforementioned), we adopt the SD-AI taxonomy to aid in making evaluation results directly comparable to text-to-CLD approaches.

3.3.3 CLD Hallucination Mitigation Approaches

Approaches that aim to reduce hallucinations through Retrieval-Augmented Generation show promise but remain constrained by retrieval quality. Zhang et al. [18] introduce LACR, an LLM-assisted causal discovery method that uses RAG over scientific papers to recover causal graphs: for each variable pair it retrieves relevant literature, prompts the LLM to infer (in)dependence and direct vs. indirect association in a constraint-based manner, aggregates these decisions to build the graph skeleton, and then uses the LLM to orient edges. Experiments on benchmark datasets show improved graph recovery and sensitivity to newer literature that may contradict outdated “ground truth” graphs.

Another approach integrates a causal graph into RAG. Causal-graph RAG (CausalRAG) [57] shows promise for reducing hallucinations by retrieving causally relevant context beyond traditional RAG limitations like chunking, but they use the LLM for causal-path inference mainly post-retrieval, and either approach remains constrained by the coverage and quality of the underlying corpus/graph.

¹SD-AI Leaderboard: <https://ub-iad.github.io/sd-ai/#/leaderboard/cld>

3.3.3.1 RAG and the constraints of corpus dependence

There are several failure modes when conditioning an LLM’s generation on retrieved evidence. First, generation quality is fundamentally bounded by retrieval: the necessary evidence may be absent from the corpus (coverage/availability), inaccessible due to licensing or indexing constraints, or fragmented by preprocessing choices such as chunking and metadata loss. Second, even when the evidence exists, it may not be retrieved due to query–document mismatch (lexical or semantic mismatch, long-tail terminology, synonymy), embedding or retriever bias, suboptimal query formulation, or limitations of top-k recall under tight context budgets. Third, retrieval can return partially relevant or misleading context—e.g., semantically similar but causally irrelevant passages, outdated or contradictory sources, or redundant snippets that crowd out decisive evidence—so the model’s response is conditioned on noise rather than signal. Finally, post-retrieval steps (reranking, summarization, graph/community aggregation) can further distort evidence by compressing away qualifiers and uncertainty, exacerbating omission and bias in the generated answer.

3.3.3.2 Arguing for corpus independence from a practicality standpoint

In CLD construction, if an expert must provide the corpus that is used as input to the CLD pipeline, the role of the model is constrained to a translation engine. The texts selected by the expert are presumably already, to some degree, familiar to the researcher, which can limit the model’s ability to reduce bias—for example by suggesting hypotheses or search directions grounded in its broader training corpus that the researcher may be unfamiliar with. In contrast, if the model can be used to propose an initial causal hypothesis or search direction, this can save time and may yield new insight, conditional on the approach being reliable enough to improve discovery efficiency on average.

This motivates and positions corpus-independent hallucination reduction methods such as LLM-as-a-judge and logprobs uncertainty quantification, which are established in the broader hallucination reduction field but, according to the literature found in this review, remain untried in the context of causal loop diagram generation. Before discussing these methods in more detail in the next part of the literature review, we first conclude with an overview of the surveyed approaches.

3.3.4 Comparison of LLM-Based CLD Approaches

Table 3.1 summarises prior work on LLM-based CLD construction, distinguishing approaches by their input requirements, hallucination mitigation strategies, and evaluation methods.

Table 3.1: Comparison of LLM-based CLD Generation and Validation Approaches

Approach	Method	Input	Corpus Req.	Hallucination Mitigation
SD-Bot [52]	Text-to-CLD extraction via LLM	Text documents	Yes	None reported
SD-AI [54]	Benchmark suite (causal translation + conformance)	Text prompts / test cases	Yes	Evaluation benchmark (not a mitigation method)
Theoraizer [34]	Variable-pair querying; multiple candidates	User-defined variables	No	Logprob-based UQ; candidate generation
Liu & Keith [58]	Curated prompting techniques	Dynamic hypothesis text	Yes	None reported
Pipeline Algebra [56]	Typed operators; iterative prompting	Abstract + web search	Partial	Expert refinement step
CausalRAG [57]	RAG-enhanced causal discovery	Scientific literature	Yes	RAG grounding (99.5% faithfulness)
Susanti & Holsm. [59]	Validation (not generation); fine-tuning vs prompting	Existing causal graphs	Yes	Fine-tuning (+20.5 F1 over prompting)
Zhang et al. [18]	RAG-based causal graph discovery	Scientific literature	Yes	RAG grounding
This thesis	Corpus-independent generation + post-hoc validation	Domain spec. + GT Lit node set (V)	No	LLM-as-a-Judge; UQ metrics; Deep Research

Overall observations:

Causal translation/extraction approaches outnumber corpus-independent generation-only approaches in the reviewed literature.

- **Corpus dependency:** Most approaches require an external corpus (text-to-CLD extraction or RAG), limiting applicability when relevant corpora are unavailable. Among the CLD-focused approaches surveyed in this review, Theoraizer [34] and this thesis are notable for operating corpus-independently and for constructing CLD-level outputs, in contrast to adjacent pairwise causal discovery work that primarily targets causal graphs (often DAGs) rather than signed feedback-loop CLDs.
- **Hallucination mitigation:** Prior work largely neglects systematic hallucination detection outside of RAG, especially corpus-independent methods such as LLM-as-a-Judge. Theoraizer [34] uses logprobs for uncertainty quantification but does not report a systematic evaluation of hallucination detection performance. RAG-based approaches (CausalRAG [57], Zhang et al. [18]) rely on retrieval grounding but are capped by retrieval quality. This thesis systematically evaluates multiple corpus-independent detection methods.
- **Evaluation scope:** SD-Bot [52] reports 60% link recall and 66–83% loop recall on controlled datasets. Liu & Keith [58] demonstrate expert-comparable quality on simple structures. Pipeline Algebra [56]

presents qualitative case studies. No prior work operationalises multiple definitions of LLM hallucinations in a CLD context (literature ground truth vs. synthetic CLD-specific hallucination types) and compares these systematically while controlling for prompt variation and LLM stochasticity.

Positioning of our approach: Given the prevalence of corpus-dependent text-to-CLD/causal translation pipelines and their reliance on retrieval quality, and the understudy of the (corpus-independent) alternative, CLD generation, combined with recent CLD-specific hallucination taxonomies, provides an opportunity for both quantifying hallucination rates of CLD generation and positioning the approach relative to causal translation. Also, this provides a platform to study whether established corpus-independent hallucination mitigation primitives—introduced later in this review, including LLM-as-a-Judge, LLM-as-a-Corrector, and logprob-derived uncertainty signals—not studied systematically in CLD generation, can reliably detect and remediate CLD hallucination types.

These CLD-specific failure modes and benchmark findings motivate an edge-level generation pipeline paired with post-hoc validation. The generator design evaluated in this thesis is described in Methods (Section 5.2).

Scope disclaimer. While comprehensive ablation of the generator component is beyond the scope of this thesis, we report baseline performance metrics and share selected generator system experimentation conducted as part of the preliminaries to contextualize hallucination detection experiments; however, these results carry their own limitations as detailed in the appendix. It should also be noted that the generator design builds on intellectual contributions from Rick Quax, with implementation contributions from Cillian Hourican and Andrew Crossley, as well as contributions from Nitai Nijholt (as stated in the claim of authorship).

3.4 Prompting strategies for LLM-based causal reasoning

Prompts have been found to be highly influential in determining LLM outputs [15, 16]. Considering the autoregressive mechanism, where each token is generated conditioned on all previous tokens including the prompt, this makes intuitive sense: the prompt establishes the context and reasoning trajectory that the model follows during generation. Even subtle variations in prompt wording, structure, or examples can lead to substantially different outputs, making prompt engineering a critical component of LLM-based systems.

3.4.1 Dominant Prompting Paradigms.

Three dominant paradigms have emerged in the prompting literature:

3.4.2 Few-Shot Prompting.

Few-shot prompting provides the LLM with a small number of input-output example pairs that demonstrate the desired task format and reasoning pattern [16]. By conditioning generation on these examples, models can adapt to novel tasks without parameter updates. Research has shown that few-shot prompting can improve performance by 20–50% on classification tasks, with effectiveness depending on the quality, diversity, and ordering of examples provided. The key insight is that examples serve as implicit task specifications, allowing models to infer the desired behaviour from demonstrated patterns rather than explicit instructions.

3.4.3 Chain-of-Thought (CoT) Prompting.

Chain-of-thought prompting encourages LLMs to generate intermediate reasoning steps before producing a final answer [15]. By decomposing complex problems into sequential reasoning traces, CoT enables models to tackle multi-step reasoning tasks that would otherwise exceed their capabilities. Kojima et al. [60] demonstrated that even zero-shot CoT—achieved by simply appending “Let’s think step by step” to prompts—can dramatically improve reasoning performance: accuracy on the MultiArith dataset increased from 17.7% to 78.7%, and on GSM8K from 10.4% to 40.7%. This finding suggests that LLMs possess latent reasoning capabilities that can be elicited through appropriate prompting, without requiring task-specific examples. Structured CoT variants, which impose explicit frameworks for breaking down tasks, have shown additional improvements—up to 16.8% increases in faithfulness to grounding documents.

3.4.4 Role-Based Prompting.

Role-based prompting assigns specific personas or expert roles to guide model behaviour (e.g., “You are a medical expert” or “Act as a causal reasoning specialist”). This approach has been shown to improve zero-shot reasoning capabilities, particularly in domain-specific tasks where role context activates relevant knowledge encoded during pretraining [61, 62]. Role assignment can be combined with other paradigms—for example, instructing a model to “act as a causal scientist and think step by step” combines role-based and CoT approaches.

3.4.5 Prompting in Causal Discovery Contexts.

In the specific context of causal discovery with LLMs, these same paradigms have been employed with varying success. Kıcıman et al. [13] provide a comprehensive overview of LLM capabilities in causal reasoning, demonstrating that prompting strategies significantly affect performance on causal inference tasks. Their work shows that LLMs can leverage domain knowledge encoded during pretraining to make causal judgements, but that this capability is highly sensitive to prompt formulation.

The “statistical causal prompting” (SCP) approach [63] combines statistical causal discovery with knowledge-based causal inference facilitated by LLMs, embedding domain expert knowledge into algorithms through carefully designed prompts. Experiments demonstrate that augmenting statistical causal discovery with LLM-derived knowledge approaches ground truth more closely than methods without prior knowledge integration. Similarly, the Autonomous LLM-Augmented Causal Discovery Framework (ALCM) [64] synergizes data-driven algorithms with LLMs through structured prompting, automating the generation of more accurate and interpretable causal graphs.

However, research has also identified systematic pitfalls. Joshi et al. [12] show that LLMs are prone to fallacies in causal inference, such as inferring causality based on the order of entity mentions or temporal sequences rather than genuine causal structure. These findings suggest that while prompting can improve LLM performance on causal tasks, careful prompt design is essential to avoid eliciting spurious causal claims.

For CLD generation specifically, Liu and Keith [58] demonstrate that curated prompting techniques can produce CLDs of comparable quality to expert-constructed diagrams on simple structures. Their approach uses few-shot examples of CLD construction combined with structured output requirements. However, this work focuses on generation quality rather than hallucination detection, and does not systematically evaluate prompt variants for verification tasks.

3.4.6 Prompting for Hallucination Detection and Correction.

Prompting for hallucination detection represents a distinct challenge from prompting for generation or causal reasoning. While generation prompts optimise for producing coherent, relevant outputs, detection prompts must elicit critical evaluation of potentially erroneous content—a fundamentally different cognitive stance.

Recent work has developed several prompting strategies specifically for hallucination detection:

- **Self-verification prompting:** Methods like Chain-of-Verification (CoVe) [65] prompt models to generate verification questions about their own outputs, answer these questions independently (to avoid confirmation bias), and then revise responses based on verification findings. This approach has shown significant hallucination reduction in free-text generation tasks.
- **Self-contradiction analysis:** By prompting models to generate deliberately contradictory responses and analysing inconsistencies, systems can identify claims where the model exhibits uncertainty or conflicting knowledge—potential hallucination indicators [9, 66].
- **Claim decomposition:** Breaking complex claims into atomic sub-claims enables more granular verification. Each sub-claim can be independently assessed for plausibility, with inconsistencies across sub-claims signalling potential errors [65].
- **Multi-agent critique:** Du et al. [67] show that prompting multiple LLM instances to debate and critique each other’s responses reduces hallucinations and improves factual accuracy, as errors are more likely to be identified through adversarial review.

However, none of these approaches have been specifically adapted to the causal discovery setting, where verification requires assessing whether a proposed causal relationship is scientifically plausible given domain knowledge. This gap motivates the development of domain-specific prompting strategies for CLD verification.

3.4.7 Bradford Hill Criteria (BHC) as a Prompting Framework.

To address the gap between general hallucination detection and CLD-specific verification, this thesis proposes a prompting method based on the BHC (BHC) for causal assessment [68]. Originally developed for epidemiological research, the BHC provide a systematic framework for evaluating evidence for causal relationships. The nine criteria are: (1) *Strength*—the magnitude of association; (2) *Consistency*—reproducibility across studies; (3) *Specificity*—specific cause leads to specific effect; (4) *Temporality*—cause precedes effect; (5) *Biological gradient*—dose-response relationship; (6) *Plausibility*—biological or mechanistic plausibility; (7) *Coherence*—alignment with existing knowledge; (8) *Experiment*—experimental evidence supports association; and (9) *Analogy*—similar associations observed elsewhere.

Shimonovich et al. [68] conducted a systematic review of causal inference frameworks, finding that a subset of these criteria—particularly plausibility, temporality, and experiment—are ubiquitous across diverse causal assessment frameworks in epidemiology, social science, and systems thinking. We adapt these findings for CLD verification by focusing on four key criteria:

- **Plausibility** assesses whether the proposed causal mechanism is scientifically reasonable given domain knowledge.
- **Temporality** verifies that the causal direction is appropriate (cause must precede effect).

- **Strength** evaluates the magnitude and significance of the association.
- **Experiment** checks for experimental evidence supporting the association. In cases where direct experimental design is not available (e.g., general knowledge verification), we operationalise **Coherence** as a proxy to check alignment with established theory and empirical findings.

Building on this framework, we develop a “Mechanistic” prompt variant that structures LLM evaluation around these criteria, providing explicit guidance for assessing causal claims. This contrasts with baseline prompts (direct evaluation without scaffolding) and pure CoT prompts (step-by-step reasoning without Shimonovich-specific criteria).

3.4.8 Application to Judge and Corrector Roles.

We apply these prompting principles across different roles and tasks in the CLD verification pipeline:

3.4.9 LLM-as-a-Judge.

For hallucination detection, the judge receives a generated causal edge (including source variable, target variable, polarity, and causal narrative) and must assess its validity. We evaluate three prompt variants:

1. **Baseline:** Direct evaluation instruction without reasoning scaffolding.
2. **Chain-of-Thought:** Step-by-step reasoning instruction [60] with systematic evaluation guidance.
3. **Mechanistic:** Structured multi-criteria evaluation using BHC-aligned criteria (Plausibility, Temporality, Strength, Coherence). In this variant, *Coherence* is operationalised as a proxy for experimental evidence, as direct experimental design details are often not accessible in the general knowledge setting.

For citation judging (assessing whether cited sources support claimed causal relationships), we additionally evaluate a **Mechanistic-Lit** variant. This variant explicitly incorporates the **Experiment** criterion from the BHC framework, replacing Coherence, as the presence of cited literature allows for the direct assessment of study design and experimental evidence.

3.4.10 LLM-as-a-Corrector.

For hallucination correction, the corrector receives edges flagged as potentially problematic by the judge and must determine appropriate remediation actions (retain, revise motivation, change polarity, or remove). Prompt variants follow a cumulative design:

1. **Baseline:** Examples only (control condition).
2. **CoT:** Examples plus step-by-step CoT reasoning instructions ($1.45 \times$ baseline length).
3. **Mechanistic:** Examples plus CoT plus Bradford Hill causal criteria ($1.91 \times$ baseline length).

This design enables systematic evaluation of whether domain-specific prompting (BHC) provides value beyond general reasoning scaffolding (CoT) for CLD verification tasks. The experimental results (Chapter 8) compare these variants on both detection accuracy and correction effectiveness.

3.5 LLM-as-a-Judge: Automated Evaluation of LLM Outputs

The **LLM-as-a-Judge** paradigm employs large language models to evaluate the outputs of other LLMs, providing scalable automated assessment where human evaluation is prohibitively expensive or time-constrained [21]. This approach has emerged as a primary evaluation methodology for open-ended generation tasks where traditional metrics (BLEU, ROUGE) fail to capture semantic quality.

3.5.1 Foundations and Empirical Validation

The paradigm was systematically established by Zheng et al. [21], who introduced MT-Bench—a multi-turn benchmark of 80 challenging questions—and Chatbot Arena, a crowdsourced platform for pairwise model comparison. Their key finding was that strong LLM judges (GPT-4) achieve over 80% agreement with human preferences on pairwise comparisons, approaching the level of inter-annotator agreement among humans themselves. This high concordance established LLM-as-a-Judge as a viable proxy for human evaluation at scale.

Subsequent studies have confirmed and extended these findings. Recent comprehensive surveys indicate that LLM-as-a-Judge-derived accuracy metrics align more closely with human evaluations than traditional UQ metrics, achieving up to 80% agreement with human evaluators across diverse tasks [69]. This makes LLM judges strong candidates for evaluating context-aware outputs—such as causal loop diagram edges—especially when expert human evaluation time is scarce.

Reference-free vs. reference-based evaluation. LLM-as-a-judge work has converged on a useful distinction between reference-free and reference-based evaluation [70, 71]. A *correctness judge* is intentionally context-independent: it evaluates whether an edge narrative is internally coherent and mechanistically plausible even when no external evidence is available or when evidence is incomplete/ambiguous. This aligns with intrinsic verification approaches that assess logical consistency and step validity without retrieval—such as critique-and-correction setups [72] and deductive verification of chain-of-thought reasoning [73]. In a causal discovery setting, this judge targets reasoning quality—directionality, polarity, conditionality, and alternative explanations—because semantic topical match alone can miss these causal specifics, and because some plausibility criteria (e.g., Bradford Hill-style coherence/plausibility) are not reducible to “is there a matching quote.”

Citation judge as evidence-based verifier. Complementarily, the literature supports a citation judge as a retrieval-augmented, evidence-based verifier that makes judgements about support, not just about relevance. Evidence-grounded paradigms like decompose-then-verify [74] and search-augmented verification [75] operationalise this by checking atomic claims against retrieved passages, while citation-focused work uses natural language inference to evaluate whether cited text actually entails the stated proposition [76]. This maps cleanly onto a judge that operates on retrieved quotes rather than model priors.

3.5.2 Systematic Biases in LLM Judges

However, using one LLM to evaluate another introduces structural biases inherent to the transformer architecture and training process. Research has identified several systematic biases that affect judge reliability:

Position Bias. LLM judges exhibit systematic preferences based on the *order* in which candidate responses are presented. In pairwise comparison settings, judges may favour responses presented first (primacy bias) or last (recency bias), independent of actual quality [21]. This positional preference can flip evaluation outcomes when response order is reversed, introducing noise unrelated to content quality.

Verbosity Bias. LLM judges tend to prefer longer, more detailed responses even when concise answers are equally or more accurate [21, 69]. This bias is particularly problematic for scientific evaluation, where precision and accuracy may be better served by focused claims rather than elaborate explanations. In CLD evaluation, this could manifest as preference for edges with verbose justifications over those with succinct but accurate mechanistic explanations.

Self-Enhancement Bias. LLM judges exhibit a tendency to rate outputs from their own model family more favourably than outputs from other models [69]. This self-preference introduces systematic bias when the same model (or model family) is used for both generation and evaluation, creating a form of circular validation.

Susceptibility to Adversarial Manipulation. LLM-as-a-Judge has been shown to be susceptible to adversarial attacks that exploit these biases [29, 69]. Adversarial prompts can be crafted to inflate judge scores without improving actual output quality, raising concerns about robustness in high-stakes evaluation settings.

3.5.3 Evaluation Modes: Pointwise vs. Pairwise

LLM judges can operate in two primary modes, each with distinct trade-offs:

Pointwise Evaluation. The judge assesses a single response against defined criteria, producing an absolute score (e.g., 0–1 scale) or categorical verdict (correct/incorrect/partially correct). This mode is computationally efficient and enables direct comparison across different evaluation sessions, but requires well-calibrated scoring criteria.

Pairwise Evaluation. The judge compares two candidate responses and selects the better one, often with explanation. This relative comparison is more robust to calibration issues—the judge need only rank, not score—but requires $O(n^2)$ comparisons for n candidates and cannot produce absolute quality estimates.

For CLD hallucination detection, pointwise evaluation is more practical: each edge requires an independent assessment of validity rather than comparison against alternatives. The challenge lies in establishing reliable scoring criteria and thresholds that distinguish hallucinations from valid relationships.

Taken together, this literature supports judge-style verification (and judge-then-correct patterns) as scalable hallucination mitigation primitives. The specific judge and corrector designs evaluated in this thesis are detailed in Methods (Sections 5.4.2, 5.4.3, and 5.6).

Connection to test-time compute. Conceptually, LLM-as-a-Judge can be viewed as a *test-time compute* primitive: it allocates additional inference-time computation to validate (and potentially reject) a candidate output that was produced by a cheaper first-pass generation step. In this thesis, judges are therefore not only evaluation instruments but also building blocks for compute-scaled validation pipelines. We discuss the broader test-time compute perspective—including serial “generate–verify–revise” patterns and parallel sampling/selection strategies that subsume judge-style verification—in more detail in Section 3.8.

3.5.4 LLM-as-a-Corrector: Beyond Detection to Remediation

Extending the judge paradigm, **LLM-as-a-Corrector** uses language models to not only identify errors but actively remediate them [19, 65]. In this setup, a judge first identifies problematic outputs, then a corrector LLM receives the original output along with the judge’s critique and generates an improved version.

This judge-then-correct pipeline has shown promise in iterative refinement settings. Chain-of-Verification (CoVe) [65] applies this pattern by: (1) generating a baseline response, (2) planning verification questions,

(3) answering questions independently to avoid bias, and (4) generating a corrected response incorporating verification findings. Similar approaches include Reflexion [19], which maintains a memory of past errors to guide future corrections. Similarly, Self-Refine [77] enables LLMs to improve their own outputs through an iterative feedback-refinement loop using a single model for both generation, feedback, and refinement, reporting average performance gains of $\sim 20\%$ across diverse tasks without requiring supervised training data.

However, correction introduces its own failure modes. Correctors may introduce new errors while fixing detected ones, particularly when the underlying knowledge is genuinely uncertain. In CLD contexts, a corrector might “fix” a valid but uncommon causal relationship by replacing it with a more stereotypical one, trading one form of error for another. Also, since the corrector receives the Judge’s feedback as input, if the Judge makes an error, this will likely propagate to the corrector, which is instructed to adhere to the Judge’s instructions.

Connection to test-time compute. LLM-as-a-Corrector instantiates *serial* test-time compute: additional inference-time steps are used to iteratively revise an initial output, conditioned on critique signals (here: judge verdicts/messages). This places the judge-corrector loop within a larger family of compute-scaled refinement methods (e.g., generate-verify-edit and debate-style revision), and clarifies why correction cost should be treated as a resource to allocate selectively rather than uniformly. The broader design space of serial vs. parallel test-time compute strategies—and how these motivate our later Deep Research validation design—is reviewed in Section 3.8.

3.5.5 Implications for CLD Hallucination Detection

The LLM-as-a-Judge and LLM-as-a-Corrector paradigms offer scalable approaches to CLD validation. In the CLD setting, prior work suggests several practical considerations and potential mitigations:

1. **Bias mitigation:** Position and verbosity biases can be partially addressed through prompt engineering (randomizing presentation order, penalizing excessive length) or ensemble methods (aggregating multiple judge calls with varied conditions) [21, 69].
2. **Complementary validation:** Given systematic judge biases and susceptibility to adversarial manipulation, LLM judges are best treated as one signal among others and combined with complementary checks (e.g., intrinsic uncertainty signals and retrieval/embedding-based grounding) rather than used in isolation [29, 69].
3. **Conservative thresholding:** Because high scores can reflect superficial plausibility rather than correctness, edges flagged as low-confidence warrant serious scrutiny, while edges receiving high scores may still benefit from additional validation in high-stakes settings [69].
4. **Domain-specific calibration:** Judge reliability and score calibration can vary with task framing and domain; thresholds and interpretation may therefore require domain-specific tuning and periodic revalidation [69].
5. **Error propagation in correction:** When correctors receive judge feedback as input, judge errors can propagate downstream—an incorrect verdict may lead the corrector to “fix” a valid edge or preserve an invalid one. This coupling implies that corrector effectiveness is bounded by judge accuracy and that correction benefits are context-dependent [19, 65, 77].
6. **Correction-induced errors:** Correctors may introduce new errors while addressing detected ones, particularly when the underlying knowledge is genuinely uncertain [19, 65]. In CLD contexts, a corrector might replace a valid but uncommon causal relationship with a more stereotypical one, trading one error type for another; when available, evidence-grounding can help reduce such failures.

3.5.6 Limitations

Several fundamental limitations constrain the applicability of judge–corrector pipelines for CLD validation:

- **Circular validation risk:** Using the same model (or model family) for generation, judgement, and correction creates potential for circular validation, where systematic biases are reinforced rather than detected [69].
- **Knowledge boundary constraints:** Without access to external evidence (e.g., retrieval or citations), judges and correctors share the same parametric knowledge limitations as generators, and therefore cannot reliably verify claims that fall outside the model’s knowledge boundary [6, 7, 9].
- **Causal reasoning limitations:** LLMs exhibit systematic weaknesses in causal reasoning, including sensitivity to variable ordering and susceptibility to spurious correlations [12, 13]. These limitations affect judges assessing causal validity and correctors attempting to repair causal claims.
- **Computational cost:** Multi-step judge–corrector pipelines multiply inference costs. For large-scale CLD generation, this overhead may be prohibitive without selective application guided by uncertainty signals.
- **Evaluation complexity:** Assessing judge and corrector performance requires ground-truth labels, which are expensive to obtain for CLD edges and may themselves reflect expert disagreement rather than objective truth.

Taken together, this literature supports hybrid detection approaches that combine LLM-based judges and correctors with complementary uncertainty and grounding signals, rather than relying on any single mechanism in isolation.

3.6 Logprob Uncertainty Quantification (LUQ) Metrics for Hallucination Detection

In the Background chapter we reviewed autoregressive decoding (including temperature, top- k , and top- p sampling) and the general mechanisms by which autoregressive generation can amplify errors. Here, we focus on *corpus-independent* predictors and grounding signals that can be computed from the generation process and lightweight comparisons, motivating the metrics evaluated in this thesis.

Observing the assigned probabilities over an LLM’s output tokens can provide computationally inexpensive uncertainty signals derived directly from the generation process without requiring external knowledge bases or retrieval systems. Relying on LLM-generated signals to quantify uncertainty requires that this model uncertainty is reflected in the model’s internal state at some level—i.e., that models ‘know what they don’t know’—in a manner that is measurable through either the internal state or the outputs, a premise supported by Kadavath et al. [78]. Yet, hallucination is associated with a model asserting something factually incorrect in a confident manner [79], so this uncertainty may not be reflected in the output text itself. It may, however, be reflected in the LLM’s output log-probability distribution over tokens, which has been shown to be directly related to hallucinations [14, 80].

While these metrics are directly accessible with white-box model access when running models locally or via open-source model providers, the proprietary OpenAI LLM APIs also expose token-level log-probabilities during

generation, enabling computation of Logprob Uncertainty Quantification (LUQ) metrics using state-of-the-art commercial models (at time of writing, GPT-4.1).

These signals contain useful information that can be employed for, e.g., knowledge distillation, where a student model is trained to replicate a teacher model’s output token probability distributions [81], thereby improving the student’s performance. Other use cases include autocomplete and evaluation for RAG. The utility of the information contained in output distribution logprobs may also explain why some providers of proprietary models (i.e., Anthropic) do not expose them, whether for data/privacy reasons [82] or to prevent model stealing [83] or distillation. Notably, OpenAI exposes only a truncated probability distribution over the vocabulary, limited to the top k tokens. Nonetheless, logprobs remain available for open-source alternatives and are thus feasible to compute in many cases. As noted, because logprobs are generated as part of the sequence generation process, they are computationally inexpensive and have the additional benefit of providing a measure of uncertainty that is independent of external knowledge bases or retrieval systems. Consequently, if logprobs prove to be useful signals for hallucination detection in a CLD generation context, they would constitute a desirable signal to employ.

3.6.1 The Uncertainty–Hallucination Link.

Crucially, the probability distribution at each generation step provides a window into the model’s epistemic state. When the model “knows” the correct continuation, the distribution tends to be peaked; when uncertain, probability mass spreads across alternatives. Xiao and Wang [79] empirically confirmed that higher predictive uncertainty - measured through the token distribution - corresponds to higher hallucination rates. This insight motivates LUQ metrics: by examining the shape of these distributions, we can identify generation steps where the model may be confabulating.

3.6.2 LUQ Metrics: From Sequence to Token Granularity

Token-level probability distributions, accessible through model logits or log-probabilities, provide direct insight into the LLM’s confidence during generation. Varshney et al. [14] demonstrated that low-confidence generations correlate with hallucinated content, proposing validation of outputs based on minimum token probability. Their “stitch in time” approach identifies potentially hallucinated spans by detecting tokens where $P(x_i|x_{<i}) < \tau$ for a calibrated threshold τ , enabling early intervention before erroneous content propagates.

Given these token-level probability distributions, we can derive uncertainty features at multiple granularities. We present three complementary metrics, ordered from coarsest (whole-sequence) to finest (single-token) aggregation:

3.6.2.1 Perplexity (Sequence-Level).

Perplexity aggregates uncertainty across the *entire generated sequence*, providing a holistic measure of how “surprising” the output was to the model. Defined as the exponentiated average negative log-likelihood with *length normalization*:

$$\text{PPL} = \exp \left(-\frac{1}{N} \sum_{i=1}^N \log P(x_i|x_{<i}) \right) \quad (3.1)$$

Length normalization (division by N) ensures comparability across sequences of different lengths, as shorter sequences generally have higher joint probability [80]. Higher perplexity indicates the model found the sequence surprising, potentially signalling confabulation. However, as a whole-sequence average, perplexity can be diluted

by confident tokens surrounding a localized hallucination, and can be inflated by domain-specific terminology or rare but valid tokens.

3.6.2.2 Maximum Window Entropy (Subsequence-Level).

To detect *localized* uncertainty spikes that whole-sequence averaging would obscure, Sriramanan et al. [80] proposed windowed aggregation. They observe that the entire sentence may not be hallucinatory, and that length-average scores may therefore be insufficient for accurate hallucination detection, motivating a windowed approach sensitive to short, localized hallucination spans.

At each token position t , entropy is computed over the top- k alternative tokens:

$$H_k(t) = - \sum_{j=1}^k \tilde{p}_j \log \tilde{p}_j \quad (3.2)$$

where $\tilde{p}_j$ represents the normalized probability of the j -th most likely token. We then apply a sliding window of size w to compute rolling averages, returning the maximum:

$$H_{\text{max-win}} = \max_i \left(\frac{1}{w} \sum_{t=i}^{i+w-1} H_k(t) \right) \quad (3.3)$$

This identifies the highest-uncertainty *subsequence* in the generated text - particularly suited for detecting hallucination onset, where uncertainty may spike locally before the model commits to a fabricated claim and in the generated sequence contingent on that claim the model may rise in confidence again.

3.6.2.3 Minimum Probability (Token-Level).

At the finest granularity, minimum probability captures the single “weakest link” in the generation chain:

$$P_{\min} = \min_{i \in [1, N]} P(x_i | x_{<i}) \quad (3.4)$$

Varshney et al. [14] demonstrated that this metric effectively identifies hallucination onset - a single highly uncertain token often marks where fabrication begins. Their “stitch in time” approach uses this signal for early intervention, detecting tokens where $P(x_i | x_{<i}) < \tau$ for a calibrated threshold τ .

3.6.3 Worked Example: Applying LUQ Metrics

To illustrate these metrics, consider an LLM generating the causal claim: “*Stress increases cortisol levels*”. At each token position t , the API returns the probability of the selected token along with probabilities for top- k alternatives. Figure 3.1 visualises a hypothetical generation with $k = 3$:

Position 3 (“cortisol”) exhibits lower confidence ($P = 0.45$) and higher distributional uncertainty. Applying the LUQ metrics at each granularity:

- **Perplexity** (Sequence): $\exp\left(-\frac{1}{4}[\log 0.92 + \log 0.88 + \log 0.45 + \log 0.95]\right) \approx 1.60$ - the uncertain token at $t = 3$ is partially masked by confident neighbours.

$t = 1$	$t = 2$	$t = 3$	$t = 4$
Stress	increases	cortisol	levels
$P = 0.92$	$P = 0.88$	$P = 0.45$	$P = 0.95$
<i>Top-k alternatives (normalized $\tilde{p}$):</i>			
Stress	increases	cortisol	levels
(0.85)	(0.80)	(0.50)	(0.90)
Chronic	elevates	adrenaline	production
(0.10)	(0.15)	(0.30)	(0.07)
Acute	raises	hormone	release
(0.05)	(0.05)	(0.20)	(0.03)

Figure 3.1: Token probabilities during LLM generation. Top row: selected tokens with probabilities $P(x_t|x_{<t})$. Below: top- k alternatives with normalized probabilities. Position $t = 3$ shows high uncertainty - alternatives “adrenaline” and “hormone” have substantial probability mass.

- **Max Window Entropy** (Subsequence): With $w = 2$, the window spanning $t = 3, 4$ would show elevated entropy due to position 3, localizing the uncertainty spike.
- **Min Probability** (Token): $\min(0.92, 0.88, 0.45, 0.95) = 0.45$ at $t = 3$ - directly identifying the weakest link.

This example illustrates the complementary nature of the three granularities: perplexity provides a global summary, windowed entropy localizes problematic subsequences, and min probability pinpoints the exact token of concern.

3.6.4 Types of Uncertainty in the Logprob Signal

The uncertainty captured by LUQ metrics is not monolithic. Following the classical decomposition in machine learning [84], uncertainty in autoregressive language models can be categorized into two fundamentally different types:

3.6.4.1 Uncertainty sources in autoregressive generation.

In the context of autoregressive language modelling, these uncertainties arise from distinct sources [85]:

- *Epistemic*: Out-of-distribution inputs, rare token combinations, knowledge gaps, extrapolation beyond training distribution
- *Aleatoric*: Genuinely ambiguous contexts, multiple valid completions, underspecified prompts
 - *Lexical* (sub-case): Synonym choices, paraphrase options, stylistic variation, tokenization alternatives

3.6.4.2 Aleatoric Uncertainty (Data Uncertainty).

This is inherent, irreducible uncertainty arising from ambiguity in the task itself. In language generation, aleatoric uncertainty manifests when multiple valid continuations exist - for instance, completing “The cat sat on the...” could legitimately yield “mat,” “sofa,” or “windowsill.” This uncertainty reflects genuine linguistic variability rather than model ignorance. High entropy at such positions is *not* indicative of hallucination; it reflects the creative freedom inherent in language.

A notable sub-case is *lexical uncertainty* (surface-form variation), where multiple tokens or phrasings convey the same semantic content. When a model shows uncertainty between “large” and “big,” or “however” and “nevertheless,” this reflects stylistic optionality rather than factual doubt. This distinction is central to semantic entropy approaches [44], which cluster semantically equivalent outputs to separate surface-level lexical uncertainty from deeper semantic uncertainty about factual content. For hallucination detection, lexical uncertainty is largely noise - what matters is whether the model is uncertain about *what to claim*, not *how to phrase it*.

3.6.4.3 Epistemic Uncertainty (Model Uncertainty).

This is reducible uncertainty arising from the model’s lack of knowledge - gaps in training data or limitations in model capacity. When a model is asked about events after its knowledge cutoff, obscure facts, or domains underrepresented in training, the resulting uncertainty reflects genuine ignorance. Xiao and Wang [79] demonstrated that epistemic uncertainty is more indicative of hallucination than aleatoric or total uncertainty, as it captures precisely those cases where the model is “guessing” rather than expressing legitimate ambiguity.

The challenge for hallucination detection is that LUQ metrics capture *total* uncertainty without distinguishing its source. A flat probability distribution may indicate either “I don’t know” (epistemic - hallucination risk) or “many options are valid” (aleatoric - creative freedom). This conflation is a fundamental limitation of single-generation uncertainty metrics, motivating approaches like semantic entropy [44] that attempt to separate these components through multiple sampling.

3.6.5 Limitations of LUQ Metrics

Despite their theoretical grounding, LUQ metrics face fundamental limitations:

Calibration assumptions. LUQ metrics assume that model confidence correlates with factual accuracy - that uncertainty signals epistemic doubt. However, this calibration is not guaranteed by standard training objectives [78]. Models can be confidently wrong, particularly on memorized but incorrect patterns.

Domain shift. Metric distributions vary with domain. A model may exhibit high perplexity on valid but specialized terminology (e.g., medical or legal jargon), producing false positives. Conversely, fluent hallucinations in the model’s “comfort zone” may show low uncertainty.

Confident hallucinations. The hallucination snowballing phenomenon [41] means that once a model commits to a fabricated premise, subsequent tokens may be generated with high confidence as the model maintains internal coherence - despite being factually wrong.

Uncertainty conflation. As discussed in Section 3.6, the logprob distribution conflates epistemic, aleatoric, and lexical uncertainty into a single signal. A high-entropy distribution may indicate genuine ignorance (hallucination risk), legitimate ambiguity (multiple valid continuations), or mere stylistic optionality (synonym choices). LUQ metrics cannot distinguish these cases from a single generation. This conflation means that high uncertainty is a necessary but not sufficient condition for hallucination - the model may be uncertain for benign reasons. Approaches like semantic entropy [44] attempt to address this by sampling multiple generations and clustering semantically equivalent outputs, thereby isolating epistemic uncertainty. However, such methods require multiple forward passes and are computationally expensive compared to single-generation LUQ metrics.

These limitations motivate hybrid approaches combining LUQ metrics with context-sensitive validation (e.g., retrieval-augmented verification) and ensemble classification over multiple uncertainty signals.

3.7 Embedding-Based Grounding Metrics

Complementing LUQ metrics, embedding-based similarity between a candidate text span t and an external reference s provides a context-sensitive grounding check. More generally, for any text span whose support is being assessed (e.g., an LLM output, a human-written claim, or an intermediate system-generated hypothesis), the cosine similarity between the candidate text t and a chosen reference text s can serve as a soft grounding signal:

$$\text{sim}(\mathbf{e}_t, \mathbf{e}_s) = \frac{\mathbf{e}_t \cdot \mathbf{e}_s}{\|\mathbf{e}_t\| \|\mathbf{e}_s\|} \quad (3.5)$$

where $\mathbf{e}_t$ and $\mathbf{e}_s$ are embedding vectors of the candidate text span t and the chosen reference text s , respectively. Low similarity may indicate that the candidate text is weakly supported by (or inconsistent with) the chosen reference—including cases where a claim is paired with an irrelevant or mismatched reference (“attribution hallucination”) [27, 86].

3.7.1 Design choices: reference text and input/query text.

The usefulness of embedding-based grounding depends on what is selected as the reference text s and what is embedded as the input/query x (when retrieval is involved). Valid choices for the *reference text* s include: (i) an explicitly cited span or paragraph, (ii) retrieved passages provided as context (e.g., RAG evidence), (iii) a gold-standard evidence snippet in an evaluation dataset, (iv) a full document section (trading precision for coverage), or (v) other machine-generated text intended as evidence (e.g., a judge’s retrieved quote). Likewise, the *input/query text* x can be the original user prompt, the model’s full answer, an extracted atomic claim, or a structured hypothesis (e.g., a causal edge narrative). In many practical pipelines, x is derived directly from the candidate span t (e.g., $x = t$ or a structured reformulation of t) when the goal is to retrieve evidence for that candidate claim. These choices induce different failure modes (e.g., dilution when embedding long answers, overconstraint when using retrieval context as the only reference) and therefore matter when comparing grounding approaches later in this thesis.

Unlike LUQ metrics, which are purely intrinsic to the generation process, embedding-based grounding requires access to external reference material. This represents both a strength (verification against source text becomes feasible) and a limitation (access to reference material is required). Furthermore, proprietary model providers do not provide access to the training corpus of their language models, making direct source attribution infeasible in such cases, as the mapping between a generated sequence and its original training sources is unavailable.

3.7.2 Retrieval-Augmented Generation

The canonical Retrieval-Augmented Generation (RAG) approach [87] uses an input sequence x to retrieve passages z from a knowledge base $\mathcal{K}$ according to a retrieval objective, typically based on embedding similarity:

$$z^* = \arg \max_{z \in \mathcal{K}} \text{sim}(\mathbf{e}_x, \mathbf{e}_z) \quad (3.6)$$

where $\mathbf{e}_x$ and $\mathbf{e}_z$ denote the embeddings of the query x and candidate passage z , respectively. The retrieved passage(s) are then provided as context to a generator, producing an output sequence y conditioned on both x and the retrieved evidence (e.g., $p_\theta(y \mid x, z^*)$), with retrieval typically parameterized as $p_\eta(z \mid x)$ [87].

Modern LLM web interfaces from providers such as Anthropic, OpenAI, Google, and Perplexity incorporate search-augmented generation, combining iteratively refined web searches with RAG to produce grounded responses [88]. This approach aims to anchor generation in retrievable sources, reducing the risk of hallucination and providing direct attribution.

3.7.3 Limitations of Retrieval-Constrained Generation

A notable limitation of the RAG paradigm is that retrieval may *overconstrain* generation, leading to a failure mode where the model, instructed to condition its response solely on retrieved content, fails to assert a true claim simply because the retrieval process did not surface supporting evidence [89]. This can occur even when an unconstrained model—drawing on its parametric knowledge—might have correctly inferred the relationship, but the retrieved context does not contain (or even contradicts) the relevant support [88].

This failure mode becomes more likely as the source that would confirm the claim may not appear in search results, with the probability of this occurrence inversely related to the depth and breadth of the search. Furthermore, evidence suggests that LLM responses may synthesize knowledge from multiple sources [90, 91], such that a valid claim may not be directly attributable to any single retrievable document. This synthesis property of LLMs exacerbates the recall limitations of RAG-constrained systems, as the supporting evidence for a synthesized claim may be distributed across multiple sources that are not jointly retrievable.

In contrast, unconstrained generation (standard LLM inference without retrieval constraints) does not suffer this recall limitation but is more susceptible to false positives (hallucinations), as the model may generate plausible-sounding but unsupported claims.

3.7.4 A Hybrid Approach: Generate-then-Search

Given the complementary strengths and weaknesses of unconstrained generation and retrieval-augmented validation, we propose a hybrid approach that maximises recall through unconstrained generation while boosting precision through post-hoc retrieval-augmented validation. This approach proceeds as follows:

1. **Hypothesis Generation:** An LLM generates a hypothesized causal link with a narrative justification (e.g., “ A causes B through mechanism M ”).
2. **Evidence Retrieval and Validation:** The hypothesis is submitted as a query to a retrieval system (e.g., web search). An LLM-as-a-judge evaluates whether the retrieved evidence supports the causal narrative.
3. **Iterative Correction:** If the judge validates the claim, the hypothesis is accepted. If not, a corrector revises the causal narrative accordingly before re-evaluation.

This generate-then-search paradigm separates the creative generation phase (optimised for recall) from the critical validation phase (optimised for precision), potentially yielding higher F1 scores with respect to ground truth than either approach alone.

3.7.5 Limitations of the Hybrid Approach

Several limitations of this hybrid approach warrant acknowledgment:

Retrieval dependency. The quality of LLM-as-a-judge validation remains contingent on retrieval system accuracy and the availability of relevant source documents, and even the position of those source texts in the context window [89]. Claims for which no supporting documentation exists in the search corpus cannot be validated, regardless of their truth value.

Confirmation bias. As formulated, the system queries only for *supporting* evidence and does not actively seek counter-evidence. This asymmetry may introduce confirmation bias, where claims are accepted based on partial supporting evidence without consideration of contradictory information [92].

Semantic gap. Embedding-based similarity may fail to capture nuanced semantic relationships. A generated claim and its purported source may have high embedding similarity while differing in critical details, or conversely, may express the same fact in sufficiently different language to register low similarity.

Despite these limitations, when a reasonable match is found between a generated sequence and a source text, this provides supporting evidence for the generated claim—a desirable starting point in scientific contexts, from which further verification or attempts at falsification can proceed.

These considerations motivate evaluating a combination of corpus-independent uncertainty features and corpus-dependent grounding signals. The concrete UQ feature set and its use in our experiments are specified in Methods (Section 5.2.2).

3.8 Test-time compute and multi-agent validation

RQ3 investigates whether adaptive test-time compute allocation—triggered by uncertainty signals—can improve CLD accuracy efficiency. Prior work demonstrates that test-time strategies can reduce hallucinations in reasoning and knowledge-intensive tasks, and that compute-optimal allocation per-prompt yields up to $\sim 4\times$ efficiency gains over uniform sampling [43]. This section synthesizes the main paradigms—serial refinement and parallel sampling—and identifies gaps in their application to CLD generation.

3.8.1 Serial Compute: Iterative Refinement

Serial test-time compute arranges LLM calls sequentially, enabling “divide and conquer” reasoning where each step can reflect on and correct errors from preceding stages. Several serial strategies have demonstrated hallucination reduction in reasoning tasks:

3.8.1.1 ReAct: Reasoning and Acting.

Yao et al. [93] introduced a paradigm that interleaves chain-of-thought reasoning with actions querying external tools (e.g., search APIs, knowledge bases). Unlike pure chain-of-thought prompting—which relies solely on parametric knowledge and is prone to hallucination—ReAct grounds reasoning in retrieved evidence, allowing models to verify claims against external sources. The framework demonstrates that reasoning and action are mutually beneficial: reasoning traces guide action selection, while action observations refine subsequent reasoning, enabling iterative error correction. Empirically, ReAct reduced hallucination rates on knowledge-intensive tasks compared to reasoning-only baselines.

3.8.1.2 Multi-Agent Debate.

Du et al. [67] introduced a method where multiple LLM instances engage in multi-round debates to enhance response accuracy. Each model proposes and critiques responses over several rounds, leading to a consensus. The authors report that this approach significantly enhances mathematical and strategic reasoning across a number of tasks and improves the factual validity of generated content by reducing fallacious answers and

hallucinations. The approach applies directly to existing black-box models using uniform procedures and prompts across different tasks.

3.8.1.3 Chain-of-Verification (CoVe).

CoVe [65], detailed in Section 3.5, exemplifies the generate-verify-correct pattern central to serial test-time compute. While CoVe reduces hallucinations through iterative self-verification, it has not been adapted specifically to verify causal claims—e.g., generating verification questions about whether relationship $X \rightarrow Y$ is supported by evidence or whether polarity is correctly assigned.

3.8.1.4 Misconception Refutation (MR).

Estornell and Liu [94] propose editing responses rather than only selecting among them: (1) prompt an LLM to identify errors, misconceptions, and inconsistencies, then (2) prompt it again to minimally correct the response. This approach draws on the insight that LLMs are often more skilled at evaluating answers than at directly providing them. The authors prove theoretically that misconception refutation increases the probability of debate converging to the correct answer. MR is particularly valuable when combined with quality and diversity pruning in multi-agent debate settings.

Gap for CLD generation: None of these serial test-time methods have been adapted specifically to verify causal claims—e.g., generating verification questions about whether relationship $X \rightarrow Y$ is supported by evidence or whether polarity is correctly assigned. This thesis explores whether the Judge-Corrector primitive can reduce hallucinations in CLD generation (RQ1).

3.8.2 Parallel Compute: Sampling and Selection

Parallel test-time compute generates multiple candidate outputs simultaneously, then selects or aggregates results.

3.8.2.1 Self-Consistency.

Wang et al. [66] introduced self-consistency, a decoding strategy that samples diverse reasoning paths and selects the most consistent answer by marginalizing over sampled paths. The key insight is that complex reasoning problems typically admit multiple different ways of thinking that lead to the correct answer. Unlike greedy decoding which commits to a single path, self-consistency generates multiple candidates in parallel and aggregates via majority voting. This achieved significant improvements on arithmetic and commonsense reasoning benchmarks: GSM8K (+17.9%), SVAMP (+11.0%), AQuA (+12.2%), StrategyQA (+6.4%), and ARC-challenge (+3.9%).

3.8.2.2 Best-of-N Sampling.

This general strategy generates N candidate responses in parallel and selects the best according to a reward model or verifier. Efficiency gains are possible through compute-optimal allocation strategies that adapt N based on problem difficulty [43]. In CLD literature, Theoraizer [34] applies prompt ensembling and logprob-based candidate selection—a form of Best-of-N with uncertainty-weighted aggregation. However, Theoraizer does not systematically evaluate hallucination detection performance of these signals.

3.8.3 Adaptive Compute Allocation

A key efficiency question is whether expensive verification can be deployed *selectively* based on signals that predict when it is needed. Snell et al. [43] demonstrate that compute-optimal strategies—adapting verification effort per-prompt—yield up to $4\times$ efficiency gains over uniform allocation. RouteLLM [95] routes queries between smaller and larger models based on predicted difficulty, achieving significant cost savings while maintaining quality. More recently, BEST-Route [96] extends this to test-time optimal compute allocation across model cascades.

Gap for CLD generation: These approaches predict query difficulty to route compute, but do not use *generator-side uncertainty metrics* (e.g., logprobs, perplexity) as triggers. Moreover, no work has explored adaptive compute allocation for CLD hallucination mitigation. This thesis investigates whether UQ metrics from CLD generation can serve as triggers for selective deployment of expensive verification (Judge, Corrector, or Deep Research), addressing RQ3.

This literature motivates adaptive test-time compute for hallucination mitigation but reveals validation gaps: the core primitives (Judge, Corrector) have not been validated for CLD generation, and UQ-triggered selective verification remains unexplored in this domain. This thesis first validates these primitives (RQ1–RQ2) before exploring adaptive allocation (RQ3). The Deep Research pipeline (Section 5.7) instantiates a ReAct-style approach adapted for causal claim validation.

3.9 Synthesis and bridge to methods

Prior work provides many ingredients (LLM-based extraction/generation, judge-style evaluation, uncertainty proxies, retrieval grounding), which have been developed and tested to some level of sophistication and success in hallucination reduction in other contexts. However, the literature lacks a systematic evaluation for the specific task of hallucination detection in CLD generation.

Rather than choosing breadth (e.g., multi-agent debate has been shown to improve reasoning accuracy and reduce hallucinations in other tasks [67, 94]), we choose an approach that first thoroughly validates the core agent primitives of LLM-as-a-Judge and LLM-as-a-Corrector in a single-agent setting. If validated individually and together, these primitives form a natural building block for multi-agent systems, supporting an observe-act-react cycle that allows for iterative error correction. While established in the literature, these have not yet been validated in the CLD context; knowledge of how these building blocks function can then inform the design of more complex multi-agent systems, chaining together generation, evaluation, and retrieval as the CLD generation task demands.

Conditional on LLM-as-a-Judge and LLM-as-a-Corrector being effective, and hallucination detection using LUQ showing discriminative signal, we can explore allocating expensive verification (judges, correctors, literature search) *selectively*—using these computationally cheap LUQ signals—rather than applying it uniformly across all candidate edges. The original RQ3 design proposed using UQ metrics as triggers for corrector deployment; the pivot to Deep Research explores an alternative: allocating test-time compute to external evidence validation rather than internal correction.

The relative contribution of LLM-as-a-Judge, LLM-as-a-Corrector, and each metric class—LUQ metrics, embedding-based grounding, and retrieval-augmented validation—to hallucination detection in a CLD generation context remains an empirical question. This thesis addresses this question through systematic evaluation across multiple experimental conditions.

Chapter 4

Problem Setup



This section defines the task, hallucination types, and evaluation metrics used throughout the experiments.

4.1 Task: Hallucination Detection and Mitigation for CLD Edges

The end-to-end system produces Causal Loop Diagrams (CLDs)—directed graphs where nodes represent system variables and edges represent causal relationships with polarity (POSITIVE, NEGATIVE, or NONE) and mechanistic explanations. However, the primary task evaluated in this thesis is not “generate a complete CLD” in the abstract; rather, it is detecting and mitigating hallucinations in CLD edge predictions produced by an LLM generator under a fixed variable set.

Unit of analysis. Throughout RQ1–RQ3, we operate on ordered *edge slots* (A, B) over a fixed node set V . For each slot, the generator outputs a relationship label in {POSITIVE, NEGATIVE, NONE} plus a short mechanistic narrative (and, where applicable, citations). Hallucination detection then assigns each slot a binary label (hallucinated vs. non-hallucinated) under a specified ground-truth construct (GT Lit vs. GT Synth), and mitigation mechanisms (corrector / Deep Research) attempt to improve alignment under that construct.

 **Role of generation.** Generation is an upstream prerequisite that defines the candidate edge set and artefacts, but it is treated as a *given input process* in the main experiments: we fix the validation node set V to the GT Lit variables and evaluate downstream verification/remediation methods on the resulting edge tables. Full generation details are provided in Appendix H (Algorithm H.1). This isolates hallucination detection/mitigation performance from confounds due to differing node sets.

Input data (ground truth): Appendix C.1 (Tables C.1 and C.2). **Output artefacts:** Appendix C.5 (Table C.36).

4.2 Hallucination Definition

Manifestations Hallucinations = **incorrectness in causal links**: erroneous *absence* (FN) or *presence* (FP) of links, wrong direction, or wrong causal narrative.

Operationalisation

Key definition (used throughout RQ1–RQ3).

- **GT Lit:** CLD from peer-reviewed literature. **Hallucination** = **FP/FN** w.r.t. literature CLD.
- **GT Synth:** LLM-corrupted edges. **Hallucination** = **corrupted edges** (corrupted=True).

Clarification (construct validity and motivation). GT Lit and GT Synth define *different* ground-truth constructs: GT Lit operationalises hallucination as deviation from a published expert reference model (FP/FN w.r.t. the literature CLD), whereas GT Synth operationalises hallucination as *known injected errors* produced by an LLM corruptor (corrupted=True). We include GT Synth experiments to test judge efficacy in a controlled setting with fully known ground truth, and to connect CLD hallucination operationalisation to broader hallucination benchmarks in the general domain (e.g., HaluEval [97]), which similarly use LLM-based corruption to generate adversarial errors. Results across GT Lit and GT Synth should therefore be interpreted as complementary: GT Synth primarily validates controlled detection of induced CLD errors, while GT Lit evaluates alignment with expert reference CLDs in a domain-specific setting.

Notation (to avoid TP/FP/FN ambiguity). Throughout this thesis, we use confusion-matrix terms in *two distinct* ways:

- **(A) Generator edge recovery vs. GT Lit (structure alignment).** We evaluate whether the generator recovers the *existence* of literature edges over the fixed node set V . Under this edge-slot representation, we treat POSITIVE and NEGATIVE as *edge present* and NONE as *edge absent*. A *TP* is a correctly predicted present edge slot, *FP* is a predicted present edge slot that is absent in GT Lit, *FN* is a missed GT Lit edge slot predicted as NONE, and *TN* is a correctly predicted NONE slot. (Polarity and narrative quality are validated separately by the judges; GT Lit does not provide ground-truth narratives.)
- **(B) Hallucination detection (judge classification).** Here, the *positive class* is “hallucination” and TP/FP/FN/TN refer to the judge correctly/incorrectly flagging hallucinations. In **GT Synth**, the hallucination label is **corrupted=True**. In **GT Lit** judge experiments, the hallucination label is *disagreement with the expert reference CLD* under the edge-slot definition above (i.e., $FP \vee FN$ w.r.t. GT Lit).

Table 4.1: Two uses of TP/FP/FN in this thesis (generator scoring vs. judge scoring).

Use	Positive class ($y=1$)	Ground truth label and interpretation
Generator edge recovery (vs. GT Lit)	“Edge present” (POSITIVE/NEGATIVE)	GT Lit provides an expert edge set over fixed nodes V . We score <i>edge existence</i> in ordered edge slots: $FP/FN/TN/TP$ are with respect to whether an edge is present in the literature CLD (polarity/narratives are evaluated separately).
Judge hallucination detection	“Hallucination”	GT Synth: $y=1$ iff corrupted=True . GT Lit judge experiments: $y=1$ iff the generator output is <i>misaligned with the expert CLD</i> under the edge-existence definition (i.e., $FP \vee FN$ w.r.t. GT Lit).



Synthetic corruption (GT Synth). To validate judge-based hallucination detection under *fully known* error labels, we construct a synthetic ground-truth regime by applying an LLM **Corruptor** to an LLM-generated base edge table. The corruptor introduces controlled, HaluEval-inspired perturbations [97] at a fixed corruption rate (used in this thesis: **rate=0.3**) and marks each perturbed edge slot with **corrupted=True**. This yields a

clean binary target for judge evaluation (`corrupted=True` vs. `False`) while keeping the node set fixed to the validation set V .

We describe the Corruptor component (inputs, process, outputs, and corruption types) in the System Architecture chapter (Section 5.3); the full algorithm is in Appendix H (Algorithm H.4).

Chapter 5

System Architecture

This section documents all system components: the generator, judges (correctness and citation), corrector, corruptor, and Deep Research system.

5.1 Pipeline Overview

Figure 5.1 presents the complete CLD generation, judging, and correction pipeline. The system operates in three phases:

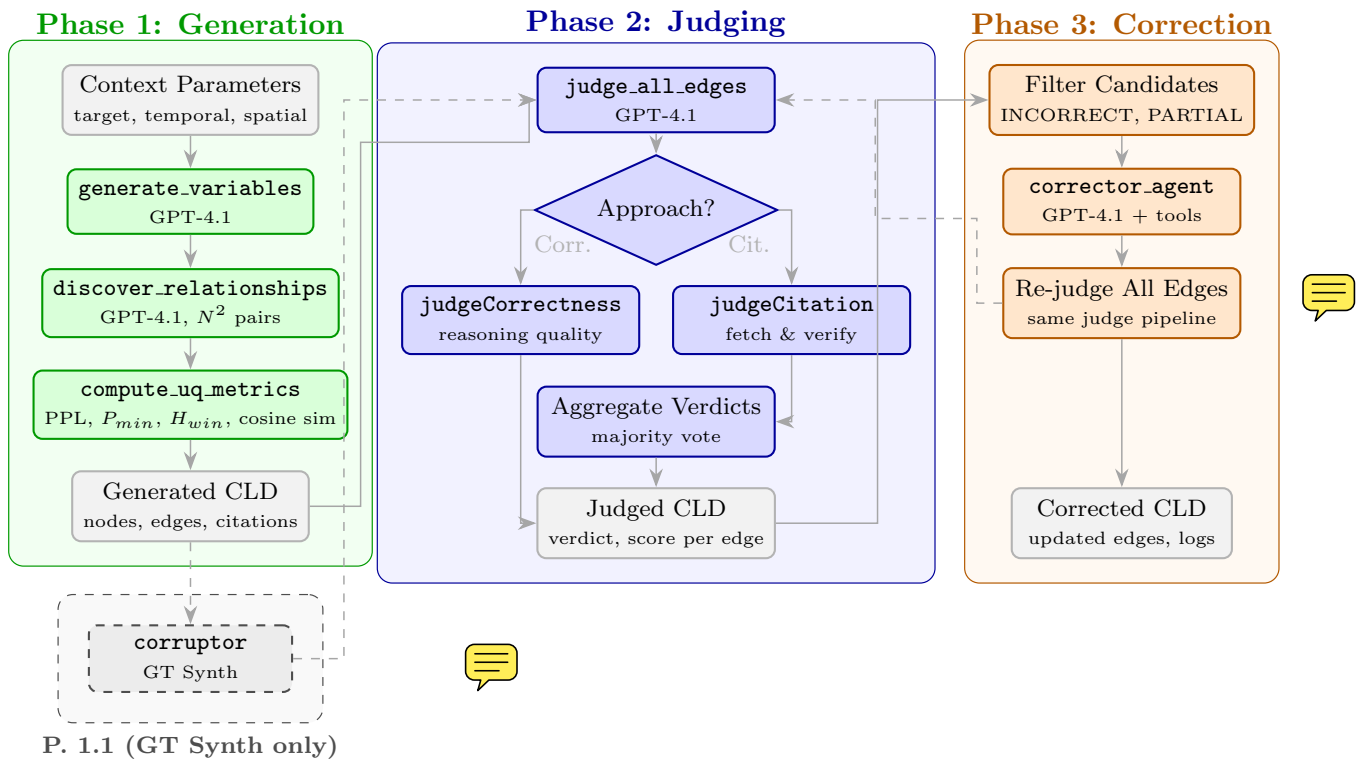


Figure 5.1: Complete CLD pipeline showing three phases: **Generation** (green) creates variables and edges with UQ metrics capture; **Judging** (blue) evaluates edge quality via correctness or citation-based approaches with majority voting; **Correction** (orange) applies LLM-guided fixes to flagged edges followed by re-judging. **Optional (GT Synth):** a corruption step (dashed) can be applied after Generation and before Judging to produce `corrupted=True/False` labels for synthetic ground truth.

Phase 1: Generation: The LLM generator (GPT-4.1) constructs CLDs through iterative node and edge generation, capturing uncertainty quantification (UQ) metrics for downstream hallucination detection.

Phase 1.1: (GT Synth only) Corruption: A Corruptor can be applied after Generation and before Judging to introduce synthetic perturbations and emit `corrupted=True/False` labels for known-ground-truth hallucinations.

Phase 2: Judging: An LLM judge evaluates each edge using either correctness-based reasoning assessment or citation-based verification, producing per-edge verdicts.

Phase 3: Correction: An LLM corrector applies tool-based fixes to flagged edges, followed by re-judging to validate corrections.

5.2 Generator

5.2.1 Design rationale: ~~Generator~~

This section briefly motivates the generator’s decomposition into edge-level decisions and the accompanying safeguards used in our experiments. Full algorithmic and prompt details are provided in the Appendix .

In our CLD generation system, given temporal and spatial scales and domain context, we generate a set of nodes (or, in the main experiments, insert validated nodes by fixing the node set to the ground-truth validation variables V), and then generate relationships pairwise as directed queries over ordered variable pairs (A, B) and (B, A) (i.e., $|V|(|V|-1)$ candidate edge slots). For each directed pair, the LLM is asked whether A *directly* causes B and to return a relationship type and a short causal narrative; in this study, relationship types are constrained to $\{\text{POSITIVE}, \text{NEGATIVE}, \text{NONE}\}$ (Appendix E.2, Listing E.1). To address the limitation of reviewing links in isolation, we perform a mediation check by passing the current node set V to the edge prompt as candidate mediators. When supported by the LLM API, we also capture token-level logprobs for the edge-generation response (relationship label + narrative) and derive logprob-based uncertainty features (LUQ metrics) used in downstream hallucination detection. Algorithmic details are traced from the implementation in Appendix H (Algorithm H.1). Relative to Crielaard et al.’s CLD-to-SDM pipeline (Section 2.3; Crielaard et al. [5]), this operationalises the *causal link identification* step and approximates *intermediary-variable documentation* via mediation checks over the current node set, while deferring richer aCLD elements such as functional forms and interaction terms to future work (i.e., these are not addressed in this thesis). Our downstream phases address evidence-quality annotation through post-hoc edge validation and remediation (LLM-as-a-Judge and LLM-as-a-Corrector) and, when allocating additional test-time compute, external evidence acquisition via Deep Research (RQ3).

5.2.2 Generator implementation

Input: Domain specification (target phenomenon, spatial/temporal scale, scope), optional seed node set.

Process: The Generator constructs CLDs through iterative node and edge generation (see Appendix H.1 for algorithmic details):

- **Node generation:** Proposes system variables relevant to the domain
- **Edge generation:** For each variable pair, proposes relationship type (POSITIVE, NEGATIVE, NONE) with causal narrative
- **UQ capture:** Records token-level logprobs during generation for downstream uncertainty quantification

Output: Complete CLD (nodes + edges with causal narratives) and per-edge UQ metrics.

Note: In all RQ1 and RQ2 experiments, nodes were seeded with the validation CLD node sets; node generation was disabled to isolate edge-level performance.

Configuration: GPT-4.1, Nitai_C prompt, T=0.7—selected via preliminary studies (Section 7.6).

Algorithm reference: Appendix H (Algorithm H.1). **Input parameters (prompt):** Appendix E.2. **Input parameters (configuration):** Appendix C.4.10 (Table C.28).

UQ Metrics Capture

UQ metrics are computed solely from the generator LLM’s internal signals during edge generation—they do not require external context. We capture four UQ metrics from generator logprobs and embeddings (see Appendix H.6 for algorithmic details):

5.2.3 Design rationale: UQ metrics



We treat UQ features as cheap, corpus-independent signals that may complement retrieval-based grounding. This section motivates why we combine LUQ metrics with embedding/retrieval signals and how they are intended to be used (e.g., flagging/triggering more expensive verification).

Taken together, embedding-based grounding and retrieval-augmented validation provide a principled way to move beyond purely intrinsic uncertainty signals by explicitly checking whether a generated claim is semantically aligned with external evidence. Crucially, these components differ in *corpus dependence*: LUQ metrics are *corpus-independent* (computed directly from the generator’s token probabilities without requiring any external text corpus), whereas embedding similarity is *corpus-dependent* because it requires candidate source texts to embed and compare against, inheriting retrieval coverage and accessibility constraints.

At the same time, the preceding limitations imply that cosine similarity should be interpreted as a *soft* proxy rather than a definitive correctness criterion: high similarity can reflect topical overlap without causal support, and low similarity can arise from benign paraphrase or construct mismatches. In a CLD setting, semantic relevance alone is insufficient: a passage can be “about” the same variables while failing to entail the specific *direction* and *polarity* of a causal edge, or while only supporting a mediated/conditional relationship. At the same time, the corpus-independent nature of logprob-based UQ metrics provides a signal that is not bounded by retrieval quality. This motivates the evaluation strategy adopted in this thesis: combining multiple approaches with complementary strengths that cover the weaker points of other methods. We treat embedding similarity as a lightweight grounding feature that complements LUQ metrics rather than as a correctness guarantee. We also pair it with more context-sensitive, correctness-based and citation-based generate-then-search judging to assess, in a qualitative and context-dependent manner, whether retrieved passages provide causal evidence for an edge narrative. Given that UQ metrics are cheaper computationally than judging or correction, if proven effective, they could be used to flag hallucinations and save compute, or to signal ambiguity and trigger deployment of a more expensive judge–corrector loop. Finally, the experiments in this thesis quantify how much each component (LUQ, embedding grounding, retrieval, and judging) contributes to hallucination detection and, if these prove to be effective enough, to adaptive allocation of test-time verification compute.

Table 5.1: UQ metrics captured during edge generation. Conceptual formulas shown; implementation uses capping for numerical stability (see Section 7.2).

Metric	Formula	Expected
Perplexity	$\text{PPL} = \exp\left(\frac{1}{n} \sum_{i=1}^n \text{nll}_i\right)$	$\uparrow \Rightarrow$ halluc
Min Probability	$P_{\min} = \min_{i \in \{1, \dots, n\}} P(x_i)$	$\downarrow \Rightarrow$ halluc
Max Window Entropy	$H_{\max\text{-}win} = \max_i \frac{1}{w} \sum_{t=i}^{i+w-1} H_k(t)$	$\uparrow \Rightarrow$ halluc
Cosine Similarity	$\text{sim} = \max_c \cos(\mathbf{e}_{\text{narr}}, \mathbf{e}_c)$	$\downarrow \Rightarrow$ halluc

Where: $\text{nll}_i = -\log P(x_i)$ with $P(x_i) = \exp(\log\text{prob}_i)$; $H_k(t) = -\sum_{j=1}^k \tilde{p}_j(t) \log \tilde{p}_j(t)$ is entropy over top- $k=5$ tokens with $\tilde{p}_j(t) = \text{softmax}(\log\text{prob}_j(t))$; $w=5$ is the sliding window size; $\mathbf{e}$ denotes embeddings (all-mpnet-base-v2, $d=768$).

5.2.4 UQ metrics implementation

Input: Token-level logprobs from generator LLM response (when available via API).

Process: Four metrics are computed from the generator’s completion tokens (see Appendix H.6 for algorithmic details):

- **Perplexity:** Average token-level uncertainty with two-stage capping (token NLL capped at 20.0, average capped at 10.0) to prevent outliers from dominating
- **Min Probability:** Confidence floor—minimum token probability across the sequence, capturing worst-case uncertainty
- **Max Window Entropy:** Highest-uncertainty region detected via sliding window ($w=5$) over per-token entropies (top- $k=5$ alternatives)
- **Cosine Similarity:** Semantic alignment between narrative and citation chunks (citation experiments only); embeddings via all-mpnet-base-v2 (768-dim)

Output: Per-edge UQ feature vector for downstream hallucination classification.

Configuration: Logprobs from OpenAI Chat Completions API (`logprobs=True`, `top_logprobs=5`); embedding model all-mpnet-base-v2 [98].

Full implementation details: Exact formulas with numerical stability measures (capping thresholds, sliding window parameters, chunking strategy) are provided in Section 7.2.

Algorithm reference: Appendix H (Algorithm H.6).

Token sequence note. When available (OpenAI Chat Completions with `logprobs=True` and `top_logprobs=5`), logprob-derived UQ metrics are computed over the model’s *completion* token stream exposed as `choice.logprobs.content` (a list of per-token entries with fields `token`, `logprob`, and `top_logprobs`). This token stream corresponds to the full assistant completion for the edge call, i.e., the entire JSON string `{"Relationship": "...", "Explanation": "..."} including JSON syntax and the Relationship label. Prompt/input tokens are not included in this API output, so these metrics reflect uncertainty in the generated output string rather than the prompt. For providers/endpoints without logprobs support (e.g., non-OpenAI models or the OpenAI Responses API), logprob-derived metrics are omitted.`

5.3 Corruptor (Synthetic corruption for GT Synth)

Input: An LLM-generated base edge table over a fixed node set V (edge slots (A, B) with relationship label and narrative), a corruption seed, and a target corruption rate $r \in [0, 1]$.

Process: Using an LLM-based hallucination injection methodology inspired by HaluEval [97] for motivation corruption, applying the main edge-level hallucination types found in SD-AI’s taxonomy [54] and applying spurious edges and polarity flips programmatically, in 1/3 equal proportions at a corruption rate of 0.3 (meaning we apply corruption to 30% of the edges):

- **Motivation corruption:** replace a valid mechanistic narrative with plausible-but-flawed reasoning while keeping the edge slot fixed.
- **Spurious edges:** convert NONE $\rightarrow$ POSITIVE or NEGATIVE to create false positive links.
- **Polarity flips:** invert POSITIVE $\leftrightarrow$ NEGATIVE.

For reproducibility, corruption is seeded; the output table preserves both original and corrupted fields for auditing.

Output: A corrupted edge table with a per-edge-slot binary label **corrupted** and the corresponding corrupted content (relationship label and/or narrative). This dataset is used as GT Synth in RQ1a to evaluate judge detection under known injected errors.

Configuration used in this thesis: model=gpt-4.1, temp=0.7, top_p=1, rate=0.3. **Algorithm reference:** Appendix H (Algorithm H.4). **Reproducibility (run commands):** Appendix C.4.

5.4 Judges

5.4.1 Design rationale: LLM-as-a-Judge

We operationalise two judge types—Correctness and Citation—based on prior work on LLM-based evaluation [21]. This section explains the specific scaffolding choices we adopt for CLD edge verification.

The LLM-as-a-Judge system evaluated here incorporates principles from both the hallucination detection literature and causal inference methodology.

We operationalise two judge types: Correctness and Citation-based judging.

Correctness Judge. The correctness judge evaluates whether an edge narrative is internally coherent, and is by design independent of external context. To address the gap between general hallucination detection and CLD-specific verification—where assessment requires evaluating scientific plausibility of causal mechanisms rather than surface-level coherence—we adapt the Bradford Hill Criteria (BHC) from epidemiological causal assessment frameworks [68] as structured evaluation scaffolding. The mechanistic prompt variant operationalises four key BHC criteria (plausibility, temporality, strength, and coherence), combined with Chain-of-Thought prompting to elicit systematic reasoning [15, 60].

Citation Judge. For citation-based judging, we incorporate self-verification patterns from Chain-of-Verification [65] and retrieval-augmented evidence assessment [99], grounding causal claims against retrieved literature. When judging evidence retrieved from multiple sources, this thesis evaluates citations *one at a time* rather than concatenating all citations into a single context window. This keeps the judging context clean and reduces ambiguity: each verdict is conditioned on a single citation’s content, avoiding cross-document interference and allowing failure cases to be attributed to specific sources. These separate judge verdicts are then aggregated via majority voting, choosing the mean score as a starting point to reflect average support in the retrieved documents, although alternatives (median, top- k /max, or explicit reranking/selection of the best citation by a verifier model) may be more robust depending on the evidence distribution and retrieval noise (see Discussion).

Following established LLM-as-a-Judge methodology [21], we employ pointwise scoring with thresholds for binary hallucination classification.

5.4.2 Correctness Judge implementation

Input: Variables, relationship type, causal narrative, prompt type.

Process: LLM-as-a-judge [100] assesses edge correctness by verifying logical/scientific coherence of the causal claim (see Appendix H.2 for algorithmic details).

Output Scores: 0=Incorrect, 0.5=Partially correct, 1=Correct.

Configuration: GPT-4.1, T=0 for reproducibility [21, 101].

Prompt Variants

1. **Baseline** (1500 max output tokens): Direct evaluation without scaffolding
2. **Mechanistic** (1500 max output tokens): Structured multi-criteria evaluation using Bradford Hill-aligned framework [68] (Plausibility, Temporality, Strength, Coherence)
3. **Chain-of-Thought** (10000 max output tokens): Explicit step-by-step reasoning [60] with systematic evaluation of causal logic

Full prompt texts are provided in Appendix E.3.

Algorithm reference: Appendix H (Algorithm H.2). **Reproducibility (run commands):** Appendix C.4 (see also Table C.5) and Appendix C.4.1.

5.4.3 Citation Judge implementation

Input: Variables, relationship type, causal narrative, prompt type.

Process: The Citation Judge implements a **generate-then-search** pipeline: it first uses Brave Search to retrieve candidate article URLs for the edge narrative (Algorithm H.8), then fetches the corresponding full text (ContentScraper-only in the thesis configuration; Algorithm H.9), and finally applies an LLM-as-a-judge [100] to determine whether each fetched source supports the causal narrative (see Appendix H.3 for algorithmic details).

Output Scores: 0=Not supported, 0.5=Partially supported, 1=Fully supported.

Aggregate Judge Score: Each retrieved citation is judged independently with a categorical verdict that is mapped to a scalar score:

$$\text{SUPPORTED} \rightarrow 1.0, \quad \text{PARTIALLY SUPPORTED} \rightarrow 0.5, \quad \text{CONTRADICTED/UNADDRESSED} \rightarrow 0.0.$$

For an edge with C retrieved citations, the aggregate citation-judge score is computed as the mean:

$$\text{aggregate_score} = \frac{1}{C} \sum_{c=1}^C \text{score}_c.$$

Note (mean aggregation vulnerabilities): The mean can be brittle: it is sensitive to the number of retrieved citations, can be dominated by a single strong/weak source, and can be skewed by retrieval/fetch failures (missing-at-random is not guaranteed). It also implicitly treats citations as exchangeable and independent, which may not hold when sources are redundant or correlated.

Qualitative Aggregation: In addition to the numeric score, an LLM aggregator synthesizes a qualitative verdict from the per-citation results (see Algorithm H.3, line 19). This thesis uses only the numeric aggregate scores for quantitative analysis.

Configuration: GPT-4.1 (or GPT-5-mini for cost reduction in GT Synth), T=0 for reproducibility.

Prompt Variants

1. **Baseline** (1500 max output tokens): Direct evaluation without scaffolding
2. **Mechanistic** (1500 max output tokens): Bradford Hill-aligned framework
3. **Chain-of-Thought** (10000 max output tokens): Explicit step-by-step reasoning
4. **Mechanistic-Lit** (1500 max output tokens): Mechanistic prompt enhanced with explicit literature grounding requirements

Full prompt texts are provided in Appendix E.4.

Algorithm reference: Appendix H (Algorithm H.3). **Fetcher algorithm/config:** Appendix H (Algorithm H.9).

Retrieval Mechanism

Step 1 (Search): Brave Search is used to obtain up to 5 candidate citation URLs per edge, optionally filtered by a trusted-domain allowlist (Algorithm H.8).

Step 2 (Fetch): For each URL, full text is fetched using the citation fetcher. In the experiments reported in this thesis, fetching was run in **ContentScraper-only** mode (BeautifulSoup-based fetching with PDF-capable extraction). See Appendix H.9 for the full fetcher pipeline and configuration details.

Retrieval policy (compact summary).

- **Domain filtering (search):** Citation URLs are obtained via Brave Search and filtered with a trusted-domain allowlist (up to 5 URLs retained per edge; Appendix H.8).
- **Query construction:** For each edge, the search query concatenates the **source**, **relationship**, and **target** variables with the generated causal narrative/claim text; queries are truncated to Brave API limits (50 words, 400 characters).

- **Paywalls / partial text:** Full-text fetching uses ContentScraper with local PDF extraction; sources that fail to fetch or yield short/error content are discarded. If no valid full text can be fetched, the edge is assigned a `NO_CITATION` outcome for citation-based judging.

Retrieval Strategy: Generate-then-Search

The Citation Judge follows a **generate-then-search** paradigm: the generator produces edges without retrieval constraints, and the judge subsequently searches for supporting literature. This decouples generation recall from retrieval coverage—the generator leverages its full parametric knowledge, while retrieval serves as post-hoc validation rather than a constraint on generation. See Section 3.7.4 for motivation and limitations of this approach.

5.5 Citation Search (Brave)

Input: An edge claim (source variable, relationship type, target variable, and causal narrative), plus an optional trusted-domain allowlist and a Brave Search API key.

Process: The citation search component issues a web search query for each edge to obtain candidate citation URLs. Queries concatenate `source`, `relationship`, `target`, and the edge narrative text, and are truncated to Brave API limits (50 words, 400 characters). Results are optionally filtered by a trusted-domain allowlist (e.g., academic/government domains) to bias toward credible sources. In this codebase, Brave search is used during citation filling when no usable citations are returned directly by an LLM (see Algorithm H.8).

Output: Up to 5 candidate citation URLs per edge, stored as `r.citations` for downstream citation fetching and citation-based judging.

Algorithm reference: Appendix H (Algorithm H.8).

5.6 Corrector

5.6.1 Design rationale: LLM-as-a-Corrector

The corrector is implemented as a post-hoc remediation stage conditioned on judge feedback, allowing detection and remediation to be measured separately [19, 65].

In this paragraph, we motivate how prior judge-corrector and iterative refinement frameworks inform the design of the corrector component implemented in this thesis. Motivated by these judge-then-correct frameworks [19, 65, 77], we implement the corrector as a post-hoc remediation phase that is explicitly conditioned on judge feedback: candidate edges are selected from a Neo4j session based on low judge verdicts or low aggregate scores, and the corrector receives the edge ($A \rightarrow B$), its current relationship type and motivation (causal narrative), and the judge’s critique (verdict + message). Crucially, we explicitly separate judging and correction into distinct phases, rather than having a single model perform both tasks simultaneously. This separation follows the principle that task decomposition improves LLM performance on complex reasoning tasks [42], and allows us to measure detection and remediation performance independently—an important methodological requirement when evaluating whether hallucinations are being correctly identified versus successfully repaired. The corrector then executes one of a small set of auditable actions—either *revise* the motivation while preserving the edge type, or *change_type* by switching the relationship label (e.g., `POSITIVE/NEGATIVE` ↔ `NONE`) while providing a new motivation—mirroring the “generate then refine” pattern in Self-Refine [77] and the final correction step

in CoVe [65] while keeping the action space constrained to reduce uncontrolled edits. To mitigate the risk of introducing new errors, we optionally re-run the judge after corrections to re-score the updated edge set, closing a lightweight verify–correct loop. Unlike Reflexion [19], we do not maintain cross-run memory; instead, correction decisions are logged back into the graph (e.g., `corrected`, `correction_action`, `corrector_message`) for reproducibility and downstream analysis. Full algorithmic details and the exact prompt variants used for the corrector are provided in Appendix H (Algorithm H.5) and Appendix E.5.

5.6.2 Corrector implementation

Input: Variables, relationship type, causal narrative, judgement from LLM-as-a-judge.

Process: Changes an edge by (see Appendix H.5 for algorithmic details):

- a) **Changing relationship type:** e.g., `None`→`Positive`, `Negative`→`None`
- b) **Revising motivation:** Rewriting causal narrative with improved reasoning/evidence

Configuration: GPT-4.1, T=0.7 for exploration [77]. Full prompt variants (Baseline, CoT, Mechanistic) are provided in Appendix E.5.

Direction Correction Mechanism Because all pairs are evaluated bidirectionally ($A \rightarrow B$ and $B \rightarrow A$), direction errors are corrected via type changes:

Example: If $A \rightarrow B$ is erroneous (should be $B \rightarrow A$): $A \xrightarrow{\text{NONE}} B$ (remove wrong) + $B \xrightarrow{\text{POS}} A$ (add correct)

Algorithm reference: Appendix H (Algorithm H.5). **Input parameters (prompts):** Appendix E.5.

5.7 Deep Research System

The Deep Research (DR) system is a multi-agent pipeline that iteratively searches scientific literature for evidence supporting or refuting causal relationships.

5.7.1 Design rationale: Deep Research

Deep Research is designed as a multi-agent, test-time compute pipeline for external evidence validation of edge claims. This section motivates the key design choices (decomposition, parallel search, reranking, and quoting) in relation to prior multi-agent and verification patterns.

In this subsection, we motivate how the multi-agent and test-time compute literature informs the Deep Research design choices evaluated in this thesis. The Deep Research system evaluated here is motivated by several recurring principles in the LLM multi-agent design literature. First, it uses iterative refinement to improve search behaviour over time, in the spirit of self-improvement loops such as Self-Refine [77]. Second, it decomposes an edge claim into multiple independent search directions—including a verbatim query of the causal claim—and executes these directions in parallel, consistent with claim-decomposition patterns (e.g., CoVe) [65] and compute-scaling strategies that benefit from parallel exploration [43]. Third, it incorporates an explicit reranking step, which is common in literature search and retrieval systems and benefits from strong embedding models [98, 102]. In our implementation, we initially operate over Brave Search result snippets as lightweight evidence proxies—leveraging the provider’s built-in retrieval and ranking via the Brave Search

API [103]—and then perform full-text article fetching only for the highest-rated candidates most likely to contain direct causal evidence. Fourth, it adopts an orchestrated agent architecture with judge/corrector style control flow, implemented using typed agent tooling (PydanticAI) [104] and aligned with broader test-time compute perspectives [43]. Finally, it requires explicit passage quoting to support evidence claims, in line with the SAFE framework, which has been shown to reduce hallucinations in retrieval-augmented settings [86]. Collectively, these choices prioritise breadth over minimality, maximizing the probability of finding direct causal evidence and enabling an initial test of whether automated literature search merits further investigation for CLD validation. Full system architecture and algorithmic details are described in Section 5.7.

5.7.2 Deep Research implementation

Input: Causal claim (source variable, target variable, relationship type, causal motivation/narrative).

Process: Multi-agent literature search pipeline with iterative refinement (see Appendix H.11 for algorithmic details):

- **Hypothesis evaluation:** Produces a focused search plan (3–4 queries) plus keywords and confounders; additionally inserts the *verbatim claim* as a search query.
- **Parallel web search:** Runs the queries in parallel; within each query thread, the search agent iteratively calls a web-search tool with a hard search budget (max 5 searches per query thread). Retrieval uses Brave Search with a trusted-domain priority list and query augmentation (appending “scientific study research evidence”).
- **Evidence scoring:** Scores each retrieved snippet/source on `relevance_score` and `quality_score` (1–10) with brief methodological justification.
- **Article selection & full-text fetch:** Selects up to 2–3 candidate articles likely to contain direct causal evidence and fetches full text via HTML parsing (BeautifulSoup), truncating to a fixed length cap to avoid token overload.
- **Direct causal evidence detection:** Applies strict criteria for *direct* causal support (e.g., RCTs/experiments, natural experiments, and meta-analyses/systematic reviews confirming causality); requires a verbatim quoted passage from the source.
- **Judgement, refinement, synthesis:** Judges aggregate support (and can propose a modified claim for a subsequent iteration), then synthesizes the final structured verdict.

Output: Final verdict (`supported`/`partially_supported`/`unsupported`/`invalid`), confidence score (1–10), `direct_causal_found` flag, strongest-evidence fields (`strongest_causal_evidence_url`, `..._passage`, `..._study_type`, `..._confidence`), and a list of retrieved sources (`evidence_urls`).

Multi-Agent Architecture

The system is implemented using the PydanticAI framework [104] with typed agent definitions and structured outputs. Seven primary LLM agents are coordinated by a Python orchestrator class in a multi-iteration pipeline (see Appendix H.11 for algorithmic details); the codebase also defines an optional history summariser for token-budget management. The full multi-agent pipeline is documented in Table H.10 in Appendix H.10.

Models: The Deep Research system uses a two-tier model architecture:

- **Heavy model (default: GPT-5):** Used for complex reasoning tasks—hypothesis evaluation, article selection, direct causal evidence detection, evidence judgement, and verdict synthesis.
- **Light model (default: GPT-5-mini):** Used for high-volume tasks—iterative web search and evidence scoring (and optional history summarization).

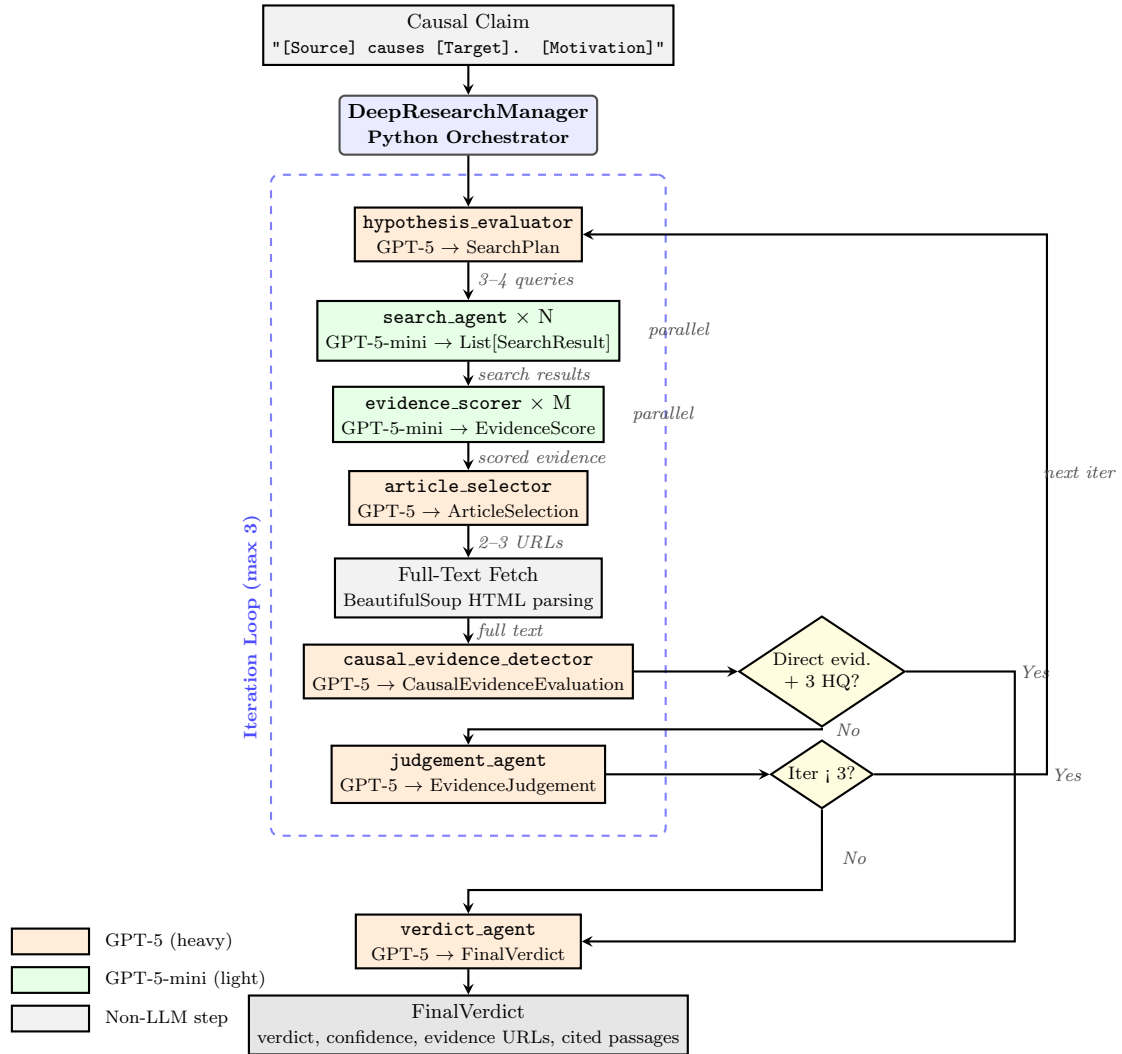


Figure 5.2: Deep Research multi-agent pipeline execution flow. The DeepResearchManager orchestrates seven LLM agents across up to three iterations. Parallel execution (dashed annotations) is used for search and scoring. Early stopping occurs when direct causal evidence is found with 3+ high-quality sources. Orange nodes indicate GPT-5 (heavy model) for complex reasoning; green nodes indicate GPT-5-mini (light model) for high-volume tasks.

Search Infrastructure

Search Provider: Brave Search API with academic domain filtering [103].

Query Enhancement: All queries are automatically appended with “scientific study research evidence” to bias toward academic content.

Retry Strategy: Retry-enabled HTTP sessions for Brave API calls and full-text fetching. Timeouts: 10s connect, 40s read (Brave); 15s for full-text fetch.

Algorithm reference: Appendix H (Algorithm H.11). **Agent components:** Appendix H.10 (Table H.10). **Input parameters (configuration):** Appendix C.4.8 (Table C.23).

Early Stopping Criteria

The system terminates early when any of the following conditions are met:

- **Strong evidence found:** Direct causal evidence detected AND ≥ 3 high-quality sources (relevance ≥ 8 , quality ≥ 7)
- **High confidence support:** Judgement supported with confidence ≥ 8
- **Time limit:** 900 seconds (15 minutes) elapsed
- **Max iterations:** Configured iteration limit reached

Chapter 6

Experimental Data

This section documents the CLD domains, context parameters, and validation datasets used across all experiments.

6.1 Evaluation Datasets (GT Lit, LLM Simulations, GT Synth)

Across experiments we distinguish three dataset types, which differ in what is treated as ground truth and how labels are defined:

1. **GT Lit (literature ground truth):** The published CLDs (nodes and edges) extracted from peer-reviewed sources (Appendix C.1, Table C.1). For each domain, GT Lit defines the validation node set V and the validation edge labels used for scoring (see edge-slot definition below). In GT Lit experiments, hallucinations are operationalised as FP/FN relative to this literature CLD.
2. **LLM simulations over GT Lit nodes (generated edges only):** For each domain and run seed, we generate a CLD by fixing the node set to the GT Lit validation variables V and generating edges only. These generated edge tables are produced by the Phase 1: Generation pipeline (Section 5.2; Figure 5.1) and are the primary *predictions* used in RQ1–RQ3 (Figure 7.1). We compute accuracy/FP/FN metrics by comparing predictions against (i) GT Lit labels (deviation from literature) and (ii) GT Synth corruption labels when synthetic corruption is applied (next item). Per-RQ output directories for these run artefacts (under `final_runs/`) are listed in Appendix C.1 (Table C.4); file types and naming conventions are in Appendix C.5 (Table C.36).
3. **GT Synth (synthetic corruption ground truth):** A controlled dataset constructed by taking an LLM simulation edge table produced by Phase 1: Generation (Section 5.2; Figure 5.1) and applying the Corruptor (Section 5.3) to introduce known errors at a fixed corruption rate. As in all edge-level experiments, the node set is fixed to the validation node set V , so corruption is defined over the same edge-slot universe. Ground truth is the `corrupted` flag per edge slot (rather than agreement with GT Lit). GT Synth is used to validate judge detection under fully known, injected error labels.

6.2 Validation and Auxiliary Datasets

In addition to the core CLD datasets above, several auxiliary datasets are used for baselines, validation, and robustness checks:

1. **TruthfulQA verification set:** General-domain hallucination benchmark (1,634 Q&A pairs) used as a judge sanity check prior to CLD evaluation (RQ1; Section 7.8). *Location:* Appendix C.1, Table C.2.
2. **Uleman’s CLD (expert GMB):** An independently expert-constructed CLD (150 edges) used to validate citation-judge reliability and search infrastructure effects (Section 7.11). *Location:* Appendix C.1, Table C.2.
3. **Human–LLM agreement set:** A small human-annotated edge set (72 edges) used to assess agreement between human judgements and citation-judge scores (Section 7.11.2). *Location:* Appendix C.1, Table C.2.
4. **Physics CLDs:** Four curated CLDs (26 nodes, 31 edges total) derived from unambiguous physical laws, used as a sanity check for judge behaviour under clear causal structure (Section 7.11.3):
 - *Thermostat Heating* (7 nodes, 8 edges): Newton’s Law of Cooling [105], First Law of Thermodynamics [106]
 - *Predator-Prey* (6 nodes, 8 edges): Lotka-Volterra equations [107, 108]
 - *RC Circuit* (7 nodes, 8 edges): Ohm’s Law [109], Kirchhoff’s Laws [110]
 - *Water Tank* (6 nodes, 7 edges): Pascal’s Law [111], Torricelli’s Law [112]*Location:* Appendix C.1, Table C.3.
5. **Deep Research human validation sample:** A stratified subset of Deep Research outputs (50 edges) used for applicability calibration and threshold selection (RQ3 validation; Section 7.14). *Location:* Appendix C.1, Table C.2.



Dataset filenames and locations for all auxiliary datasets are summarised in Appendix C.1 (Table C.2); per-experiment output locations are in Table C.4.

6.3 CLD Domains

Three domain-specific Causal Loop Diagrams (CLDs) representing distinct scientific disciplines:

1. **Depressive Symptoms [113]:** Biopsychosocial model (14 variables, 34 edges) capturing feedback loops between cognitive, behavioural, and physiological factors in depressive states. Constructed via triangulation of group model building, literature review, and causal discovery.
2. **Social Norms [114]:** Public health model (10 variables, 12 edges) representing interplay between social norms, body weight perception, and behavioural dynamics. Developed using system dynamics modelling with the HELIUS cohort.
3. **Older Persons ED [115]:** Healthcare systems model (34 variables, 66 edges) of factors influencing older persons’ ED utilisation patterns. Constructed through group model building with interdisciplinary clinical experts.

The exact ground-truth CLD datasets (including filenames and locations) are listed in Appendix C.1 (Table C.1).

Note on “edges” and validation edges used in evaluation. Throughout Results, an **edge** is defined as an ordered (source, target) *edge slot* with a relationship label in {POSITIVE, NEGATIVE, NONE}, i.e., the full directed pair set with $Edges = |V|(|V| - 1)$ represented by the **All Edges** table. The effective number of ground-truth validation edges used for scoring is the count of rows with **In Validation Graph = Yes** (so $TP + FN$ equals the validation-edge count under this representation).

Note on node generation vs. edge-only evaluation. The system supports both node generation and edge generation (Appendix H.1) and we evaluate node generation prompt robustness in Appendix F; however,

in the edge-level experiments (RQ1a/RQ1b/RQ2/RQ3) we **fix the node set to the validation CLD’s ground-truth variables** and generate **edges only** to isolate edge discovery and downstream validation

6.4 Context Parameters for LLM Generation

Each CLD was configured with domain-specific context parameters extracted from the source Excel files. These parameters are provided to the LLM generator to guide causal reasoning within appropriate temporal, spatial, and domain boundaries:

Table 6.1: LLM Context Parameters by CLD (Extracted from Validation Excel Files)

Parameter	Value
<i>Depressive Symptoms CLD</i>	
Target	How biopsychosocial factors interact to influence the development and persistence of depressive symptoms in healthy adults
Spatial Scale	Psychological, social, behavioural, and biological variables
Temporal Scale	Medium- to long-term processes, capturing changes over weeks to months as stressors accumulate and symptoms evolve
Scope	Individual-level stress response mechanisms
<i>Social Norms CLD</i>	
Target	Variations in social norms and cultural perceptions of body weight across different ethnic and gender groups
Spatial Scale	Amsterdam, focusing on specific sociocultural groups within the city
Temporal Scale	Monthly time steps
Scope	Adults aged 18+ from six sociocultural groups in Amsterdam: Dutch, Moroccan, and South-Asian Surinamese men and women
<i>Older Persons ED CLD</i>	
Target	Why individuals aged 65 and older in Amsterdam visit the emergency department
Spatial Scale	Local healthcare context and experiences of older adults within this urban setting
Temporal Scale	Hours to days (inferred from healthcare decision-making timeline)
Scope	Patient-level factors, healthcare system factors, social environment, and socioeconomic/contextual influences

For repository paths to the underlying ground-truth Excel files from which these parameters are extracted, see Appendix C.1 (Table C.1).

Following system dynamics principles [1], variables acting on smaller temporal scales than the variable of interest can be averaged out, while variables on larger scales can be treated as constants. By specifying explicit temporal boundaries, the LLM generator is guided to focus on causal mechanisms operating within the relevant timeframe—e.g., acute stressor responses (weeks–months) for depressive symptoms versus monthly social dynamics for obesity prevalence. Similarly, spatial scale constrains whether causal relationships operate at the individual, community, or healthcare system level. The scope parameter further constrains the domain of variables the LLM should consider.

6.5 CLD Selection Rationale

Ground truth CLDs were selected from peer-reviewed publications with explicitly defined causal structures (named variables, directed edges, polarities). The three CLDs span distinct subdomains within health—biopsychosocial health, public health behaviour, and healthcare systems—which aligns with the thesis’s *Mechanistic* (Bradford Hill–aligned) prompting framework for evaluating causal claims in health literature [68]. CLD

sizes range from 12 to 66 edges (validation sets: Social Norms 12, Depressive Symptoms 34, Older Persons ED 66), providing variation in system complexity.

Limitation: All three CLDs fall within the health domain, limiting generalizability to other fields (e.g., economics, ecology, engineering). Additionally, these CLDs represent expert-hypothesized relationships rather than empirically validated causal effects—a common limitation in complex systems where controlled experiments are infeasible [113–115]. Evaluation therefore measures reproduction of published expert models, which may or may not be causally correct. 

6.6 Edge Structure

Each CLD edge contains: **source/target variables** (named entities with definitions), **relationship polarity** (POSITIVE/NEGATIVE/NONE), **causal motivation** (natural language mechanism explanation), and **scientific citations** (peer-reviewed references). The standardised output artefacts produced per run (including judged Excel files, JSON metrics, and naming conventions) are documented in Appendix C.5 (Table C.36).

Example: Edge row. Table 6.2 shows a single edge as stored in the evaluation spreadsheets, including source/target variables, relationship type, classification against the validation CLD, judge verdict, causal motivation, and retrieved citations.

Source	Target	Type	Class.	Judge	Motivation (excerpt)	Citation(s) (excerpt)
Language barriers	Health literacy	NEG	TP	Fully supp.	Language barriers reduce access to health information...	pmc.../PMC9205365, pmc.../PMC5091931, ...

Table 6.2: Example edge row from the evaluation spreadsheets (Older Persons ED; mechanistic-lit citation judge).

Example: Corrector columns (same row, continued). Table 6.3 shows the corrector-related columns for the same edge. When the judge flags an edge as potentially problematic, the corrector either revises the motivation or changes the relationship type.

Source	Target	...	Action	Original motivation (excerpt)	Corrected motivation (excerpt)
Language barriers	Health literacy	...	revise	Language barriers reduce access to health information...	Language barriers impede comprehension of medical instructions, reducing health literacy...

Table 6.3: Corrector columns for the same edge row (continued). The “...” indicates columns from Table 6.2.

Example: UQ metric columns (same row, continued). Table 6.4 shows the generation-time UQ metrics for the same edge. These logprob-derived signals capture token-level uncertainty: *perplexity* measures average surprisal; *min probability* captures the least confident token; *max window entropy* reflects local uncertainty spikes; and *cosine similarity* measures semantic alignment between the motivation and retrieved citations.

Source	Target	...	Perplexity	Min Prob	Max Entropy	Cosine Sim.
Language barriers	Health literacy	...	1.242	0.041	0.630	0.840

Table 6.4: UQ metric columns for the same edge row (continued). The “...” indicates columns from Tables 6.2–6.3.

Chapter 7

Experimental Design

This section presents the experimental design for each research question, including the master overview, random baseline, and detailed RQ specifications.

Chapter outline (top-down). To facilitate a clear overview given the volume of experiments, this chapter follows a hierarchical, top-down approach: we begin with the core definitions and evaluation constructs used throughout, then describe the system components and a helicopter view of the end-to-end pipeline (Figure 7.1). We then provide a chapter-level experimental overview table that summarises the full cross-RQ design space and analysis choices (Table 7.1) and a complementary overview of where the per-RQ datasets and outputs live for reproducibility (Appendix C.1, Table C.4). Finally, we move to a more granular overview of the per-RQ designs and drill down into the detailed specifications for each experiment (Sections 7.9–7.14).

Preliminaries (setup & baselines)

P1: Generator configuration

Model selection + prompt ablation + temperature sensitivity

Data: GT Lit CLDs (3)

Outputs: LLM simulations (edge tables/runs) + best model/prompt

P2: Random baseline

Erdős-Rényi graphs $G(n, m)$ matched to each CLD

Data: GT Lit CLDs (3)

Outputs: random baseline CLD edge tables + baseline F1 distribution

P3: Judge sanity check

TruthfulQA verification (general-domain hallucination detection)

Data: TruthfulQA (1,634 Q&A)

Outputs: Accuracy / F1 sanity-check

Main Experiments (RQ1–RQ3)

RQ1a: LLM-as-a-Judge

Detect hallucinations in CLD edges

Data: LLM simulations + GT Lit + GT Synth corruptions

Outputs: AUC/F1 by prompt (blocked design)

RQ1b: LLM-as-a-Corrector

Repair judge-flagged edges and re-evaluate

Data: LLM simulations + GT Lit + GT Synth (from RQ1a)

Outputs: Δ F1, action breakdowns

RQ2: Context-insensitive detection (UQ)

Use generation-time UQ signals to predict hallucinations

Data: LLM simulations (deduplicated runs); logprobs/embeddings

Outputs: single-metric AUCs + ensemble results

RQ3a: Deep Research validation

Find direct causal evidence for edge claims

Data: Stratified edges from LLM simulations labelled vs GT Lit (TP/FP/FN)

Outputs: detection rates, enrichment analysis

Validation & Robustness Checks

V1: Search provider comparison

Citation-judge reliability on external expert CLD

Data: Uleman's CLD (150 edges)

Outputs: coverage/score by provider

V2: Human-LLM agreement

Inter-rater agreement on citation judging

Data: Human validation set (72 edges)

Outputs: κ , agreement %, r

V3: Physics sanity check

Judge behaviour on unambiguous causal edges

Data: Physics CLDs (31 edges)

Outputs: score distributions

V4: Retrieval success analysis

Separate infrastructure failures vs evidence failures

Data: Physics/Uleman's + GT Lit retrieval logs

Outputs: fetch success rates

V5: Deep Research human validation

Applicability calibration + threshold extrapolation

Data: Human DR sample (50 edges)

Outputs: calibrated enrichment bounds

Dataset legend: LLM simulations | GT Lit | GT Synth | TruthfulQA | Uleman's CLD | Physics CLDs | Human validation

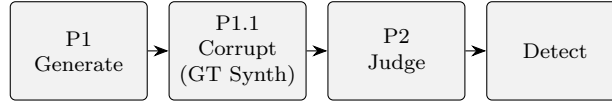
Figure 7.1: End-to-end experimental pipeline overview. The workflow is organised into three phases: **Preliminaries** (configuration selection and baselines), **Main experiments** (RQ1–RQ3), and **Validation** (RQ reliability/robustness checks). Gray boxes indicate datasets and where they enter the pipeline. **Direct dataset links:** GT Lit CLD files are listed in Appendix C.1 (Table C.1); validation/auxiliary datasets (TruthfulQA, Uleman's, Physics, human validation) are in Appendix C.1 (Table C.2); and the per-RQ run outputs (including LLM simulation edge tables generated in Phase 1 and used throughout RQ1–RQ3) are mapped in Appendix C.1 (Table C.4).

7.1 Experimental Design Framework

Per-RQ Experimental Design (RQ1a, RQ1b, RQ2, RQ3)

The design framework is instantiated in four main experiments (RQ1a, RQ1b, RQ2, RQ3). We summarise the **unit of analysis**, **treatments**, **datasets**, and **outputs** here to make the per-RQ design explicit (full specifications appear in Sections 7.9, 7.12, 7.13, and 7.14). The blocking structure and unit of inference for the main repeated-measures experiments are defined immediately below (Section 7.1).

RQ1a (Judge detection).



Goal: measure whether LLM judges detect hallucinated edges.

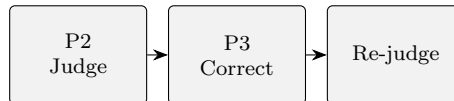
Design: a **2×2 factorial design** crossing **Ground Truth type** $\in \{\text{GT Lit}, \text{GT Synth}\}$ with **Judge type** $\in \{\text{Correctness}, \text{Citation}\}$, yielding four experimental cells. Within each cell we use a blocked repeated-measures structure with blocks = CLD×run ($N = 9$), and prompt variant as the within-block factor.

Unit of Analysis: Block-level (CLD×run) means ($N = 9$).

Datasets: **GT Lit** and **GT Synth** (see Section 6.1). Hallucination: FP/FN (GT Lit) or corrupted status (GT Synth).

Outputs: per-edge judge scores and binary hallucination labels; block-level (CLD×run) aggregates for inference.

RQ1b (Corrector efficacy).



Goal: test whether a corrector improves edge quality on judge-flagged edges.

Design: blocked repeated-measures with blocks = CLD×run ($N = 9$), within-block factor = corrector prompt.

Unit of Analysis: Block-level (CLD×run) means ($N = 9$).

Datasets: judge-flagged edges from RQ1a (see Section 6.1 for base datasets).

Outputs: pre/post correction edge tables and re-judged scores; block-level improvements $\Delta\text{Precision}/\Delta\text{Recall}/\Delta\text{F1}$ and $\Delta\text{judge score}$.

RQ2 (Context-insensitive detection via UQ).



Goal: evaluate whether generator-side uncertainty metrics predict hallucinations without external context.

Design: blocked aggregation with blocks = CLD×run ($N = 9$); inference uses block means.

Unit of Analysis: Block-level (CLD×run) means ($N = 9$).

Datasets: deduplicated **GT Lit** runs (see Section 6.1). Hallucination labels defined as FP/FN w.r.t. GT Lit.

Outputs: per-edge UQ metrics (perplexity, min probability, max window entropy; plus cosine similarity for citation runs), per-file and per-block AUCs, and (optionally) ensemble classifier performance under group-aware splits.

RQ3 (Deep Research literature validation).



Goal: test whether the Deep Research system can distinguish valid from hallucinated causal claims by finding **direct causal evidence** in the scientific literature.

Design: exploratory pilot study (single-run, no replication) due to computational cost; analysis is descriptive with confirmatory tests limited to class-wise differences in detection rates (see Section 7.14).

Unit of Analysis: Edge-level (descriptive statistics and proportion tests).

Datasets: a stratified sample of edges from RQ1a GT Lit experiments, labelled by GT Lit validation class (TP/FP/FN).

Outputs: per-edge Deep Research verdicts (**direct_causal_found**, confidence, evidence list/URLs) and class-wise detection rates (TP/FP/FN) with associated uncertainty and discrimination tests.

Design Rationale and Unit of Inference

- **GT Lit experiments:** CLD generation varies (seeds 10, 20, 30) → tests judge generalisation across CLD instances
- **GT Synth experiments:** Corruption varies (seeds 1, 2, 3), base CLD fixed → isolates corruption detection ability
- **Seed separation:** Methodological independence between paradigms (10/20/30 vs 1/2/3)

Confound Prevention: The seed structure explicitly controls for potential confounds. In GT Synth experiments, fixing the base CLD while varying only the corruption seed ensures that performance differences reflect the judge’s ability to detect different corruption patterns—not differences in underlying CLD structure. In GT Lit experiments, CLD structure variation is intentional: we deliberately test whether the judge generalises across different valid CLD realizations. The seed separation (10/20/30 for generation vs. 1/2/3 for corruption) maintains methodological independence.

Experiments use a **blocked (repeated-measures) design** with 9 independent blocks: 3 CLDs × 3 runs (seeds 10, 20, 30 for generation; 1, 2, 3 for corruption). This structure:

- **Controls for CLD-level variance:** Performance is confounded by domain difficulty (Depressive Symptoms vs. Social Norms vs. Older Persons ED); blocking isolates treatment effects.
- **Controls for run-to-run variance:** Different seeds produce different CLD instances; blocking treats each (CLD, run) pair as a subject.
- **Enables within-subject analysis:** Prompt variants or methods are compared within the same block, increasing statistical power.

Unit of inference: The block-level (CLD×run) summary statistic is the unit of analysis for meta-level statistical tests. Where multiple prompt variants exist within a block, prompt is treated as a within-block factor (paired comparisons); for pooled summaries we use the block mean. This yields $N = 9$ independent observations for hypothesis testing.

Pipeline Components (Summary)

The experimental pipeline consists of four core components (see Section 5 for full details):

- **Generator (GPT-4.1, T=0.7):** Generates CLDs by iteratively proposing nodes and edges with causal narratives. Captures UQ metrics (perplexity, min probability, max window entropy) during generation for RQ2.
- **Judge (GPT-4.1/GPT-5-mini, T=0.0):** Evaluates each edge via correctness-based reasoning or citation-based literature verification. Outputs scores: 0.0 (INCORRECT / NOT_SUPPORTED), 0.5 (PARTIALLY_CORRECT / PARTIALLY_SUPPORTED), 1.0 (CORRECT / SUPPORTED).
- **Corrector (GPT-4.1, T=0.7):** Attempts to repair judge-flagged hallucinations by revising causal narratives or changing edge types.
- **Deep Research (GPT-5, T=0.0):** Multi-agent literature search pipeline that finds direct causal evidence for edge claims (RQ3 only).

7.2 Metrics

Definitions and Output Metrics

This section consolidates all metrics used across experiments.

Classification Metrics (Judge Evaluation; RQ1a).

- **AUC-ROC:** Area under the ROC curve (judge score vs. hallucination label). Baseline: AUC = 0.5 (random classification).
- **Precision:**

$$\text{Precision} = \frac{TP}{TP + FP} \quad (7.1)$$

- **Recall:**

$$\text{Recall} = \frac{TP}{TP + FN} \quad (7.2)$$

- **F1:**

$$F1 = \frac{2 \cdot TP}{2 \cdot TP + FP + FN} \quad (7.3)$$

- **Correlation (r):** Point-biserial correlation between judge score s and binary hallucination label $y \in \{0, 1\}$:

$$r_{pb} = \frac{\bar{s}_1 - \bar{s}_0}{s_s} \sqrt{pq} \quad (7.4)$$

where $\bar{s}_1$ and $\bar{s}_0$ are mean scores for the $y=1$ and $y=0$ groups, s_s is the sample standard deviation of scores, and $p = \Pr(y=1)$, $q = \Pr(y=0)$.

- **Classification threshold:** Judge score ≤ 0.5 predicts hallucination.
- **Confusion matrix terms:** $TP = \#\{y=1, \hat{y}=1\}$, $FP = \#\{y=0, \hat{y}=1\}$, $FN = \#\{y=1, \hat{y}=0\}$, $TN = \#\{y=0, \hat{y}=0\}$.

Threshold justification. Scores are ordinal and discrete by design (0.0/0.5/1.0): **0.0** indicates an explicit error, **0.5** indicates a detected partial flaw/uncertainty, and **1.0** indicates full correctness/support. Under our operational definition of hallucination as *edge-level incorrectness*, any detected error (i.e., INCORRECT or

PARTIALLY_CORRECT / NOT_SUPPORTED or PARTIALLY_SUPPORTED) implies the edge should be flagged as hallucinated, which yields the natural cutpoint ≤ 0.5 . We additionally report ROC curves from a discrete threshold sweep over the only meaningful operating points induced by these three score levels (Appendix I; Figures I.3–I.4, I.1–I.2); for judges that exceed chance discrimination ($\text{AUC} > 0.5$), the ≤ 0.5 operating point corresponds to the intended “any detected error” decision rule used throughout the main thresholded Precision/Recall/F1 analyses.

Judge Output Scores.

- **Correctness Judge (per-edge):** 0.0 (INCORRECT), 0.5 (PARTIALLY_CORRECT), 1.0 (CORRECT)
- **Citation Judge (per-citation):** 0.0 (NOT_SUPPORTED), 0.5 (PARTIALLY_SUPPORTED), 1.0 (SUPPORTED)
- **Citation Judge (per-edge):** The mean of per-edge aggregate scores. For a single edge with C retrieved citations, the edge-level score is:

$$\text{score}_{\text{edge}} = \frac{1}{C} \sum_{c=1}^C \text{verdict}_c \quad (7.5)$$

where $\text{verdict}_c \in \{0.0, 0.5, 1.0\}$ corresponds to CONTRADICTED/UNADDRESSED, PARTIALLY SUPPORTED, and SUPPORTED respectively.

Correction Metrics (RQ1b).

- **ΔF1 , $\Delta\text{Precision}$, ΔRecall :** Change from pre- to post-correction w.r.t. ground truth
- **$\Delta\text{Judge Score}$:** $\text{mean}(\text{corrected edges}) - \text{mean}(\text{original edges})$

UQ Metrics (RQ2). Context-insensitive metrics captured during generation:

- **Perplexity:** Average token-level uncertainty (capped NLL)
- **Min Probability:** Confidence floor—minimum token probability
- **Max Window Entropy:** Highest-uncertainty region (5-token sliding window)
- **Cosine Similarity:** Semantic alignment between narrative and citations (citation experiments only)

Perplexity computation. Two-stage capping improves robustness and avoids numerical instability: (1) individual token NLL values are capped at 20.0 before averaging,

$$\text{nll}_i = \min(-\log P(x_i), 20.0),$$

preventing single outlier tokens ($P < 2 \times 10^{-9}$) from dominating; (2) the average NLL is capped at 10.0, and perplexity is computed as:

$$\text{PPL} = \exp\left(\min\left(\frac{1}{n} \sum_{i=1}^n \text{nll}_i, 10.0\right)\right),$$

bounding perplexity at $e^{10} \approx 22,026$ to prevent extreme values from overwhelming comparisons.

Min Probability. The minimum token probability across the entire generated sequence:

$$P_{\min} = \min_{i \in \{1, \dots, n\}} P(x_i),$$

representing the confidence floor of the single least-confident token prediction. Low minimum probability indicates at least one token was generated with high uncertainty, which we treat as a candidate indicator of localized uncertainty. This metric is complementary to perplexity: while perplexity captures average uncertainty, min probability captures worst-case uncertainty.

Max Window Entropy. Two-step aggregation detects localized uncertainty spikes: (1) at each token position, compute entropy over top- $k=5$ alternatives returned by the API,

$$H_k(t) = - \sum_{j=1}^k \tilde{p}_j(t) \log \tilde{p}_j(t), \quad \tilde{p}_j(t) = \text{softmax}(\text{logprob}_j(t));$$

(2) apply a sliding window of size $w=5$ to compute rolling averages (via `np.convolve`), then return the maximum:

$$H_{\text{max-win}} = \max_i \frac{1}{w} \sum_{t=i}^{i+w-1} H_k(t).$$

This identifies the highest-uncertainty region in the generated text.

Cosine Similarity. Embeddings are computed locally using `all-mpnet-base-v2` [98] (768-dim, 384-token limit). Citation texts are chunked with an adaptive window: `chunk.size` = $\max(256, \min(4000, |\text{motiv.}|))$ characters, 20% overlap, de-duplicated. This adaptive sizing ensures chunks remain within the model’s 384-token context (motivations average 56 tokens, max 137; implied chunks 57–176 tokens). For each edge, $\text{sim} = \max_c \cos(\mathbf{e}_{\text{narr}}, \mathbf{e}_c)$ over all citation chunks. Available only for citation experiments.

Model Selection. We selected `all-mpnet-base-v2` based on benchmarking against Qwen3-Embedding-8B (Appendix G; Tables G.1 and G.2): $22\times$ faster (3.0 vs. 67.3 ms/doc) in our setup. Because cosine similarity scoring is throughput-bound in our pipeline, we prioritise latency; published MTEB results are included in Appendix G for context only (not as a direct cross-model comparison).

Deep Research Metrics (RQ3).

- **Detection Rate:** Fraction of edges with `direct_causal_found=True`. For an edge class $A \in \{\text{TP}, \text{FP}, \text{FN}\}$, we define

$$\text{DR}_A = \frac{1}{|E_A|} \sum_{e \in E_A} \mathbb{1}\{\text{direct_causal_found}(e) = \text{True}\},$$

where E_A is the set of edges in class A .

- **Pairwise DR Gaps (pp):** $\text{DR}_A - \text{DR}_B$ for $A, B \in \{\text{TP}, \text{FP}, \text{FN}\}$ (reported for TP–FP, TP–FN, FP–FN). As a practical heuristic, $> 20\text{pp}$ indicates meaningful discrimination.
- **Confidence Score:** Deep Research self-assessed confidence (1–10; reported as mean by class and CLD).
- **Uncertainty for proportions:** Wilson score 95% confidence intervals for detection rates (see Appendix D).
- **Discrimination tests:** Fisher’s exact test for differences in detection rates (with Bonferroni-adjusted p-values for the TP–FP / TP–FN / FP–FN family; see Appendix D).
- **Effect sizes (RQ3):** Cohen’s h for detection-rate differences; odds ratio (OR) with 95% CI from the 2×2 contingency table.

Human Validation Metrics (RQ1a Validation, RQ3).

- **Cohen’s κ (kappa):** Chance-corrected inter-rater agreement:

$$\kappa = \frac{p_o - p_e}{1 - p_e} \quad (7.6)$$

where p_o is observed agreement and p_e is expected agreement by chance. Interpretation (Landis & Koch [116]): <0.20 slight, 0.21–0.40 fair, 0.41–0.60 moderate, 0.61–0.80 substantial, >0.80 almost perfect.

- **Agreement Rate:** Exact match percentage between raters (before chance correction).
- **Pearson r :** Correlation between continuous judge scores and human ratings; used to assess score-level (not categorical) alignment.
- **ICC (Intraclass Correlation Coefficient):** For continuous applicability scores (RQ3 inter-rater check), ICC(2,1) (two-way random effects, absolute agreement, single measures):

$$\text{ICC} = \frac{\sigma_{\text{between}}^2}{\sigma_{\text{between}}^2 + \sigma_{\text{within}}^2} \quad (7.7)$$

Interpretation [117]: <0.50 poor, 0.50–0.75 moderate, 0.75–0.90 good, >0.90 excellent.

RQ3 Human Validation Metrics (Applicability-Based Calibration).

- **Applicability Score:** Human-assigned continuous score $\in [0, 1]$ indicating how well retrieved evidence matches the exact edge claim. Rubric: 1.0 = exact match; 0.8–0.9 = different operationalisations; ≈ 0.7 = partial claim; ≈ 0.6 = proxy constructs; ≤ 0.5 = weak/non-causal evidence.
- **Coverage:** Fraction of edges with applicability $\geq t$ for threshold t :

$$\text{Coverage}(t) = \frac{|\{e : \text{applicability}(e) \geq t\}|}{|E|} \quad (7.8)$$

- **Correct-causal Rate:** Fraction of passing edges labelled **causal** by human validator:

$$\text{Correct-causal}(t) = \frac{|\{e : \text{applicability}(e) \geq t \wedge \text{verdict}(e) = \text{causal}\}|}{|\{e : \text{applicability}(e) \geq t\}|} \quad (7.9)$$

Aggregation Methods (Generator Evaluation).

- **Micro-average:** Pooled TP/FP/FN across all CLDs before computing Precision/Recall/F1. Gives equal weight to each edge.
- **Macro-average:** Compute Precision/Recall/F1 per CLD, then average. Gives equal weight to each CLD regardless of size.
- **Graph Density:** Fraction of possible edges present in the CLD:

$$\text{Density} = \frac{|E|}{|V| \times (|V| - 1)} \quad (7.10)$$

where $|E|$ is edge count and $|V|$ is node count. Used for random baseline contextualisation.

Accuracy (TruthfulQA Verification).

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (7.11)$$

Used to verify judge implementation on the TruthfulQA benchmark before CLD-specific experiments.

Specificity (TruthfulQA Verification). Specificity (true negative rate) measures how well the judge avoids false hallucination flags on truthful answers:

$$\text{Specificity} = \frac{TN}{TN + FP}. \quad (7.12)$$

Citation Retrieval Metrics (Validation; Citation Judge). Because citation-based judging depends on successful full-text fetching, we report retrieval and conditional approval metrics:

- **Retrieval success rate (fetch coverage):** fraction of edges with at least one successfully fetched citation text:

$$\text{RetrievalSuccess} = \frac{|\{e : C_e \geq 1\}|}{|E|} \quad (7.13)$$

where C_e is the number of usable fetched citations for edge e .

- **NO_CITATION rate:** fraction of edges where no citation text could be fetched ($C_e = 0$), i.e., $1 - \text{RetrievalSuccess}$.
- **Score difference (fetcher comparison):** difference in average citation-judge score between two fetchers/configurations (e.g., ContentScraper minus Jina AI):

$$\Delta\text{Score} = \bar{s}_A - \bar{s}_B. \quad (7.14)$$

- **Approval rate (overall):** fraction of edges with citation-judge score above the acceptance threshold (as defined in the corresponding experiment).
- **Approval rate (conditional on retrieval):** approval restricted to retrieved edges ($C_e \geq 1$), used to separate infrastructure failures from validation failures.

RQ2 Model Interpretation Metrics (UQ ensembles). To analyse which UQ features drive classifier performance, we use:

- **Recursive Feature Elimination (RFE) selection frequency:** fraction of cross-validation folds in which a feature is selected by RFE (reported as a stability diagnostic).
- **Permutation importance (AUC drop):** decrease in test AUC when a feature is randomly permuted (shuffled) while holding other features fixed; larger drops indicate greater reliance on that feature.

Cross-Domain Generalisation Gap (RQ2). When comparing in-distribution vs. leave-one-CLD-out evaluation, we report the AUC drop:

$$\Delta\text{AUC} = \text{AUC}_{\text{in}} - \text{AUC}_{\text{out}}. \quad (7.15)$$

Computational Efficiency Metrics (RQ3, Scaling, Parallelization). We report cost/latency metrics for selective deployment and parallel execution:

- **ΔF1 per dollar:** improvement in expected F1 divided by total compute cost (often scaled as ΔF1 per \$100 for readability).
- **Cost per +0.1 F1:** extrapolated dollars required for a +0.1 increase in expected F1 under a specified scaling model.
- **Wall-clock time:** elapsed runtime (seconds) for a fixed workload under a given worker count.
- **Speedup:** parallel speedup relative to single worker, $\text{Speedup} = T_1/T_W$.
- **Parallel efficiency:** speedup per worker, $\text{Efficiency} = (\text{Speedup}/W) \times 100\%$.

7.3 Statistical Analysis

Non-Parametric Testing Framework (Main Results)

For **prompt sensitivity** (RQ1a, RQ1b), we use non-parametric repeated-measures tests because the number of independent replicates is small ($n = 9$ blocks) and distributional assumptions are difficult to verify. In addition, several primary outcomes are **bounded** (e.g., $\text{AUC} \in [0, 1]$, $\text{Precision/Recall/F1} \in [0, 1]$) and judge scores are **discrete** (0.0/0.5/1.0), which can induce non-normality and ceiling/floor effects under small- n aggregation; rank-based tests reduce sensitivity to these issues:

- **Friedman test** (see Appendix D): Non-parametric equivalent of repeated-measures ANOVA; ranks observations within each block and tests whether rank sums differ across treatments.
- **Post-hoc Wilcoxon signed-rank tests** (see Appendix D): Pairwise comparisons between specific prompts within matched blocks.
- **Effect size**: Kendall’s W (<0.1 negligible, $0.1\text{--}0.3$ small, $0.3\text{--}0.5$ medium, ≥ 0.5 large).

This framework is robust to small sample sizes, non-normality, and outliers. The trade-off is that it primarily answers *whether prompts differ* (in a rank-based sense), but it does not yield the same kind of variance decomposition and sensitivity measures as parametric repeated-measures models.

Sensitivity Analysis (Appendix B; Methods §7.10). For sensitivity-style interpretation (e.g., “how much variability is explained by prompts”), we also report **Repeated-Measures ANOVA** (RM-ANOVA) with variance decomposition (e.g., η_p^2). The trade-off is the opposite of the non-parametric approach: RM-ANOVA is more interpretable for sensitivity, but it relies on stronger distributional assumptions that are hard to validate convincingly at $n = 9$. We therefore treat RM-ANOVA as a *secondary* sensitivity summary (reporting F , df , p , and η_p^2), and keep the non-parametric results as the primary inferential evidence for prompt differences.

RQ2 hypothesis tests (single-metric UQ detection). For each UQ metric, we compute a **block-level AUC-ROC** (blocks = CLD $\times$ run, $n = 9$) and test whether discrimination exceeds chance:

- **Primary test**: one-sample t -test on block-level AUCs vs. 0.5 (Appendix D). $H_0 : \mu_{\text{AUC}} = 0.5$, $H_1 : \mu_{\text{AUC}} \neq 0.5$.
- **Assumption check**: Shapiro–Wilk test on block-level AUCs (Appendix D); as a robustness check we also report a one-sample Wilcoxon signed-rank test on $(\text{AUC} - 0.5)$ over blocks when distributional assumptions are questionable.
- **Correlation aggregation**: Pearson r is computed per experimental file and aggregated using Fisher z -transformation across files (Appendix D).

RQ3 hypothesis tests (Deep Research discrimination). RQ3 compares **detection rates** (Found vs. Not Found for `direct_causal_found`) between edge classes:

- **Primary test**: Fisher’s exact test on 2×2 contingency tables for TP–FP, TP–FN, and FP–FN (Appendix D); p-values are Bonferroni-adjusted within this family.
- **Uncertainty**: Wilson score 95% confidence intervals for detection-rate proportions.
- **Effect sizes**: Cohen’s h for proportion differences, and odds ratio (OR) with 95% CI from the contingency table.

Multiple Comparison Corrections

Bonferroni corrections control family-wise error rate, grouped by research question (see Appendix D):

- **RQ1a:** 4 omnibus Friedman tests (one per judge×GT setting) $\rightarrow \alpha_{\text{adj}} = 0.0125$; *post-hoc* prompt comparisons within each setting: **Correctness Judge** uses $k = 3$ prompts $\rightarrow 3$ pairwise comparisons, $\alpha_{\text{adj}} = 0.0167$; **Citation Judge** uses $k = 4$ prompts (including **GT Lit** and **GT Synth**) $\rightarrow 6$ comparisons, $\alpha_{\text{adj}} = 0.0083$
- **RQ1b:** 2 omnibus Friedman tests $\rightarrow \alpha_{\text{adj}} = 0.025$; post-hoc 3 pairs $\rightarrow \alpha_{\text{adj}} = 0.0167$ (see Appendix D)
- **RQ2:** 4 single-metric tests $\rightarrow \alpha_{\text{adj}} = 0.0125$
- **RQ3:** 3 Fisher’s exact tests $\rightarrow \alpha_{\text{adj}} = 0.017$ (see Appendix D)

Scope of correction families. Unless stated otherwise, we apply Bonferroni corrections *within each research question* (RQ) and within the corresponding family of statistical tests listed above. For RQ1a and RQ1b prompt-sensitivity analyses, corrections apply to post-hoc prompt comparisons within each experiment/metric (e.g., a given judge×GT setting and metric such as F1 or AUC). We do not apply a single global correction across all RQs because each RQ tests a distinct hypothesis and is reported as a separate confirmatory family.

Statistical Hypothesis and Assumption Testing

For transparency and reproducibility, the formal hypotheses, assumption diagnostics, and interval/correction procedures for all statistical tests used throughout the thesis are provided in Appendix D.

Effect Size Reporting

All significant findings include standardised effect sizes [118]:

- **Cohen’s f (ANOVA):** <0.10 negligible, $0.10\text{--}0.25$ small, $0.25\text{--}0.40$ medium, ≥ 0.40 large
- **Cohen’s d (paired differences):** <0.20 negligible, $0.20\text{--}0.50$ small, $0.50\text{--}0.80$ medium, ≥ 0.80 large
- **Cohen’s h (proportions):** Same thresholds as d
- **Pearson r :** <0.10 negligible, $0.10\text{--}0.30$ small, $0.30\text{--}0.50$ medium, ≥ 0.50 large

Uncertainty Reporting Rule

For repeated measurements (across CLDs, runs/seeds, or CV folds), we report **mean $\pm$ 95% CI** using the t -distribution with $df = N - 1$. This accounts for variance uncertainty in small samples (e.g., $N = 3$ runs or $N = 9$ blocks). For proportions (e.g., detection rates), we use Wilson score 95% intervals.

Interpretation: Readers should interpret small- N intervals ($N < 10$) as approximate rather than precise; confirmatory conclusions rely on block-based non-parametric tests (Friedman/Wilcoxon) rather than visual CI overlap. CV fold scores are not independent draws (folds share training data), so fold-based intervals should not be interpreted as strict confidence intervals.

Statistical Power. Given the small number of independent blocks ($N = 9$), our main repeated-measures analyses have limited sensitivity to small effects; null results should therefore not be over-interpreted as evidence of no effect. We emphasise standardised effect sizes alongside p -values (Table 8.26, Results).

Computational Efficiency Analysis

Time and cost scaling characterise: (1) CLD size vs. computational requirements, (2) correctness vs. citation cost trade-off, (3) model efficiency (GPT-4.1 vs. GPT-5-mini). Since each edge is judged independently, time complexity is $O(E)$ where E is edges; for dense CLDs, $O(N^2)$ w.r.t. nodes.

Cost Estimation (December 2024):

- GPT-4.1: \$2.00/1M input, \$8.00/1M output tokens
- GPT-5-mini: \$0.15/1M input, \$0.60/1M output (97% cheaper)

Full scaling results in [Appendix C.4.14](#).

7.4 Master Experimental Overview

Table 7.1 provides a complete overview of all experimental configurations, statistical methods, and analysis scripts across research questions.

Table 7.1: Master Experimental Configuration: Design, Statistical Methods, and Analysis References

RQ	Task	Dataset	GT	Varies	Fixed	Model Design	N	Unit	Metric	Test	Details	Out	Hypothesis
<i>Preliminary & Validation Studies</i>													
Pre	Generator selection	GT CLDs	Lit	Model (3)	3 CLDs, 3 runs	3 LLMs 3C×3r×3m	27	Run	F1/Prec./Rec.	Descr.	§7.6	Tab. 8.1	Best F1 + logprobs
Pre	TruthfulQA verification	TruthfulQA	TQA	Q&A pairs	sj=42	4.1 817q×2v	1634	Q&A	Acc./F1	Descr.	§7.8	Tab. 8.4	Acc > 80%
Val	Uleman’s provider study	Uleman’s CLD	Expert	Prov., fetcher	150 edges	4.1 4prov×2fetch×150e	1200	Edge	Score, cov.	Descr.	§7.11	Tab. 8.5	Infra. limits
Val	Human–LLM agreement	Human val. set	Human Rater		72 edges	4.1 72e×2raters	72	Edge	κ , agree.%	κ	§7.11.2	Tab. 8.6	$\kappa > 0.6$
Val	Physics sanity check	Physics CLDs	Phys.	4 CLDs	Laws fixed	4.1 4C×31e	31	Edge	Score	Descr.	§7.11.3	Tab. C.33	Sanity pass
<i>Main Experiments (RQ1–RQ3)</i>													
1a	Correctness judge (Lit)	LLM sims + GT CLDs	Lit	CLD s∈{10,20,30}	Tj=0; sj=42; cr=0%	4.1 3C×3r×3p	27	CLD×run	AUC,F1	Fried.+Wile.	§7.9.0.1	Fig. 8.4–8.5	H1: Low score⇒halluc.
	Correctness judge (Synth)	LLM sims + GT Synth corruptions	Synth	Corr s∈{1,2,3}	base CLD; cr=30%; Tj=0; sj=42	4.1 3C×3r×3p	27	CLD×run	AUC,F1	Fried.+Wile.	§7.9.0.2	Fig. 8.2–8.3	
	Citation judge (Lit)	LLM sims + GT CLDs	Lit	CLD s∈{10,20,30}	Tj=0; sj=42; cr=0%	4.1 3C×3r×4p	36	CLD×run	AUC,F1	Fried.+Wile.	§7.9.0.3	Fig. 8.8–8.9	H2: Low score⇒halluc.
	Citation judge (Synth)	LLM sims + GT Synth corruptions	Synth	Corr s∈{1,2,3}	base CLD; cr=30%; Tj=0; sj=42	5m 3C×3r×4p	36	CLD×run	AUC,F1	Fried.+Wile.	§7.9.0.4	Fig. 8.6–8.7	
1b	Correctness corrector (Synth)	Judge-flagged LLM sims + GT Synth	Synth	Prompt (3)	baseline judge: Tj=0; sj=42	4.1 3C×3r×3p	27	CLD×run	ΔF1	Fried.+Wile.	§7.12.0.1	Tab. 8.10	H3: Corr.↑F1
	Correctness corrector (Lit)	Judge-flagged LLM sims + GT CLDs	Lit	Prompt (3)	baseline judge: Tj=0; sj=42	4.1 3C×3r×3p	27	CLD×run	ΔF1	Fried.+Wile.	§7.12.0.2	Tab. 8.12	
Sens.	Prompt sensitivity analysis	LLM sims + GT CLDs	Lit	Prompt (k=3–4)	blocks=CLD×run	4.1 9blk×kp	9	CLD×run	η_p^2	RM-ANOVA	§7.10	Fig. B.1	Sensitivity
2	UQ (logprobs)	LLM sims (RQ2 dedup)	Lit	CLD s∈{10,20,30}	Tj=0; sj=42	4.1 3C×3r×(3–4)p	63	CLD×run	AUC vs. 0.5	t (blocks)	§7.13.0.1	Fig. 8.10; Tab. 8.14	H4: ↑PPL/↓Pmin/↑Hwin⇒halluc.
	CosSim (embeddings)	LLM sims (RQ2 dedup)	Lit	CLD s∈{10,20,30}	mpnet	3C×3r×4p	36	CLD×run	AUC vs. 0.5	t (blocks)	§7.13.0.2	Fig. 8.10; Tab. 8.14	H5: Low sim.⇒halluc.
3	Deep Research	DR outputs	Lit	TP/FP/FN strata 1 iter, 5 src		GPT-5 3 CLDs × TP/FP/FN	285	Edge	Det. rate	Fisher	§7.14.0.1	Tab. 8.17; Fig. 8.12	H6: Det _{TP} >Det _{FP}
	Human validation	Val. sample	DR+	t ∈{0.5–0.9}	Stratified	Human strat. (TP/FP/FN×CLD)	50	Edge	Applic., valid.	Wilson	§7.14.0.5	Fig. 8.13; Tab. 8.23	H7: Thresh⇒valid.
	Smart trigger	DR outputs	Lit	Judge score	mech. ≤ 0.5	GPT-5 triggered DR subset	142	Edge	ΔF1/\$	Extrap.	§7.14.0.7	Fig. 8.14	H8: Sel. DR↑eff.

Legend. **RQ:** Pre=Preliminary, Val=Validation, Sens.=Sensitivity. **Models:** 4.1=GPT-4.1, 5m=GPT-5-mini, GPT-5=GPT-5 (heavy), mpnet=all-mpnet-base-v2, Human=human rater. **GT:** Lit=literature GT, Synth=synthetic corruption (30%), TQA=TruthfulQA, Expert=Uleman’s GMB, Phys.=physics laws, DR+=DR output with human applicability. **Datasets:** LLM sims=LLM simulation edge tables (Phase 1 outputs) used throughout RQ1–RQ3; RQ2 dedup=deduplicated RQ1a GT Lit LLM sims; DR outputs=Deep Research edges; Val. sample=stratified validation sample. **Varies/Fixed:** sj=judge seed (fixed at 42), Corr seed=run number, Tj=judge temperature, cr=corruption rate. **Unit:** CLD×run = block (3 CLDs × 3 runs = 9 independent replicates); Edge = individual causal relationship. **Metrics:** AUC=AUC-ROC, PPL=Perplexity, Pmin=Min Prob, Hwin=Max Window Entropy, κ =Cohen’s kappa, η_p^2 =partial eta-squared, Det.=Detection Rate. **Tests:** Friedman=omnibus; Wilcoxon=post-hoc (Bonferroni); RM-ANOVA=repeated-measures; t-test=one-sample on block means; Fisher’s=exact test; Wilson CI=score interval (see Appendix D). **Multiple-comparison families:** Bonferroni corrections are defined per research question (RQ); for prompt-sensitivity analyses (RQ1a/RQ1b), post-hoc prompt comparisons are corrected within each experiment/metric family (e.g., per judge×GT setting and metric). **Design:** C=CLDs, r=runs, p=prompts, m=models, q=questions, v=variants, e=edges, blk=blocks, prov=providers, fetch=fetchers; strat.=stratified. **Task:** Prov.=Provider. **Symbols:** ↑/↓=increase/decrease; ⇒=implies. **Common:** Generator T=0.7, Judge T=0.0, Corruptor T=0.7. halluc.=hallucination (FPvFN w.r.t. GT). **Details:** section with experiment design/procedure (see Appendix C for scripts/data). **Out:** key Result figure/table links. **Seed structure:** GT Lit uses CLD seeds 10/20/30 (varies generation); GT Synth uses corruption seeds 1/2/3 with base CLD fixed (isolates corruption detection). **Data access:** All datasets, run artefacts, and analysis code are in the project repository (<https://github.com/causalix/causalix.ai>); see also final_runs/DATA_README.md.

7.5 Experimental Details

Structure of Experimental Details

The remainder of Methods expands each row of Table 7.1 into a reproducible specification. To make experiments easy to scan and compare, each experiment section follows a consistent structure:

- **RQ & Goal / Hypotheses:** What the experiment is testing and the expected direction of effects.
- **Source Data:** Which datasets/edge tables are used and how ground truth labels are defined (GT Lit vs. GT Synth, TP/FP/FN).
- **Simulation Parameters:** What varies vs. what is fixed (models, prompts, seeds, corruption rate, judge temperature), aligned with the **Varies/Fixed** columns in Table 7.1.
- **Simulation Procedure:** The operational steps (generation → judging → correction / Deep Research) and any filtering/deduplication rules.
- **Outputs:** The artefacts produced (Excel/JSON tables, scores, plots) and where they enter downstream analyses.
- **Metrics / Statistical Analysis:** The reported metrics and inference procedure (unit of analysis, tests, and multiple-comparison family).
- **Reproducibility:** The exact scripts/entry points and appendix pointers needed to rerun the experiment end-to-end.

When an RQ contains multiple experimental cells (e.g., RQ1a’s judge×GT combinations), we list the shared setup first, then provide a short **Experiment Details** breakdown for each cell.

7.6 Preliminary: Generator Selection

Overview.

Before running the main experiments, we conducted preliminary studies to select the generator model, prompt, and temperature. These configuration choices are fixed across all subsequent experiments.

7.6.0.1 Generator Model Selection

RQ & Goal.

Select a single frontier LLM for all CLD generation experiments, prioritizing edge recovery performance and availability of token-level log probabilities for downstream uncertainty quantification.

Source Data.

Three validation CLDs (Depressive Symptoms, Social Norms, Older Persons ED) with literature-derived ground truth. Node sets are fixed to ground truth variables. Ground-truth CLD files in Appendix C.1 (Table C.1).

Simulation Parameters.

- **Models compared:** GPT-4.1 (OpenAI), Sonar-Pro (Perplexity), Claude Sonnet 4 (Anthropic)
- **Prompt:** Nitai_C (fixed across models)
- **Temperature:** T=0.7
- **Corruption/judging:** Disabled (pure edge recovery comparison)

Simulation Procedure.

For each model-CLD combination, generate a complete **A11 Edges** table using the same node set as the literature ground truth. Models are compared purely on edge recovery against GT Lit. Generation procedure in Appendix H (Algorithm H.1).

Simulation Outputs.

Per-model **A11 Edges** tables with relationship types and causal narratives. Result directory and scripts in Appendix C.4.10 (Table C.28).

Unit of Analysis.

Run-level: each (model, CLD, seed) combination constitutes one observation ($N = 27$ total: 3 models $\times$ 3 CLDs $\times$ 3 runs).

Metrics.

Confusion-matrix counts (TP/FP/FN) computed from **A11 Edges** sheets, yielding precision, recall, and F1. Aggregate micro-averaged F1 computed by pooling TP/FP/FN across all three CLDs.

Selection Rule.

Select the generator with highest aggregate (micro-averaged) F1, prioritizing balanced precision/recall. Secondary constraint: token-level logprobs availability for RQ2; among compared models, this is available for GPT-4.1.

Reproducibility.

Full configuration and script locations in Appendix C.4.10 (Table C.28).

Results.

Table 8.1 (Chapter 8).

7.6.0.2 Generator Prompt Selection (Nitai_C)

RQ & Goal.

Select a fixed generator prompt variant that maximises edge recovery performance.

Source Data.

Three validation CLDs (Depressive Symptoms, Social Norms, Older Persons ED) with literature-derived ground truth (edge ablation), plus the node-generation ablation dataset described in Appendix F. Ground-truth CLD files in Appendix C.1 (Table C.1).

Simulation Procedure.

Two ablation studies were conducted:

- **Edge Generation Ablation:** Nine prompt variants evaluated across three CLDs, testing CoT, task decomposition, RAG/citation requirements, and relationship taxonomy complexity.
- **Node Generation Ablation:** Seven prompt variants evaluated in a $7 \times 3 \times 5$ factorial design (105 runs total).

Generation procedure in Appendix H (Algorithm H.1).

Simulation Outputs.

Per-variant F1 scores for edge and node generation tasks. Edge ablation appendix in Appendix E.1; node ablation appendix in Appendix F.

Unit of Analysis.

Prompt variant-level: Edge ablation uses 9 prompts $\times$ 3 CLDs = 27 observations; node ablation uses 7 prompts $\times$ 3 CLDs $\times$ 5 runs = 105 observations.

Key Findings.

- **Edge generation:** Nitai_C (dual-level CoT, three relationship types) achieved highest F1 (0.460), outperforming baseline (0.371) by 24%. CoT improved progressively (Nitai_A→B→C); mandatory citations reduced performance by 25%; overly restrictive grounding caused complete failure (Cillian_C: F1=0.00).
- **Node generation:** ANOVA revealed no significant differences ($F = 0.81$, $p = 0.57$; see Appendix D), with F1 ranging 0.617–0.677. Node generation is robust to prompt engineering choices.

Reproducibility.

Exact Nitai_C prompt in Appendix E.2 (Listing E.1). Edge ablation: Appendix E.1. Node ablation: Appendix F.

Results.

Edge ablation: Appendix E.1. Node ablation: Appendix F.

7.6.0.3 Generator Temperature Selection

RQ & Goal.

Validate generator temperature $T=0.7$ through sensitivity analysis measuring edge recovery performance.

Source Data.

Three validation CLDs: Social Norms (10 nodes, 12 edges), Depressive Symptoms (14 nodes, 34 edges), Older Persons ED (34 nodes, 66 edges). Ground-truth CLD files in Appendix C.1 (Table C.1).

Simulation Parameters.

- **Generator:** GPT-4.1 with Nitai_C prompt
- **Ground Truth:** Literature-derived CLDs (GT Lit)
- **Temperatures:** $T \in \{0.0, 0.3, 0.5, 0.7, 1.0\}$ (5 levels)
- **Runs:** 3 per temperature per CLD (seeds 10, 20, 30), totaling 45 experiments

Simulation Procedure.

For each temperature–CLD–seed combination, generate edges and compute F1 against GT Lit. Generation procedure in Appendix H (Algorithm H.1).

Simulation Outputs.

Per-run edge F1 scores (45 total). Temperature sensitivity configuration in Appendix Table C.27.

Unit of Analysis.

Run-level: each (temperature, CLD, seed) combination constitutes one observation ($N = 45$ total: 5 temperatures $\times$ 3 CLDs $\times$ 3 runs).

Metrics.

Edge F1 per run; aggregated by temperature level.

Statistical Analysis.

One-way ANOVA tested whether temperature significantly affects edge F1 (see Appendix D). Results in Table 8.3.

Selection Rule.

Temperature $T=0.7$ retained as it shows no significant performance difference from other values.

Results.

Table 8.3 (Chapter 8).

Reproducibility.

Full configuration in Appendix Table C.27.

7.7 Preliminaries: Generator Performance

RQ & Goal:

Contextualize generator performance via a structural random baseline—the expected edge-recovery for random graphs with the same (n, m) as each CLD.

Source Data:

Three validation CLDs with known node and edge counts (Appendix C.1, Table C.1).

Parameters:

Random baseline: 1000 Erdős–Rényi directed graphs $G(n, m)$ per CLD; LLM generator: 3 runs per CLD (seeds 10, 20, 30). Full configuration in Appendix C.4.11 (Table C.29).

Unit of Analysis:

Graph-level for random baseline ($N = 1000$ per CLD); run-level for LLM ($N = 3$ per CLD).

Simulation Procedure:

For each random graph, compute edge precision/recall/F1 vs. the ground truth; for the LLM runs, compute edge F1 likewise (Appendix C.4.11, Table C.29).

Outputs:

Random-baseline per-graph F1 distributions (1000 samples per CLD) and per-run LLM F1 values; directory mapping in Appendix C.1 (Table C.4).

Metrics/CI:

Precision, recall, F1; 95% CI uses z for random ($n=1000$) and t for LLM ($n=3$).

Results:

Table 8.2 and Figure 8.1 (Chapter 8).

Reproducibility:

Full configuration in Appendix C.4.11 (Table C.29).

7.8 Preliminary: TruthfulQA Verification

RQ & Goal:

Sanity-check the correctness-based judge on a general-domain hallucination benchmark before CLD-specific evaluation; strong TruthfulQA performance with weak CLD performance would suggest CLD hallucinations require domain grounding or citation validation beyond correctness judging.

Hypothesis:

H_0 : Accuracy ≤ 0.80 ; H_1 : Accuracy > 0.80 .

Source Data:

TruthfulQA [119] (817 questions). We construct a balanced set of 1,634 QA pairs: 817 truthful answers and 817 imitative falsehoods (Appendix C.1, Table C.2).

Parameters:

GPT-4.1, temperature 0.0, seed 42, baseline correctness prompt (same structure as RQ1a), max tokens 250. Full configuration in Appendix C.4.2 (Table C.11).

Unit of Analysis:

Q&A pair-level ($N = 1634$ total: 817 truthful + 817 hallucinated).

Simulation Procedure:

The judge labels each answer as CORRECT vs. INCORRECT based solely on factual accuracy (Algorithm H.2).

Outputs:

Per-answer verdicts with reasoning (artefact schema: Appendix C.5, Table C.36).

Metrics:

Accuracy; Precision/Recall/F1 treating Hallucinated as the positive class; Specificity for truthful answers.

Reproducibility:

Data in `final_runs/RQ1_verification_Truth_QA/`; execution/configuration in Appendix C.4.2 (Table C.11).

Results:

Table 8.4 (Chapter 8).

7.9 RQ1a: LLM-as-a-Judge Hallucination Detection

RQ & Goal:

Test whether LLM judges can detect hallucinated CLD edges: Correctness Judge (Section 5.4.2) and Citation Judge (Section 5.4.3) under two ground-truth regimes (GT Lit and GT Synth), yielding four experiment cells.

Hypotheses:

H1 (Correctness): lower correctness score implies hallucination (FP/FN w.r.t. GT).

H2 (Citation): lower citation score implies hallucination (FP/FN w.r.t. GT).

Source Data:

Three validation CLDs (Depressive Symptoms, Social Norms, Older Persons ED) with literature-derived ground truth; node sets fixed to GT variables (edge-only evaluation; node generation ablation in Appendix F). Ground-truth files and output directories: Appendix C.1 (Tables C.1, C.4).

Parameters:

Correctness judge: 3 prompts (Baseline, CoT, Mechanistic) $\times$ 3 CLDs $\times$ 3 runs = 27.

Citation judge: 4 prompts (+ Mechanistic-Lit) $\times$ 3 CLDs $\times$ 3 runs = 36.

GT Lit seeds 10/20/30 (generation variation); GT Synth seeds 1/2/3 (corruption variation) with 30% corruption.

Citation retrieval in ContentScraper-only mode (Jina AI Reader disabled; Appendix H.9). Full config in Table 7.1.

Simulation Procedure:

For each CLD–seed–prompt: generate edges (GPT-4.1 with Nitai_C); for GT Synth apply Corruptor; run the judge to score each edge; compare scores to GT labels (Algorithms H.1, H.2, H.3, and for GT Synth Algorithm H.4).

Outputs:

Per-edge scores (0.0/0.5/1.0) with verdicts and reasoning; schemas in Appendix C.5 (Table C.36).

Metrics:

AUC-ROC (baseline 0.5), point-biserial correlation r , and thresholded classification (score ≤ 0.5 = hallucination): Precision/Recall/F1.

Unit of Analysis:

Block-level (CLD $\times$ run, $N = 9$); edge-level ($N \approx 10,000$ –31,000 depending on experiment cell).

Stats:

Prompt effects under blocked repeated-measures (block = CLD $\times$ run, $n=9$): Friedman omnibus + paired Wilcoxon with Bonferroni correction (Section 7.3; Appendix D); RM-ANOVA sensitivity decomposition (η_p^2) in Appendix B.

Reproducibility:

Data directories: `final_runs/RQ1a_ground_truth_*` and `final_runs/RQ1a_corruption_*`; see Appendix C.1 (Table C.4) and Appendix C.2 (Tables C.5, C.6).

Results:

GT Lit Correctness: Figures 8.4, 8.5. GT Synth Correctness: Figures 8.2, 8.3. GT Lit Citation: Figures 8.8, 8.9. GT Synth Citation: Figures 8.6, 8.7 (Chapter 8).

Experiment Details

7.9.0.1 RQ1a: Correctness Judge (Lit)

RQ & Goal:

Evaluate Correctness Judge on GT Lit edges (literature-derived ground truth). See shared RQ1a setup (Section 7.9).

Source Data:

GT Lit edges from 3 CLDs; ground truth = FP/FN w.r.t. expert CLDs. GT files: Appendix Table C.1.

Simulation Parameters:

Appendix Table C.13. Fixed: GPT-4.1, T=0.0. Varies: 3 prompts $\times$ 3 CLDs $\times$ 3 runs = 27.

Simulation Procedure:

Algorithm H.1 (generation) $\rightarrow$ Algorithm H.2 (judging).

Outputs:

Directory: `final_runs/RQ1a_gt_lit_correctness/`. Schemas: Appendix Table C.36.

Unit of Analysis:

CLD $\times$ run $\times$ prompt ($N = 27$: 3 prompts $\times$ 3 CLDs $\times$ 3 runs).

Metrics / Stats:

See shared RQ1a setup; Friedman + Wilcoxon.

Reproducibility:

Scripts: Appendix Tables C.5, C.6.

Results:

Figures 8.4, 8.5 (Chapter 8).

7.9.0.2 RQ1a: Correctness Judge (Synth)

RQ & Goal:

Evaluate Correctness Judge on GT Synth edges (30% corruption). See shared RQ1a setup (Section 7.9).

Source Data:

GT Synth edges from 3 CLDs; ground truth = corruption status (corrupted = hallucination).

Simulation Parameters:

Appendix Table C.14. Fixed: GPT-4.1, T=0.0, 30% corruption. Varies: 3 prompts $\times$ 3 CLDs $\times$ 3 runs = 27.

Simulation Procedure:

Algorithm H.1 (generation) $\rightarrow$ Algorithm H.4 (corruption) $\rightarrow$ Algorithm H.2 (judging).

Outputs:

Directory: `final_runs/RQ1a_gt_synth_correctness/`. Schemas: Appendix Table C.36.

Unit of Analysis:

CLD $\times$ run $\times$ prompt ($N = 27$: 3 prompts $\times$ 3 CLDs $\times$ 3 runs).

Metrics / Stats:

See shared RQ1a setup; Friedman + Wilcoxon.

Reproducibility:

Scripts: Appendix Tables C.5, C.6.

Results:

Figures 8.2, 8.3 (Chapter 8).

7.9.0.3 RQ1a: Citation Judge (Lit)

RQ & Goal:

Evaluate Citation Judge on GT Lit edges (literature retrieval). See shared RQ1a setup (Section 7.9).

Source Data:

GT Lit edges from 3 CLDs; ground truth = FP/FN w.r.t. expert CLDs. GT files: Appendix Table C.1.

Simulation Parameters:

Appendix Table C.15. Fixed: GPT-4.1, T=0.0, ContentScraper. Varies: 4 prompts $\times$ 3 CLDs $\times$ 3 runs = 36.

Simulation Procedure:

Algorithm H.1 (generation) $\rightarrow$ Algorithm H.9 (retrieval) $\rightarrow$ Algorithm H.3 (judging).

Outputs:

Directory: `final_runs/RQ1a_gt_lit_citation/`. Schemas: Appendix Table C.36.

Unit of Analysis:

CLD $\times$ run $\times$ prompt ($N = 36$: 4 prompts $\times$ 3 CLDs $\times$ 3 runs).

Metrics / Stats:

See shared RQ1a setup; Friedman + Wilcoxon.

Reproducibility:

Scripts: Appendix Tables C.5, C.6.

Results:

Figures 8.8, 8.9 (Chapter 8).

7.9.0.4 RQ1a: Citation Judge (Synth)

RQ & Goal:

Evaluate Citation Judge on GT Synth edges (30% corruption, GPT-5-mini). See shared RQ1a setup (Section 7.9).

Source Data:

GT Synth edges from 3 CLDs; ground truth = corruption status (corrupted = hallucination).

Simulation Parameters:

Appendix Table C.12. Fixed: GPT-5-mini, T=0.0, 30% corruption. Varies: 4 prompts $\times$ 3 CLDs $\times$ 3 runs = 36.

Simulation Procedure:

Algorithm H.1 (generation) $\rightarrow$ Algorithm H.4 (corruption) $\rightarrow$ Algorithm H.9 (retrieval) $\rightarrow$ Algorithm H.3 (judging).

Outputs:

Directory: `final_runs/RQ1a_gt_synth_citation/`. Schemas: Appendix Table C.36.

Unit of Analysis:

CLD $\times$ run $\times$ prompt ($N = 36$: 4 prompts $\times$ 3 CLDs $\times$ 3 runs).

Metrics / Stats:

See shared RQ1a setup; Friedman + Wilcoxon.

Reproducibility:

Scripts: Appendix Tables C.5, C.6.

Results:

Figures 8.6, 8.7 (Chapter 8).

7.10 Sensitivity: Prompt Sensitivity Analysis

RQ & Goal.

Quantify prompt engineering impact on judge performance using variance-based sensitivity analysis inspired by Sobol indices [120].

Source Data.

Prompt-level and block-level results from the four RQ1a judge experiments (summarised in Table 7.1). Sensitivity output directory in Appendix C.1 (Table C.4).

Simulation Parameters.

- **Design:** Blocked with CLD×run as subjects, prompt as within-subject factor
- **Prompt distance:** MPNet [121] embeddings (768-dim); cosine distance from baseline: $d(p_i) = 1 - \cos(\mathbf{e}_{\text{baseline}}, \mathbf{e}_i)$
- **Scope:** 4 experiments × 3–4 prompt variants

Simulation Procedure.

Repeated-measures ANOVA with CLD×run as subjects and prompt as within-subject factor. Full RM-ANOVA tables and diagnostics in Appendix B.

Simulation Outputs.

RM-ANOVA effect sizes (η_p^2), assumption diagnostics, and sensitivity figures for each judge×GT setting. Output artefacts in Appendix C.5 (Table C.36); sensitivity output directory in Appendix C.1 (Table C.4).

Metrics.

Following Sobol’s variance-based sensitivity framework, for a categorical factor $A \in \{1, \dots, K\}$ (here: prompt choice) the first-order index is:

$$S_A := \frac{\text{Var}(\mathbb{E}[Y \mid A])}{\text{Var}(Y)}. \quad (7.16)$$

For categorical A , this reduces to $S_A = \eta^2$, i.e., the population one-way ANOVA effect size (correlation ratio) for a categorical factor [122]. In the main experiments, we report η_p^2 (partial eta-squared) because our design is blocked by CLD×run (repeated measures), so the sensitivity measure is computed after controlling for these subject-level differences. Partial eta-squared: $\eta_p^2 = \frac{SS_{\text{prompt}}}{SS_{\text{prompt}} + SS_{\text{error}}}$. Measures proportion of within-subject variance in F1 explained by prompt choice. Interpretation (Cohen): <0.01 negligible, 0.01–0.06 small, 0.06–0.14 medium, ≥ 0.14 large.

Unit of Analysis.

Block-level (CLD×run, $N = 9$) for RM-ANOVA variance decomposition; prompt as within-subject factor.

Statistical Analysis.

RM-ANOVA assumption diagnostics: Shapiro–Wilk (residual normality), Mauchly (sphericity), Greenhouse–Geisser correction when violated, and Bonferroni correction (see Appendix D). Full tables in Appendix B.

Reproducibility.

Data: `final_runs/Sensitivity_analysis_simple/`. Scripts: `sensitivity_analysis_simple_rm_anova.py`. Dataset mapping in Appendix C.1 (Table C.4).

Results.

Appendix B (Figure B.1).

7.11 Validation: Uleman’s Provider Study

Overview.

Four validation studies assess judge reliability: (1) search provider comparison, (2) human-LLM agreement, (3) physics CLD sanity check, and (4) retrieval success analysis.

Source Data.

Validation datasets (Uleman’s CLD, physics CLDs, human validation set) and their output directories. Dataset locations in Appendix C.1 (Table C.2).

Simulation Procedure.

Apply the relevant judge pipeline(s) (correctness/citation) to validation edges; citation judging includes retrieval and fetching. Algorithms in Appendix H (Algorithms H.2, H.3, H.9).

Simulation Outputs.

Per-edge verdicts/scores (and retrieval diagnostics for citation judging) stored in the corresponding validation output directories. Output artefacts in Appendix C.5 (Table C.36); validation directories in Appendix C.1 (Table C.2).

Reproducibility.

Dataset specifications: Appendix C.1 (Tables C.2, C.3). Scripts and configurations: Appendix C.4.6 (Tables C.18, C.19).

7.11.1 Search Provider Comparison (Uleman’s CLD)

RQ & Goal.

Validate citation-based judge on an independently expert-validated dataset. Determine whether low GT Lit scores are due to search infrastructure limitations.

Hypothesis.

If low scores are infrastructure-driven, we expect significant variation across providers/fetchers. Consistently low scores would confirm a fundamental validation gap between expert reasoning and literature-based verification.

Source Data.

Uleman’s CLD: 150-edge causal loop diagram for Alzheimer’s disease, constructed through Group Model Building (GMB) with 17 domain experts [113]. Unlike primary CLDs (extracted from literature), represents direct expert consensus with bibliographic references.

Simulation Parameters.

- **Search Providers (4):** OpenAlex, Brave Search (with/without domain whitelist), Perplexity, PubMed
- **Citation Fetcher:** ContentScraper (BeautifulSoup-based, PDF-capable); Jina AI Reader disabled
- **Design:** 4 providers $\times$ 2 fetchers $\times$ 150 edges = 1,200 judgements
- **Judge:** Citation-based (GPT-4.1, baseline prompt)

Full configuration in Appendix C.4.6 (Table C.18).

Outputs.

Per-edge scores and provider comparison summaries; output artefacts in Appendix C.5 (Table C.36).

Metrics.

Average score (mean 0–1), coverage (successful fetch %), score difference (ContentScraper – Jina AI).

Unit of Analysis.

Edge-level ($N = 150$); crossed with 4 providers $\times$ 2 fetchers = 1,200 total judgements.

Reproducibility.

Data: `final_runs/validation_RQ1a_ulemans_external/`. Configuration: Appendix Table C.18.

Results.

Table 8.5 (Chapter 8).

7.11.2 Human–LLM Agreement

RQ & Goal.

Validate that LLM citation judges produce reliable verdicts by measuring inter-rater agreement with human experts.

Hypothesis.

Expect substantial agreement ($\kappa > 0.60$), performance comparable to human baseline, and consistency across CLDs/generators.

Source Data.

72 causal edges across 3 validation files (2 CLDs, 2 generators): 30 edges Depressive Symptoms/GPT-4.1, 30 edges Depressive Symptoms/Claude 4 Sonnet, 12 edges Social Norms/GPT-4.1.

Simulation Parameters.

Human validator (domain expert) and LLM judge (GPT-4.1, T=0, baseline prompt) each provide categorical verdicts: Not supported (0), Partially supported (0.5), Fully supported (1). Judge settings follow RQ1a Citation Judge configuration. Full configuration in Appendix C.4.6 (Table C.19).

Outputs.

Per-edge human and LLM verdicts with agreement statistics; output artefacts in Appendix C.5 (Table C.36).

Metrics.

- **Cohen’s κ :** Chance-corrected agreement. Interpretation (Landis & Koch): <0.20 slight, 0.21–0.40 fair, 0.41–0.60 moderate, 0.61–0.80 substantial, >0.80 almost perfect.
- **Agreement Rate:** Exact match percentage
- **Pearson r :** Score correlation

Baseline: Human inter-rater reliability from prior GMB studies ($\kappa \approx 0.60$, 80% agreement).

Unit of Analysis.

Edge-level ($N = 72$ total across 3 validation files: 30 + 30 + 12 edges).

Reproducibility.

Data: `final_runs/RQ1_human_validation_citation_judge/`. Configuration: Appendix Table C.19.

Results.

Tables 8.6, 8.7 (Chapter 8).

7.11.3 Physics Sanity Check

RQ & Goal.

Verify that judges correctly identify valid causal relationships when unambiguous (sanity check under curated inputs).

Source Data.

Four physics-based CLDs derived from fundamental laws (31 edges, 26 variables):

1. Thermostat Heating System: Newton’s Law of Cooling [105], First Law of Thermodynamics [106]
2. Predator-Prey Dynamics: Lotka-Volterra equations [107, 108]
3. RC Circuit: Ohm’s Law [109], Kirchhoff’s Laws [110]
4. Water Tank: Pascal’s Law [111], Torricelli’s Law [112]

Edge rationales generated by Claude 4.5 Opus, verified against source texts. Per-CLD breakdown: Appendix Table C.3.

Simulation Parameters.

Full configuration in Appendix C.4.15 (Table C.33). Judge settings follow RQ1a configuration (GPT-4.1, T=0.0).

Simulation Procedure.

Both Correctness Judge and Citation Judge applied to curated physics edges (Algorithms H.2, H.3).

Outputs.

Per-edge verdicts and scores; output artefacts in Appendix C.5 (Table C.36).

Unit of Analysis.

Edge-level ($N = 31$ edges across 4 physics CLDs).

Reproducibility.

Data: `final_runs/validation_RQ1a_physics_clds/`. Configuration: Appendix Table C.33.

Results.

Table 8.8 (Chapter 8).

7.11.4 Retrieval Success Analysis

RQ & Goal.

Investigate whether low citation scores are driven by retrieval failures (missing documents) or evidence characteristics (weak/hedged claims).

Source Data.

1. Expert-provided citations: Physics CLDs, Uleman’s CLD (150 edges)
2. Automated web search: Three main validation CLDs via generate-then-search pipeline

For Uleman’s and Physics CLDs, causal narratives generated by Claude 4.5 Opus.

Metrics.

Retrieval success rate = percentage of edges with successfully fetched content. Discrepancies between retrieval success and judge approval distinguish infrastructure failures from validation failures. Fetch success criteria: Appendix H.9.

Unit of Analysis.

Edge-level (diagnostic analysis comparing retrieval success vs. approval rates across datasets).

Reproducibility.

Analysis integrated with Uleman’s and Physics CLD validation studies; see Appendix C.4.6 for scripts and configurations.

Results.

Table 8.9 (Chapter 8).

7.12 RQ1b: LLM-as-a-Corrector Evaluation

RQ & Goal:

Evaluate whether the Corrector (Section 5.6) can repair judge-flagged hallucinations.

Hypothesis:

H3: correction improves edge quality for flagged edges ($\Delta F1 > 0$).

Source Data:

Judge-flagged edges from RQ1a (GT Synth and GT Lit); output directories in Appendix C.1 (Table C.4).

Parameters:

3 corrector prompts (Baseline, CoT, Mechanistic); design = 3 prompts $\times$ 3 CLDs $\times$ 3 runs = 27 runs per GT type; evaluation judge = baseline Correctness Judge; scope = correctness-based correction only (citation-corrector omitted due to retrieval cost). Full configuration in Table 7.1.

Simulation Procedure:

For each flagged edge, apply the Corrector (revise relationship type and/or narrative), re-judge the corrected edge, and compare pre/post metrics against ground truth (Algorithms H.5 and H.2).

Outputs:

Corrected edges plus re-judge scores; artefact schema in Appendix C.5 (Table C.36).

Metrics:

$\Delta F1/\Delta Precision/\Delta Recall$ (pre $\rightarrow$ post vs. GT) and $\Delta JudgeScore = \text{mean}(\text{corrected}) - \text{mean}(\text{original})$.

Unit of Analysis:

Block-level (CLD $\times$ run, $N = 9$) for statistical inference; edge-level for individual correction evaluation.

Stats:

Efficacy via one-sample Wilcoxon signed-rank on block-level $\Delta F1$ (blocks = CLD $\times$ run, $n=9$) testing median($\Delta F1$) = 0; prompt effects via Friedman omnibus + paired Wilcoxon with Bonferroni correction (Section 7.3; Appendix D); RM-ANOVA sensitivity in Appendix B.

Reproducibility:

Data in `final_runs/RQ1b_correction_experiment_*/`; scripts in Appendix C.2 (Table C.5); analysis script `analyse_corrector_ablation.py`.

Results:

GT Synth: Tables 8.10, 8.11. GT Lit: Tables 8.12, 8.13 (Chapter 8).

Experiment Details**7.12.0.1 RQ1b: Correctness Corrector (Synth)****RQ & Goal:**

Evaluate Corrector on GT Synth flagged edges. See shared RQ1b setup (Section 7.12).

Source Data:

Judge-flagged edges from RQ1a GT Synth experiments (corrupted CLDs).

Simulation Parameters:

Appendix Table C.16. Fixed: GPT-4.1, T=0.0, baseline judge. Varies: 3 prompts $\times$ 3 CLDs $\times$ 3 runs = 27.

Simulation Procedure:

Algorithm H.5 (correction) $\rightarrow$ Algorithm H.2 (re-judging).

Outputs:

Directory: `final_runs/RQ1b_corrector_gt_synth_correctness/`. Schemas: Appendix Table C.36.

Unit of Analysis:

Block-level (CLD $\times$ run, $N = 9$); see shared RQ1b setup (Section 7.12).

Metrics / Stats:

$\Delta F1$ pre $\rightarrow$ post; Wilcoxon signed-rank.

Reproducibility:

Scripts: Appendix Tables C.5, C.6.

Results:

Tables 8.10, 8.11 (Chapter 8).

7.12.0.2 RQ1b: Correctness Corrector (Lit)**RQ & Goal:**

Evaluate Corrector on GT Lit flagged edges. See shared RQ1b setup (Section 7.12).

Source Data:

Judge-flagged edges from RQ1a GT Lit experiments. GT files: Appendix Table C.1.

Simulation Parameters:

Appendix Table C.17. Fixed: GPT-4.1, T=0.0, baseline judge. Varies: 3 prompts $\times$ 3 CLDs $\times$ 3 runs = 27.

Simulation Procedure:

Algorithm H.5 (correction) $\rightarrow$ Algorithm H.2 (re-judging).

Outputs:

Directory: `final_runs/RQ1b_corrector_gt_lit_correctness/`. Schemas: Appendix Table C.36.

Unit of Analysis:

Block-level (CLD $\times$ run, $N = 9$); see shared RQ1b setup (Section 7.12).

Metrics / Stats:

$\Delta F1$ pre $\rightarrow$ post; Wilcoxon signed-rank.

Reproducibility:

Scripts: Appendix Tables C.5, C.6.

Results:

Tables 8.12, 8.13 (Chapter 8).

7.13 RQ2: Context-Insensitive Hallucination Detection

RQ & Goal:

Test whether UQ metrics captured during generation (Section 5.2.2) can predict hallucinations without external context.

Hypotheses:

H4 (UQ): higher perplexity / lower $P_{\min}$ / higher window entropy $\Rightarrow$ hallucination (block-level AUC > 0.5).

H5 (CosSim): lower narrative-citation cosine similarity $\Rightarrow$ hallucination (block-level AUC > 0.5).

Source Data:

63 deduplicated GT Lit files (36 citation, 27 correctness), 31,704 edges, hallucination rate 15.2%. Hallucination label: (FP or FN) w.r.t. GT Lit. Ensemble phases use a 16,507-edge subset (35 citation files) where all four metrics are available. Outputs and GT files: Appendix C.1 (Tables C.4, C.1).

UQ Metrics + Parameters:

Computed from generator logprobs (Algorithms H.6, H.7). Perplexity uses capped NLL averaging (token cap 20.0; average cap 10.0, bounding PPL to $\leq e^{10} \approx 22,026$); $P_{\min}$ is the minimum token probability; H_{win} is max sliding-window entropy (window size 5 over top- k entropies with $k=5$); CosSim is the max narrative-citation-chunk embedding similarity for citation experiments (embedding model all-mpnet-base-v2 from [98]).

Outputs:

Per-edge metric values, per-file AUCs, and classifier predictions; schemas in Appendix C.5 (Table C.36).

Metrics:

AUC-ROC, point-biserial correlation r , and classifier CV/test AUC. Because hallucinations are the minority class (15.2%), AUC is used as a threshold-free separation metric; thresholded Precision/Recall depend on the chosen operating point and class prevalence.

Unit of Analysis:

Block-level (CLD $\times$ run, $N = 9$) for statistical inference; edge-level ($N = 31,704$) for descriptive analysis.

Stats:

Block-level aggregation (CLD $\times$ run, $N=9$) as the unit of inference; single-metric one-sample t -tests on block-mean AUC vs. 0.5 with Bonferroni correction; edge-level Mann-Whitney U reported descriptively. Block means are approximately normal (Shapiro-Wilk $p=0.67$); 95% CI via t ($df=8$). Correlations aggregated via Fisher z (Appendix D).

Results:

Figures 8.10, 8.11 (Chapter 8).

Experiment Details

7.13.0.1 RQ2: UQ (Logprobs)

RQ & Goal:

Test whether logprob-derived UQ metrics can predict hallucinations. See shared RQ2 setup (Section 7.13).

Source Data:

RQ1a GT Lit experiments (27 correctness + 36 citation files, 31,704 edges); hallucination = FP/FN.

Simulation Parameters:

Appendix Table C.21. Metrics: PPL, $P_{\min}$, H_{win} (computed during generation).

Simulation Procedure:

Algorithm [H.6](#) (metric extraction from logprobs).

Outputs:

Directory: `final_runs/RQ2_hallucination_detection/`. Schemas: Appendix Table [C.36](#).

Unit of Analysis:

Block-level (CLD×run, $N = 9$); see shared RQ2 setup (Section [7.13](#)).

Metrics / Stats:

Block-level AUC; one-sample t -test vs. 0.5.

Reproducibility:

Scripts: Appendix Tables [C.5](#), [C.6](#).

Results:

Figure [8.10](#) (Chapter [8](#)).

7.13.0.2 RQ2: CosSim (Embeddings)

RQ & Goal:

Test whether narrative–citation cosine similarity can predict hallucinations. See shared RQ2 setup (Section [7.13](#)).

Source Data:

RQ1a GT Lit Citation experiments only (36 files); ensemble subset = 16,507 edges.

Simulation Parameters:

Appendix Table [C.22](#). Model: all-mpnet-base-v2; max narrative–citation-chunk similarity.

Simulation Procedure:

Embedding computation + cosine similarity calculation. Benchmark: Appendix [G](#).

Outputs:

Directory: `final_runs/RQ2_hallucination_detection/`. Schemas: Appendix Table [C.36](#).

Unit of Analysis:

Block-level (CLD×run, $N = 9$); see shared RQ2 setup (Section [7.13](#)).

Metrics / Stats:

Block-level AUC; one-sample t -test vs. 0.5.

Reproducibility:

Scripts: Appendix Tables [C.5](#), [C.6](#).

Results:

Figure [8.10](#) (Chapter [8](#)).

7.13.0.3 RQ2: Analysis Pipeline (6 Phases)

Simulation Procedure (6 phases):

Phases 1–3 (single-metric): compute per-file AUC-ROC, aggregate to blocks ($N=9$), and test above-chance discrimination via one-sample t -test.

Phase 4 (RFE): Recursive Feature Elimination [[123](#)] with `GroupKFold(n_splits=3)` over blocks; classifiers: LogReg, RF, GradBoost.

Phase 5 (ensemble): block-level train/test split (67%/33%) via `GroupShuffleSplit` and model selection on training blocks via `GroupKFold`; four classifiers including a neural network.

Classifier families: brief algorithm notes for LR/RF/GB/MLP appear in Appendix [D.0.0.4](#).

Phase 6 (LOCO): leave-one-CLD-out evaluation for cross-domain generalisation. Full formal specification in Appendix [H.7](#).

Reproducible splitting protocol (Phases 4–6):

Grouping key is `block_id=cld.run` ($3 \text{ CLDs} \times 3 \text{ runs} \Rightarrow N=9$); rows are deterministically sorted by `(block_id, source_file)` prior to any group split; scaling uses `StandardScaler` fit on training blocks only. Phase 4 holds out 3 blocks and trains on 6; evaluates $k \in \{1, 2, 3, 4\}$ and selects the k maximizing validation

AUC per fold.

Phase 5 holds out 3 blocks for testing and uses the remaining 6 for training; CV on training uses 3 folds (2 blocks held out per fold).

Phase 6 holds out an entire CLD as test and trains on the other two CLDs (scaling fit on training CLDs only).

Unit of Analysis:

Block-level (CLD $\times$ run, $N = 9$) for Phases 1–5; CLD-level ($N = 3$) for Phase 6 (LOCO).

Limitations:

UQ metrics are not applicable for GT Synth (corruption is post-generation); cosine similarity is only defined for Citation Judge runs.

Reproducibility:

Data in `final_runs/RQ2_hallucination_detection/`; dataset mapping in Appendix C.1 (Table C.4). Analysis scripts in Appendix C.2 (Table C.6).

Results:

Figures 8.10, 8.11; Section 8.12.4 (Chapter 8).

7.14 RQ3: Deep Research Literature Validation

Methodological Note: The original RQ3 design proposed *smart test-time compute allocation*—using UQ metrics to selectively deploy the LLM-as-a-corrector. This was predicated on: (1) RQ2 UQ metrics generalizing across CLDs, and (2) RQ1b correctors effectively repairing hallucinations. Neither condition was met (see Discussion, Section 9.10). We therefore pivoted to **Deep Research validation** (system documented in Section 5.7).

This section presents Deep Research as an **exploratory pilot study**. Given time constraints from the late pivot, the experimental design is less comprehensive than RQ1–RQ2: single-run (no replication), limited parameter ablation, and no end-to-end pipeline integration.

7.14.0.1 Deep Research

RQ & Goal.

Evaluate whether Deep Research (Section 5.7) can distinguish valid from hallucinated causal claims by finding *direct causal* evidence in the scientific literature.

Hypothesis.

H6: high detection for TP, low detection for FP, and a TP–FP gap > 20 percentage points.

Source Data.

285 edges from RQ1a GT Lit experiments (TP=44, FP=178, FN=63). Output directory: Appendix C.1 (Table C.4).

Parameters.

Single run per edge ($\sim \$0.87/\text{edge}$) with `max_iterations=1`, `max_sources=5`, `MAX_SEARCHES=5`, `time_limit=900s`.

Simulation Procedure.

Generate search queries from the causal claim, run parallel Brave searches, score evidence, apply strict direct-causality criteria, and synthesize a verdict (Algorithm H.11).

Outputs.

Per-edge verdict, confidence (1–10), `direct_causal_found` flag, and source URLs; schemas in Appendix C.5 (Table C.36).

Metrics.

Detection rate of `direct_causal_found`; TP–FP gap $\Delta > 20\text{pp}$; mean confidence by class.

Stats.

Pairwise Fisher exact tests (Bonferroni: $m = 3$, $\alpha_{\text{adj}} = 0.017$); effect size Cohen’s h (Appendix D).

Unit of Analysis.

Edge-level ($N = 285$ edges: TP=44, FP=178, FN=63).

Reproducibility.

Config in Appendix C.4.8 (Table C.23); script `run_rq3_analysis.py`.

Results.

Figure 8.12, Table 8.18 (Chapter 8).

7.14.0.2 Computational Cost**Cost summary (reported in Results).**

See Results Section 8.13.1 (Table 8.18) for the Deep Research computational cost breakdown by CLD and the overall cost comparison (\$0.87/edge; $\sim 119\times$ citation judging; $\sim 512\times$ correctness judging).

7.14.0.3 Limitations

- **No replication:** Single run per edge prevents uncertainty quantification
- **Evidence applicability:** Retrieved causal evidence may be only partially aligned to specific edge claims
- **Full-text access:** Paywalled articles limit evidence quality
- **Search API dependency:** Results depend on Brave Search indexing

7.14.0.4 Exploratory Analysis: Ground Truth Enrichment**RQ & Goal.**

Explore whether DR results could enrich expert-constructed GT by validating “hallucinations” that may represent valid causal relationships experts omitted.

Extrapolation Procedure.

Two enrichment scenarios:

- **Scenario 1 (Enriched GT):** Promote DR-validated FPs to TP in ground truth; recompute Precision/Recall/F1.
- **Scenario 2 (LLM+DR):** Additionally add DR-validated FNs to system output.

Caveats.

Domain variation (28% Depressive Symptoms vs 76% Older Persons ED FP promotion), evidence specificity concerns, circularity risk (LLM validating LLM), no expert review. Results exploratory.

Results.

Section 8.13.3, Table 8.20 (Chapter 8).

Reproducibility.

```
python final_runs/RQ3_deep_research/compute_enriched_gt_metrics.py
```

7.14.0.5 Human Validation**RQ & Goal.**

Validate DR verdicts via human review to establish applicability thresholds and enable extrapolation.

Source Data.

50 edges (29.6% of 169 eligible) via proportional stratified sampling across CLD $\times$ classification. Seed 42 for reproducibility.

Annotation Procedure.

Human reviewer assigns categorical verdict (`causal`, `different_vars`, `invalid`) and continuous applicability score $\in [0, 1]$ using the applicability rubric in Appendix Table C.25: 1.0 = exact match; 0.8–0.9 = different operationalisations; ≈ 0.7 = partial claim; ≈ 0.6 = proxy constructs; ≤ 0.5 = weak/non-causal evidence.

Results.

Figure 8.13, Table 8.23, Table 8.21 (Chapter 8).

Reproducibility.

```
python final_runs/RQ3_deep_research/scripts/sample_edges_for_validation.py
```

Inter-rater reliability check.

To assess rubric reliability, a second rater independently annotated an overlapping subset of 13 edges using the same rubric. Agreement for the categorical verdict was quantified using Cohen’s κ (chance-corrected; interpretation per Landis & Koch as defined in Section 7.11.2). Agreement for the continuous applicability score was quantified using the Intraclass Correlation Coefficient, ICC(2,1) (two-way random effects, absolute agreement, single measures) [117]. The ICC is defined as:

$$\text{ICC} = \frac{\sigma_{\text{between}}^2}{\sigma_{\text{between}}^2 + \sigma_{\text{within}}^2}$$

where $\sigma_{\text{between}}^2$ is the variance between subjects (edges) and σ_{within}^2 is the variance within subjects (across raters). Interpretation thresholds [117]: <0.50 poor, 0.50–0.75 moderate, 0.75–0.90 good, >0.90 excellent.

Reproducibility: `python final_runs_backup.../RQ3_deep_research/validation/calculate_inter_rater_re`

7.14.0.6 Threshold Selection and Extrapolation

RQ & Goal.

Determine applicability threshold for filtering DR verdicts and extrapolate enrichment potential to full dataset.

Extrapolation Procedure.

Sensitivity analysis across $t \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$. For each threshold: compute coverage (pass rate) and correct-causal rate with Wilson 95% CI. Select threshold via gradient-based elbow rule on coverage–correctness curve.

Metrics.

- **Coverage:** Fraction with applicability $\geq t$
- **Correct-causal rate:** Fraction of passing edges labelled `causal`

Extrapolation.

Two-stage: (1) compute pass rate and causal rate from validated sample at t^* ; (2) project to full dataset ($n = 285$) with conservative lower CI bounds. Compute projected F1 under Scenarios 1 and 2.

Caveats.

Small strata (e.g., Social Norms $n = 3$) have high sampling variance. Results reported with point estimates and conservative bounds.

Results.

Figure 8.13, Table 8.23, Table 8.24 (Chapter 8).

Reproducibility.

```
python final_runs/RQ3_deep_research/scripts/enrichment_extrapolation.py
```

7.14.0.7 Smart Trigger

Estimate a cost-efficient DR deployment policy by triggering DR only on edges flagged as uncertain by the strongest GT Lit Correctness Judge (Mechanistic prompt). We compute an aggregate per-edge judge score (mean over runs) and mark edges with score ≤ 0.5 as triggered. We then evaluate the triggered subset using DR outputs and extrapolate expected $\Delta F1/\$$ under human-calibrated applicability thresholds (see Results Figure 8.14 and Methods Section 7.14.0.6).

Reproducibility: Scripts in `final_runs/RQ3_deep_research/scripts/` (see Appendix C.2).

7.15 Reproducibility

To ensure full reproducibility of all experimental results, we provide unified analysis scripts that orchestrate the complete evaluation pipeline for each research question. All analysis scripts and raw data are available in the project repository. For an end-to-end reproduction index (data availability, code locations, environment specification, and exact execution commands), see Appendix C (especially Appendix C.2, Appendix C.3, Appendix C.4, and Appendix C.5). For a single-command reproduction entry point per research question and a complete inventory of all runnable scripts under `final_runs/`, see Appendix C.4.1 and Table C.7.

7.16 Environment Setup

The project uses `uv` for dependency management. To reproduce the analysis environment:

```
# Install dependencies
uv sync

# Activate virtual environment
source .venv/bin/activate
```

Environment specification: The hardware/runtime environment and key pinned dependencies used for the experiments are documented in Appendix C.3 (Tables C.9 and C.10). *Note:* Analysis scripts run on the provided raw data and do not require API keys. Re-running the generation or judging experiments (data creation) requires a `.env` file with OpenAI and Brave Search API keys.

7.17 Unified Analysis Entry Points

The following single-entry scripts reproduce the analyses, tables, and figures reported in the Results section:

```
# RQ1 Series (TruthfulQA, Judge, Corrector)
# Reproduces: TruthfulQA metrics, RQ1a Friedman/Wilcoxon tests,
# RQ1b ablation analysis
python3 final_runs/RQ1_unified_analysis.py

# RQ2 Context-Insensitive Metrics
# Reproduces: Phase 1-3 meta-analysis, Phase 4 RFE,
```

```
# Phase 5 Ensemble, Phase 6 Cross-CLD
python3 final_runs/RQ2_unified_analysis.py

# RQ3 Deep Research
# Reproduces: Fisher’s exact tests, Cohen’s h, per-CLD breakdown,
# enrichment scenarios, and master report
python3 final_runs/RQ3_unified_analysis.py
```

7.18 Component-Specific Reproduction

Individual component scripts (temperature sensitivity, computational cost analysis, RQ1a detailed analysis) are documented in Appendix C.4.

Data Availability: Raw experiment results are in `final_runs/`. Ground-truth CLD inputs (Table C.1), validation datasets (Table C.2), and per-RQ result locations (Table C.4) are documented in Appendix C.1; output file types and naming conventions are in Appendix C.5.

7.19 Threats to Validity

This section summarises key threats to validity that graders commonly expect to see *explicitly* addressed in the Methods. A longer, interpretive discussion (including qualitative failure mechanisms) appears in the Discussion chapter.

7.19.1 Internal Validity (leakage, pseudo-replication, and dependence)

Our data are highly structured: many edges come from the same generation scenario, and multiple prompt variants are evaluated within the same (CLD×run) block. To reduce pseudo-replication and leakage, we adopt **block-aware** design and analysis choices throughout:

- **Blocked (repeated-measures) design:** We treat (CLD×run) as the independent unit for prompt comparisons and meta-inference ($N = 9$ blocks; Section 7.1).
- **Group-aware splitting:** For RQ2 ensemble models, all train/test and CV splits keep entire blocks together (GroupKFold / GroupShuffleSplit; Section 7.13.0.3), preventing leakage from correlated edges within the same scenario.
- **Edge-level results as descriptive:** When we plot pooled edge distributions, we treat them as diagnostics rather than edge-level inferential claims to avoid overstating significance under dependence.

7.19.2 Construct Validity (what “hallucination” measures)

Two complementary ground-truth regimes are used (Section 4.2), which implies different constructs:

GT Lit (literature ground truth): “Hallucination” is operationalised as disagreement with an expert reference CLD (FP/FN relative to the published edge set over a fixed variable set). This is best interpreted as *alignment with a specific expert model* rather than universal factuality. A “false positive” (edge absent

from the reference CLD) can reflect either a genuinely invalid relationship or a scope/construct-mapping decision (e.g., the relationship may hold for a related construct but not for the exact operationalised variable), and a “false negative” can reflect omission rather than confabulation.

GT Synth (synthetic corruption): “Hallucination” is the `corrupted=True` label from an LLM-based corruptor applied to an **LLM-generated base CLD** (Phase 1 output). This yields high internal validity (known injected errors) but may not fully reflect real disagreements between generators and expert modellers (e.g., synthetic polarity flips can be more detectable than subtle scope disagreements).

Measurement choices can also influence construct validity: (i) the fixed threshold (judge score ≤ 0.5) is an operational decision that trades precision vs. recall; (ii) citation judging evaluates *retrieved evidence support* for the generated narrative rather than the causal relationship in the abstract, so failures in evidence specificity or variable-term mapping can appear as “unsupported” edges even when the relationship is plausible.

7.19.3 External Validity (generalisation across domains, models, and time)

Generalisation is constrained by several design choices:

- **Domain coverage:** All primary GT Lit CLDs are in the health domain. Results may not generalise to domains with different causal-evidence norms (e.g., economics, ecology, engineering) or different causal-criterion suitability (e.g., mechanistic criteria originating in epidemiology).
- **Model dependence:** Generator and judges are specific model snapshots (primarily GPT-4.1; GPT-5-mini for cost reduction in GT Synth citation judging). Observed differences may conflate method effects with model effects when models differ between cells.
- **Retrieval dependence:** Citation-based judging depends on the retrieval and fetching stack (search index, query formulation, paywalls/HTML structure). Even with fixed code, search results and website content can drift over time, limiting strict reproducibility and external validity of citation-based scores.
- **Language and terminology:** CLD variable naming and narrative phrasing can mismatch the terminology used in the literature, potentially depressing retrieval quality and creating domain-specific failure modes.

7.19.4 Conclusion Validity (inference limits, power, and multiple comparisons)

Several factors limit the strength of statistical conclusions:

- **Small N for meta-inference:** Prompt comparisons and block-level hypothesis tests use $N = 9$ independent blocks, reducing statistical power and increasing uncertainty; null results should not be over-interpreted as evidence of no effect.
- **Power / sensitivity interpretation:** Because the independent unit is $N=9$ blocks, our confirmatory tests have limited sensitivity to small effects. We therefore emphasise standardised effect sizes (Table 8.26, Results) alongside p -values.
- **Multiple comparisons:** We apply Bonferroni corrections within each research question (Section 7.3), which controls false positives but further reduces power.
- **Dependence structure:** Edge-level measurements within a block are correlated; we therefore treat edge-level tests as descriptive and base confirmatory inference on block-level aggregates (Section 7.1).
- **RQ3 is exploratory:** Deep Research is a single-run pilot per edge (no replication) due to cost, so uncertainty quantification is limited and conclusions are framed descriptively (Section 7.14).

7.19.5 Reliability and Instrumentation (judge and retrieval behaviour)

Finally, several reliability threats apply to LLM- and retrieval-mediated measurement:

- **LLM stochasticity and drift:** We mitigate randomness by fixing seeds where supported and using $T=0$ for judges, but model/provider updates can still change outputs over time.
- **Systematic judge biases:** LLM judges can be systematically lenient or sceptical depending on prompt framing; citation judging is especially sensitive to confirmation bias because queries are generated from the model’s own narrative (generate-then-search).
- **Asymmetry for negative claims:** Validating **NONE** edges is intrinsically difficult; absence of retrieved evidence can be misinterpreted as support for “no relationship” claims (“evidence of absence” failure mode), which can inflate scores for missed expert edges.

Chapter 8

Experiments and Results

For reproduction commands and a complete mapping from thesis results to the executable analysis pipelines, see Appendix C (especially Appendix C.4, Appendix C.4.1, and Table C.7).

8.1 Generator Model Selection

Experimental design: Section 7.6.0.1. Algorithm: Algorithm H.1 (CLD Generation).

Before establishing baseline performance, we compare three frontier LLMs for CLD edge generation: GPT-4.1 (OpenAI), Sonar-Pro (Perplexity), and Claude Sonnet 4 (Anthropic). All models were evaluated on the same three validation CLDs using identical prompts (Nitai_C, T=0.7), with three independent runs per CLD. Full configuration details (including run scripts and parameters) are provided in Appendix C.4.10.

Table 8.1: Generator Model Comparison: Edge Recovery Performance. Bold indicates highest F1 per CLD; in aggregate row, bold indicates highest Precision, Recall, and F1 separately. **Edges processed** refers to the number of ordered (source, target) edge slots evaluated (including NONE).

Model	CLD	Edges processed	TP	FP	FN	Precision	Recall	F1
GPT-4.1	Social Norms	270	17	24	16	0.415	0.515	0.459
GPT-4.1	Depressive Symptoms	546	80	106	22	0.430	0.784	0.556
GPT-4.1	Older Persons ED	3168	19	356	80	0.051	0.192	0.080
Sonar-Pro	Social Norms	268	14	34	19	0.292	0.424	0.346
Sonar-Pro	Depressive Symptoms	540	71	162	31	0.305	0.696	0.424
Sonar-Pro	Older Persons ED	3095	65	1184	34	0.052	0.657	0.096
Claude-Sonnet-4	Social Norms	270	30	74	3	0.288	0.909	0.438
Claude-Sonnet-4	Depressive Symptoms	546	96	329	6	0.226	0.941	0.364
Claude-Sonnet-4	Older Persons ED	3168	92	1970	7	0.045	0.929	0.085
<i>Micro-averaged (pooled TP/FP/FN across all CLDs):</i>								
GPT-4.1	Total	3984	116	486	118	0.193	0.496	0.278
Sonar-Pro	Total	3903	150	1380	84	0.098	0.641	0.170
Claude-Sonnet-4	Total	3984	218	2373	16	0.084	0.932	0.154

Note. All models used identical prompts (Nitai_C variant) and temperature ($T=0.7$). Results pool TP/FP/FN counts across 3 independent runs per CLD (same configuration; stochastic sampling). Nodes (variables) were directly retrieved from the **GT lit CLDs**. **Edges processed** denotes the number of ordered (source, target) edge slots evaluated (including NONE), i.e., ideally $|V|(|V| - 1)$; it can be slightly lower if some rows are missing due to parsing/processing errors.

Table 8.1 shows the edge recovery performance of the three generator models. GPT-4.1 achieves the best aggregate F1 (0.278) through balanced precision-recall. Sonar-Pro performs competitively and wins the Older Persons ED CLD ($F1=0.096$), but its micro-average aggregate F1 is still lower than GPT-4.1 due to substantially more false positives overall. Claude-Sonnet-4 shows highest recall (0.932) but lowest precision (0.084), generating many false positives and yielding a lower aggregate F1. Based on best micro-average F1 score, **GPT-4.1 was selected** as the generator model for all subsequent experiments. Additionally, GPT-4.1 is the only model that provides token-level log probabilities (logprobs), enabling confidence-based edge filtering which is necessary for RQ2 experiments.

8.2 Generator Performance Baseline

Experimental design: Sections 7.7 and 7.6.0.3. *Algorithm:* Algorithm H.1.

Having selected GPT-4.1 based on the model comparison above, we now establish its baseline performance. These results show that the CLD generator recovers substantially more ground-truth edges than a structural random baseline (under our GT Lit operationalisation) using the Nitai_C prompt at $T = 0.7$.

8.2.1 Random Baseline Comparison

Experimental design: Section 7.7.

To confirm the generator produces meaningful causal structure rather than random edge configurations, we compare LLM-generated edges against a Monte Carlo random baseline. Full random-baseline configuration details are provided in Appendix C.4.11.

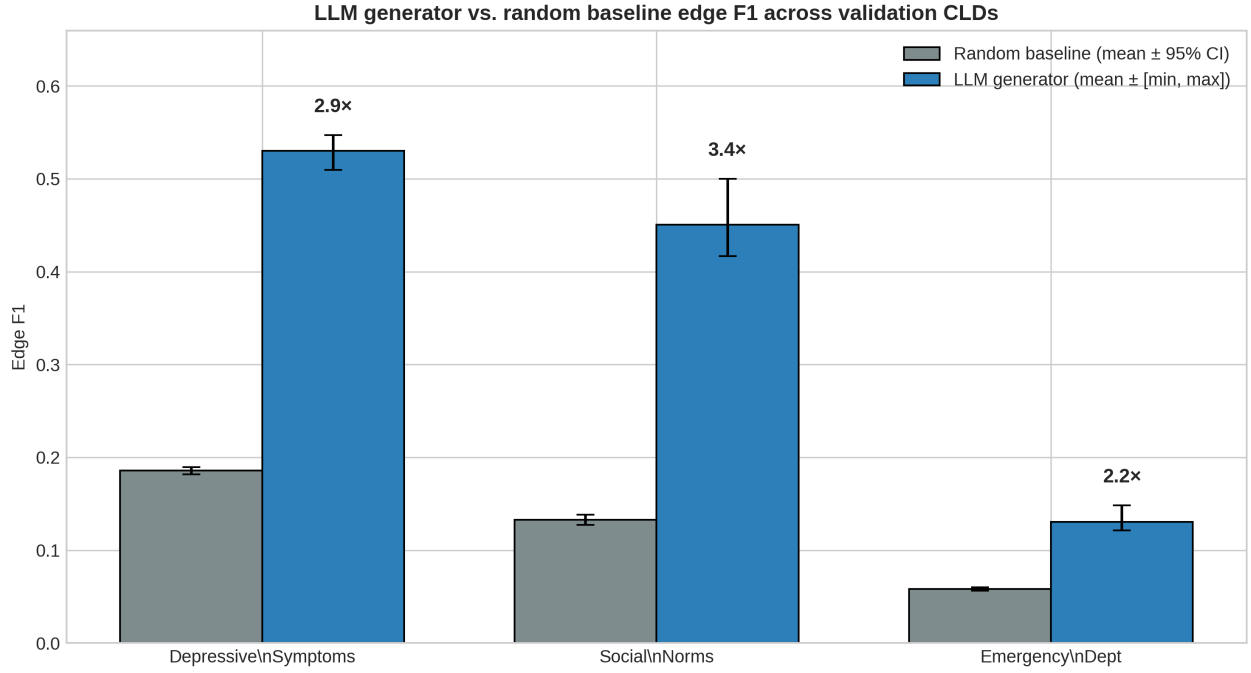


Figure 8.1: LLM generator (GPT-4.1, T=0.7, Nitai.C prompt) vs. random baseline edge F1 across all three validation CLDs. Error bars show min-max range across 3 runs.

Table 8.2: Random Baseline Comparison for Generator Edge Recovery

CLD	$ V $	$ E $	Density	Random F1 ($\pm$ 95% CI)	LLM F1 ($\pm$ 95% CI)	Improv.
Depressive Symptoms	14	34	0.187	0.186 ± 0.004	0.530 ± 0.050	2.8×
Social Norms	11	12	0.109	0.123 ± 0.005	0.451 ± 0.110	3.7×
Older Persons ED	36	66	0.052	0.055 ± 0.005	0.131 ± 0.040	2.4×
<i>Average</i>	—	—	—	0.121	0.370 ± 0.020	3.1×

Note. Random F1 via Monte Carlo (1000 iterations); LLM F1 from 3 generation runs per CLD (seeds 10, 20, 30). Density = $|E|/(|V| \times (|V| - 1))$. 95% CI uses z for random ($n=1000$) and t for LLM ($n=3$, $df=2$).

Figure 8.1 and Table 8.2 show the LLM consistently outperforms random chance (2.2–3.4× improvement), supporting that it captures non-random structure aligned with the published CLDs. Performance varies by domain complexity: Social Norms shows the largest relative improvement (3.4×) while Older Persons ED shows the smallest (2.2×) due to lower absolute F1 against a Monte Carlo random baseline which wires edges given the same variables with similar density.

8.2.2 Temperature Sensitivity Analysis

Experimental design: Section 7.6.0.3.

To validate the generator temperature choice (T=0.7), we tested whether temperature significantly affects edge recovery performance across all three validation CLDs.

Table 8.3: Generator Temperature Sensitivity: Edge F1 Score by CLD

Temperature	Edge F1 Score (Mean $\pm$ 95% CI)			
	Social Norms	Depressive	Emergency Dept	Overall
T=0.0	0.325 $\pm$ 0.099	0.455 $\pm$ 0.055	0.113 $\pm$ 0.031	0.298 $\pm$ 0.116
T=0.3	0.373 $\pm$ 0.087	0.445 $\pm$ 0.014	0.125 $\pm$ 0.043	0.314 $\pm$ 0.113
T=0.5	0.314 $\pm$ 0.102	0.480 $\pm$ 0.051	0.105 $\pm$ 0.012	0.299 $\pm$ 0.126
T=0.7	0.348 $\pm$ 0.148	0.446 $\pm$ 0.014	0.104 $\pm$ 0.032	0.299 $\pm$ 0.120
T=1.0	0.381 $\pm$ 0.054	0.444 $\pm$ 0.027	0.105 $\pm$ 0.013	0.310 $\pm$ 0.120

Note. Edge F1 = $\frac{2 \cdot \text{TP}}{2 \cdot \text{TP} + \text{FP} + \text{FN}}$ comparing generated edges against literature ground truth.

Design. 5 temperatures $\times$ 3 runs $\times$ 3 CLDs = 45 total experiments. Seeds: 10, 20, 30 per temperature.

Per-CLD columns: Mean $\pm$ 95% CI across 3 runs per CLD at each temperature.

Overall column: Mean $\pm$ 95% CI computed across all 9 individual runs (3 CLDs $\times$ 3 runs) per temperature. The larger CI reflects variability both within and between CLDs.

Per-CLD ANOVA (temperature effect): Social Norms $F(4, 10) = 1.49$, $p = 0.28$; Depressive $F(4, 10) = 3.08$, $p = 0.07$; Emergency Dept $F(4, 10) = 1.76$, $p = 0.21$. None significant at $\alpha = .05$.

Temperature (T) = 0.7 was chosen for the generator LLM as initial default to strike a balance between diversity and determinism in sampled outputs. Subsequent sensitivity analysis shown in Table 8.3 shows that temperature does not significantly affect Edge F1: one-way ANOVA $F(4, 40) = 0.035$, $p = 0.998$, $\eta^2 = 0.004$ (negligible effect). Shapiro-Wilk [124] normality and Levene’s [125] homogeneity assumptions were satisfied (all $p > .05$). Temperature does not significantly affect Edge F1 ($p > 0.05$), justifying T=0.7 as a reasonable default.

8.3 Correctness Judge Verification

Experimental design: Section 7.8. *Algorithm:* Algorithm H.2 (Correctness Judge).

Before evaluating the LLM-as-a-Judge approach on CLD edges, we sanity-check our correctness-based judge implementation on TruthfulQA—a commonly used benchmark for distinguishing truthful from false-but-plausible answers—to verify that the evaluation pipeline behaves as expected on a controlled setting.

Table 8.4: TruthfulQA Judge Verification
(n=1,634)

Metric	Value	
Accuracy	0.876	
Precision	0.890	
Recall	0.859	
F1 Score	0.874	
Specificity	0.894	
	Pred: Halluc.	Pred: Truthful
Actual: Halluc.	702 (TP)	115 (FN)
Actual: Truthful	87 (FP)	730 (TN)

Note. Judge: GPT-4.1, temp=0.0, seed=42, baseline correctness prompt. Dataset: 1,634 samples (817 hallucinated, 817 truthful; 50% base rate). Classification threshold: judge_score < 0.5 $\Rightarrow$ Hallucinated.

Table 8.4 shows the performance of the correctness judge on the TruthfulQA dataset using the baseline prompt. Classification performance across both Recall (85.9%) and Precision (89.0%) are both high, indicating the judge neither over-flags nor under-flags. These results support that our correctness judge implementation works correctly on this benchmark. The subsequent RQ1a experiments will reveal whether this capability transfers to the more challenging domain of CLD edge validation.

8.4 RQ1a: Corruption Detection - Correctness

Experimental design: Section 7.9.0.2. *Algorithms:* Algorithm H.4 (Corruptor), Algorithm H.2 (Correctness Judge).

8.4.1 Performance Metrics Overview

This section presents hallucination detection performance across all CLDs and prompt variants using the Correctness Judge. Ground truth is established by generating CLDs with an LLM and then introducing controlled corruptions. A *true positive* (TP) occurs when the judge correctly flags a corrupted edge, while a *false positive* (FP) occurs when an uncorrupted edge is incorrectly flagged as corrupted. A *false negative* (FN) is a corrupted edge that the judge fails to detect, and a *true negative* (TN) is an uncorrupted edge that the judge correctly does not flag. The classification threshold is set at 0.5: edges receiving a judge score ≤ 0.5 are classified as hallucinations. Figure 8.2 shows the performance metrics overview for the correctness-based corruption detection task. A consolidated sensitivity analysis of judge prompt variants (effect sizes and assumption checks across all RQ1a prompt ablations) is provided in Appendix B.1.

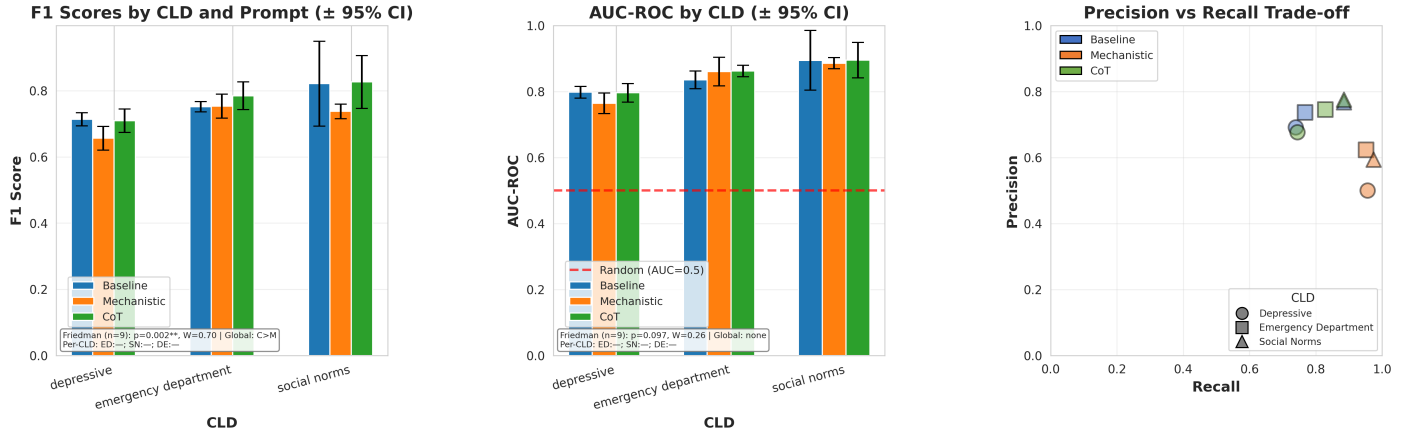


Figure 8.2: Performance metrics overview for corruption detection (Correctness Judge). Shows F1 scores (Left), ROC-AUC with random (AUC=0.5) baseline (Middle), and aggregate performance (Right) by prompt. Friedman test (see Appendix D) (below each panel) tests prompt effect with (CLD, run) as blocks; per-CLD Wilcoxon tests (see Appendix D) show which pairs differ within each CLD (e.g., “DE:M>B” = Depressive Symptoms: Mechanistic>Baseline). W = Kendall’s W . DE=Depressive Symptoms, ED=Older Persons ED, SN=Social Norms; B=Baseline, M=Mechanistic, C=CoT. $*** p < 0.001$.

The leftmost plot of Figure 8.2 shows the correctness judge obtaining F1 scores in the range 0.65 to 0.82 across all CLDs and prompt variants. Depressive Symptoms has the lowest average F1, followed by Older Persons ED, with Social Norms the highest. CIs overlapping per prompt variants within each CLD suggest the differences per prompt are not statistically significant.

The middle plot of Figure 8.2 shows the AUC-ROC with a random (AUC = 0.5) baseline, indicating that the correctness judge can discriminate corrupted from non-corrupted edges in each CLD and prompt. AUC ranges from 0.77 at the lowest end (Mechanistic prompt in Depressive Symptoms) to 0.83 at the highest end (Baseline prompt in Social Norms). Again, the highest AUC on average is obtained for Social Norms, followed by Older Persons ED, with Depressive Symptoms the lowest; CIs overlap, suggesting no statistically significant differences.

The right plot of Figure 8.2 shows the precision-recall trade-off per prompt variant and CLD. The Mechanistic prompt achieves the highest recall (it flags the highest proportion of corrupted edges), while Baseline and CoT show higher precision. CoT has slightly higher recall and precision for the Older Persons ED CLD than Baseline, but performance is otherwise similar across CLDs and does not differ significantly overall.

8.4.2 Detailed Analysis

In-depth analysis of corruption type detection patterns and score discrimination ability.

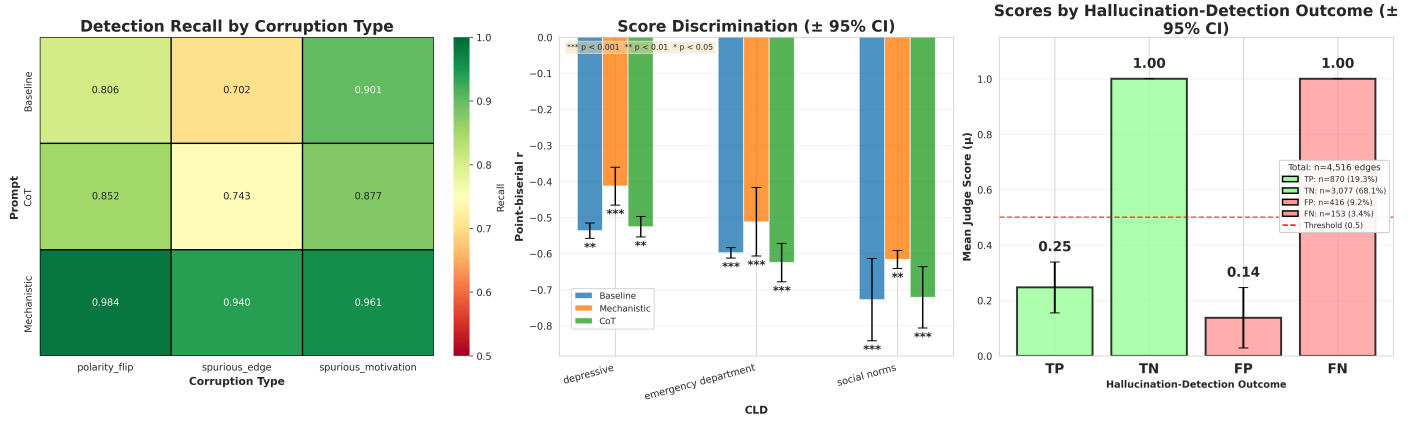


Figure 8.3: Detailed analysis for correctness-based corruption detection. Recall by corruption type with colour scale from 0.5 to 1.0 (Left). Score discrimination showing point-biserial correlation between ground truth hallucinations (corruption=True) and judge scores (Middle). Judge scores by hallucination-detection outcome (Right): mean judge scores (μ) and 95% confidence intervals, with correct outcomes shown in green and incorrect outcomes shown in red.

Figure 8.3 shows the detailed analysis of the correctness judge’s performance. The leftmost plot of Figure 8.3 shows the recall by corruption type aggregated across all CLDs and 3 runs, and shows that the correctness judge with mechanistic prompt operates with higher recall for each corruption type than baseline and CoT prompts, with CoT outperforming baseline on spurious and polarity flip corruption types but outperforming CoT on spurious motivation, while still scoring less than Mechanistic overall.

The middle plot of Figure 8.3 shows the point-biserial correlation between the ground truth corruption label (`corrupted=True`) and judge scores (threshold judge score ≤ 0.5). We observe statistically significant negative correlations (w.r.t. $H_0 : \rho = 0$), which is the expected direction: lower judge scores should be associated with corrupted edges. Confidence intervals per prompt still overlap for Older Persons ED and Social Norms, suggesting no statistically significant differences between prompt variants; we also find a statistically significant lower relative correlation for Mechanistic on Depressive Symptoms w.r.t. the other prompt variants. Overall, correlations are large, ranging from -0.55 to -0.7 for Mechanistic and Baseline (lowest for Depressive Symptoms and highest for Social Norms), with CoT showing slightly lower correlations from -0.4 to -0.6.

The rightmost plot of Figure 8.3 shows mean judge score (μ) by *hallucination-detection outcome* (TP/FP/TN/FN), using the same threshold (≤ 0.5) and corruption label (`corruption=True`) as the reported F1 scores. As expected under thresholding, outcomes predicted as “clean” (TN and FN) have higher mean scores than outcomes predicted as “hallucination” (TP and FP). The substantive failure mode is the FN bar: a non-trivial fraction of corrupted edges are missed yet still receive the maximum judge score (near 1.0), indicating that some synthetic corruptions are plausibility-preserving under correctness judging.

Full ROC curves are provided in Appendix I (note: scores are discrete at 0, 0.5, and 1, so only a small number of operating points are meaningful).

8.5 RQ1a: Ground Truth - Correctness

Experimental design: Section 7.9.0.1. *Algorithm:* Algorithm H.2 (Correctness Judge).

For this experiment, the variables from ground truth from literature are passed and LLM generated edges are judged. In this scenario, ground truth from literature (GT Lit.) is used, such that a hallucination is defined as an FP/FN with respect to the ground truth.

8.5.1 Performance Metrics Overview

Detailed view of core performance metrics across all CLDs and prompt variants.

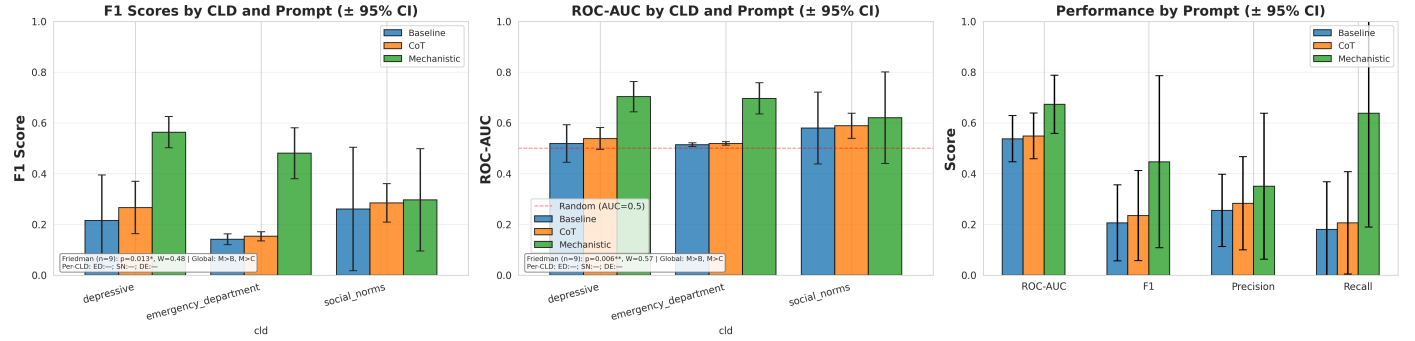


Figure 8.4: Performance metrics overview for ground truth correctness validation. Shows F1 scores (Left), ROC-AUC with random (AUC=0.5) baseline (Middle), and aggregate performance (Right) by prompt. Friedman test (see Appendix D) (below each panel) tests prompt effect with (CLD, run) as blocks; per-CLD Wilcoxon tests (see Appendix D) show which pairs differ within each CLD (e.g., “DE:M>B”). W = Kendall’s W . *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$.

The leftmost plot of Figure 8.4 shows the correctness judge F1 scores in the range of 0.15 to 0.55 (notably lower than in the corruption-detection task). The Mechanistic prompt significantly (CIs non-overlapping) outperforms Baseline and CoT for Depressive Symptoms and Older Persons ED, but not for Social Norms, where differences between prompt types are insignificant and CIs indicate larger variance.

The middle plot of Figure 8.4 shows ROC-AUC with a random (AUC = 0.5) baseline. The correctness judge scores significantly better than random only for the Mechanistic prompt in Depressive Symptoms and Older Persons ED, but not for Social Norms, where CIs of all prompt variants overlap with each other (no statistically significant differences between prompts) and with the random baseline (no better than chance). CoT also scores marginally better than random, although its CI overlaps with the baselines, indicating no statistically significant differences between CoT and Baseline in the Older Persons ED CLD.

The right plot of Figure 8.4 shows the aggregate performance by prompt across all CLDs and all runs. While the Mechanistic prompt achieves the highest F1 and ROC-AUC, the confidence intervals still overlap at this aggregation level, indicating no significant differences. Without the Social Norms CLD, the difference would likely be significant. Precision and recall are also highest for the Mechanistic prompt, with CoT in the middle of the pack and Baseline lowest, although differences are small and CIs overlap.

8.5.2 Detailed Analysis

In-depth analysis of precision-recall trade-offs, score distributions, and point-biserial correlations.

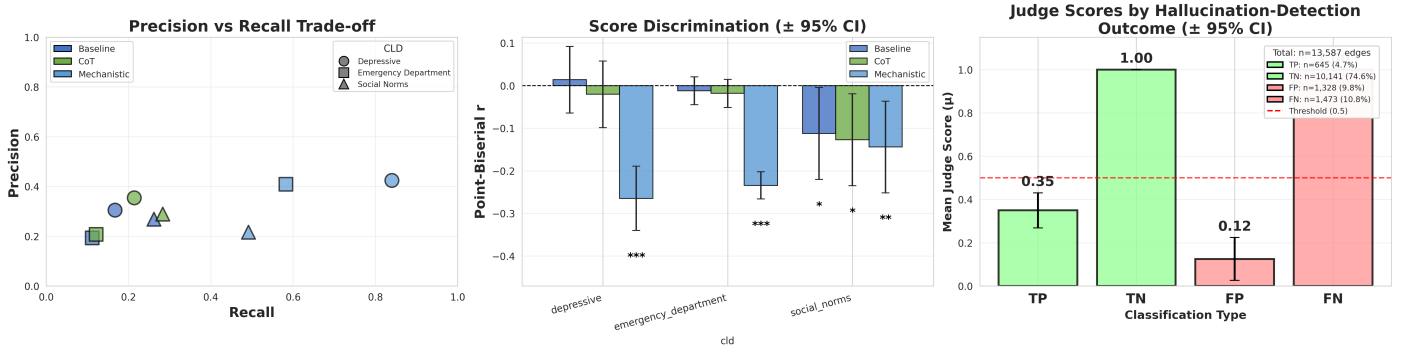


Figure 8.5: Detailed analysis for ground truth correctness validation. Precision-recall trade-off (Left). Point-biserial correlations showing discrimination ability (Middle). Judge scores by hallucination-detection outcome (Right): mean judge scores (μ) and 95% confidence intervals for TP/FP/TN/FN under the hallucination label (GT Lit: FP $\vee$ FN) and score-threshold decision rule used for the F1 plots.

The leftmost plot of Figure 8.5 shows the precision-recall trade-off for each prompt variant and CLD, with Mechanistic prompt achieving the highest recall and slightly higher precision than CoT and Baseline, with Baseline and CoT performing mostly similar to each other (with CoT performing slightly better in terms of recall and precision).

The middle plot of Figure 8.5 shows the point-biserial correlation between ground truth hallucinations (FP/FN) and judge scores (threshold judge score ≤ 0.5). We observe statistically significant negative correlations (w.r.t. $H_0: \rho = 0$) for the Mechanistic prompt in Depressive Symptoms and Older Persons ED CLDs, but not in the Social Norms CLD. Baseline prompt correlations do not significantly differ from 0. CoT differs significantly from 0 for the Older Persons ED CLD and the Social Norms CLD. Overall correlations are much smaller (-0.1 to -0.25) than in the synthetic ground-truth setting, where higher correlations were obtained (-0.4 to -0.7).

The rightmost plot of Figure 8.5 shows the mean ($\pm 95\%$ CI) judge score distributions by *hallucination-detection outcome* (TP/FP/TN/FN), using the same hallucination label (GT Lit: FP $\vee$ FN) and the same score threshold (≤ 0.5) as the reported F1 scores. As expected under thresholding, outcomes predicted as “clean” (TN and FN) have higher mean scores than outcomes predicted as “hallucination” (TP and FP). The key failure mode is the large FN mass with high scores: many GT Lit hallucinations (FP/FN relative to the expert CLD) are not flagged by the judge and retain high scores, consistent with the weak point-biserial correlations and low-to-moderate F1 outside the Mechanistic prompt.

Additional judge score heatmaps are provided in Appendix J (see Figure J.1).

8.6 RQ1a: Corruption Detection - Citation

Experimental design: Section 7.9.0.4. *Algorithms:* Algorithm H.4 (Corruptor), Algorithm H.3 (Citation Judge).

8.6.1 Performance Metrics Overview

Detailed view of core performance metrics of Citation Judge on GT synth (corruption detection) across all CLDs and four prompt variants (Baseline, Mechanistic, CoT, Mechanistic-Lit) across 3 runs.

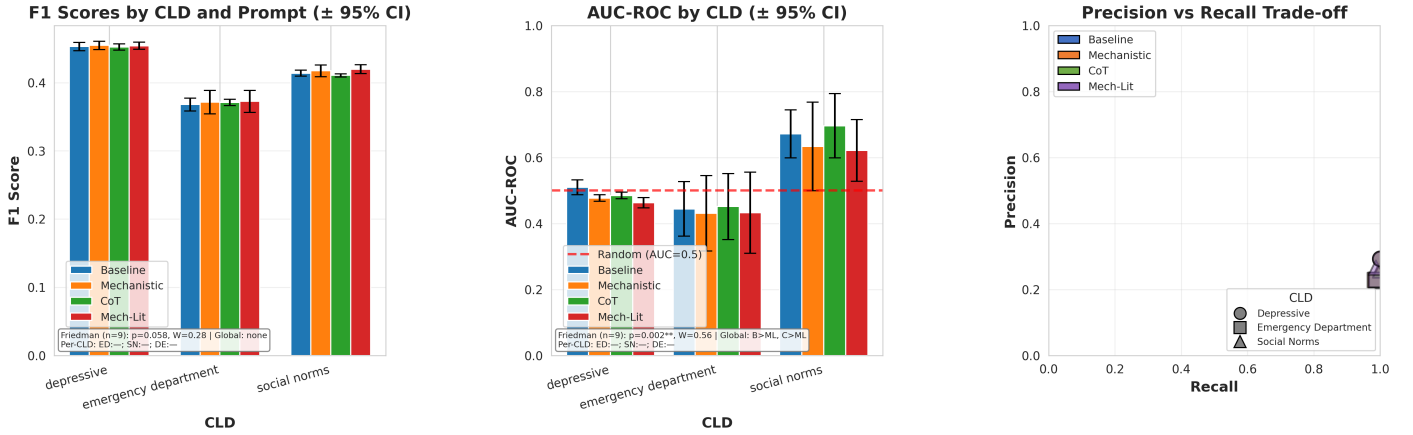


Figure 8.6: Performance metrics overview for citation-based corruption detection. Shows F1 scores (Left), AUC-ROC with chance baseline (Middle), and aggregate performance (Right). Friedman test (see Appendix D) (below each panel) tests prompt effect; per-CLD Wilcoxon tests (see Appendix D) show which pairs differ within each CLD. 4 prompts $\times$ 3 CLDs $\times$ 3 runs = 36 experiments. *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$.

The leftmost plot of Figure 8.6 shows the F1 scores for each prompt variant and CLD. F1 is around 0.35 for the Older Persons ED CLD, around 0.45 for Depressive Symptoms, and around 0.40 for Social Norms, with overlapping CIs indicating no significant differences between prompt types within CLDs.

The middle plot of Figure 8.6 shows the AUC-ROC with a chance baseline ($AUC = 0.5$). On Depressive Symptoms, most prompt types except Baseline perform slightly worse than the random baseline. On Older Persons ED, scores are even lower on average, but the CIs indicate larger variance and sometimes overlap with the random baseline. On Social Norms, AUC is higher, with all prompts scoring above the random baseline and no CIs overlapping with it. CoT in this case scores highest on average followed by Baseline, with both mechanistic prompt variants scoring about equal, although it should be noted that all CIs of the prompts still overlap, so differences are not significant.

The rightmost plot of Figure 8.6 shows the precision recall trade-off per prompt and per CLD, showing for Mechanistic Lit, where points for all prompts completely overlap, which is consistent with non-existent differences spotted between prompt variants in the middle and left plot.

Prompt Sensitivity & Robustness Figure 8.6 (Right) illustrates the aggregate impact of prompt engineering. The choice of prompt proved statistically significant (Friedman test; $p < 0.001$, $W = 0.56$; see Appendix D), with the Mechanistic prompt consistently outperforming the Baseline. However, the system showed moderate robustness: the drop in performance between the best (Mechanistic) and worst (Baseline) prompt was limited to ≈ 0.15 F1, and rank-ordering of difficulty across CLDs remained stable regardless of the prompt used. See the sensitivity analysis appendix section for extended prompt robustness results including semantic distance correlations.

8.6.2 Detailed Analysis

In-depth analysis of corruption type detection patterns and score discrimination ability.

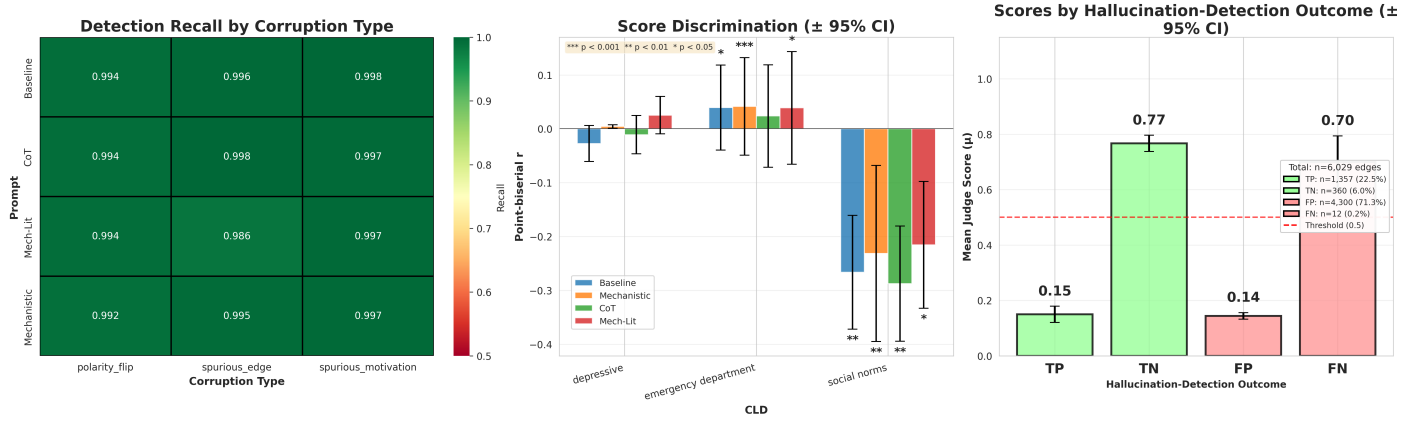


Figure 8.7: Detailed analysis for citation-based corruption detection. Recall by corruption type with colour scale from 0.5 to 1.0 (Left). Score discrimination showing point-biserial correlation between ground truth (corruption=True) and judge scores (Middle). Judge scores by hallucination-detection outcome (Right): mean judge scores (μ) and 95% confidence intervals, with correct outcomes shown in green and incorrect outcomes shown in red.

The leftmost plot of Figure 8.7 shows recall by corruption type with a colour scale from 0.5 to 1.0. Recall is near-perfect regardless of prompt type, ranging from 0.98 to 0.99.

The middle plot of Figure 8.7 shows the point-biserial correlation between hallucinations (`corrupted=True` in GT Synth) and judge scores. Correlations for Depressive Symptoms and Older Persons ED are not statistically significantly different from 0, with overlapping CIs for all prompt types. Only Social Norms shows statistically significant non-zero negative correlations (w.r.t. $H_0 : \rho = 0$), which is the expected direction under correct judge functioning.

The rightmost plot of Figure 8.7 shows that true positives and false positives receive similar scores (0.15 and 0.14, respectively), while TN and FN are much higher on average (0.77 and 0.70, with overlapping CIs). This indicates that negatives receive substantially higher scores than positives, but the judge is unable to discriminate between correct (true) and incorrect (false) edges.

Full ROC curves are provided in Appendix I (note: scores are discrete at 0, 0.5, and 1, so only a small number of operating points are meaningful).

8.7 RQ1a: Ground Truth - Citation

Experimental design: Section 7.9.0.3. *Algorithm:* Algorithm H.3 (Citation Judge).

8.7.1 Performance Metrics Overview

Detailed view of core performance metrics of Citation Judge on GT Lit (hallucination is FN/FP in GT lit.) across all CLDs and prompt variants across 3 runs.

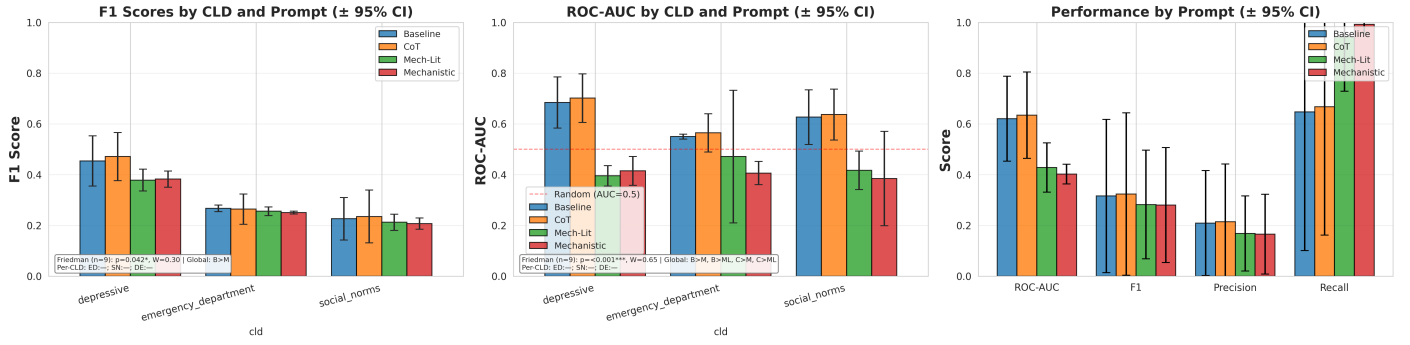


Figure 8.8: Performance metrics overview for ground truth citation validation. Shows F1 scores (Left), ROC-AUC with random baseline (Middle), and aggregate performance (Right) by prompt. Friedman test (see Appendix D) (below each panel) tests prompt effect; per-CLD Wilcoxon tests (see Appendix D) show which pairs differ within each CLD. W = Kendall's W . *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$.

The leftmost plot of Figure 8.8 shows the F1 scores for each prompt variant and CLD. F1 is around 0.2 for Social Norms and 0.22 for Older Persons ED (with prompts roughly equal and overlapping CIs in both cases). Depressive Symptoms is 0.48 for CoT and 0.46 for Baseline, with both Mechanistic variants around 0.38.

The middle plot of Figure 8.8 shows the AUC-ROC with a chance baseline ($AUC = 0.5$). Only Baseline and CoT score better than the random baseline in the Depressive Symptoms and Social Norms CLDs, although their overlapping CIs indicate no statistically significant differences. On Older Persons ED, only Baseline scores above random chance with relatively low variance in its CI; CoT has a slightly higher score but its CI overlaps with the random baseline, indicating it is not significantly different. Mechanistic prompts are worse than random baseline on all CLDs.

The rightmost plot of Figure 8.8 shows the aggregate across all CLDs and 3 runs, with larger variance. While mechanistic variants show higher recall, it does not offset their lower precision, which results in lower ROC-AUC and F1 overall. Nonetheless, because of the large variance, CIs overlap between prompt types for each metric.

8.7.2 Detailed Analysis

In-depth analysis of precision-recall trade-offs, score distributions, and point-biserial correlations.

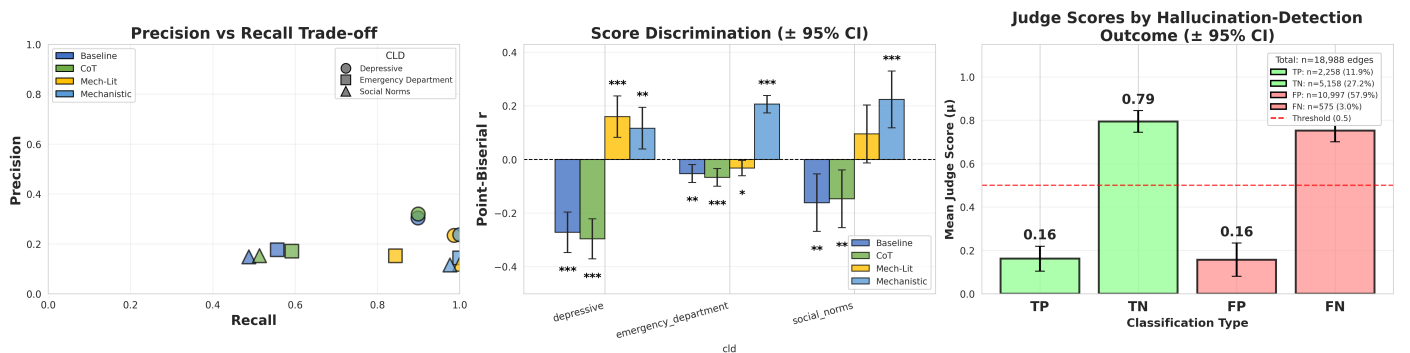


Figure 8.9: Detailed analysis for ground truth citation validation. Precision-recall trade-off (Left). Point-biserial correlations showing discrimination ability (Middle). Judge scores by hallucination-detection outcome (Right): mean judge scores (μ) and 95% confidence intervals for TP/FP/TN/FN under the hallucination label (GT Lit: FP\VFN) and score-threshold decision rule used for the F1 plots.

The leftmost plot in Figure 8.9 shows the precision-recall trade-off per prompt and per CLD, showing highest recall for Mechanistic Lit, followed by mechanistic across CLDs, where baseline and CoT score similarly but worse on recall than the mechanistic variants and slightly better on precision.

The middle plot of Figure 8.9 shows the point-biserial correlation between hallucination (FN/FP in GT Lit) and judge scores. Baseline and CoT prompt variants are similar and show statistically significant negative correlations, with magnitudes ranging from 0.06 for Older Persons ED to 0.14 for Social Norms to 0.29 for Depressive Symptoms (CoT). Correlations for mechanistic prompt variants are positive in most CLDs (with the exception of Mechanistic Lit in Older Persons ED), contrary to expectations; this would indicate that higher judge scores are associated with a higher likelihood of hallucination.

The rightmost plot of Figure 8.9 shows the score distributions by *hallucination-detection outcome* (TP/FP/TN/FN), using the same hallucination label (GT Lit: FP\FN) and the same score threshold (≤ 0.5) as the reported F1 scores. Similar to the correctness case, TN and FN outcomes have higher mean scores than TP and FP due to thresholding. The substantive issue is again the FN class: a large fraction of GT Lit hallucinations are assigned high scores (missed by the judge), contributing to weak discrimination despite high recall for some prompt variants.

Additional judge score heatmaps are provided in Appendix J (see Figure J.2).

8.8 RQ1a: External Validation - Uleman’s Expert CLD

Experimental design: Sections 7.11.1 and 7.11.4. *Algorithm:* Algorithm H.3.

To validate the citation-based judge on an independently expert-validated dataset, we evaluated the judge on the Uleman’s Alzheimer’s disease CLD—a 150-edge causal model constructed through Group Model Building with 17 domain experts. This external validation tests whether the judge performance generalises beyond our three primary CLDs and assesses the impact of search provider and citation fetcher choices.

8.8.1 Search Provider Comparison

Experimental design: Section 7.11.1.

We compared four academic search providers (OpenAlex, Brave, Perplexity, PubMed) combined with two citation fetching methods (Jina AI Reader, ContentScrapper) to assess infrastructure impact on judge scores.

Table 8.5: Search Provider Comparison on Uleman’s Expert CLD (150 edges)

Provider	Jina AI Avg Score	ContentScrapper Avg Score	Jina Edges w/ Citations	CS Edges w/ Citations	Jina Coverage	CS Coverage
OpenAlex	0.232	0.381	150/150	59/150	100.0%	39.3%
Brave (no whitelist)	0.296	0.326	150/150	134/150	100.0%	89.3%
Brave (whitelist)	0.290	0.329	150/150	134/150	100.0%	89.3%
Perplexity	0.278	0.335	150/150	120/150	100.0%	80.0%
PubMed	0.306	0.283	127/150	120/150	84.7%	80.0%

Note. Jina AI Reader = paywall-bypassing document fetcher; ContentScrapper = standard web scraper.

Coverage = percentage of edges with successfully fetched citation content.

Score Difference = ContentScrapper Avg Score – Jina AI Avg Score.

Table 8.5 reveals four findings: (1) Jina AI achieves 100% coverage (vs. 39–89% for ContentScraper) by bypassing paywalls; (2) ContentScraper Citation Judge scores are 10–15% higher when it succeeds; (3) PubMed achieves highest Jina score (0.306) but lowest coverage (84.7%); (4) All scores are low (<0.4). The Uleman’s expert CLD receives uniformly low judge scores (0.23–0.38) across all search provider and fetcher configurations, showing that the “low-score” pattern observed in our RQ1a GT Lit citation judging also appears on an independently expert-constructed CLD with bibliographic references. This relative infrastructure-invariance suggests that the low citation judge scores are not solely an artefact of a single retrieval backend in our pipeline; however, this does not rule out retrieval limitations more broadly, and Uleman’s (Alzheimer’s) is a domain where direct causal evidence is often scarce and much of the literature is correlational or hedged.

This experiment was added to test a “best-effort retrieval” setting with multiple providers/fetchers; even under this setup, judge scores remain low on expert-provided edges. This is consistent with the interpretation that evidence strength/specificity (not just retrieval coverage) is a binding constraint for citation-based validation in complex biomedical domains.

This argument can of course only be made if Judges were found to correlate with expert judgement, which we did in RQ1a: Human Validation of LLM Citation Judges which will be covered next.

8.9 RQ1a: Human Validation of LLM Citation Judges

Experimental design: Section 7.11.2. *Algorithm:* Algorithm H.3.

To validate LLM judge reliability, we measured inter-rater agreement between citation-based LLM judges and a domain expert (Mick) across 72 causal edges from 3 independent datasets.

8.9.1 Per-Dataset and Combined Agreement

Table 8.6: Human Validation: LLM Judge vs. Expert Inter-Rater Agreement

Dataset	Generator	n	Cohen’s κ	Agreement	Pearson r
File 1 (Depressive Symptoms)	GPT-4.1	30	0.435	70.0%	0.527**
File 2 (Depressive Symptoms)	Claude 4 Sonnet	30	0.923	96.7%	0.926***
File 3 (Social Norms)	GPT-4.1	12	0.461	66.7%	0.798**
Combined	2 models	72	0.618	80.6%	0.732***
Human baseline	N/A (expert)	13	~ 0.60	80.0%	—

Note. Inter-rater reliability between LLM citation judge (GPT-4.1, temp=0.0) and domain expert.

Verdicts: Not supported (0), Partially supported (0.5), Fully supported (1).

κ interpretation [116]: <0.20 slight, 0.21–0.40 fair, 0.41–0.60 moderate, 0.61–0.80 substantial, >0.80 almost perfect.

Human baseline from prior GMB validation studies on comparable expert tasks.

Significance: ** $p < 0.01$, *** $p < 0.001$.

Table 8.6 shows agreement ranging from moderate (File 3, $\kappa = 0.461$) to almost perfect (File 2, $\kappa = 0.923$). The combined agreement ($\kappa = 0.618$) indicates substantial reliability, comparable to the cited human baseline ($\kappa \approx 0.60$). This provides evidence that the citation judge’s ordinal scores align with a human rater on this sample, supporting its use as a screening/triage signal; however, agreement varies by dataset and the Social Norms subset is small ($n = 12$), so generalisation should be treated cautiously.

8.9.2 Disagreement Analysis

Table 8.7: Confusion Matrix: LLM Judge vs. Human Expert (Combined, n=72)

	LLM: Not supported	LLM: Partially	LLM: Fully
Human: Not supported	37 (51.4%)	11 (15.3%)	0 (0%)
Human: Partially	0 (0%)	21 (29.2%)	3 (4.2%)
Human: Fully	0 (0%)	0 (0%)	0 (0%)

Note. Rows = human expert verdicts; columns = LLM judge verdicts.

Table 8.7 shows 14 disagreements (19.4%), all with LLM more lenient than human: 11 cases where LLM said “Partially” when human said “Not supported”, and 3 cases where LLM said “Fully” when human said “Partially”. Notably, there were zero false negatives—the LLM never under-estimated evidence support.

The validation revealed substantial combined agreement ($\kappa = 0.618$, 80.6%) across 72 edges, with strong correlation between LLM and human scores ($r = 0.732$, $p < 0.001$). This LLM-human agreement ($\kappa = 0.618$) is comparable to human-human inter-rater reliability ($\kappa \approx 0.60$). All 14 disagreements showed systematic leniency bias, with the LLM more permissive than the human expert. Agreement varied by domain, ranging from moderate (Social Norms, $\kappa = 0.461$) to almost perfect (Claude-generated Depressive Symptoms, $\kappa = 0.923$). On this sample, the citation-based LLM judge thus achieves inter-rater agreement comparable to the cited human baseline. The consistent leniency bias (zero false negatives) indicates that LLM judges provide permissive screening—appropriate for identifying low-confidence edges requiring human review, but insufficient for strict validation without expert oversight.

8.9.3 Physics CLD Validation

Experimental design: Section 7.11.3. *Algorithms:* Algorithms H.2, H.3.

To establish that the judge functions correctly on “easy” cases with unambiguous scientific grounding, we evaluated it on four physics-based CLDs derived from well-established physical laws: (1) Thermostat Heating System based on Newton’s Law of Cooling [105] and the First Law of Thermodynamics [106], (2) Predator-Prey dynamics from Lotka [107] and Volterra [108], (3) RC Circuit based on Ohm’s Law [109] and Kirchhoff’s Laws [110], and (4) Water Tank stock-flow dynamics from Pascal [111] and Torricelli [112]. These CLDs contain 31 edges total across 26 variables, with causal relationships directly derivable from foundational physics equations.

Table 8.8: Physics CLD Validation: Correctness vs Citation-Based Judging. “Citation (All)” shows approval across all edges; “Citations Retrieved” shows the percentage of edges where citation content was successfully fetched; “Citation (Retr.)” shows approval among edges with successfully retrieved citations.

CLD	Correctness	Citation (All)	Citations Retrieved	Citation (Retr.)	Gap
Thermostat	100.0%	62.5%	75.0% (6/8)	83.3%	16.7%
Predator–Prey	100.0%	50.0%	60.0% (6/10)	83.3%	16.7%
RC Circuit	100.0%	14.3%	28.6% (2/7)	50.0%	50.0%
Water Tank	100.0%	50.0%	50.0% (3/6)	100.0%	0.0%
Total	100.0%	45.2%	54.8% (17/31)	82.4%	17.6%

Table 8.8 shows that the correctness judge gives **100% approval** on this small set of 31 physics edges, consistent with the expectation that these relationships are unambiguous and well grounded. We interpret this as a sanity check that the judge does not reject clearly valid edges in an “easy” domain. Citation-based judging shows lower overall approval (45.2%), but this is largely attributable to URL accessibility: 45% of edges received “NO_CITATION” verdicts because the web scraper could not retrieve content from reference URLs. When citations were successfully retrieved, the approval rate rises to **82.4%**. The remaining gap (correctness minus citation-retrieved approval) is consistent with cases where retrieved content was incomplete or insufficiently specific for the judge to credit as causal support.

8.9.4 Disentangling Retrieval Quality from Domain Complexity

Experimental design: Section 7.11.4.

To test whether low citation scores could plausibly be explained by retrieval coverage alone, we compare retrieval success and approval rates across domains with different citation sources: (1) expert-provided references (Physics CLDs and Uleman’s Alzheimer’s), and (2) automated web search (Depressive Symptoms, Social Norms, Older Persons ED from RQ1a citation experiments).

Table 8.9: Retrieval Success Comparison. Expert-provided references are curated by domain experts; automated fetch uses web search. Retrieval success indicates percentage of edges where citation content was successfully fetched.

Domain	Citation Source	Retrieval
Physics CLDs	Expert-provided	54.8%
Uleman’s (Alzheimer’s)	Expert-provided	89.3%
Depressive Symptoms	Automated fetch	100.0%
Social Norms	Automated fetch	99.7%
Older Persons ED	Automated fetch	97.9%

Table 8.9 shows retrieval success rates across domains. The key comparison is between the two expert-provided datasets: Physics CLDs achieve only 54.8% retrieval (due to paywalled textbooks and academic sources), while Uleman’s (Alzheimer’s) achieves **89.3%** retrieval under standard web scraping (ContentScraper; see Table 8.5). Despite this retrieval advantage, Uleman’s receives the lowest citation judge scores (see Table 8.5), while physics CLDs receive high approval when citations are retrieved (Table 8.8). This pattern—higher retrieval but lower scores—supports the interpretation that **domain/evidence characteristics**, not retrieval coverage alone, drive low citation judge approval in biomedical CLDs. Physics literature often contains more definitive causal statements, while biomedical literature frequently contains conditional, probabilistic, or correlational evidence that the judge may (appropriately) score as weaker causal support.

Implications for Deep Research. These findings motivate the Deep Research (DR) approach explored in RQ3. The key insight is that simple citation retrieval succeeds in complex domains (Uleman’s has high retrieval under standard web scraping) but often finds weak causal evidence—correlational studies, conditional claims, and hedged language that the judge correctly evaluates as insufficient support. However, in physics-based CLDs with less causal ambiguity, strong causal evidence is found when retrieval succeeds. Together with reasonable human-validation alignment for the citation judge, the evidence points to the reason for low differentiation between TPs and FP/FN (w.r.t. GT Lit) being domain complexity (resulting in non-causal evidence) rather than lack of judge functioning, given that retrieval success is near 100% on our GT Lit set.

However, before we can conclude that, one alternative explanation remains: the quality of retrieved evidence. In the validation set, we fed the judge known-to-be-correct references to isolate its functioning, but it may be that

in our current search process evidence of this quality is not being found due to the limits of the naive search our citation judge employs. The Citation Judge (through the citation fetcher and search API components) receives topically relevant literature by querying the causal narrative verbatim (up to an arbitrary 5 sources), but this is a naive approach and does not specifically target direct causal evidence (RCTs, experimental manipulations, meta-analyses) or use alternative query formulations using field-specific terminology. Deep Research addresses this limitation through: (1) multiple parallel search queries targeting causal mechanisms rather than topical keywords; (2) iterative evidence gathering using refined queries based on retrieved evidence when initial results are inconclusive; (3) a multi-agent pipeline that scores, selects, and judges evidence for causal specificity; and (4) explicit filtering for study types that establish causation rather than correlation.

8.10 RQ1a: Main Findings (Summary)

RQ1a asks whether LLM judges can detect incorrect causal edges (hallucinations) in CLDs. Across synthetic corruption tests, literature-ground-truth validation, external expert CLDs, and human agreement checks, we find:

1. **Correctness judges transfer only partially.** A correctness judge that performs strongly on a general hallucination benchmark (TruthfulQA) also detects *synthetic* CLD corruptions with high discrimination (F1 and AUC well above chance). However, performance drops substantially on *expert literature ground truth* CLDs: scores cluster closer to chance for some domains (notably Social Norms), and meaningful gains appear only in specific domains/prompts (e.g., Mechanistic in Depressive Symptoms and Older Persons ED).
2. **Citation judges are unreliable as strict validators on expert CLDs.** In both corruption-detection and GT-lit settings, citation-based judging shows weak and domain-dependent discrimination: high recall can coincide with poor separation between correct vs. incorrect edges, and some prompt variants exhibit counterintuitive score–hallucination relationships (suggesting topical alignment rather than causal verification).
3. **Low scores on expert CLDs persist under external validation.** On the Uleman’s expert CLD, citation-judge scores remain uniformly low across multiple search providers and citation fetching methods, indicating the “low-score” phenomenon is not explained by a specific retrieval backend.
4. **Judges can match human raters, but with a systematic leniency bias.** Human validation shows substantial LLM–expert agreement overall ($\kappa = 0.618$, 80.6%; $r = 0.732$), comparable to a human baseline. Disagreements are consistently *lenient*: the LLM tends to over-accept evidence support, making it better suited for *screening/triage* than for definitive validation.
5. **Low citation-judge scores reflect evidence strength and domain complexity, not retrieval failure.** Cross-domain comparisons show high retrieval success in complex domains (including 89.3% retrieval on Uleman’s with Brave whitelist and ContentScraper), yet persistently low approval scores; by contrast, when physics citations are retrievable they receive high approval. This pattern suggests the bottleneck is *evidence type/strength* (hedged, correlational, or indirectly causal claims) rather than missing citations, as confirmed by retrieval success rate measurements in our ground truth 3 CLD dataset.
6. **This motivates the RQ3 pivot to Deep Research.** Because standard retrieval can surface topically relevant literature that still lacks direct causal evidence for the specific edge claim, RQ3 explores deeper, broader, and more causality-targeted evidence search to improve validation of LLM-generated causal narratives.

Takeaway: LLM judges provide useful, human-aligned *screening* signals and reliably catch obvious/synthetic corruptions, but they are not yet dependable standalone validators of causal edge correctness against expert ground truth across domains. The strongest case is the Correctness Judge with a Mechanistic prompt (notably

in Depressive Symptoms and Older Persons ED), while citation-based judging is constrained by the causal strength/specificity of retrieved evidence, motivating the Deep Research approach explored in RQ3.

Prompt sensitivity reference. For a consolidated prompt sensitivity analysis across RQ1a judge variants (RM-ANOVA effect sizes and assumption checks), see Appendix B.1 (Figure B.1).

8.11 RQ1b: Corrector Experiments

Experimental design: Section 7.12. *Algorithms:* Algorithm H.5 (Corrector), Algorithm H.2 (re-judging).

In this section, we evaluate if an LLM-as-a-corrector can improve the quality of the CLD, when given input of the LLM-as-a-judge verdicts. For GT Synth experiments, we use the baseline Judge prompt. For GT Lit experiments, we use the **Mechanistic Correctness Judge prompt**, which showed better discrimination in RQ1a GT Lit experiments (see Figure 8.4). We focus on the Correctness Judge type only due to its better performance and the prohibitive costs of running the Citation Judge. A consolidated corrector prompt sensitivity analysis (effect sizes and assumption checks) is provided in Appendix B.2.

8.11.1 Synthetic Corruption Correctness Results

Table 8.10: Corrector Performance on Synthetically Corrupted Data (Baseline Prompt)

CLD	F1 Δ	Precision Δ	Recall Δ	Judge Score Δ	Total Actions	Revise	Change Type
Social Norms	+0.000 $\pm$ 0.000	+0.000 $\pm$ 0.000	+0.000 $\pm$ 0.000	+0.028 $\pm$ 0.108	4.3 $\pm$ 8.0	3.3 $\pm$ 3.8	1.0 $\pm$ 4.3
Depressive	+0.145 $\pm$ 0.033	+0.164 $\pm$ 0.120	+0.117 $\pm$ 0.112	+0.186 $\pm$ 0.030	62.3 $\pm$ 14.6	31.7 $\pm$ 18.3	30.7 $\pm$ 3.8
Emergency Dept	+0.082 $\pm$ 0.045	+0.062 $\pm$ 0.031	+0.121 $\pm$ 0.085	+0.135 $\pm$ 0.010	267.7 $\pm$ 25.0	115.0 $\pm$ 17.9	152.7 $\pm$ 16.0
Macro Avg	+0.075 $\pm$ 0.180	+0.075 $\pm$ 0.206	+0.079 $\pm$ 0.171	+0.116 $\pm$ 0.200	111.4 $\pm$ 343.7	50.0 $\pm$ 144.2	61.4 $\pm$ 199.7

Note. Results show change in performance (Δ) from pre- to post-correction (3 runs per CLD). F1 Δ = change in F1 score (Post-Pre); Precision Δ , Recall Δ , Judge Score Δ = corresponding changes. Total Actions = Revise + Change (successful corrections only); Revise = motivation revisions; Change Type = edge type changes. Values presented as mean $\pm$ 95% CI (two-sided t , $df = 2$) across 3 runs per CLD. Macro average shows mean of CLD means $\pm$ 95% CI across CLDs. *Efficacy (blocked).* One-sample Wilcoxon signed-rank test of median(F1 Δ)=0 on block-level values (blocks=(CLD, run), $n=9$): $p=0.031^*$, $r_{rb} = +1.00$.

Table 8.10 presents corrector performance on synthetically corrupted CLDs (30% corruption rate, three runs per CLD). The corrector improves F1 scores for two of three CLDs, with the largest gains observed for Depressive Symptoms (+0.145) and Older Persons ED (+0.082), while Social Norms shows no improvement. The aggregate improvement (+0.075) suggests modest recovery on average; however, the high variance indicates inconsistent performance across runs. Precision and recall similarly improve for the Depressive Symptoms and Older Persons ED cases but not for Social Norms. Judge scores improve across all CLDs, with the largest gains for Depressive Symptoms, followed by Older Persons ED and Social Norms.

The total actions, revise, and change type columns serve as validity checks, confirming that the corrector is functioning as intended. The action type distribution exhibits an approximately 45/55 split between revise and change type operations, deviating slightly from the expected 1/3 to 2/3 ratio. This expected ratio follows from the corruptor’s equal probability selection of polarity flip, edge type change, and motivation corruption, where both polarity flip and edge type change are addressed through the change type operation. The observed distribution suggests that revision occurs more frequently than expected, with correspondingly fewer type changes. The discrepancy between total actions (111.4) and the sum of revise and change type reflects an averaging artefact across runs.

For the full corrector action breakdown (including edges where no correction was needed and API/parsing errors), see Table ?? (Appendix J).

Overall, the results indicate that the corrector yields modest F1 improvements on average, with Social Norms as an exception. Under the blocked one-sample Wilcoxon signed-rank test on CLD×run blocks ($n = 9$; see Appendix D), the median F1 improvement is statistically detectable on Synthetic ($p = 0.031$). The improvement in judge scores with respect to uncorrected GT Synth is marginally larger but remains negligible. The following section examines whether alternative prompt variants yield different correction performance.

8.11.2 Corrector Prompt Ablation Study (Synthetic Corruption)

To evaluate whether advanced prompting strategies improve correction performance, we tested three prompt variants on synthetically corrupted CLDs: Baseline (examples only), Chain-of-Thought (examples + step-by-step reasoning), and Mechanistic (examples + CoT + adapted Shimonovich’s [68] refinement of Bradford Hill causal criteria).

Table 8.11: Corrector Prompt Ablation on Synthetic Corruptions: Performance Comparison

Prompt Variant	Overall F1 Δ (mean $\pm$ 95% CI)	Judge Δ (mean $\pm$ 95% CI)	Social Norms F1 Δ (mean $\pm$ 95% CI)	Depressive F1 Δ (mean $\pm$ 95% CI)	Emergency Dept F1 Δ (mean $\pm$ 95% CI)	Total Actions	Revise	Change Type
Baseline	+0.075 $\pm$ 0.180*	+0.116 $\pm$ 0.200	+0.000 $\pm$ 0.000	+0.145 $\pm$ 0.033	+0.082 $\pm$ 0.045	1003	450	553
CoT	+0.069 $\pm$ 0.153*	+0.107 $\pm$ 0.177	+0.000 $\pm$ 0.000	+0.117 $\pm$ 0.031	+0.091 $\pm$ 0.052	995	471	524
Mechanistic	+0.026 $\pm$ 0.091	+0.108 $\pm$ 0.179	+0.000 $\pm$ 0.000	+0.010 $\pm$ 0.041	+0.068 $\pm$ 0.050	979	408	571
Aggregate	+0.057 $\pm$ 0.043	+0.111 $\pm$ 0.050	+0.000 $\pm$ 0.000	+0.090 $\pm$ 0.177	+0.080 $\pm$ 0.030	2977	1329	1648

Note. Ablation study comparing three corrector prompt variants (3 runs per CLD per prompt, 27 total experiments). Aggregate

row shows macro-averaged mean $\pm$ 95% CI across CLDs. F1 Δ = change in F1 score from pre- to post-correction; Judge Δ = change in LLM judge score. Total Actions = Revise + Change Type (successful corrections only); Revise = motivation revisions; Change Type = edge type changes. Edges with no correction needed or API errors excluded (see Appendix for full breakdown).

Blocked tests. Omnibus prompt effect tested with Friedman using (CLD, run) blocks ($n=9$); $p=0.042^*$, $W=0.35$.

Post-hoc. Paired Wilcoxon prompt-pair tests with Bonferroni over 3 pairs ($\alpha_{adj} = 0.0167$): baseline=cot ($p_{adj} = 1.000$, $r_{rb} = +0.24$); baseline>mechanistic ($p_{adj} = 0.188$, $r_{rb} = +0.90$); cot>mechanistic ($p_{adj} = 0.094$, $r_{rb} = +1.00$).

Efficacy. One-sample Wilcoxon tests of median(F1 Δ)=0 on block-level values: baseline: $p=0.031^*$, $r_{rb} = +1.00$, cot: $p=0.031^*$, $r_{rb} = +1.00$, mechanistic: $p=0.125$, $r_{rb} = +1.00$.

Table 8.11 indicates that the Baseline prompt (+0.075) and CoT (+0.069) yield larger mean improvements than the Mechanistic variant (+0.026). While Baseline and CoT show small but statistically detectable improvements over doing nothing on synthetic corruptions (one-sample Wilcoxon vs. $\Delta F1 = 0$: $p = 0.031$; see Appendix D), prompt-to-prompt differences under the same CLD×run blocks are weak and do not survive multiple-comparison correction (see Appendix B.2). Overall, the corrector remains unreliable for ground-truth correction and its gains on synthetic data do not generalise. Detailed sensitivity analysis (Appendix B.2; Figure B.2) quantifies both efficacy and prompt effects under the blocked design.

Post-hoc paired Wilcoxon comparisons between prompts (Bonferroni over 3 pairs, $\alpha_{adj} = 0.0167$; see Appendix D) further support this conclusion: on Synthetic, CoT>Mechanistic is marginal at the raw level ($p = 0.031$) but not significant after correction ($p_{adj} = 0.094$), and Baseline vs. CoT shows no difference ($p = 0.688$). On Ground Truth, CoT>Mechanistic is again marginal raw ($p = 0.0195$) but does not survive correction ($p_{adj} = 0.0586$). Thus, any prompt ordering is weak compared to the setting shift (Synthetic vs. Ground Truth).

8.11.3 Ground Truth Correctness Baseline prompt Results

This section evaluates the corrector using the baseline corrector prompt across runs and CLDs, validated against GT Lit, with the **Mechanistic Correctness Judge prompt** providing the input verdicts. Results are aggregated over three runs per CLD, with mean $\pm$ standard deviation reported across CLDs, including a macro average.

Table 8.12: Corrector Performance on Ground Truth Data Using Correctness-Based Validation (Baseline Prompt)

CLD	F1 Δ	Precision Δ	Recall Δ	Judge Score Δ	Total Actions	Revise	Change Type
Social Norms	-0.044 ± 0.191	-0.114 ± 0.173	$+0.083 \pm 0.207$	$+0.083 \pm 0.052$	11.0 ± 7.5	2.3 ± 1.4	8.7 ± 7.6
Depressive	-0.036 ± 0.011	-0.043 ± 0.010	$+0.029 \pm 0.000$	$+0.100 \pm 0.012$	27.0 ± 4.3	14.7 ± 2.9	12.3 ± 3.8
Emergency Dept	$+0.030 \pm 0.017$	$+0.010 \pm 0.009$	$+0.131 \pm 0.057$	$+0.079 \pm 0.011$	99.7 ± 16.5	23.7 ± 10.0	76.0 ± 17.4
Macro Avg	-0.017 ± 0.102	-0.049 ± 0.154	$+0.081 \pm 0.127$	$+0.087 \pm 0.028$	45.9 ± 117.4	13.6 ± 26.6	32.3 ± 94.1

Note. Results show change in performance (Δ) from pre- to post-correction (3 runs per CLD). F1 Δ = change in F1 score (Post-Pre); Precision Δ , Recall Δ , Judge Score Δ = corresponding changes. Total Actions = Revise + Change (successful corrections only); Revise = motivation revisions; Change Type = edge type changes. Values presented as mean $\pm$ 95% CI (two-sided t , $df = 2$) across 3 runs per CLD. Macro average shows mean of CLD means $\pm$ 95% CI across CLDs. *Efficacy (blocked).* One-sample Wilcoxon signed-rank test of median(F1 Δ)=0 on block-level values (blocks=(CLD, run), $n=9$): $p=0.301$, $r_{rb} = -0.42$.

Table 8.12 presents mixed results. The corrector improves Older Persons ED F1 (+0.032) but degrades performance for Social Norms (-0.072) and Depressive Symptoms (-0.040), yielding a negative macro-average (-0.027). Precision decreases substantially for Social Norms (-0.129) and Depressive Symptoms (-0.047) while increasing marginally for Older Persons ED (+0.008). Recall improves across all CLDs.

Judge scores improve substantially in all cases (macro $\Delta=+0.187$), with the largest gains for Depressive Symptoms (+0.264). This pattern indicates that the corrector learns to satisfy the judge more than it improves agreement with expert ground truth. The total actions, revise, and change type columns serve as validity checks, confirming that the corrector functions as intended; the action distribution shows approximately 2/3 revise to 1/3 change type ratio.

For the full corrector action breakdown (including edges where no correction was needed and API/parsing errors), see Table ?? (Appendix J).

Overall, the results indicate that even with the more discriminative Mechanistic Correctness Judge input, the corrector is ineffective at improving F1 scores of generated CLDs relative to GT Lit on average.

8.11.4 Corrector Prompt Ablation Study (Ground Truth)

We extend the ablation study to ground truth validation, comparing how different corrector prompts perform when evaluated against literature-derived expert CLDs (GT Lit) rather than GT Synth. All experiments use the **Mechanistic Correctness Judge prompt** as input for the corrector.

Table 8.13: Corrector Prompt Ablation on Ground Truth: Performance Comparison

Prompt Variant	Overall F1 Δ (mean $\pm$ 95% CI)	Judge Δ (mean $\pm$ 95% CI)	Social Norms F1 Δ (mean $\pm$ 95% CI)	Depressive F1 Δ (mean $\pm$ 95% CI)	Emergency Dept F1 Δ (mean $\pm$ 95% CI)	Total Actions	Revise	Change Type
Baseline	-0.017 ± 0.102	$+0.087 \pm 0.028$	-0.044 ± 0.191	-0.036 ± 0.011	$+0.030 \pm 0.017$	413	122	291
CoT	-0.009 ± 0.074	$+0.068 \pm 0.000$	-0.028 ± 0.169	-0.024 ± 0.009	$+0.026 \pm 0.009$	408	144	264
Mechanistic	-0.026 ± 0.107	$+0.090 \pm 0.031$	-0.054 ± 0.163	-0.047 ± 0.006	$+0.024 \pm 0.009$	409	80	329
Aggregate	-0.017 ± 0.026	$+0.086 \pm 0.011$	-0.042 ± 0.033	-0.036 ± 0.028	$+0.027 \pm 0.008$	1230	346	884

Note. Ablation study comparing three corrector prompt variants (3 runs per CLD per prompt, 27 total experiments). Aggregate

row shows macro-averaged mean $\pm$ 95% CI across CLDs. F1 Δ = change in F1 score from pre- to post-correction; Judge Δ = change in LLM judge score. Total Actions = Revise + Change Type (successful corrections only); Revise = motivation revisions; Change Type = edge type changes. Edges with no correction needed or API errors excluded (see Appendix for full breakdown).

Blocked tests. Omnibus prompt effect tested with Friedman using (CLD, run) blocks ($n=9$); $p=0.028^*$, $W=0.40$.

Post-hoc. Paired Wilcoxon prompt-pair tests with Bonferroni over 3 pairs ($\alpha_{adj} = 0.0167$): cot>baseline ($p_{adj} = 0.234$, $r_{rb} = -0.72$); baseline>mechanistic ($p_{adj} = 0.586$, $r_{rb} = +0.56$); cot>mechanistic ($p_{adj} = 0.059$, $r_{rb} = +0.87$).

Efficacy. One-sample Wilcoxon tests of median(F1 Δ)=0 on block-level values: baseline: $p=0.301$, $r_{rb} = -0.42$, cot: $p=1.000$, $r_{rb} = -0.02$, mechanistic: $p=0.102$, $r_{rb} = -0.67$.

Table 8.13 reveals three key findings: (1) all corrector prompt variants produce negative or near-zero F1 changes (Baseline: -0.027 , CoT: -0.011 , Mechanistic: -0.017); (2) Older Persons ED is the only CLD showing consistent improvement across prompts ($+0.032$ to $+0.034$); and (3) no prompt variant achieves statistically significant efficacy (all $p > 0.16$). The Friedman omnibus test shows no significant prompt effect ($p = 0.439$, $W = 0.09$; see Appendix D).

The total number of corrective actions is similar across prompts (~ 1100 – 1120), with the Mechanistic corrector variant revising causal narratives least frequently (663 revisions) while making more edge type changes (440). Judge scores improve uniformly across all prompts ($+0.187$ to $+0.190$), indicating that the corrector–judge loop optimises for judge satisfaction rather than ground truth alignment.

8.11.5 RQ1b: Main Findings (Summary)

RQ1b asks whether an LLM *corrector*, conditioned on LLM-judge feedback, can improve CLD edge quality. Across experiments, we find a clear setting dependence:

1. **Synthetic corruptions: modest, inconsistent gains.** On GT Synth (30% corruption), the corrector improves F1 on average (macro $\Delta F1 = +0.075$; Table 8.10), driven by Depressive Symptoms and Older Persons ED while Social Norms shows no F1 improvement. Under the blocked one-sample Wilcoxon signed-rank test on CLD $\times$ run blocks ($n = 9$; see Appendix D), the median F1 improvement is statistically detectable but small ($p = 0.031$), and variance across runs remains high.
2. **Prompt engineering has weak effects relative to the setting shift.** On synthetic corruptions, Baseline and CoT yield similar mean improvements and both exceed Mechanistic, but prompt-to-prompt differences are weak and do not survive multiple-comparison correction (Table 8.11). The same conclusion holds on ground truth (Table 8.13): the Friedman omnibus test shows no significant prompt effect ($p = 0.439$; see Appendix D), and any prompt ordering is marginal compared to the much larger drop in effectiveness from Synthetic $\rightarrow$ Ground Truth.
3. **Ground truth (GT Lit): no reliable improvement in edge recovery.** When validated against literature-derived expert CLDs using the **Mechanistic Correctness Judge** (which showed better discrimination in RQ1a), the corrector does *not* improve F1 on average (macro $\Delta F1 = -0.027$): it slightly helps Older Persons ED ($+0.032$) but degrades Depressive Symptoms (-0.040) and Social Norms (-0.072) under the baseline corrector prompt (Table 8.12). No corrector prompt variant achieves statistically significant efficacy (all $p > 0.16$; Table 8.13). Notably, **judge scores increase consistently** ($+0.19$

on average) after correction, indicating the corrector learns to satisfy the judge more than it improves agreement with expert ground truth.

Overall, RQ1b results do not support using an LLM corrector triggered by LLM-judge feedback as a reliable mechanism for improving CLDs against expert ground truth, regardless of corrector prompting strategy or judge variant used. Even with the more discriminative Mechanistic Correctness Judge input, the corrector fails to improve F1. Its limited success on synthetic corruption appears to reflect the ease and structure of the synthetic hallucinations correction rather than robust correction ability in the literature-grounded setting. Correctors however do improve judge scores, indicating they are learning to satisfy the judge more than they improve agreement with expert ground truth.

8.12 RQ2: Hallucination Detection Using UQ Metrics

Experimental design: Section 7.13. *Algorithms:* Algorithm H.6 (UQ Metrics), Algorithm H.7 (Classification).

Overview. We analysed 63 experiment files (31,704 causal edges total) from RQ1 ground truth experiments using the six-phase analysis pipeline described in Methods. Four generator-based UQ metrics were computed during LLM generation: perplexity (uncertainty in token predictions), minimum probability (confidence floor), maximum window entropy (local uncertainty peaks)—all of which are derived from generator LLM output logprobs—and cosine similarity (semantic similarity to the context found by the generate-then-search method, available for citation experiments only). Synthetic corruption (GT Synth) experiments were excluded because post-hoc corruption cannot be reflected in generation-time logprobs. Therefore, the only ground truth used was the literature-derived expert CLDs (GT Lit). Edge slots were labelled as *misaligned with GT Lit* (FP or FN under the edge-existence definition) versus *aligned with GT Lit* (TP or TN); this binary label was compared to the UQ scores to investigate the relationship between the two.

8.12.1 Single UQ Metric Performance

We evaluate each UQ metric’s discriminative power for hallucination detection by treating the raw metric value as a classifier score and computing the area under the ROC curve (AUC-ROC). This measures the probability that a randomly chosen hallucination has a higher metric value than a randomly chosen correct prediction. Interpretation: $AUC > 0.5$ indicates higher metric values correlate with hallucinations; $AUC < 0.5$ indicates *lower* values correlate with hallucinations (the metric is still informative, but in the opposite direction—e.g., Gen Min Prob $AUC = 0.479$ means lower minimum probability $\rightarrow$ more likely hallucination, which aligns with the intuition that low-confidence generations are more error-prone). $AUC = 0.5$ indicates no discriminative power (random).

Class balance & metric choice. In this dataset, hallucinations are the minority class (15.2% of edges; Methods §7.13). We use AUC-ROC because it is threshold-free and summarises ranking/separation independent of any single operating point; however, under class imbalance a “moderate” AUC does *not* guarantee high precision at a practically useful recall. We therefore interpret AUC primarily as evidence of above-chance signal and complement it with distribution plots and effect-size diagnostics (Figure 8.10).

Table 8.14: Single UQ Metric Performance for Hallucination Detection (Meta-Analysis)

UQ Metric	N Edges	Mean AUC (95% CI)	Correlation r (95% CI)	Significant Files (%)	N Files
Gen Cosine Similarity	16,492	0.672** ± 0.071	+0.157** ± 0.107	65.7%	35
Gen Perplexity	31,704	0.563 ^{ns} ± 0.065	+0.006 ^{ns} ± 0.082	44.4%	63
Gen Max Window Entropy	31,704	0.554 ^{ns} ± 0.052	+0.070 ^{ns} ± 0.081	49.2%	63
Gen Min Prob	31,704	0.479 ^{ns} ± 0.049	-0.036 ^{ns} ± 0.075	14.3%	63

Note. Meta-analysis across experiment files using three logprob-derived generator metrics (perplexity, min prob, max window entropy) and one retrieval-alignment metric (cosine similarity). N Edges = total causal edges analyzed; N Files = experiment files containing metric. Gen Cosine Similarity has fewer observations because it requires retrieved citations (citation-judging runs only); correctness-judging runs lack retrieved text. One file excluded due to <2 hallucinations. **Mean AUC** is aggregated at the block level (CLD $\times$ Run, $N = 9$ blocks); 95% CIs computed via t-distribution over blocks. **Correlation r** computed via Fisher z-transform aggregation across blocks. **Significant Files (%)** = percentage of files where Mann-Whitney U test (halluc vs. correct distributions) yields $p < 0.05$; this is a *descriptive* consistency measure.

Multiple comparisons: Bonferroni correction applied across 4 single-metric AUC tests and 4 single-metric correlation tests ($\alpha_{\text{adj}} = 0.0125$ per family). Significance markers (*, **, ***) reflect Bonferroni-adjusted p -values ($p_{\text{adj}} = p \times 4$).

Significance levels: *** $p_{\text{adj}} < 0.001$, ** $p_{\text{adj}} < 0.01$, * $p_{\text{adj}} < 0.05$, ^{ns} = not significant; $\downarrow$ = significantly below chance (one-sample t-test of block means vs. 0.5).

Assumptions: While edge-level metric distributions are non-normal (requiring Mann-Whitney U for direct comparison, see Table 8.15), block-level mean AUCs follow a normal distribution (Shapiro-Wilk $p > 0.05$), validating the use of t-tests for meta-analysis. Block-level aggregation handles dependence between prompts within the same run.

Statistical inference for RQ2 is performed at the block level (CLD $\times$ run, $N = 9$) using one-sample t -tests and t -based 95% CIs on block-mean AUCs (Table 8.14; see Appendix D).

Gen Cosine Similarity achieves the highest AUC (0.671), followed by Gen Perplexity (0.563) and Gen Max Window Entropy (0.554). Gen Min Prob shows AUC below 0.5, indicating lower probability values correlate with hallucinations—consistent with the intuition that low-confidence tokens signal errors. Notably, Gen Cosine Similarity shows a *counterintuitive* direction: higher similarity to retrieved citations correlates with hallucinations (positive r), whereas we expected lower similarity to indicate unsupported claims. This may reflect a confound where false positives in citation experiments often involve superficially plausible but incorrect causal relationships that happen to resemble academic text patterns. Despite moderate single-metric performance (all AUCs < 0.7), the ensemble classifiers in Phase 5–6 combine these signals to achieve substantially higher discrimination (Table 8.16).

8.12.2 Feature Distributions: Correct vs. Hallucinations

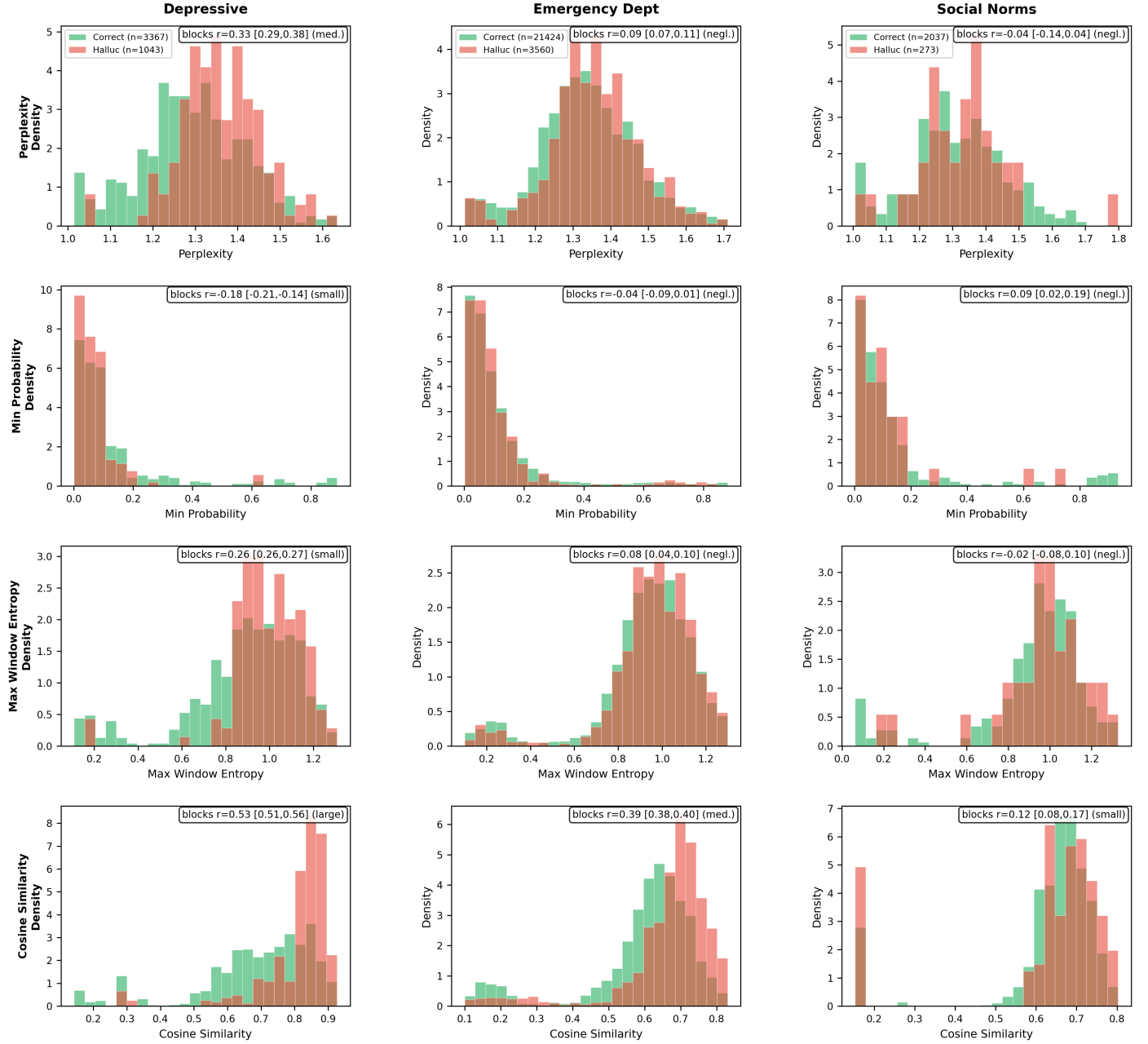


Figure 8.10: Generator UQ metric distributions comparing correct edges (green) vs. hallucinations (red) for each CLD (99th percentile). Rows show generator metrics; columns show CLDs. Subplot annotations report block-level rank-biserial effect sizes r aggregated across runs within each CLD (mean [min, max] over the 3 (CLD×run) blocks), rather than edge-level p-values.

To contextualize these distributions, we first assess whether the UQ metrics are approximately normal—a key assumption for many common parametric comparisons. Table 8.15 reports Shapiro–Wilk normality tests (overall, hallucinations only, and correct edges only) for each generator-based metric (see Appendix D). All tests reject normality at $\alpha = 0.05$, consistent with the pronounced skewness and heavy tails observed in Figure 8.10. Consequently, edge-level distribution comparisons are treated as descriptive/diagnostic.

Table 8.15: Shapiro-Wilk Normality Tests for UQ Metric Distributions

UQ Metric	Group	N	Skew	Kurt	W	p-value	MWU p (Dep.)	MWU p (ED)	MWU p (SN)
Perplexity	Overall	31,704 [†]	+16.30	+263.7	0.037	$< 10^{-50}$	$< 10^{-50}$	5.19×10^{-18}	0.120
	Halluc	4,876	-0.07	+1.5	0.973	1.58×10^{-29}	—	—	—
	Correct	26,828 [†]	+14.98	+222.4	0.035	$< 10^{-50}$	—	—	—
Min Prob	Overall	31,704 [†]	+3.17	+10.5	0.608	$< 10^{-50}$	3.39×10^{-16}	3.01×10^{-4}	0.964
	Halluc	4,876	+3.71	+14.8	0.542	$< 10^{-50}$	—	—	—
	Correct	26,828 [†]	+3.09	+10.0	0.598	$< 10^{-50}$	—	—	—
Max Window Entropy	Overall	31,704 [†]	-1.60	+2.9	0.853	$< 10^{-50}$	1.59×10^{-30}	1.41×10^{-13}	0.051
	Halluc	4,876	-1.88	+4.9	0.831	$< 10^{-50}$	—	—	—
	Correct	26,828 [†]	-1.56	+2.6	0.850	$< 10^{-50}$	—	—	—
Cosine Similarity	Overall	16,507 [†]	-1.54	+2.4	0.842	$< 10^{-50}$	$< 10^{-50}$	$< 10^{-50}$	0.036
	Halluc	2,514	-1.87	+4.5	0.827	5.63×10^{-46}	—	—	—
	Correct	13,993 [†]	-1.56	+2.4	0.836	$< 10^{-50}$	—	—	—

Note. Shapiro-Wilk test for normality (null hypothesis: data is normally distributed). [†]Subsampled to $n = 5,000$ due to Shapiro size limit. Skew = skewness (0 = symmetric; positive = right-tailed); Kurt = excess kurtosis (0 = normal; positive = heavy-tailed). (Dep./ED/SN) = two-sided Mann-Whitney U p -values comparing Correct vs Hallucination distributions within each CLD (filtered by 99th percentile). All metrics reject normality at $\alpha = 0.05$, justifying non-parametric methods.

Interpretation: The normality diagnostics confirm that the generator-based UQ metrics are strongly non-Gaussian at the edge level (both overall and within hallucination/correct strata). This cautions against over-interpreting visual separation alone and motivates non-parametric, distribution-robust summaries for within-CLD diagnostics. For inferential claims about RQ2 (above-chance discrimination across independent generation scenarios), we rely on block-level aggregation (CLD $\times$ run) rather than edge-level p -values to avoid pseudo-replication.

Figure 8.10 compares LLM generator uncertainty metric distributions for correct edges (green) versus hallucinations (red) across the three CLDs (99th percentile clipped; Mann-Whitney U effect sizes r shown per subplot; see Appendix D). Columns show different CLDs; rows show different metrics. Overall, the figures show unclear separation between hallucinated and non-hallucinated edges visually, though statistical tests show differences.

Perplexity distributions shift slightly left for correct edges in Depressive Symptoms and show less positive kurtosis than hallucinations in Older Persons ED and Social Norms, although p -values show significance in Depressive Symptoms and Older Persons ED but not in Social Norms.

Minimum token probability exhibits the most consistent separation (Depressive Symptoms $p = 3.39 \times 10^{-16}$; Older Persons ED $p = 3.01 \times 10^{-4}$; Social Norms $p = 0.964$). Valid edges tend to have a tail of higher minimum probability across all CLDs, although most of the probability mass of the correct distribution is also concentrated in the low-probability region.

Maximum window entropy shows significance in Older Persons ED and Depressive Symptoms but not in Social Norms, with hallucinations skewing toward higher local entropy peaks in Depressive Symptoms and Older Persons ED (Depressive Symptoms $p = 1.59 \times 10^{-30}$; Older Persons ED $p = 1.41 \times 10^{-13}$). In contrast, no separation is observed for Social Norms (max window entropy $p = 0.051$), where the distribution appears slightly bimodal. Finally, cosine similarity to the retrieved context (available only for citation judging runs) differs significantly in all three CLDs (Depressive Symptoms and Older Persons ED $p < 10^{-50}$; Social Norms $p = 0.036$), with hallucinations tending toward higher similarity. This suggests that semantic proximity to retrieved context does not guarantee causal correctness and may reflect topical alignment rather than faithful edge generation.

Overall, evidence suggests that these metrics carry some signal, but it appears domain-dependent, uneven, and subtle, and does not yield clear visual separation. Moreover, because of the relatively large sample size, statistically significant effects are easy to observe even when effect sizes are small, so significance does not necessarily imply practical usefulness.

Even if no single metric yields a clear univariate threshold, the metrics may still be jointly informative: multivariate models can capture interactions and non-linear combinations that define a decision boundary not evident in marginal distributions. Therefore, we next train an ensemble classifier using all generator metrics jointly.

8.12.3 Recursive Feature Elimination Analysis

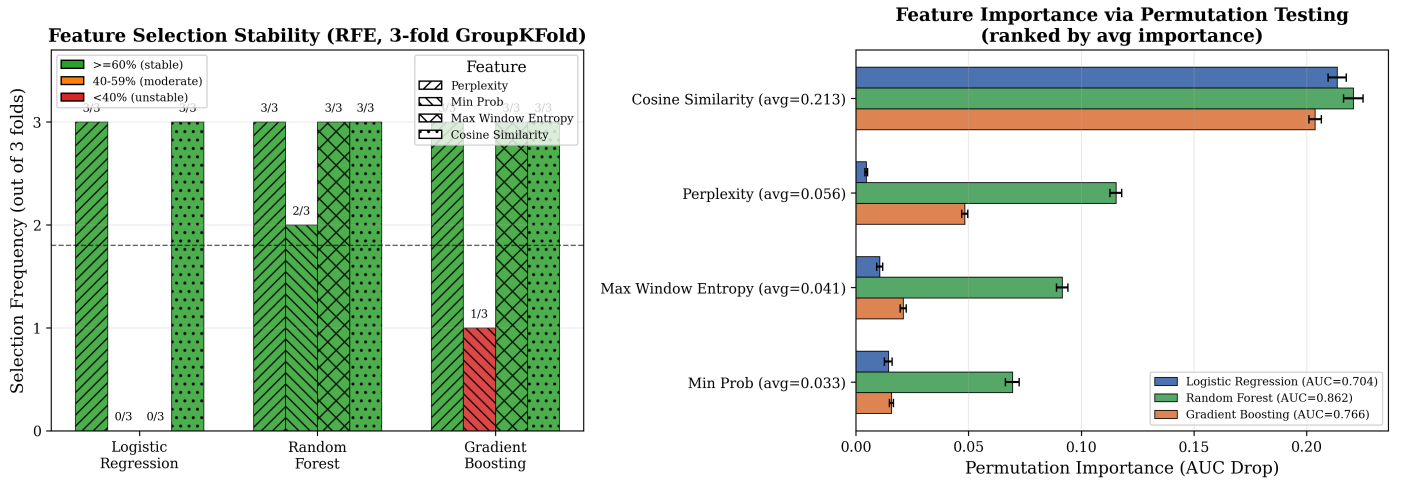


Figure 8.11: RFE analysis across classifiers with 3-fold block-level GroupKFold (blocks = CLD × run). **Left:** Feature selection stability with colour-coded thresholds (green $\geq 60\%$ = stable, orange 40–59%, red $< 40\%$ = unstable); labels show selection frequency. **Right:** Permutation importance (AUC drop when feature is shuffled) for each classifier; error bars show standard deviation across 30 permutations.

Figure 8.11 reveals substantial differences in feature selection and importance across classifiers. Gen Cosine Similarity is the most important feature for all classifiers (permutation importance 0.21–0.29), consistent with its stable selection across all models. For Logistic Regression (AUC = 0.704), Gen Min Prob, Gen Max Window Entropy, and Gen Perplexity show near-zero permutation importance, explaining why RFE never selects them (0/3 folds). In contrast, Random Forest (AUC = 0.862) shows high permutation importance for all 4 features (0.10–0.29), justifying its selection of all features. However, Random Forest’s superior in-distribution performance comes at the cost of poor cross-domain generalisation (Table 8.16), suggesting that the additional features capture CLD-specific patterns rather than generalizable hallucination signals.

Having identified which features contribute to classification, we now evaluate ensemble classifiers using a cross-validation framework designed to detect overfitting and assess cross-CLD generalisation.

8.12.4 Ensemble Classification Results

Ensemble classifiers require a consistent feature vector per edge. Therefore, Phases 4–6 are computed on the subset of edges for which all four UQ metrics are available (i.e., citation-judging runs only), yielding 16,507

edges across 35 experiment files—a reduction from the 31,704 edges across 63 files used in the single-metric analysis.

Table 8.16 presents a cross-phase validation analysis designed to detect overfitting. Each phase uses a progressively stricter validation strategy: Phase 4 tests feature selection on all pooled data; Phase 5 evaluates on held-out samples from the same distribution (all CLDs); Phase 6 tests generalisation to entirely unseen CLDs (leave-one-CLD-out cross-validation). Performance drops between phases indicate the degree to which models exploit CLD-specific patterns rather than learning generalizable hallucination signals.

Table 8.16: Ensemble Classifier Performance Across Validation Phases (Block-Level Cross-Validation)

Classifier	Phase 4: RFE CV AUC ($\pm$ 95% CI)	Phase 5: CV AUC ($\pm$ 95% CI)	Phase 5: Test AUC	Phase 6 (leave-one-CLD-out): AUC ($\pm$ 95% CI)	Performance F1@0.5 ($\pm$ 95% CI)	Drop (5→6)
Logistic Regression	0.704 $\pm$ 0.061	0.695 $\pm$ 0.067	0.702	0.664 $\pm$ 0.239	0.304 $\pm$ 0.306	0.039
Random Forest	0.862 $\pm$ 0.216	0.832 $\pm$ 0.304	0.809	0.629 $\pm$ 0.179	0.174 $\pm$ 0.345	0.179
Gradient Boosting	0.766 $\pm$ 0.019	0.743 $\pm$ 0.059	0.765	0.645 $\pm$ 0.296	0.194 $\pm$ 0.650	0.120
Neural Network	N/A	0.709 $\pm$ 0.032	0.739	0.648 $\pm$ 0.211	0.132 $\pm$ 0.473	0.092

Note. Block-level cross-validation using 3-fold GroupKFold with blocks = (CLD $\times$ run, $N = 9$). Entire blocks stay together in train OR test to prevent pseudo-replication. **Phase 4 (RFE CV AUC):** 3-fold block-level CV on all pooled edges; measures feature selection performance. **Phase 5 (CV AUC):** 3-fold block-level CV on training blocks ($\sim 67\%$ of blocks); measures in-distribution performance. **Phase 5 (Test AUC):** Single evaluation on held-out test blocks ($\sim 33\%$ of blocks); validates generalization within same distribution. **Phase 6 (Cross-CLD AUC/F1@0.5):** Leave-one-CLD-out CV—train on 2 CLDs, test on held-out 3rd CLD, averaged across all 3 CLDs; AUC uses predicted probabilities; F1@0.5 uses the fixed deployment threshold $p(\text{hallucination}) \geq 0.5$. Phase 4 evaluates all 4 UQ metrics via RFE for each classifier; the best-performing classifier (Random Forest) selected all 4 features, which are then used in Phases 5–6. **Uncertainty:** All phases show mean $\pm$ 95% CI using the t -distribution with $df = n - 1$, per the uncertainty reporting rule (Methods Section 7.3). Performance Drop = Phase 5 Test AUC – Phase 6 Mean AUC.

Table 8.16 shows cross-phase validation results using block-level GroupKFold (blocks = CLD $\times$ run, $N = 9$). Random Forest achieves the highest in-distribution performance (Phase 5 Test AUC = 0.809) but exhibits substantial degradation under leave-one-CLD-out evaluation (Phase 6 mean AUC = 0.629, drop = 0.179). Gradient Boosting shows similar overfitting (Phase 5 = 0.765, Phase 6 = 0.645, drop = 0.120). In contrast, Logistic Regression and the Neural Network show smaller cross-domain degradation (drops of 0.039 and 0.092) and similar Phase 6 mean AUC (0.664 vs 0.648; overlapping 95% CIs). The block-level CV prevents pseudo-replication from correlated edges within the same generation scenario, yielding more conservative but trustworthy performance estimates. Overall, these results suggest that simpler models generalise more robustly across CLDs, while higher-capacity tree ensembles are more prone to CLD-specific overfitting.

8.12.5 RQ2: Main Findings (Summary)

RQ2 asks whether generator-time uncertainty (UQ) metrics can predict hallucinated edges (FP/FN w.r.t. GT Lit) in CLD generation. Across the single-metric and ensemble analyses, we find:

1. **Single metrics carry only weak-to-moderate signal.** The strongest single metric is Gen Cosine Similarity (AUC = 0.671; $r = 0.153$; Table 8.14), while the remaining generator-time metrics provide weaker discrimination (AUC ≈ 0.55). Notably, cosine similarity exhibits a counterintuitive direction (higher similarity associated with hallucinations), consistent with topical overlap rather than causal faithfulness.

2. **High-capacity ensembles overfit across domains.** Random Forest achieves the highest in-distribution performance (Phase 5 Test AUC = 0.809) but degrades under leave-one-CLD-out evaluation (Phase 6 mean AUC = 0.629), indicating CLD-specific overfitting (Table 8.16).
3. **Simpler models generalise better, but remain limited.** Logistic Regression and the Neural Network show the smallest cross-domain performance drops (0.039 and 0.092) and comparable Phase 6 mean AUC (~ 0.65 – 0.66 ; Table 8.16), implying some generalizable signal, but not strong enough to support reliable cross-domain hallucination detection in isolation.

Takeaway: Single generator UQ metrics provide a weak signal for hallucination detection. When used in an ensemble as input features for classifiers, they perform well in-distribution for some models (Gradient Boosting, Random Forest) but show limited cross-CLD generalisation, reducing practical applicability. In this study, UQ metrics are better viewed as auxiliary features for triage/monitoring rather than as a standalone, domain-robust hallucination detector.

RQ3 Pivot Rationale. The original RQ3 design—using generator UQ metrics to selectively deploy the corrector—is not supported by our findings: RQ2 shows that UQ metrics do not generalise across CLDs (Phase 6 AUC ≈ 0.63 – 0.67), and RQ1b shows the corrector does not reliably improve GT Lit quality (macro $\Delta F1 \approx -0.02$). We therefore pivot to an alternative form of smart test-time compute: using the Mechanistic Correctness Judge score (which showed more reliable discrimination in RQ1a GT Lit; Table 8.13) as the uncertainty signal, and **Deep Research** (DR)—automated multi-agent literature search—as the form of deploying more (expensive) test-time compute that can measurably validate causal claims.

This preserves the spirit of RQ3: a computationally cheap uncertainty signal triggers selective deployment of more expensive but more effective test-time compute. The RQ1a physics CLD validation (Section C.4.15) provides empirical motivation for DR: simple citation retrieval achieves 89.5% retrieval coverage on Uleman’s CLD but yields low judge scores, while physics CLDs with lower retrieval (55%) achieve high approval when retrieved (82.4%). The bottleneck is not retrieval success but **evidence type**—standard search finds correlational literature in complex domains while physics literature contains definitive causal statements. DR addresses this by specifically targeting direct causal evidence (RCTs, experimental manipulations, meta-analyses) through iterative, multi-agent search that filters for study types establishing causation rather than correlation. Due to time constraints from the late pivot, we did not explore the DR design space as systematically as in RQ1a/RQ1b with a component-wise ablation study. The DR system uses multiple search queries in parallel, iterative evidence gathering, and a multi-agent pipeline for scoring, selection, and judgement (see Methods, Section 5.7, for full system architecture).

Limitations of the exploratory design. The DR system has many degrees of freedom (query count, iteration depth, scoring thresholds, agent prompts, and components) that would ideally require a full ablation study with local and global sensitivity analysis. Additionally, searching for evidence *for* a claim introduces confirmation bias risk, and the system inherits publication bias from the underlying literature. These limitations are acknowledged; the following results should be interpreted as an exploratory pilot rather than a definitive evaluation.

8.13 RQ3: Deep Research Validation (Exploratory Pilot)

Experimental design: Section 7.14. *Algorithm:* Algorithm H.11 (Deep Research).

Methodological pivot. The original RQ3 (smart test-time compute using UQ metrics to selectively deploy correctors) was contingent on RQ2 generalisation and RQ1b corrector efficacy—conditions not supported by our findings. We therefore repurposed RQ3 as an exploratory pilot of **Deep Research (DR)**: using automated literature search to *validate* (rather than correct) LLM-generated causal claims. This pilot is single-run and non-ablated; see Discussion (RQ3 pivot section) for rationale and limitations.

We deployed DR on 285 edges (TP + FP + FN from the RQ1a GT Lit labels across all three CLDs; TN omitted to reduce cost) and asked whether automated literature search can validate the LLM’s causal *narrative* for each edge. The key metric is **detection rate**: the fraction of edges for which DR retrieves *direct causal evidence* (RCTs, experimental manipulations, or meta-analyses; not purely correlational studies). We additionally report DR’s self-assessed confidence (1–10).

Table 8.17: Deep Research Detection Rate and Discrimination Analysis (n=285 edges)

Classification	n	Detection Rate	Confidence	TP–X Gap	Cohen’s h	p -value
TP (Expert-accepted)	44	70.5%	6.59	—	—	—
FP (LLM hallucinations)	178	62.4%	6.64	8.1 pp	0.17	.38
FN (Expert-rejected)	63	42.9%	6.78	27.6 pp	0.56	.006**

Note. Detection rate = % edges with direct causal evidence found. Confidence = self-assessed (1–10 scale). TP/FP/FN from

RQ1a GT Lit validation across all 3 CLDs.

TP–FP test. Fisher’s exact $p = .38$; OR = 1.44 [95% CI: 0.70, 2.94]; Cohen’s $h = 0.17$ [95% CI: -0.16, 0.50] (negligible); power = 17%.

TP–FN test. Fisher’s exact $p = .006^{**}$; Cohen’s $h = 0.56$ [95% CI: 0.18, 0.95] (medium); power = 82%.

FP–FN test. Fisher’s exact $p = .008^{**}$; Cohen’s $h = 0.39$ [95% CI: 0.11, 0.68] (small); power = 76%.

Multiple comparisons. Bonferroni correction for 3 pairwise tests. TP–FP $p_{\text{adj}} = 1.00$; TP–FN $p_{\text{adj}} = .02^{*}$; FP–FN $p_{\text{adj}} = .02^{*}$.

Table 8.17 shows that TP edges have a higher detection rate than FP edges, but the TP–FP gap (8.1 pp) is not statistically significant ($p = .38$; low power, effect-size CI includes zero). In contrast, detection differs substantially between TP and FN (27.6 pp; significant after correction; medium effect) and between FP and FN (19.5 pp; significant after correction; small effect).

Given these aggregate results, we next report per-CLD detection rates to assess whether DR’s discriminative behaviour is domain-dependent.

8.13.1 Computational Cost

Experimental design: Section 7.14.0.2.

Table 8.18: Deep Research Computational Cost by CLD

CLD	Vars	Possible	Analysed	TN	Tokens	Cost
Depressive	14	182	84	98	21.8M	\$58
Social Norms	10	90	17	73	4.4M	\$12
Older Persons	34	1122	184	938	44.3M	\$177
Total	—	1,394	285	1,109	70.4M	\$247

Cost at GPT-5 rates: \$247 for 285 edges (\$0.87/edge). DR is $\sim 119\times$ more expensive than citation judging, $\sim 512\times$ more expensive than correctness judging. True Negatives (1,109 edges) not analysed due to cost.

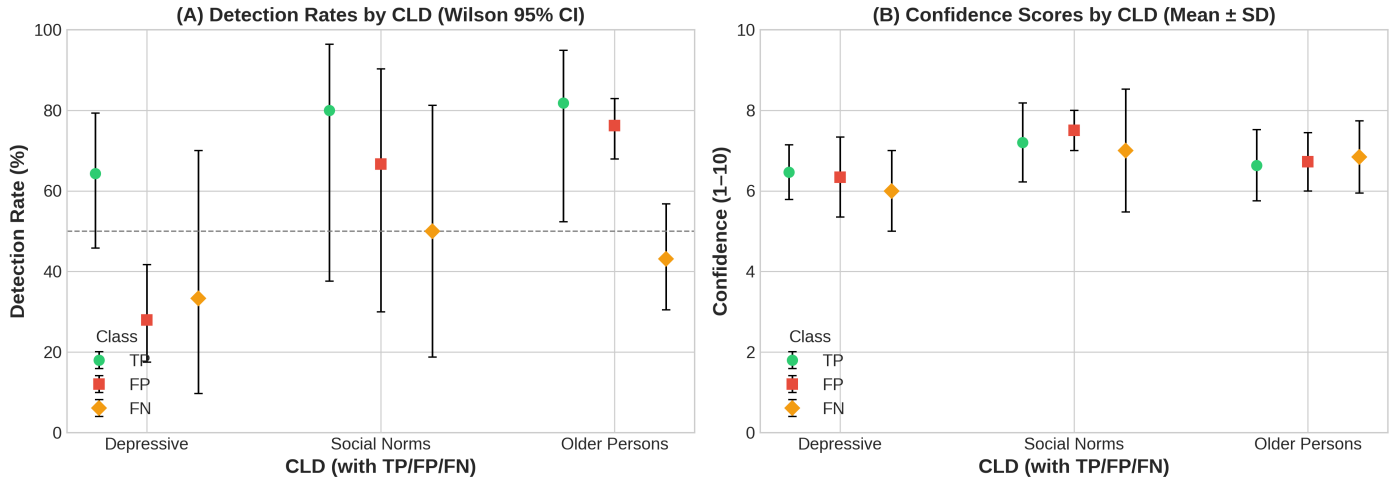


Figure 8.12: Deep Research results. (A) Detection rates by CLD and classification (TP/FP/FN) with Wilson 95% CIs. (B) Confidence scores by CLD and classification (mean $\pm$ SD).

Figure 8.12A shows substantial between-CLD variation in detection gaps between edge classes: Depressive Symptoms exhibits meaningful discrimination (TP–FP gap = +36 pp), while Social Norms (+13 pp) and Older Persons ED (+6 pp) show weak discrimination; uncertainty is substantial for Social Norms due to small n . Figure 8.12B shows DR’s self-reported confidence provides little discrimination between edge classes: means cluster tightly (range < 1 point on a 10-point scale) with substantial overlap across TP/FP/FN within each domain.

In aggregate, DR does not significantly discriminate between valid and hallucinated (as defined as FP in GT Lit) causal claims (TP–FP gap = 8.1 pp; $p = .38$; Cohen’s $h = 0.17$, negligible). Nonetheless, DR retrieves direct causal evidence for 62.4% of FP edges and 42.9% of FN edges. This is consistent with multiple explanations: (i) some “false positives” may reflect literature-supported relationships that experts excluded for scope/theoretical reasons, and/or (ii) DR may retrieve evidence that is not sufficiently specific or causal for the exact claim—a motivation for the human validation causal evidence applicability analysis below. The FP–FN gap (+19.5 pp, $p = .008^{**}$, $p_{\text{adj}} = .024^{*}$) survives correction: FP edges have higher detection than FN edges.

8.13.2 Per-CLD Detection Rate Analysis

Experimental design: Section 7.14.0.1.

Table 8.19: Per-CLD Pairwise Discrimination Tests (Fisher’s Exact, Bonferroni $m = 3$)

CLD	Comparison	Δpp	Cohen’s h	Interpretation	p	p_{adj}
Depressive	TP–FP	+36.3	0.75	medium	.004**	.01*
	TP–FN	+31.0	0.63	medium	.20	.61
	FP–FN	-5.3	-0.12	negligible	1.00	1.00
Social Norms	TP–FP	+13.3	0.30	small	1.00	1.00
	TP–FN	+30.0	0.64	medium	.55	1.00
	FP–FN	+16.7	0.34	small	1.00	1.00
Older Persons	TP–FP	+5.6	0.14	negligible	1.00	1.00
	TP–FN	+38.7	0.83	large	.04*	.13
	FP–FN	+33.1	0.69	medium	.001***	.001***

Note. All pairwise comparisons (TP–FP, TP–FN, FP–FN) within each CLD using Fisher’s exact test; Bonferroni correction applied within each CLD ($m = 3$, $\alpha_{adj} = 0.017$). Effect size interpretation: $|h| < 0.2$ negligible, 0.2–0.5 small, 0.5–0.8 medium, > 0.8 large. Detection rates and confidence distributions shown in Figure 8.12.

Table 8.19 reveals substantial domain variation in DR’s TP–FP discriminative ability, relatively smaller variation in TP–FN, and substantial variation in the FP–FN gap. In the Depressive Symptoms CLD, DR clearly distinguishes valid from hallucinated edges (TP–FP: +36.3 pp, $h = 0.75$, $p_{adj} = .01$). The Social Norms CLD shows a smaller, non-significant gap (TP–FP: +13.3 pp, $h = 0.30$). In contrast, the Older Persons ED CLD shows negligible TP–FP discrimination (+5.6 pp, $h = 0.14$), but a significant FP–FN gap (+33.1 pp, $h = 0.69$, $p_{adj} < .001$), indicating DR validates most edges—including false positives—while consistently missing edges the LLM failed to generate.

8.13.3 Exploratory: Ground Truth Enrichment Analysis

Experimental design: Section 7.14.0.4.

Given that 62.4% of FP edges have `direct_causal_found=True` under DR, we explore an optimistic upper bound: *if* DR-validated FPs were treated as true causal relationships, how would generation performance change? This assumption is intentionally strong and is used only to bound the potential impact; the human-validation analysis below provides a more conservative calibration.

Table 8.20: LLM Generation Metrics: Two Enrichment Scenarios (Aggregate and Per-CLD)

Dataset	Original	Scenario 1: Enriched GT		Scenario 2: LLM+DR System	
	F1	F1	FPs → TP	F1	FNs found
Aggregate	0.267	0.705 (+163%)	111/178 (62%)	0.779 (+191%)	27/63 (43%)
Conservative	–	0.662	97/178 (54%)	0.719	19/63 (30%)
Depressive	0.500	0.667 (+33%)	14/50 (28%)	0.687 (+37%)	2/6 (33%)
Conservative	–	0.600	8/50 (16%)	0.600	0/6 (0%)
Social Norms	0.455	0.692 (+52%)	4/6 (67%)	0.828 (+82%)	3/6 (50%)
Conservative	–	0.522	1/6 (17%)	0.583	1/6 (17%)
Older Persons	0.113	0.722 (+540%)	93/122 (76%)	0.813 (+621%)	22/51 (43%)
Conservative	–	0.671	82/122 (67%)	0.740	15/51 (29%)

Scenario 1 (Enriched GT): Promotes FP edges with DR evidence to TP. Tests: “What if experts missed valid relationships?”

Scenario 2 (LLM+DR System): Scenario 1 + DR discovers FN edges with literature support. Tests: “What could a complete LLM+DR pipeline achieve?”

Aggregate confusion matrices: Original: TP=44, FP=178, FN=63. Scenario 1: TP=155, FP=67, FN=63. Scenario 2: TP=182, FP=67, FN=36.

Conservative rows: Use Wilson-score 95% lower bounds on the FP→TP and FN-found proportions (floored to integer counts) to produce conservative Scenario 1/2 estimates.

Of the 63 FN edges (expert-included, LLM-missed), 27 (42.9%) have DR causal evidence. Under Scenario 2, a complete LLM+DR pipeline would recover these edges as positive relationships, reducing FN from 63 to 36. The remaining 36 FN edges (57.1%) lack searchable evidence under our DR settings, consistent with tacit domain knowledge or evidence types not captured by the retrieved literature. Scenario 1 quantifies the impact of promoting FP→TP (F1: 0.267 → 0.705), while Scenario 2 estimates an upper bound when additionally recovering FN edges with DR evidence (F1: 0.267 → 0.779). The 36 residual FNs represent expert knowledge that neither LLM generation nor literature search recovers in this setting.

Human validation sampling and labels. We stratified-sampled 50 edges from the Deep Research output restricted to `direct_causal_found=True` and manually labelled each edge with a categorical verdict (`human_verdict` ∈ {`causal`, `different_vars`, `causal`, `Not relevant` / `non-causal`}) and a human-assigned evidence applicability score `Fully applicable` ∈ [0, 1]. The sampling protocol and applicability rubric are defined in Methods (Section 7.14.0.5).

Human validation evidence verdicts. Table 8.21 summarises the distribution of human verdicts by edge class. All TP edges are supported by direct causal evidence, but a majority of FP edges also have causal evidence (either exact-match `causal` or `different_vars`, `causal`). This corroborates that the primary failure mode is *variable mismatch* (causal evidence for related constructs/measures), rather than outright non-causal evidence.

Table 8.21: Human Validation: Evidence Verdict Distribution by Edge Classification ($n = 50$)

Verdict	TP ($n = 9$)	FP ($n = 32$)	FN ($n = 9$)	Total
causal	9 (100.0%)	21 (65.6%)	5 (55.6%)	35 (70.0%)
different_vars, causal	0 (0.0%)	8 (25.0%)	2 (22.2%)	10 (20.0%)
Not relevant / non-causal	0 (0.0%)	3 (9.4%)	2 (22.2%)	5 (10.0%)
Any causal	9 (100.0%)	29 (90.6%)	7 (77.8%)	45 (90.0%)

Note. `causal` = exact match to edge claim; `different_vars, causal` = causal evidence for related but not identical variables/constructs; `Not relevant / non-causal` = evidence does not support a causal relationship. “Any causal” = `causal` $\cup$ `different_vars, causal`.

Table 8.22: Applicability Score Rubric for Human Validation

Score	Description
1.0	Causal evidence for exact variable constructs and measures
0.9	Causal evidence, measure differing for one variable
0.8	Causal evidence, measure differing for two variables
0.7	Causal evidence, one variable differing but believably related
0.6	Causal evidence, two variables differing but believably related
0.5	Non-causal evidence for variables in question
≤ 0.4	Progressively less related evidence

Applicability-based calibration (thesis-ready filtering). Because DR’s binary `direct_causal_found=True` flag only indicates that *some* direct causal evidence was retrieved—and often fails via variable/operationalisation mismatch—we calibrated it using the human `Fully_applicable` $\in [0, 1]$ score (see Methods Section 7.14.0.6 for threshold selection methodology). For thresholds $t \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$, we computed the pooled coverage–correctness tradeoff (Figure 8.13/Table 8.23).

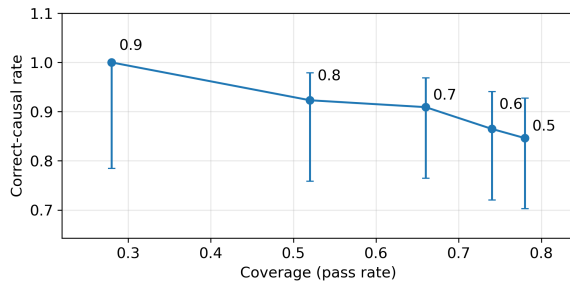


Figure 8.13: Human validation tradeoff curve (pooled; Wilson 95% CI). Each point corresponds to an applicability threshold $t \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$ applied to the validated sample ($n = 50$). Let $x(t)$ be *coverage* = $\Pr(\text{Fully_applicable} \geq t)$ and $y(t)$ be *correct-causal rate* = $\Pr(\text{human_verdict} = \text{causal} \mid \text{Fully_applicable} \geq t)$. Points are annotated by t . Selected threshold: $t = 0.7$.

Table 8.23: Human validation tradeoff curve (pooled): coverage vs. correct-causal rate

Threshold	Coverage	Correct-causal rate	n pass
0.5	0.780	0.846	39
0.6	0.740	0.865	37
0.7	0.660	0.909	33
0.8	0.520	0.923	26
0.9	0.280	1.000	14

Coverage = fraction of validated edges with applicability $\geq t$. Correct-causal rate = fraction of passing edges labelled `causal`.

Bold row indicates selected threshold $t = 0.7$.

We then selected $t = 0.7$ via the script’s elbow/gradient decision rule and used the human-validated pass rates to extrapolate expected FP promotions and FN discoveries to the full DR dataset (Table 8.24).

Table 8.24: LLM Generation Metrics with Human-Validated Ground Truth Enrichment (Aggregate, extrapolated)

Scenario	Precision	Recall	F1
Original (expert GT)	0.198	0.411	0.267
Scenario 1: Enriched GT (FP→TP)	0.459	0.618	0.527
Scenario 2: LLM+DR System (+FN found)	0.494	0.709	0.582
Conservative Scenario 1	0.396	0.583	0.472
Conservative Scenario 2	0.415	0.629	0.500

Threshold selected by elbow/gradient rule (Option B) on validation tradeoff curve: $t = 0.7$. FP promotions (point / conservative): 58 / 44. FN discoveries (point / conservative): 15 / 7. Counts are extrapolated from a stratified human-validation sample and should be interpreted as estimates.

Critical Caveats: Both scenarios are **exploratory and preliminary**. Key limitations: (1) DR may validate edges as being directly causal irrespective of other CLD specific context such as mediators. (2) LLM-based DR validating LLM-generated edges risks circularity; (3) The review of promoted/discovered edges was solely done by the author of this thesis and not a domain expert and has no inter rater reliability reporting; (4) **True Negatives not analysed**—DR covered only 20.4% of the edge space (285 of 1,394 possible edges); the 1,109 TN edges were not analysed due to cost (\$0.87/edge, \$269 for all TNs). These metrics represent **upper bounds** on achievable performance, pending domain expert validation. Dataset available for community verification (see Data Availability).

8.13.4 Smart-Trigger DR Deployment Estimate

Experimental design: Section 7.14.0.7.

The enrichment analysis above applies DR to *all* edges. In practice, DR is expensive (\$0.87/edge), motivating a **smart-trigger** deployment that runs DR selectively. We estimate expected F1 gains if DR is triggered only for edges where the most reliable RQ1a Ground Truth Correctness judge (Mechanistic prompt) assigns a low aggregate score (≤ 0.5), indicating uncertainty about the edge (as opposed to the original RQ3 intention of using RQ2 UQ metrics as the trigger).

Procedure. We parsed per-edge Mechanistic judge scores from the 9 judged workbooks (3 CLDs $\times$ 3 runs) and computed the mean score per edge. We then joined these to the 285 DR edges and marked edges as *triggered* if their mean score ≤ 0.5 . Of the triggered edges, we counted how many had `direct_causal_found=True` in the DR oracle. Finally, we applied the human-validated calibration rates at $t = 0.7$ (FP: 89.5%, FN: 83.3%) to estimate expected promotions/discoveries.

Results. Of 285 DR edges, 142 triggered (50%). Among triggered edges with DR evidence: 59 FP edges and 9 FN edges had `direct_causal_found=True`. Applying calibration:

Table 8.25: Smart-trigger Deep Research enrichment estimates. Trigger: Mechanistic GT correctness score ≤ 0.5 . Calibration: human-validated applicability ≥ 0.7 .

Scenario	Precision	Recall	F1 (95% CI)
Original	0.198	0.411	0.267
Enriched GT (+52.8 FP→TP)	0.436	0.606	0.507 [0.457, 0.524]
LLM+DR (+7.5 FN discovered)	0.454	0.653	0.536 [0.474, 0.557]

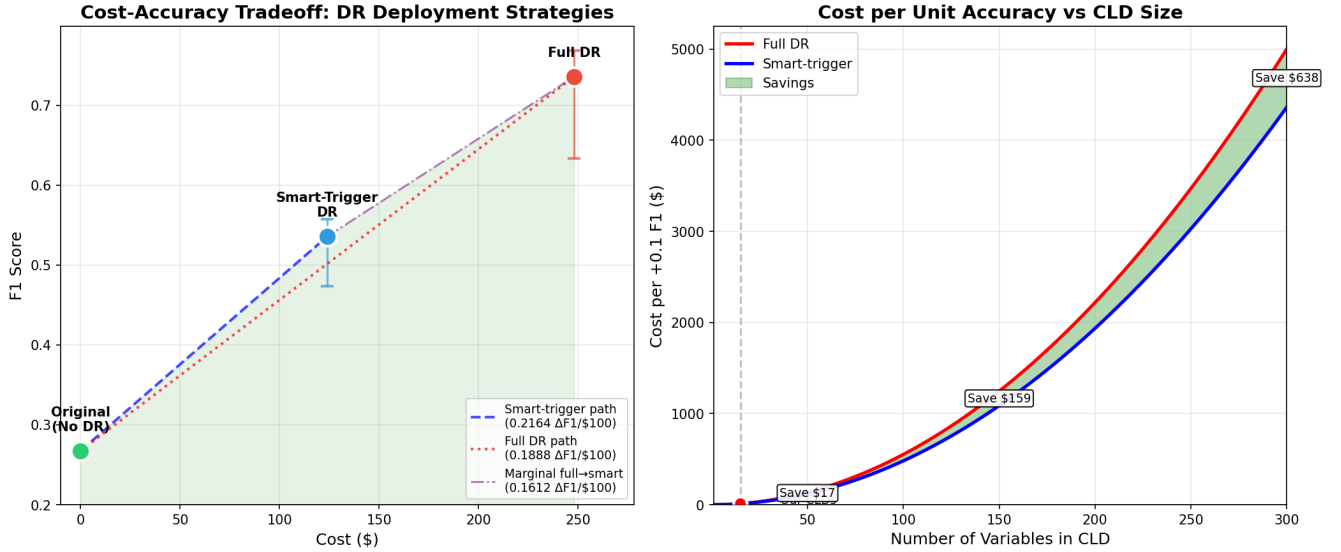


Figure 8.14: Cost-accuracy tradeoff and cost-per-unit-accuracy scaling for Deep Research (DR) deployment. Left: observed cost vs. expected F1 for always-on DR vs. smart-trigger DR (judge-triggered deployment). Right: extrapolated cost per +0.1 F1 improvement as CLD size increases under a sparse-edge growth assumption, highlighting the widening cost gap.

Note. Figure 8.14 shows two-sided uncertainty bands (Wilson 95% bounds) propagated from the human-calibrated promotion/discovery rates (see Methods, Sections 7.14.0.5 and 7.14.0.4). For interpretation, we prefer the conservative estimate obtained by using the Wilson 95% lower bound, because it reduces the risk of overstating gains due to sampling variability in the human validation rates. While this lower bound addresses sampling uncertainty in the calibration step, it does not account for other sources of uncertainty (e.g., generalisation of applicability to non-validated edges, edge dependence, domain shift, and not considering CLD mediators). Further research should assess how the estimated gains change when these assumptions are relaxed.

We evaluated whether a selective test-time DR deployment policy could improve cost-efficiency relative to always-on DR, using the RQ1a Ground Truth Correctness judge (Mechanistic prompt) as a trigger (mean aggregate score ≤ 0.5). On the 285-edge RQ3 dataset, 142 edges (50%) triggered; among these, DR reported `direct_causal_found=True` for 59 FP and 9 FN edges. Using the human-validated calibration at applicability ≥ 0.7 (FP: 17/19 causal; FN: 5/6 causal; Wilson 95% CIs reported above), the smart-trigger policy yields an estimated $\Delta F1$ of 0.268 at $\sim \$124$ total compute (judge on all edges + DR on triggered edges), compared to $\Delta F1$ of +0.468 at $\sim \$248$ for full DR.

Under these assumptions, smart-trigger achieves higher $\Delta F1$ per dollar (0.216 vs. 0.189 $\Delta F1$ per \$100; $\sim 15\%$ relative improvement) while recovering $\sim 57\%$ of the full-deployment F1 gain at $\sim 50\%$ of the DR cost. The scaling panel should be interpreted as a cost extrapolation, not a validated performance claim: it assumes the observed trigger rate and per-edge applicability distribution generalise to other edges and other CLDs. Under this stylized model, the absolute dollar gap between full and smart-trigger deployment increases with CLD size due to quadratic scaling of the edge space, while the efficiency ratio remains approximately constant.

8.14 RQ3: Main Findings (Summary)

RQ3 asks whether automated literature search (Deep Research; DR) can validate LLM-generated causal edges and improve end-to-end CLD quality. As an exploratory pilot (single-run, non-ablated), we find:

1. **DR detection is not a reliable FP filter in aggregate.** Across 285 TP/FP/FN edges, DR retrieves `direct_causal_found=True` for many FP edges (62.4%), yielding a small, non-significant TP–FP detection gap (8.1 pp; $p = .38$; Cohen’s $h = 0.17$; Table 8.17). DR does, however, distinguish FN from both TP and FP (TP–FN and FP–FN gaps significant after correction; Figure 8.12).
2. **Utility is domain-dependent and DR confidence is weakly informative.** Per-CLD results show strong TP–FP discrimination in Depressive but weak discrimination in Social Norms and Older Persons (Table 8.19; Figure 8.12). DR’s self-reported confidence shows substantial overlap across TP/FP/FN classes within domains (Figure 8.12B), limiting its value as a standalone accept/reject criterion.
3. **Enrichment effects can be large if gains extrapolated from the human validation sample hold.** A naive upper bound (treat all DR-positive edges with `direct_causal_found=True` as correct) yields very large gains (Scenario 1/2; Table 8.20). However, DR sometimes finds causal evidence for highly related, but not directly equivalent, concepts or using different measures. Human validation shows that the dominant failure mode is *variable mismatch*—causal evidence for different constructs than the edge’s stated variables. To form a more realistic estimate based on the human validation sample, an optimal trade-off between evidence quality and coverage can be found by applying a threshold on the pooled tradeoff curve ($t = 0.7$; Figure 8.13), which yields more calibrated enrichment estimates than the naive “any causal evidence found by DR is valid” assumption: F1 improves from 0.267 (expert GT) to 0.527 (Scenario 1) and 0.582 (Scenario 2), with conservative estimates of 0.472 and 0.500 (Table 8.24). Based on the quality of causal evidence observed in the human validation sample, the human-calibrated enrichment extrapolation seems mostly reasonable, but it relies on strong assumptions: (i) the within-CLD edge-class applicability distribution in the validated sample generalises to the remaining (out-of-sample) edges for that CLD, and (ii) edges are evaluated largely in isolation rather than in the full CLD causal context (e.g., mediators/confounders) that could disqualify a relationship from being directly causal.
4. **Smart-trigger DR slightly improves cost-efficiency (exploratory).** Triggering DR only on edges flagged as uncertain by the strongest correctness judge (Mechanistic; score ≤ 0.5) recovers a notable fraction of the full-deployment DR enrichment gains at roughly half the DR cost, with an estimated $\Delta F1$ of 0.268 after human-calibrated applicability filtering (Figure 8.14). This provides a proof of concept that DR efficiency can be improved by deploying DR selectively using lower-compute uncertainty estimates.

Takeaway: In this pilot, DR is best interpreted as a high-cost, domain-dependent *enrichment signal* that requires human-calibrated applicability thresholds; it is not a drop-in validator that cleanly separates expert-true edges from LLM false positives across domains. It can, however, yield direct causal evidence for a majority of edges in our CLD ground truth sample (under strong assumptions) that, if validated, could be used to improve CLD quality, with smart triggering used to partially offset DR’s computational cost and achieve higher accuracy per dollar spent.

8.15 Effect Size Summary

Table 8.26 summarises the standardised effect sizes observed across all research questions, linking each finding to its stated hypothesis, quantitative formulation, statistical test, and conclusion. This provides a unified view of which null hypotheses were rejected and the magnitude of effects detected.

Table 8.26: Effect Size Summary: Hypothesis Operationalisation, Statistical Tests, and Conclusions

RQ	Hypothesis	H_0 / H_1	Test	Effect	Value	p	Conclusion (decision; effect)
<i>RQ1a: LLM-as-a-Judge Validation</i>							
1a	LLM-human agreement	$H_0: \kappa = 0$ vs $H_1: \kappa > 0$	Cohen's κ (n=72)	κ	0.618	—	H_0 rejected; substantial
1a	Prompt affects F1 (Corr. Citation)	H_0 : equal medians $H_1: \geq 1$ differs	Friedman (9 blocks)	W	0.28	.058	H_0 not rejected; weak
	Prompt affects F1 (Corr. Correctness)			W	0.70	.001**	H_0 rejected; strong
	Prompt affects F1 (GT Citation)			W	0.30	.042*	H_0 rejected; moderate
	Prompt affects F1 (GT Correctness)			W	0.48	.013*	H_0 rejected; moderate
<i>RQ1b: LLM-as-a-Corrector Evaluation</i>							
1b	Corrector efficacy (Synthetic)	H_0 : med($\Delta F1$)=0 H_1 : med($\Delta F1$) $\neq$ 0	Wilcoxon signed-rank (9 blocks)	d	+1.18	.031*	H_0 rejected; large
	Corrector efficacy (Ground Truth)	H_0 : med($\Delta F1$)=0 H_1 : med($\Delta F1$) $\neq$ 0	Wilcoxon signed-rank (9 blocks)	d	−0.32	.301	H_0 not rejected; medium
<i>RQ2: Context-Insensitive Hallucination Detection</i>							
2	UQ discriminates hallucinations	H_0 : AUC=0.5 H_1 : AUC $\neq$ 0.5	One-sample t -test (9 blocks)	AUC	0.672	<.001***	H_0 rejected; moderate
	Cross-CLD generalisation	—	Descriptive (Phase 5→6)	ΔAUC	0.18	—	Poor generalisation
<i>RQ3: Deep Research Literature Validation*</i>							
3	DR discriminates (TP–FP)	H_0 : $p_{TP} = p_{FP}$ H_1 : $p_{TP} \neq p_{FP}$	Fisher's exact (Bonf. $m=3$)	h	0.17	0.382	H_0 not rejected; negl.
	DR discriminates (TP–FN)	H_0 : $p_{TP} = p_{FN}$ H_1 : $p_{TP} \neq p_{FN}$	Fisher's exact (Bonf. $m=3$)	h	0.56	0.006	H_0 rejected; medium
	DR discriminates (FP–FN)	H_0 : $p_{FP} = p_{FN}$ H_1 : $p_{FP} \neq p_{FN}$	Fisher's exact (Bonf. $m=3$)	h	0.39	0.008	H_0 rejected; small
<i>RQ3: Enrichment Analysis (Exploratory)*</i>							
	Enrichment baseline	—	Descriptive	F1	0.267	—	Baseline
	Scenario 1: Enriched GT	—	Descriptive	$\Delta F1$	+0.438	—	Exploratory
	Scenario 2: LLM+DR	—	Descriptive	$\Delta F1$	+0.512	—	Exploratory
	Human-val. Scenario 1	—	Extrapolation	$\Delta F1$	+0.260	—	Exploratory
	Human-val. Scenario 2	—	Extrapolation	$\Delta F1$	+0.315	—	Exploratory
	Smart compute	—	Descriptive	$\Delta F1/\$100$	+0.027	—	Exploratory

Effect size interpretation. κ : Landis-Koch (0.61–0.80 = substantial). W : <0.3 weak, 0.3–0.5 moderate, >0.5 strong. d : <0.2 small, 0.2–0.8 medium, >0.8 large.

r : <0.2 weak, 0.2–0.5 moderate, >0.5 strong. h : <0.2 negligible, 0.2–0.5 small, 0.5–0.8 medium, >0.8 large.

Significance. * $p < .05$, ** $p < .01$, *** $p < .001$. RQ3 p -values are Bonferroni-adjusted ($\alpha_{adj} = 0.0167$).

* *RQ3 exploratory.* Enrichment estimates are upper-bound/extrapolated; not independently validated ground truth.

RQ1a prompt tests. Friedman with (CLD×run) blocking; W = Kendall's concordance.

RQ1b efficacy. One-sample Wilcoxon signed-rank on 9 blocks.

RQ2 discrimination. AUC = Area Under Curve; One-sample t -test on 9 block means (CLD×run) vs 0.5.

In summary, Table 8.26 shows: **RQ1a**—LLM-human agreement is substantial ($\kappa=0.618$, H_0 rejected); prompt effects on citation judging are weak ($W=0.28$ – 0.30) while correctness judging shows strong prompt sensitivity ($W=0.70$, H_0 rejected). **RQ1b**—corrector efficacy is confirmed for synthetic corruptions (H_0 rejected, $d=+1.18$, large effect) but not for ground truth (H_0 not rejected, $d=-0.32$, medium effect). **RQ2**—UQ metrics show statistically significant but weak discrimination (H_0 rejected, $r=0.15$) with poor cross-CLD generalisation ($\Delta AUC=0.18$). **RQ3**—no TP–FP discrimination (H_0 not rejected, $h=0.17$, negligible), but DR distinguishes TP and FP from FN (TP–FN: H_0 rejected, $h=0.56$, medium; FP–FN: H_0 rejected, $h=0.39$, small; both survive Bonferroni correction). Exploratory enrichment analyses indicate large aggregate improvements ($\Delta F1=+0.438$ to $+0.512$), with human-validated extrapolations remaining substantial ($\Delta F1=+0.260$ to $+0.315$).

8.16 Computational Scaling Analysis

Experimental design: Section 7.3.

Full scaling methodology, scripts, and supplementary scaling results are provided in Appendix C.4.14.

8.16.1 Time and Cost Scaling

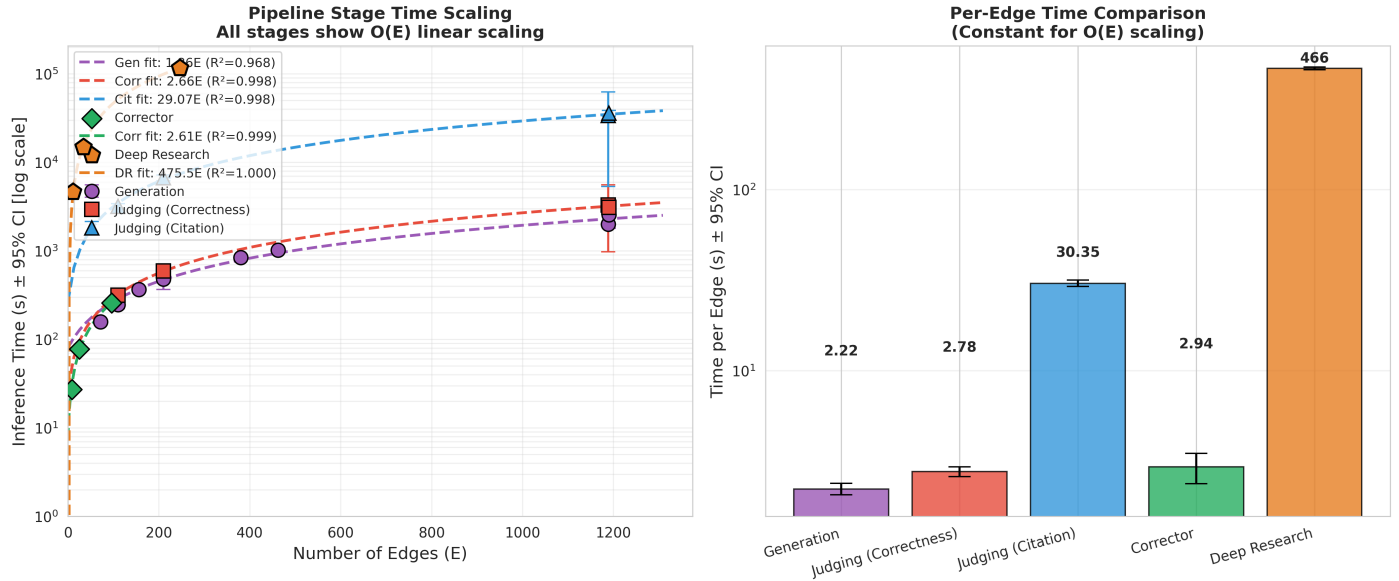


Figure 8.15: Generation vs. judging scaling comparison (mean $\pm$ SD). **Left:** Time scaling vs. number of edges. **Right:** Per-edge time comparison across pipeline stages. Error bars show SD.

Figure 8.15 shows that both generation and correctness judging exhibit $O(E)$ linear scaling with similar slopes (~ 2 s/edge). Generation (2.2 ± 0.5 s/edge) and correctness judging (2.8 ± 0.2 s/edge) have comparable costs, while citation judging (30.1 ± 1.4 s/edge) dominates the pipeline.

8.16.2 Token Cost Analysis

Table 8.27: Computational Efficiency Summary by Experiment

Experiment	Model	Files	Edges	Tok/Edge	¢/Edge	Total (\$)
<i>RQ1a Judge Experiments (actual)</i>						
Corruption Citation	5m	36	18,120	42,992	1.37	247.67
Ground Truth Citation	4.1	36	18,117	43,933	9.22	1669.69
Corruption Correctness	4.1	27	13,590	480	0.16	22.28
Ground Truth Correctness	4.1	27	13,587	470	0.16	21.34
Human Validation	4.1	3	260	42,639	8.70	22.61
<i>RQ1b Corrector Experiments (estimated)</i>						
Synth Baseline	4.1	9	4,527	980	0.33	14.74
Synth CoT	4.1	9	4,527	980	0.33	14.74
Synth Mechanistic	4.1	9	4,527	980	0.33	14.74
GT Baseline	4.1	9	4,529	980	0.33	14.75
GT CoT	4.1	9	4,529	980	0.33	14.75
GT Mechanistic	4.1	9	4,529	980	0.33	14.75
<i>RQ3 Deep Research (actual)</i>						
Deep Research	5/5m	3	291	240,913	30.12	87.65
Total						2159.70

Note. RQ1a costs from actual LLM Usage Stats. RQ1b estimated: correction + rejudging = $2 \times$ correctness tokens/edge. RQ3 from Deep Research logs (mix of GPT-5 + GPT-5-mini). Models: 4.1 = GPT-4.1 (\$2.00/\$8.00 per 1M tokens), 5 = GPT-5 (\$1.25/\$10.00 per 1M tokens), 5m = GPT-5-mini (\$0.25/\$2.00 per 1M tokens).

Reproduction: `python3 final_runs/cost_analysis/calculate_all_costs.py`

Table 8.27 reveals that Deep Research dominates per-edge cost (247K tokens/edge, \$0.87/edge) due to multi-agent orchestration and iterative search. Additional cost-analysis scripts and breakdowns are provided in Appendix C.4.16.

For a 66-edge CLD (Older Persons ED), generation plus correctness judging costs approximately \$0.29 and takes 6 minutes cumulative API time. Citation validation adds significant overhead (33 minutes, \$3.68 for GPT-4.1 or \$0.13 for GPT-5-mini). Linear $O(E)$ scaling enables predictable cost estimation for arbitrary CLD sizes.

8.16.3 Parallelization Speedup

We benchmarked parallelization using the configuration described in Methods: 34 edges (Depressive Symptoms CLD), GPT-4.1 (temperature=0.3), worker counts of 1, 3, 5, and 10, with 3 runs per configuration for both correctness judging (short prompts) and citation judging (long prompts with retrieved citations).

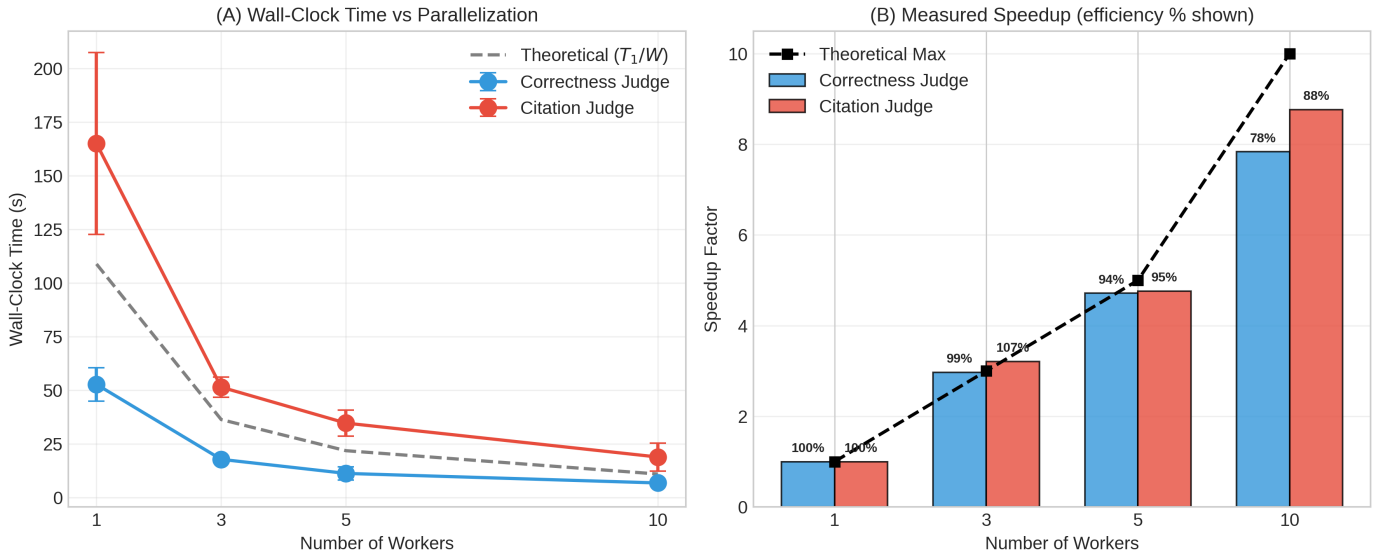


Figure 8.16: Parallelization benchmark (34 edges, GPT-4.1, 3 runs per config, mean $\pm$ SD). **(A)** Wall-clock time vs. worker count; dashed line shows theoretical perfect scaling (T_1/W). **(B)** Measured speedup with efficiency percentages.

Figure 8.16A shows citation judging (165s sequential) takes $3.1\times$ longer than correctness (52.7s) due to longer prompts with retrieved citations. At 10 workers (Figure 8.16B), correctness achieves $7.8\times$ speedup (78% efficiency) while citation achieves $8.8\times$ speedup (88% efficiency). Citation judging shows slightly better parallel efficiency due to higher per-call latency dominating overhead.

Table 8.28: Parallelization Benchmark: Wall-Clock Time and Speedup by Judge Type

Workers	Correctness Judge			Citation Judge		
	Time (s)	Speedup	Eff.	Time (s)	Speedup	Eff.
1	52.7 ± 7.8	$1.00\times$	100%	165.0 ± 42.4	$1.00\times$	100%
3	17.7 ± 1.3	$2.97\times$	99%	51.4 ± 4.6	$3.21\times$	107%
5	11.2 ± 3.1	$4.71\times$	94%	34.6 ± 6.1	$4.76\times$	95%
10	6.7 ± 1.1	$7.84\times$	78%	18.8 ± 6.5	$8.77\times$	88%

Note. Model: GPT-4.1, temp=0.3. CLD: Depressive symptoms (34 edges). Time = wall-clock (mean $\pm$ SD, n=3). Speedup = T_1/T_W . Efficiency = Speedup/ $W \times 100\%$.

Chapter 9

Discussion

This chapter synthesizes the experimental findings from RQ1a, RQ1b, RQ2, and RQ3, discussing their implications for LLM-powered CLD generation and validation, the limitations encountered, and directions for future research. We begin with preliminary results that informed model selection, proceed through each research question, and conclude with overall insights and practical recommendations.

9.1 Direct Answers to the Research Questions

For reader convenience, we provide a direct, concise answer to each research question before the deeper interpretive discussion that follows.

RQ1: Can LLM-as-a-Judge methods detect hallucinations in LLM-generated CLDs?

Answer: Partially. LLM judges reliably detect *synthetic* CLD corruptions (GT Synth) with strong discrimination, but their ability to detect deviations from *expert literature ground truth* (GT Lit) is substantially weaker and domain-dependent. Overall, LLM-as-a-Judge is best interpreted as a *screening/triage* signal rather than a dependable standalone validator across domains.

RQ1a: How effectively can correctness-based and citation-based judges distinguish valid from hallucinated causal relationships?

Answer: Correctness-based judging outperforms citation-based judging, but both have important failure modes.

- **Correctness judge:** Strong on GT Synth, but performance drops sharply on GT Lit; mechanistic prompting provides the most robust GT Lit discrimination in our experiments, though it fails in some domains (notably Social Norms).
- **Citation judge:** Unreliable as a strict validator on expert CLDs; high retrieval coverage does not imply strong causal evidence, and the judge can over-accept or mis-rank edges due to retrieval confounds (e.g., “evidence of absence” for NONE claims and topical alignment without causal specificity).

RQ1b: Can LLM-as-a-Corrector improve CLD accuracy when guided by judge feedback?

Answer: Not reliably on literature ground truth. The corrector yields modest, inconsistent improvements on synthetic corruptions, but does not improve (and can degrade) alignment with expert GT Lit on average. Judge scores increase consistently after correction, suggesting a Goodhart-like dynamic where the corrector learns to satisfy the judge more than it improves expert-grounded accuracy.

RQ2: Can uncertainty quantification (UQ) metrics predict hallucinations in generated CLDs?

Answer: Only weakly, and not in a domain-robust way. Single generator-time UQ metrics provide weak-to-moderate discrimination, and while ensembles can perform very well in-distribution, cross-CLD generalisation remains limited. In practice, these metrics are better suited to *flagging/triage* than to standalone hallucination detection across domains.

RQ3: Can adaptive test-time compute allocation improve CLD accuracy efficiency?

Answer: The original UQ-triggered judge+corrector design is not supported; a modified judge-triggered Deep Research strategy shows proof-of-concept efficiency gains. Because (i) UQ metrics do not generalise reliably across CLDs (RQ2) and (ii) the corrector does not reliably improve GT Lit accuracy (RQ1b), the original “adaptive triggering” pathway is untenable as specified. However, selectively deploying higher-cost test-time compute for *literature validation* (Deep Research) when a lower-cost signal flags uncertainty (e.g., the Mechanistic correctness judge score) recovers a substantial fraction of the full-deployment gains at roughly half the Deep Research cost in our exploratory analysis, albeit with strong assumptions and clear domain dependence.

9.2 Roadmap for the Detailed Discussion

The previous subsection gave direct answers to each research question. The remainder of this chapter explains *why* the results look the way they do and what they imply, following the structure of the empirical study. We begin with preliminary findings that shape interpretation throughout: (i) **which generator model** was selected and why, (ii) **how strong the generator is relative to a random baseline**, and (iii) whether **key hyperparameters** (notably temperature) materially affect CLD generation quality. We then discuss the core findings in sequence—**RQ1a** (judge detection performance), **RQ1b** (judge-guided correction), **RQ2** (UQ metrics), and **RQ3** (test-time compute and the Deep Research pivot)—before examining **mechanisms of judge failure** that clarify recurring error patterns. We conclude by synthesizing implications across research questions, offering practical recommendations, and outlining limitations and future work.

9.3 Preliminary Results: Generator Selection and Baseline Performance

9.3.1 Model Selection Rationale

GPT-4.1 was selected as the primary generator over Sonar-Pro (F1=0.170) and Claude-Sonnet-4 (F1=0.154) for two principal reasons. First, GPT-4.1 achieved the best micro-average F1 (0.278) across all CLDs through balanced precision-recall trade-offs (results pooled across 3 independent runs per CLD). While Claude-Sonnet-4

demonstrated extreme recall (0.932), its poor precision (0.084) indicated a tendency to generate many spurious edges. Sonar-Pro showed competitive per-CLD performance (winning Older Persons ED) but accumulated substantially more false positives in aggregate. Second, GPT-4.1 was unique among the models in providing logprob access through its API, which was essential for the RQ2 experiments on uncertainty quantification (UQ) metrics.

9.3.2 Random Baseline Comparison

The generator’s $2.2\text{--}3.4\times$ F1 improvement over the Monte Carlo random baseline (which randomly wires edges between variable pairs) provides evidence that LLM-based CLD generation is not purely random and is partially aligned with our ground truth dataset of published CLDs, rather than producing arbitrary connectivity patterns. Social Norms exhibited the largest relative improvement ($3.4\times$), while Older Persons ED showed the smallest ($2.2\times$) due to its lower absolute F1. This improvement over random wiring is consistent with the interpretation that the generator leverages patterns captured in its learned representations and could reproduce (part of) the causal structure encoded in the published CLDs.

This result provides an initial (and limited) validation signal for the “generate-only” approach to CLD construction, which contrasts with text-to-CLD approaches where an LLM parses researcher-selected authoritative texts. The advantage is that it is independent of a researcher-provided corpus (though of course not independent of the LLM training corpus) and can therefore generate CLDs without explicit textual grounding. This shifts the retrieval burden of finding the right source text away from the researcher and enables the LLM’s contribution to be additive to what the researcher already knows, since the model is likely trained on (at least part of) literature the researcher has not read or may not be able to reproduce. Whether the LLM can reproduce this performance across domains remains an open question. While these generator results provide encouraging initial evidence, future work should validate this approach across diverse domains such as the social sciences, psychology, economics, and physics, as the health-domain focus of these CLDs limits the current evidence.

9.3.3 Temperature Hyperparameter Robustness

Multiple parameters beyond the prompt can be tuned to affect LLM outputs, including temperature, top-p, seed, and maximum token count. Our ANOVA analysis ($F(4,40)=0.035$, $p=0.998$; see Appendix D) revealed no statistically significant generator temperature effect on generation quality, suggesting that CLD generation is robust to this hyperparameter choice within the tested range, chosen model, and dataset. This finding justifies our selection of temperature=0.7 as a reasonable default and indicates that prompt engineering may be more consequential than temperature tuning for this task.

Other temperature defaults (judge=0.0) were chosen based on established recommendations from the LLM-as-a-Judge literature [21, 101]. The corruptors temperature default was chosen to be the same as the generator temperature to minimise stylistic distributional shifts, ensuring that hallucinations must be detected based on content rather than stylistic artefacts. The corrector’s temperature default was chosen to be 0.7 to align with that of the generator, in line with the Self-Refine framework which also selects similar temperatures for generator and corrector [77]. A comprehensive sensitivity analysis exploring interactions between temperature, top-p, and other parameters would be valuable but was cost-prohibitive for this dataset. We recommend future research conduct such analyses on smaller CLDs to reduce computational costs but explore a larger section of the hyperparameter space, with subsequent validation to find out if findings generalise to larger graphs.

9.3.4 TruthfulQA Verification

Prior to domain-specific experiments, we validated our correctness-judge *infrastructure* on the TruthfulQA benchmark [119], achieving $F1=0.874$ and $Accuracy=0.876$. This benchmark tests whether models can distinguish truthful from false-but-plausible statements, providing a basic sanity check that our evaluation pipeline behaves as intended. However, TruthfulQA performance does not directly predict CLD-specific performance, as the task structure differs substantially: TruthfulQA involves classifying isolated factual claims, whereas CLD judging requires evaluating causal relationships within a domain-specific variable ontology. Nonetheless, this verification step increased confidence that our judging setup was implemented correctly before proceeding to the more CLD-specific controlled synthetic corruption experiments.

9.3.5 Definitions of Ground Truth, Hallucination, and Transfer Validity

Before discussing judge performance, we clarify the operational definitions used throughout this thesis and acknowledge their limitations.

Ground Truth from Literature (GT Lit): Published expert CLDs define the reference edge set over a fixed variable set. Under this operationalisation, an edge *present* in the expert CLD is treated as a reference-positive relationship; an edge *absent* is treated as reference-negative. For evaluation we define “hallucination” as disagreement with this reference: **FP** = generated edge absent from the expert CLD, and **FN** = expert edge the generator missed. (Throughout Results, “TP/TN/FP/FN” may refer either to generator-vs-GT classes or to the hallucination-detection confusion matrix; where relevant we explicitly tie it to the detection setup used for the F1 plots.)

Ground Truth from Synthetic Corruption (GT Synth): Controlled corruptions (polarity flips, spurious edges, motivation changes) applied to LLM-generated CLDs provide known error labels (**corrupted=True**) with high internal validity. Because we control the corruption process, we know precisely which edges are erroneous. This corruption procedure was adapted from the HaluEval benchmark [97] and therefore provides a comparable reference point separate from our GT Lit definition for CLD hallucination detection.

Hallucination: We operationalise hallucination as disagreement with the chosen ground truth. In GT Lit this includes both false positives (edges generated but absent from the expert CLD) and false negatives (expert edges not generated). This differs from the broader literature, which typically restricts “hallucination” to false positives (confabulations) rather than omissions.

While necessary for quantitative evaluation, treating expert CLDs as absolute ground truth is an acknowledged oversimplification. Expert models synthesize domain knowledge through rigorous GMB processes [5], but they inevitably encode the experts’ biases, knowledge gaps, and scope decisions. An edge absent from an expert CLD (making an LLM generation a false positive) may reflect genuine expert knowledge that the relationship does not exist, but it could also reflect scope limitations, disciplinary blind spots, incomplete literature review, or construct/variable-mapping choices (where a relationship holds for a related construct but not for the specific operationalised variable in the CLD). RQ3 directly explores this issue by using Deep Research to find literature evidence for so-called “hallucinations,” finding that 62.4% of false positives have retrievable causal evidence—suggesting that the ground truth itself may be incomplete. This result is highly interesting, but it is predicated on strong assumptions (relationship isolation, human validation set extrapolation, and applicability), which future work should relax and validate across additional datasets.

Furthermore, the CLD-specific operationalisation of hallucination limits transfer validity to other domains. In free-text generation, “hallucination” typically refers to confabulations or logical inconsistencies. In our context, it refers to deviation from a specific expert consensus model within a constrained variable space.

Readers should therefore interpret our hallucination results as measuring *alignment with expert causal models* rather than universal measures of LLM factuality.

9.4 RQ1a (Correctness Judge): Judge Performance

We compared two judging approaches—correctness-based (evaluating logical consistency) and citation-based (evaluating literature support)—across two ground truth definitions: synthetic corruption (GT Synth) and literature ground truth (GT Lit).

9.4.1 Synthetic Corruption Detection

The Correctness Judge’s strong performance on GT Synth ($F1=0.65\text{--}0.82$ across prompts) suggests that LLMs can often detect *structural* and *semantic* inconsistencies when they are artificially induced—a pattern aligned with the HaluEval benchmark [97]. However, the negligible practical difference between Chain-of-Thought (CoT) [60] and Baseline prompts (despite statistical significance on Friedman tests; see Appendix D) suggests that simply adding reasoning steps may not materially improve detection of synthetic errors in this setup. The Mechanistic prompt’s high recall but lower precision indicates that forcing the model to consider formal causal criteria (adapted from the BHC) biases it toward scepticism, making it a useful screening tool but a weak standalone validator in isolation as precision drops.

9.4.2 Literature Ground Truth Performance

When the ground truth definition shifts from synthetic corruptions to expert literature (GT Lit), detection performance drops substantially ($F1 \approx 0.70 \rightarrow 0.35$). This degradation suggests that while LLMs are adept at spotting the *artificial* inconsistencies produced by corruption strategies (like polarity flips or obviously spurious edges), they struggle to align with the subtler, domain-specific inclusion criteria of expert modellers.

Notably, the Mechanistic prompt—which forces explicit evaluation of causal criteria—was the only variant to maintain significant discriminative power ($AUC > 0.5$) on the Depressive Symptoms and Older Persons ED CLDs, while Baseline and CoT prompts collapsed to near-random performance. This is consistent with the value of structured causal reasoning over generic verification prompts in our experiments.

However, on the Social Norms CLD, even the Mechanistic prompt failed to discriminate, highlighting a potential boundary condition: structured causal criteria appear less effective in domains where causal mechanisms are behavioural and context-dependent rather than physiological. This limitation may stem from the BHC themselves, which were originally designed to assess causality in epidemiology and health sciences. Future research should investigate the generalisation of mechanistic prompting across diverse fields (e.g., economics, physics, psychology) before definitive conclusions about domain-specificity can be drawn.

9.4.3 GT Synth vs. GT Lit Performance Gap

The performance gap between GT Synth ($F1=0.65\text{--}0.82$) and GT Lit ($F1=0.15\text{--}0.55$) represents nearly a halving of detection capability. This gap has several possible explanations:

1. **Systematic corruption patterns:** Synthetic corruptions may introduce more detectable patterns (e.g., polarity flips create logical contradictions) than the subtle disagreements between LLM outputs and expert judgement.
2. **Tacit domain knowledge:** GT Lit includes relationships reflecting tacit expert knowledge not fully captured in judge prompts or retrievable via search.
3. **Scope vs. factual disagreement:** Many GT Lit “errors” may reflect scope decisions (experts chose not to include a valid relationship) rather than factual errors the judge could detect. Note that our system incorporates a mediation check and scope constraints in the generation pipeline; nonetheless, remaining disagreements may reflect limitations in how well such constraints capture expert scoping decisions (an important direction for future research).

This gap has implications for evaluating hallucination detection systems: performance on synthetic benchmarks may overestimate real-world capabilities.

9.4.4 Corruption Type Detection Patterns

The Mechanistic prompt achieved highest recall across all corruption types, with CoT and Baseline substantially lagging behind. This indicates the Mechanistic prompt can detect both structural hallucinations (spurious edges, polarity flips) and narrative hallucinations (motivation corruptions), while maintaining reasonable precision and AUC scores. Polarity flips were generally easier to detect than spurious edges, likely because they create more obvious logical inconsistencies (“A increases B” when the correct relationship is “A decreases B”).

9.4.5 Point-Biserial Correlations

Correlation scores between judge scores and hallucination status dropped significantly when moving from synthetic ($r = -0.4$ to -0.7) to literature ground truth ($r = -0.1$ to -0.25). In some cases, correlations were both small and noisy, failing to attain statistical significance for Baseline and CoT prompts on Depressive Symptoms and Older Persons ED CLDs. Only the Mechanistic prompt retained relatively good performance across CLDs, further supporting its superiority for literature ground truth settings.

9.4.6 Social Norms CLD Anomaly

In the Social Norms CLD, all prompts failed to reliably discriminate hallucinations from valid edges, with high variance and confidence intervals barely exceeding chance level in AUC. This could reflect the complexity of the Social Norms domain, where causal mechanisms are behavioural and context-dependent rather than physiological, or it could indicate limitations in the literature ground truth quality for this domain. The failure mode warrants further investigation into whether domain-adapted prompts or alternative causal frameworks (beyond the BHC) would improve performance.

9.4.7 Judge Score Distributions

Interpreting “judge score by TP/FP/FN/TN” requires care because **TP/FP/FN/TN denote the hallucination-detection confusion matrix** (the same labels and threshold that produce the plotted F1 scores), while the **definition of the true label differs** between GT Synth and GT Lit. In GT Synth (corruption detection), the true label is **corrupted=True**. In GT Lit (literature ground truth), the true label is **hallucination = (FP $\vee$ FN)** with respect to the expert CLD.

In both settings, the basic score ordering (TN/FN high; TP/FP low) is *partly mechanical* because the prediction is defined by thresholding the same score (score ≤ 0.5). The substantive signal comes from *errors*: large or high-scoring **FNs** indicate missed hallucinations/corruptions that the judge still rates as supported, while large **FPs** indicate over-flagging. Comparing GT Synth to GT Lit therefore reflects a shift in difficulty: synthetic corruptions are more detectable (higher AUC/F1), whereas GT Lit disagreements with expert CLDs are harder to separate from true edges, yielding many high-scoring misses and weaker discrimination overall.

9.5 RQ1a (Citation Judge): Judge Performance

The Citation Judge operates on the expectation that retrievable literature evidence is more likely for valid causal claims than for invalid ones. However, this expectation may fail when causal narratives are not well-documented in searchable literature, or when the specific claim has never been studied.

9.5.1 Retrieval Implementation Considerations

The specific retrieval implementation is highly consequential for results. Our implementation searches for the causal narrative (verbatim) via the Brave Search API, retrieves the first five articles (constrained by cost), fetches content using a custom scraper, and passes snippets to the judge for evaluation. This shallow, cost-constrained retrieval may not represent the best possible approach.

This setup also introduces a confirmation-bias risk: because queries are generated from the model’s own causal narrative, retrieval is biased toward finding *supporting* text rather than disconfirming evidence. Future work should investigate broader and deeper search strategies, including actively seeking counter-evidence (where available) and incorporating alternative query formulations (e.g., related constructs, confounders, or competing mechanisms), as explored more extensively in RQ3.

Finally, the generate-then-search pipeline was chosen to remove the constraint of search-then-generate approaches typical of RAG-style literature synthesis: it prioritises recall (generate freely first) at the cost of precision (since post-hoc evidence may be correlational or only partially applicable). The post-hoc search step then retroactively prunes or down-weights weakly supported edges, with results discussed in RQ1b. Adding strict citation and LLM based websearch, following a search-then-generate paradigm, (Nitai_E, mandatory citations) reduced average F1 by 25% relative to the best-performing prompt (Nitai_C) with prompt structure otherwise identical. These results are based on a single-run, small-sample ablation and should be treated as suggestive rather than conclusive.

9.5.1.1 Limitations of generate-then-search in this thesis.

While generate-then-search improves relationship recall by decoupling generation from retrieval, it also shifts burden onto post-hoc validation. First, unconstrained generation can increase false positives: plausible causal narratives may be produced even when the literature is weak, correlational, or non-specific to the exact claim. Second, negative claims (e.g., NONE) are difficult to validate because absence of retrieved evidence can be misinterpreted as support, creating an “evidence of absence” failure mode. Third, the approach remains retrieval-bounded at the validation step: claims with no accessible or discoverable sources cannot be grounded, and shallow, cost-constrained retrieval amplifies false negatives and topical confounds. In particular, terminology mismatch between CLD variable names (and the generator’s narrative phrasing) and the language used in the literature can create systematic false negatives: a true edge may be penalized simply because relevant evidence is described using different constructs, operationalisations, or domain-specific synonyms. Similarly, evidence

may be effectively inaccessible (paywalled, blocked, or only available as indirect/secondary discussion), causing valid edges to appear unsupported. These constraints help explain why citation-based judging did not reliably separate TP/FP on expert CLDs despite high retrieval coverage.

9.5.1.2 Majority voting per citation limitations and future work.

In this thesis, per-citation verdicts are mapped to numeric scores and aggregated with a simple mean (see Section 5.4.3; Appendix Algorithm H.3). While convenient, the mean has important vulnerabilities in a retrieval-based pipeline. It is sensitive to the number of successfully fetched citations (which can vary across edges), and to selection effects introduced by search ranking and fetch failures (missing-at-random is not guaranteed). It can also be dominated by a single outlier source (very strong or very weak), and implicitly treats citations as exchangeable and independent even when sources are redundant or correlated (e.g., press releases, review articles that repeat the same claim).

Future work could explore more robust aggregation schemes, such as using the median per-citation score, or using a top- k / max-based rule that treats the task as “find at least one strong causal source” rather than averaging across mixed-quality evidence. Notably, the implementation already supports an explicit “one causal source found” judging mode that searches for direct causal evidence in at least one citation (Appendix Algorithm H.3; `modules.py` approach `one_causal_source_found`), which may better align with how experts validate causal claims in practice. Finally, the system also computes a qualitative, edge-level aggregate verdict via an LLM aggregator over the per-citation results; evaluating this qualitative aggregation (and comparing it to numeric aggregation rules) is a natural direction for future research. In this thesis, we use the numeric aggregate scores only for quantitative analysis. More broadly, the full distributions of per-citation scores/verdicts (and their relationship to retrieval success, domain, and edge type) could be analysed in more detail; combined with human validation, such analyses can guide which aggregation approach is most effective.

An additional direction, directly connected to the test-time compute framing (Section 3.8), is to introduce an explicit *selection* step: use a separate model (e.g., a lightweight reranker or verifier) to choose the single most informative citation (or a small shortlist) from the retrieved set, and then run the more expensive causal judgement only on that selected evidence rather than averaging across all retrieved items.

9.5.2 Synthetic Corruption Setting

In the synthetic corruption setting (using GPT-5-mini for cost reduction), hallucination detection showed AUC scores below 0.5 on Depressive Symptoms and Older Persons ED CLDs, and only slightly above 0.5 on Social Norms for all prompts. F1 scores ranged from 0.20 to 0.48, substantially worse than the Correctness Judge (0.63–0.82) on the same GT Synth definition. This indicates that citation-based judging is less effective than correctness judging for detecting synthetic corruptions across all prompt variants. However, using GPT-5-mini introduces a model confound relative to the GPT-4.1 judging conditions: differences may reflect model-specific behaviour rather than purely the effect of citation-based verification. Future work should repeat the GT Synth citation-judge experiments on the same judge model as GT Lit (or run both models on both settings) to isolate method effects from model effects.

While prompt variants showed overlapping confidence intervals, Friedman tests (see Appendix D) detected statistically significant differences between prompt variants across CLDs, with Baseline and CoT both outperforming the Mechanistic prompt. This reversal (Mechanistic performing worse for citation judging) may reflect that formal causal criteria interact poorly with the noisy or correlational or otherwise non-causal evidence provided by web search results.

9.5.3 The “Evidence of Absence” Paradox

The hypothesized “evidence of absence” failure mode is that **NONE** (“No Relationship”) claims can be spuriously rewarded when retrieval fails: if the search system does not surface linking evidence, the judge may interpret missing evidence as support for a negative claim. This mechanism is clearly visible in qualitative inspection of individual GT Lit false-negative cases (generator predicts **NONE** on an expert edge) where retrieval returns no direct linking evidence and the judge treats this as partial support for the negative claim. Importantly, this asymmetry *does* also appear in the aggregate score-by-outcome plots: across both correctness- and citation-based GT Lit validation, **false negatives receive high mean scores and often sit close to true negatives** (i.e., they are treated as “clean”), consistent with the judge over-crediting **NONE** rationales under weak evidence conditions (Figures 8.5 and 8.9).

The qualitative asymmetry remains a plausible mechanism at the *instance* level: **NONE** claims require no affirmative evidence, whereas positive causal claims are penalized when retrieved text is merely topically related or lacks mechanistic specificity. This can bias the judge toward over-accepting “no link” rationales under weak retrieval, even if part of the aggregate score ordering is mechanically induced by thresholding on the same judge score.

When the generator incorrectly predicts “No Relationship” (an FN), it provides a motivation stating that “A does not cause B” or “no information is available.” The Citation Judge evaluates this negative claim against retrieved literature. When the retrieval system fails to find specific linking evidence, the judge treats the *absence of evidence* as *evidence of absence*, marking the generator’s “No Relationship” claim as supported.

This creates a perverse incentive: the judge rewards the generator for missing edges whenever retrieval is weak. Conversely, when the generator correctly identifies a relationship (TP), the judge demands explicit affirmative evidence. If retrieved text supports the general link but misses the specific mechanism, the judge penalizes the claim. This asymmetry explains the Citation Judge’s inability to effectively discriminate between valid and invalid edges.

9.5.4 Below-Random AUC Performance

The Citation Judge scoring below 0.5 AUC on Depressive Symptoms and Older Persons ED CLDs runs counter to the expectation that citation evidence would improve detection. A likely explanation is topical alignment confound: semantically similar but factually incorrect relationships may retrieve topically relevant but non-supportive literature, inflating scores for invalid edges.

9.5.5 High Recall Masking Poor Discrimination

The Citation Judge achieved recall of 0.98–0.99 regardless of prompt variant, but with no meaningful TP/FP separation. This indicates the judge flags almost everything as potentially problematic—high recall that is trivially achievable but not useful for selective intervention. The precision-recall trade-off in citation-based approaches remains a significant challenge.

9.5.6 Uleman’s External Validation

To investigate whether low citation judge scores reflected retrieval failures, we conducted a post-hoc validation study on the Uleman’s CLD, an independently expert-validated CLD with researcher-provided references [126].

Across search providers (OpenAlex, Brave, Perplexity, PubMed), all produced uniformly low scores (0.28–0.38), confirming that low scores are not retrieval artefacts specific to our pipeline.

Our chosen combination (ContentScraper + Brave Search) achieved 89% coverage with average judge score 0.290—comparable to alternatives. The uniformly low scores across providers may indicate that the Alzheimer’s domain (Uleman’s CLD focus) has primarily correlational evidence available, as establishing causality in neurodegenerative disease is notoriously difficult.

Using Jina AI Reader (a paywall-bypassing document fetcher with LLM-optimised content extraction) would achieve 100% coverage with similar judge scores, but its cost precluded use in this study. We note this as a limitation that could be addressed with specialized fetchers in future work.

9.5.7 Physics CLD Validation and Retrieval Limitations

To disentangle judge quality from retrieval effects, we also evaluated the judge on a small suite of physics-based CLDs with highly reliable causal structure (Thermostat control, Lotka–Volterra predator–prey dynamics, RC circuit charging, and water tank draining), totaling 31 edges grounded in well-established physical laws and standard textbook treatments. In this setting, the Correctness Judge achieved perfect agreement (31/31 edges judged **CORRECT**), providing a strong sanity check that the judge can recognise clear causal relationships when the underlying mechanisms are unambiguous.

In contrast, citation-based evaluation on the same physics CLDs was dominated by content accessibility rather than causal ambiguity. Although references were expert-curated, many sources are paywalled (publisher textbooks, institutional DOI landings) or otherwise difficult to scrape, leading to **NO_CITATION** for 14/31 edges (45%). Under our half-credit scheme for partial support, overall citation approval was 35.5%; however, conditional on successful retrieval, 82.4% of edges were at least partially supported (Fully/Partially supported) by the fetched excerpts, meaning they would not be flagged as hallucinations using our Judge score ≤ 0.5 = hallucination thresholding. The most severe failure case was the RC circuit CLD, where the majority of candidate sources are modern textbooks or restricted publisher pages, illustrating that even scientific domains where causality is unambiguous can yield low citation scores under shallow, web-only retrieval.

In our main GT Lit citation-judging dataset (Depressive Symptoms, Social Norms, Older Persons ED; baseline runs), retrieval success was near-saturated: 100.0% (Depressive Symptoms), 99.7% (Social Norms), and 97.9% (Older Persons ED). This suggests that, in the health-domain settings central to this thesis, citation judging is typically *not* bottlenecked by outright retrieval failure; instead, performance hinges on whether retrieved snippets contain sufficiently specific causal evidence rather than merely topically related (often correlational) discussion. Combined with the confirmation-bias risk of narrative-based querying, high retrieval rates can therefore coexist with weak discrimination: the system almost always finds *something*, but “something retrieved” is not equivalent to “causal mechanism evidenced.”

Together with the Uleman’s provider comparison, these results clarify two distinct sources of low citation-judge performance: (i) *retrieval/fetching limitations* can depress citation scores even when the causal claim is correct (physics CLDs), and (ii) *domain evidence limitations* can produce uniformly low scores even when retrieval coverage is high (Uleman’s). Both evidence points motivate investigating a deeper and broader search strategy, as carried out in RQ3: (a) to increase the chance of finding *direct causal* evidence (which is rarer and harder to retrieve than correlational discussion), and (b) to surface alternative accessible sources (e.g., preprints, open-access versions, reviews, meta-analyses) when primary references are fetch-constrained.

9.6 Mechanisms of Judge Failure

Investigation into edge-level reasoning reveals three distinct failure modes explaining poor performance of both Citation and Correctness judges. These mechanisms—Evidence of Absence, Plausibility Bias, and Partial Evidence Acceptance—create structural blind spots that quantitative metrics alone fail to capture. All qualitative examples in this section are drawn directly from the per-edge judged spreadsheets and run logs saved under `final_runs/` (see footnotes for exact paths).

9.6.1 The “Evidence of Absence” Paradox (Citation Judge)

The Citation Judge exhibits a perverse incentive structure when evaluating negative claims, and the same asymmetry can also surface in the Correctness Judge. For the Citation Judge, when retrieval fails to find relevant literature—often due to keyword mismatches, variable specificity, or simply the rarity of *direct* causal evidence—the judge can treat *absence of evidence* as *evidence of absence*, validating NONE with non-trivial confidence. For the Correctness Judge, an analogous failure mode arises because NONE claims are hard to falsify without external evidence: a coherent “no clear mechanism” rationale often contains no internal contradiction for the judge to seize on, so it can assign partial (or even high) support despite the edge being present in the expert graph.

For example, in the Older Persons ED CLD evaluation (CoT prompt, run 2), the judge evaluated an expert edge that the generator missed (a GT Lit **false negative** under the generator-vs-GT definition) where the generator asserted a NONE relationship between “Functioning of healthcare professionals” and “Availability of alternatives for ED”.¹

- **Motivation:** “Functioning of healthcare professionals does not directly cause availability of alternatives for ED. There is no clear causal mechanism...”
- **Judge reasoning:** “...it does not report that changes in how healthcare professionals function directly create or make available alternative care options...”
- **Outcome:** Aggregate verdict **Partially supported**, score **0.5**

This edge exists in the validation graph (hence FN), yet the judge assigned a non-trivial score by treating “lack of direct evidence in retrieved snippets” as support for the negative claim.

This pattern is visible in the judge-score distributions: across multiple settings, **TN and FN outcomes receive substantially higher mean scores than TP and FP**, and critically, the FN bar is often close to TN (e.g., in citation-based corruption detection, $FN \approx TN$ in the score-by-outcome panel; Figure 8.7). In other words, many missed edges are scored as “clean” rather than being pushed below the hallucination threshold, which is exactly what the evidence-of-absence asymmetry predicts.

9.6.2 Plausibility Bias (Correctness Judge)

The Correctness Judge, lacking external retrieval, relies entirely on internal logical consistency. This makes it highly susceptible to “Plausible Hallucinations”—spurious edges that are factually unverified but logically coherent.

¹Source artefact (GT Lit, citation judge, Older Persons ED, run 2, CoT prompt): `final_runs/RQ1a_ground_truth_citation/emergency_department/run_2/judged_older_persons_emergency_department_visits_and_interactionstitled_spreadsheet_cot_20251118_100404_20251118_105236.xlsx`.

For example, in the Older Persons ED CLD (mechanistic prompt, run 2), a corrupted edge between “Healthy lifestyle” and “Acute care demand” received the maximum score.²

- **Spurious Motivation:** “Healthy lifestyle... promotes greater health awareness... reduces likelihood of escalating... direct negative effect.”
- **Judge Reasoning:** “The explanation clearly articulates a plausible causal mechanism... Reasoning is internally consistent... CORRECT.”
- **Outcome:** Score **1.0**, despite being corrupted

The judge validated the *logic* of the corruption, not its factuality. In this evaluation file, 204/257 corrupted edges (79%) received aggregate scores ≥ 0.5 . Under synthetic “spurious motivation” corruptions designed to sound plausible, correctness-only judging collapses into a coherence check.

9.6.3 Partial Credit and Non-Scientific Evidence Acceptance (Citation Judge)

Even when edges are labelled corrupted, the Citation Judge can assign high scores because: (i) it grants partial credit for plausibly related mechanisms, and (ii) retrieval may return non-peer-reviewed sources containing supportive statements. Although whitelisting of scientific domains and BHC prompting partially mitigates this, the degree to which this was mitigated was not systematically evaluated. Future work should systematically quantify this effect by stratifying retrieved sources by evidence tier (e.g., systematic reviews/RCTs vs. observational studies vs. expert opinion) and measuring judge discrimination and calibration under controlled retrieval settings, with and without domain whitelisting and BHC prompting.

In the Older Persons ED evaluation, we found high-scoring corrupted edges.³

- **Non-scientific source:** An edge between “Proactive attitude” and “Language and cultural barriers” received score 1.0 (Fully supported) based on a non-peer-reviewed blog providing prescriptive “strategies” rather than empirical evidence.
- **Partial credit on intermediate mechanisms:** An edge between “Accessibility of information” and “Knowledge” received score 0.625 by validating the intermediate concept of “information overload,” though direct causal evidence for the final claim was absent.

These cases demonstrate that citation judging is sensitive to retrieval artefacts (including non-scientific sources) and partial-credit aggregation on plausible intermediate mechanisms.

²Source artefacts (GT Synth, correctness judge, Older Persons ED, run 2, mechanistic prompt): judged spreadsheet `final_runs/RQ1a_corruption_detection_correctness_final/emergency_department/run_2/judged_older_persons_emergency_department_visits_and_interactionstitled_spreadsheet_mechanistic_20251116_020717_20251116_021236.xlsx`; run log shows repeated max-score assignments for Healthy lifestyle → Acute care demand leading to an ED visit, e.g. `final_runs/RQ1a_corruption_detection_correctness_final/emergency_department/run_2/judging_20251116_015224.log`.

³Source artefacts (GT Synth, citation judge, Older Persons ED, run 2): judged spreadsheet `final_runs/RQ1a_corruption_detection_citation_gpt5mini/emergency_department/run_2/judged_citation_older_persons_emergency_department_visits_and_interactionstitled_spreadsheet_mechanistic_20251204_092254_20251204_102947.xlsx`; run log contains per-edge updates including (Proactive attitude)-[:POSITIVE]->(Language and cultural barriers) with `aggregate_score=1.0` and (Accessibility of information)-[:NEGATIVE]->(Knowledge) with `aggregate_score=0.625`: `final_runs/RQ1a_corruption_detection_citation_gpt5mini/emergency_department/run_2/judging_20251204_065138.log`.

9.7 Empirical Validation: LLM–Human Agreement in CLD Evaluation

To validate the applicability of LLM-as-a-Judge to causal loop diagram evaluation, we conducted human validation experiments measuring inter-rater agreement between LLM citation judges and a human validator across 72 causal edges from three independent datasets spanning two CLDs and two judge models (GPT-4.1 and Claude Sonnet 4). We compare our LLM–human agreement to a human–human inter-rater baseline ($\kappa \approx 0.60$) from prior GMB validation studies on comparable expert tasks.

9.7.1 Results.

The combined analysis yielded substantial agreement: Cohen’s $\kappa = 0.618$ with 80.6% exact agreement rate, and Pearson correlation $r = 0.732$ ($p < 0.001$) between LLM and human scores. This level of agreement meets the human inter-rater baseline ($\kappa \approx 0.60$) established in prior GMB validation studies [116].

9.7.2 Systematic leniency bias.

Analysis of disagreements revealed a consistent pattern: 100% of disagreements (14/14 cases) showed the LLM judge being more permissive than the human validator (i.e., the judge scored evidence higher). The LLM never under-estimated evidence support (zero false negatives), but did over-estimate support where the human assigned lower scores. This asymmetric error profile suggests LLM citation judges provide useful but permissive screening—appropriate for identifying low-confidence edges requiring human review, but insufficient for strict validation without expert oversight.

9.7.3 Domain and model variability.

Agreement varied substantially across datasets: from moderate ($\kappa = 0.461$, 66.7% agreement) on Social Norms edges to almost perfect ($\kappa = 0.923$, 96.7% agreement) on Depressive Symptoms edges judged by Claude. Notably, judge model had a larger effect than domain: on the same Depressive Symptoms CLD, the Claude judge achieved almost perfect agreement with the human validator ($\kappa = 0.923$) while the GPT-4.1 judge showed only moderate agreement ($\kappa = 0.435$, 70.0% agreement). This pattern suggests that certain judge models may assess causal evidence in ways that align more closely with human judgement—an important consideration for deployment across diverse CLD construction settings.

9.8 RQ1b: Corrector Experiments

9.8.1 Corrector Action Mix and Repair Limitations

Action logs indicate the corrector predominantly uses *revision* and *type-change* operations, with an action mix that broadly matches the expected distribution given the corruption process (motivation corruption vs. structural/type errors). On synthetic corruptions, revisions occur somewhat more often than the nominal 1/3 rate (with correspondingly fewer type changes), while on GT Lit the macro-average action mix shows $\sim 2/3$ revise to 1/3 type-change ratio.

However, *action execution* does not reliably translate into *quality improvement*. Across prompts, the corrector shows clear setting dependence: modest improvements on GT Synth do not generalise to GT Lit, and judge-score gains are more consistent than F1 gains. This supports interpreting the corrector as an inconsistent repair mechanism that can satisfy the judge without reliably improving expert-grounded alignment.

9.8.2 Goodhart’s Law Manifestation

The corrector improves F1 scores slightly in the GT Synth setting but not for GT Lit. Instead, it appears to optimise for improving judge scores—which it succeeds at, albeit marginally—rather than improving ground truth alignment. This is consistent with a Goodhart-like dynamic: “When a measure becomes a target, it ceases to be a good measure.” In this setup, the corrector appears to satisfy the judge rather than reliably improve CLD quality.

Different prompts showed no statistically significant performance differences in either setting, suggesting that prompt engineering has limited impact on corrector behaviour. This points toward the need for architectural solutions (e.g., reinforcement learning from human feedback on correction quality) rather than prompting improvements.

9.8.3 Per-CLD Variation in Corrector Effectiveness

Corrector effectiveness varied substantially across CLDs:

- **Older Persons ED:** Consistent F1 improvement across all prompts
- **Social Norms:** F1 degradation ($\Delta F1 = -0.072$)
- **Depressive Symptoms:** F1 degradation across all prompts ($\Delta F1 \approx -0.040$)

This heterogeneity is expected under our pipeline design: the corrector is *supervised by judge feedback*. When the judge provides informative signals, correction can repair real errors; when the judge is weak or systematically biased, correction becomes a “learning-from-noisy-labels” problem and can amplify the judge’s failure modes rather than improve expert-grounded alignment.

This interpretation is consistent with the RQ1a results: the Social Norms CLD is a boundary case where even the Mechanistic correctness judge fails to discriminate on GT Lit, while Older Persons ED retains meaningful discrimination. In such a setting, the corrector is more likely to perform harmful edits (e.g., over-pruning plausible edges or rewriting relationships toward generic-but-judge-pleasing formulations), which would reduce GT Lit F1 even if judge scores improve. Depressive Symptoms sits between these extremes: it contains a mixture of more “biomedical/mechanistic” relationships (where judge feedback may be useful) and more context-dependent psychosocial relationships (where the judge may be less reliable), making net effects prompt- and edge-mix-dependent.

Two concrete analyses could strengthen this explanation in future work. First, test whether per-CLD $\Delta F1$ correlates with judge quality (e.g., AUC on GT Lit) and whether the correlation is mediated by *which component* of F1 moves (precision vs. recall). Second, characterise correction edits by outcome class: for each CLD, measure how often edits convert $FP \rightarrow TN$ or $FN \rightarrow TP$ versus causing regressions ($TP \rightarrow FN$, $TN \rightarrow FP$), and relate these transitions to judge confidence and error type (polarity, spurious edge, motivation/revision). Together, these would directly validate whether correction success is primarily limited by judge signal quality and domain-specific causal ambiguity.

9.8.4 Implications for Production Pipelines

The setting dependence and Goodhart-like effects suggest that deploying current LLM correctors in production CLD pipelines would not reliably improve expert-grounded CLD quality. In particular, the corrector’s consistent judge-score improvements alongside weak or negative GT Lit F1 changes indicate a risk of optimizing for the evaluation signal rather than true causal accuracy. In practice, this motivates conservative deployment (human-in-the-loop review, strong audit logging, and domain-specific validation) rather than fully automated correction.

9.9 RQ2: UQ Metrics

9.9.1 Single Metric Performance

Individual uncertainty quantification metrics achieved only modest discrimination:

- **Cosine Similarity:** AUC=0.671 (highest, but counterintuitive direction)
- **Perplexity:** AUC=0.563
- **Max Window Entropy:** AUC=0.554
- **Min Probability:** AUC=0.479 (below chance)

Logprob-derived metrics (Perplexity, Max Window Entropy, Min Probability) perform near chance level, indicating weak discriminative power for hallucination detection in isolation.

Several factors may explain this limited performance. First, token-level logprobs conflate different uncertainty types—lexical, syntactic, and ontological—that are not necessarily indicative of hallucinations. Grammatical uncertainty about word choice does not imply factual uncertainty about the causal relationship. Second, we employed relatively naive sequence-level aggregations (mean, max, min over token sequences), which may obscure more informative signals. Related work [34] has shown that examining logprobs for specific semantically meaningful tokens (e.g., the probability assigned to “causes” in “A causes B”) may be more informative than whole-sequence statistics.

For our purposes, sequence-level indicators were motivated by the original RQ3 design: if effective, these cheap metrics could trigger more expensive citation retrieval, with the causal narrative serving as input to that process. The failure of these metrics undermined this design.

9.9.2 Counterintuitive Cosine Similarity Direction

Cosine similarity between embedded articles and causal narratives was intended as a baseline metric, with the hypothesis that higher similarity would correlate with lower hallucination probability (finding semantically aligned evidence should indicate factual grounding). However, the observed correlation was positive ($r=+0.153$, $p<0.001$)—higher similarity associated with *higher* hallucination probability.

Several explanations are plausible:

1. **Semantic-causal conflation:** Semantic similarity may capture topical alignment without establishing causal evidence.

2. **Language mismatch:** True positives validated by experts may use different terminology than appears in searchable literature, yielding lower similarity scores despite factual correctness.
3. **False positive literature support:** False positives (the majority of errors in our experiments) may actually have literature support—higher cosine similarity could indicate that these “hallucinations” are in fact literature-supported relationships that experts chose to exclude for scope reasons.

The third explanation connects directly to RQ3 findings, where Deep Research found evidence for 62.4% of false positives.

9.9.3 Non-Normality and Statistical Approach

All UQ metric distributions rejected normality at the 0.05 significance level (Shapiro-Wilk tests; see Appendix D), consistent with the visual distributions, which show bimodality, skewness, and long tails in some cases, and thus justify non-parametric Mann-Whitney U tests for group comparisons (see Appendix D). Mann-Whitney U computes a rank-sum statistic measuring how often values from one group exceed values from the other, with statistical significance indicating distributional shift. Most metrics showed significant differences across CLDs, suggesting some exploitable signal exists despite weak individual AUC values.

9.9.4 Ensemble Classifier Performance

Given weak single-metric performance, we investigated whether ensembles could achieve better discrimination. Four classifiers were trained: Logistic Regression (LR), Random Forest (RF), Gradient Boosting (GB), and Neural Network (NN).

Feature Selection: Recursive Feature Elimination (RFE) revealed that feature importance varies by classifier under block-level cross-validation. Logistic Regression relies primarily on **Perplexity** and **Cosine Similarity** (both selected in all folds), while Random Forest and Gradient Boosting typically select a broader set of features. Permutation testing confirmed Cosine Similarity as the most important feature across all classifiers (measured by average AUC drop when shuffled), followed by Perplexity, Min Probability, and Max Window Entropy. The consistent inclusion of at least one logprob-derived feature alongside Cosine Similarity suggests complementary information content.

In-Distribution Performance: With block-level splitting (held-out *blocks*, not edges), Random Forest achieved Test AUC=0.809 on the held-out blocks, indicating strong discrimination when training and test data are drawn from the same set of CLDs.

9.9.5 Cross-Domain Generalisation Failure

Leave-one-CLD-out cross-validation revealed clear performance drops:

- **RF:** 0.809 $\rightarrow$ 0.629 ($\Delta=0.179$)
- **GB:** 0.765 $\rightarrow$ 0.645 ($\Delta=0.120$)
- **NN:** 0.739 $\rightarrow$ 0.648 ($\Delta=0.092$)
- **LR:** 0.702 $\rightarrow$ 0.664 ($\Delta=0.039$)

The stronger in-distribution RF performance remains partly misleading as a proxy for cross-domain usefulness: leave-one-CLD-out results show that these models still exploit CLD-specific patterns that do not generalise.

Neural Network and Logistic Regression show more graceful degradation, with LR slightly exceeding NN’s out-of-CLD performance (0.664 vs. 0.648) despite lower in-distribution scores.

For practical deployment, we recommend: (1) Neural Networks when sufficient training data is available from the target domain; (2) Logistic Regression when data is limited, as it requires less data and provides nearly equivalent out-of-domain generalisation.

However, even the best out-of-CLD AUC (≈ 0.66) is insufficient for reliable hallucination detection. These metrics are better suited to flagging edges for human review than for automated accept/reject decisions. Looking at the uncertainty estimates (95% CIs), performance is consistent with barely above chance in some out-of-CLD settings, confirming that LUQ metrics are not sufficiently discriminant for cross-domain hallucination detection.

9.9.6 Implications for Original RQ3 Design

The poor cross-CLD generalisation undermined the original RQ3 design of using UQ metrics as early hallucination indicators to trigger expensive test-time compute. Optimal thresholds varied substantially across domains (e.g., a cosine similarity threshold optimal for Depressive Symptoms would misclassify edges in Social Norms), making universal threshold-based triggering unreliable. Per-domain calibration would require enough labelled edges to estimate stable metric distributions in the target CLD—data that is unavailable early in CLD construction and likely to drift as new edges are generated—thereby negating the efficiency gains. This finding motivated the pivot to Deep Research as an alternative approach.

9.10 RQ3: Deep Research Validation

9.10.1 Methodological Pivot and Rationale

9.10.1.1 Original Design and Its Prerequisites.

The original RQ3 design proposed *adaptive test-time compute allocation*: using UQ metrics as early hallucination indicators to selectively deploy the LLM-as-a-corrector, improving efficiency while maintaining accuracy. This design was predicated on two assumptions:

1. **RQ2 Prerequisite:** UQ metrics would generalise across CLDs, enabling threshold-based triggering.
2. **RQ1b Prerequisite:** The LLM-as-a-corrector would effectively repair detected hallucinations.

Our findings undermined both prerequisites. RQ2 showed that while UQ metrics can achieve strong within-CLD discrimination (RF Test AUC=0.809 under block-level splitting), leave-one-CLD-out validation revealed poor generalisation, with optimal thresholds varying substantially across domains. RQ1b showed that corrective actions do not reliably improve expert-grounded (GT Lit) alignment and that judge-score gains are more consistent than F1 gains, making “adaptive triggering” on an unreliable downstream repair step untenable.

9.10.1.2 The Pivot.

Given these findings, we pivoted RQ3 to address: *Can automated literature search validate causal claims?* This reframes the problem from “correcting hallucinations” to “externally validating claims against scientific

literature”—a capability valuable regardless of UQ metric generalisation. The Deep Research system represents an alternative test-time compute strategy: rather than correcting errors, it seeks external evidence to validate claims, potentially flagging low-evidence edges for human review. Importantly, while UQ metrics failed as a reliable trigger, the Mechanistic Correctness Judge provides a *compute-cheap* uncertainty signal that can be used to selectively deploy Deep Research in an exploratory “smart-trigger” policy, partially recovering the original efficiency motivation (see Adaptive-Trigger Deployment Analysis below).

9.10.2 Design Rationale

While we did not perform component-wise ablation studies to inform the design of the Deep Research system, we intentionally prioritised breadth over minimality given the exploratory nature of the late pivot. In the spirit of exploration, we therefore forgo Occam’s razor—the principle of choosing the simplest explanation consistent with the data—and its modelling analogue, the Minimum Description Length (MDL) principle: choosing the model that minimises summed model and data complexity. The multi-agent design principles underlying the system are reviewed in Section 3.8, with full architectural details in Section 5.7. This substantial per-edge compute budget was intended to maximise the probability of finding causal evidence, establishing whether this direction merits further investigation.

9.10.3 Detection Rate Findings

Deep Research found direct causal evidence for 70.5% of true positives versus 62.4% of false positives—a moderate TP–FP discrimination gap of 8.1 percentage points (Cohen’s $h = 0.17$). The high FP validation rate indicates that many “false positives” (relative to the expert CLD) have retrievable causal evidence of some form, and that automated literature search alone does not cleanly separate expert-included from expert-excluded edges.

This has practical implications: literature retrieval alone is insufficient for hallucination filtering. Many relationships excluded by domain experts nonetheless have retrievable evidence, whether because experts made scope decisions, because evidence supports a related-but-not-identical construct mapping, or because the evidence matches only part of the edge claim (mechanism specificity), motivating the human applicability analysis reported below.

9.10.4 Per-CLD Detection Rate Variation

Detection rates varied substantially by domain (*pp* denotes percentage points):

- **Depressive Symptoms:** TP–FP gap = +36 pp ($h = 0.75$, significant)—Deep Research provides meaningful discrimination
- **Social Norms:** TP–FP gap = +13 pp ($h = 0.30$)—modest discrimination
- **Older Persons ED:** TP–FP gap = +6 pp ($h = 0.14$)—negligible discrimination

This domain dependence suggests that Deep Research utility correlates with literature availability and causal evidence density in each field. The Depressive Symptoms CLD, drawing on well-studied psychological relationships, benefits most from automated literature search in terms of the TP–FP discrimination gap. However, since false positives frequently retrieved causal evidence, one could also argue that CLDs with the smallest gap “benefit” the most if Deep Research is interpreted primarily as an enrichment signal rather than as a hallucination filter.

9.10.5 Confidence Score Uninformativeness

Deep Research’s self-reported confidence scores provided no discriminative power between edge classes. Mean confidence scores clustered within 1 point on a 10-point scale, with substantial overlap across TP/FP/FN within each domain. Self-reported confidence is therefore not useful as an accept/reject criterion—the system cannot self-assess the validity of its findings.

9.10.6 Human Validation and Applicability Analysis

We conducted human validation on a stratified random sample of 50 edges drawn from the subset where Deep Research reported direct causal evidence (`direct_causal_found=True`). Stratification was performed jointly across CLDs (Depressive Symptoms, Social Norms, Older Persons ED) and edge classes (TP/FP/FN), using proportional allocation across CLD×class strata with a minimum of one edge per non-empty stratum (random seed = 42). The final sample comprised 9 TP, 32 FP, and 9 FN edges (Depressive Symptoms $n = 10$, Social Norms $n = 3$, Older Persons ED $n = 37$). The dominant failure mode was variable mismatch: Deep Research often found causal evidence for related but different constructs or measures (10/50 edges). Invalid or irrelevant evidence was uncommon (5/50 edges).

Using an elbow-selected threshold of applicability ≥ 0.7 on the human-scored “Fully applicable” metric, 33/50 edges (66%) passed the threshold; among passing edges, 30/33 (90.9%) were judged correct-causal. This calibration enables more realistic extrapolation of enrichment effects.

9.10.7 Ground Truth Enrichment Analysis

Under the strong assumption that DR-validated edges represent genuine causal relationships that experts omitted, we computed optimistic upper-bound F1 improvements:

- **Scenario 1 (FP→TP promotion):** F1 increases from 0.267 to 0.705
- **Scenario 2 (+ FN recovery):** F1 increases from 0.267 to 0.779

Human-calibrated estimates (accounting for the 66% pass rate at $t = 0.7$) yield more conservative but still substantial improvements. The 36 residual FNs (edges in GT that were neither generated nor found by DR) likely represent tacit expert knowledge not captured in searchable literature.

9.10.8 Adaptive-Trigger Deployment Analysis

To evaluate selective deployment efficiency, we triggered Deep Research only on edges flagged by the Mechanistic Correctness Judge (score ≤ 0.5). This triggered 50% of edges while achieving 57% of the full-deployment F1 gain—a 15% relative efficiency improvement (0.216 vs. 0.189 Δ F1 per \$100 compute). This proof-of-concept demonstrates that judge-triggered Deep Research can improve cost-efficiency while retaining most of the validation benefit.

9.10.9 Limitations of the Pivot

We acknowledge that the RQ3 pivot occurred late in the research timeline, constraining investigation depth:

- **Single-run design:** No replication prevents uncertainty quantification for detection rates.
- **Limited ablation:** Search parameters, iteration counts, and evidence thresholds were not systematically varied.
- **Model confound:** Deep Research uses GPT-5/GPT-5-mini, while CLD generation used GPT-4.1. Superior DR performance may partially reflect GPT-5’s enhanced reasoning capabilities rather than the multi-agent architecture. A controlled ablation comparing single-model direct search against the full Deep Research pipeline on identical model versions would be required to isolate the agentic contribution.
- **Confirmation bias risk:** Literature search inherently favours finding supporting evidence over disconfirming evidence.
- **TN edges unanalysed:** Cost constraints prevented analysing true negative edges.

Despite these constraints, the findings are informative: the moderate TP-FP discrimination gap and high FP validation rate point to important limitations of literature-based validation for CLD hallucination detection.

9.10.10 Framing: Exploratory Pilot Study

We present the Deep Research investigation as an **exploratory pilot study** rather than a fully validated methodology. The 285-edge evaluation establishes baseline detection rates and highlights key constraints (domain dependence, limited power from single-run deployment, and evidence applicability/specificity), providing groundwork for future research on evidence-based causal claim validation.

9.11 Synthesis Across Research Questions

Taken together, our findings paint a nuanced picture of automated CLD validation feasibility:

LLMs can generate CLDs that substantially exceed random baselines ($2.2\text{--}3.4\times$ F1 improvement), suggesting that the models encode non-trivial domain-relevant structure. However, absolute performance remains modest ($F1 \approx 0.27$), indicating substantial room for improvement.

LLM judges can detect synthetic corruptions ($F1=0.65\text{--}0.82$) but struggle with literature ground truth ($F1=0.15\text{--}0.55$). The performance gap suggests that artificial inconsistencies are easier to detect than subtle expert-model disagreements. Mechanistic prompting with explicit causal criteria appears to provide the most robust discrimination in our experiments.

LLM correctors tend to optimise for judge satisfaction rather than ground truth improvement. Judge scores improve more consistently than GT Lit F1, and the action mix (revise vs. type change) does not guarantee quality gains. This Goodhart-like behaviour suggests that human-in-the-loop correction remains important.

UQ metrics provide weak, domain-specific signals that do not generalise well across CLDs. While ensemble classifiers can achieve strong in-distribution performance (RF Test AUC=0.809 under block-level splitting), leave-one-CLD-out validation reveals substantial degradation ($AUC \approx 0.63\text{--}0.66$). These metrics are better suited to flagging edges for review than automated filtering.

Literature retrieval finds evidence for a majority of “hallucinations” (62.4% of FPs), suggesting that expert CLDs may be incomplete rather than representing absolute ground truth. The distinction between “hallucination” and “expert scope decision” can be unclear in practice.

The original adaptive test-time compute vision remains theoretically motivated but practically challenged. Cross-domain threshold calibration and improved correctors would be required to realise efficiency

gains. Deep Research offers an alternative approach: while it cannot reliably discriminate expert-validated from LLM-generated edges in aggregate (TP–FP gap = 8.1 pp, $p = .38$), it does distinguish generated edges (TP and FP) from missed edges (FN), with significant TP–FN ($h = 0.56$) and FP–FN ($h = 0.39$) gaps. Utility is domain-dependent: Depressive Symptoms shows strong TP–FP discrimination (+36 pp), while Social Norms (+13 pp) and Older Persons ED (+6 pp) show weaker separation.

- Ground truth enrichment analysis suggests substantial potential improvements—human-calibrated extrapolations yield $\Delta F1$ of +0.26 to +0.32 when promoting DR-validated false positives and recovering missed edges, though these rely on strong assumptions about applicability generalisation. Adaptive-trigger deployment (using Mechanistic Judge scores to selectively invoke DR) achieves 57% of the full-deployment enrichment gains at 50% of the cost, demonstrating a viable path to improved test-time compute efficiency.

9.12 Practical Recommendations

Based on our findings, we offer the following recommendations for practitioners using LLMs for CLD generation:

1. **Avoid relying solely on automated correction.** Current LLM correctors show setting dependence and Goodhart-like effects: judge-score gains do not reliably translate into GT Lit quality improvements. Human review remains important for quality assurance.
2. **Treat UQ metrics as flagging tools, not filters.** Logprob-based and embedding-based metrics can help identify edges for human review but should not be used for automated accept/reject decisions, especially across domains.
3. **Expect domain-dependent performance.** Judge and metric effectiveness varies substantially by CLD domain. Validate on held-out data from your target domain before deployment.
4. **Consider literature retrieval for enrichment, not filtering.** Deep Research can identify potentially valid relationships that experts may have omitted, but evidence often varies in applicability/specificity to the exact edge claim, requiring calibration and expert review.
5. **Maintain human domain expertise.** The gap between synthetic and literature ground truth performance, combined with the high rate of DR-validated “hallucinations,” underscores that human expert judgement remains essential for CLD validation.

9.13 Limitations

9.13.1 Single Model Dependency

All experiments used GPT-4.1 as the primary LLM for generation and GPT-4.1/GPT-5-mini for judging. Performance characteristics may differ substantially for other foundation models (Claude, Gemini, open-source alternatives). The judge-corrector framework’s effectiveness is likely model-dependent, and our conclusions may not transfer to other LLM families with different training data, architectures, or alignment procedures.

9.13.2 Health Domain Focus

All three CLDs focus on health-related domains (Depressive Symptoms, Older Persons ED visits, Social Norms in health contexts). Generalisation to other scientific domains (physics, economics, ecology) cannot be assumed without validation on domain-diverse CLD corpora. We have, however, done some validation on physics CLDs of citation and correctness judge functioning, finding positive results, to reduce the risk of domain-specific biases in our findings.

9.13.3 Ground Truth Construct Validity

Our GT Synth (synthetic corruption) and GT Lit (literature ground truth) represent different constructs. GT Synth provides high internal validity (we know exactly what was corrupted) but may not capture the full spectrum of natural hallucination types. GT Lit provides ecological validity (expert CLDs represent real-world targets) but conflates genuine errors with scope decisions. The performance gap between these conditions reflects this construct difference as much as detection difficulty.

9.13.4 Sensitivity Analysis Constraints

Due to computational costs, systematic local and global sensitivity analyses were not conducted. A defensible global sensitivity analysis would require varying not only inference hyperparameters (e.g., temperature and top- p), but also *problem instances* (CLD domains/variants), because optimal settings may be domain-dependent. Even a modest 5-level grid over temperature and top- p yields $5^2 = 25$ configurations; replicated across three seeds and three CLDs this becomes $25 \times 3 \times 3 = 225$ full CLD generations, before accounting for additional prompt variants or more CLD datasets. Because each CLD run evaluates a dense edge space ($|V|(|V| - 1)$ ordered pairs) and can trigger downstream judging and correction, total API calls and cost scale multiplicatively, rendering comprehensive sensitivity mapping infeasible within our budget constraints.

We did not explore interactions between temperature and top- p , prompt variants, and CLD domain (i.e., whether the same settings generalise across domains), or their joint effects on generation quality. While our generator temperature ANOVA (Section 9.3; see Appendix D) showed no significant effect within the tested range, this represents a limited probe rather than comprehensive sensitivity mapping.

LLM parameter choice justifications. Key LLM parameters—generator model (GPT-4.1), judge temperature ($T = 0.0$), corrector temperature ($T = 0.7$), and corruptor configuration—were selected through theoretical arguments and limited empirical validation rather than exhaustive search (see Chapter 7). Generator model selection was based on a single-run comparison across three candidates; judge/corrector temperatures followed established recommendations from the LLM-as-a-Judge literature [21, 77, 101]; corruptor settings matched generator parameters to minimise stylistic confounds. These choices are defensible but not optimised.

Classifier hyperparameters. The ensemble classifiers (Random Forest, Gradient Boosting, Logistic Regression, Neural Network) used fixed hyperparameters without systematic tuning: RF and GB used 100 estimators; MLP used hidden layers (32, 16); Logistic Regression used sklearn defaults. For performance estimation and feature selection we used **block-level** cross-validation (3-fold GroupKFold over CLD $\times$ run blocks) and a single held-out block split for Phase 5; no hyperparameter search (e.g., GridSearchCV, RandomizedSearchCV) was conducted. Classifier performance could potentially improve with tuned hyperparameters, though the leave-one-CLD-out generalisation gap (Phase 5 $\rightarrow$ Phase 6) suggests that overfitting to CLD-specific patterns—rather than suboptimal hyperparameters—is the primary limitation.

Temperature relevance for future work. Temperature sensitivity analysis may become less relevant as frontier models evolve. Several recent models (e.g., OpenAI’s o1/o3 reasoning models, some Claude configurations) do not expose temperature controls or use alternative sampling strategies (e.g., best-of- n with internal verifiers). Future replication studies should assess whether temperature remains a meaningful hyperparameter for the specific models employed.

9.13.5 Human Validation Sample Size

Human validation was conducted on 72 edges (LLM-human agreement) and 50 edges (DR applicability). While sufficient for estimating effect sizes and identifying patterns, larger samples would provide tighter confidence intervals and enable subgroup analyses.

9.13.6 Statistical Power Asymmetry & Sample Size Considerations

The nested data structure creates a power asymmetry: edge-level analyses ($n = 31,704$ edges) achieve power ≈ 1.0 , enabling detection of even trivially small effects that may lack practical significance. Conversely, CLD-level analyses ($n = 3$ domains) are substantially underpowered. Under our edge-slot representation, CLDs contribute different numbers of ordered (source, target) pairs ($|V|(|V| - 1)$): Social Norms has $10 \times 9 = 90$, Depressive Symptoms has $14 \times 13 = 182$, and Older Persons ED has $34 \times 33 = 1,122$. As a result, edge-level statistics are dominated by larger CLDs while still representing only single independent observations at the domain level. This structure means that statistically significant edge-level findings may not generalise across domains, as confirmed by leave-one-CLD-out validation showing substantial AUC degradation. Interpreting edge-level p-values requires attention to effect sizes and cross-CLD consistency rather than significance alone.

While our multi-CLD validation spans diverse domains, the total number of CLDs ($n=3$) limits statistical power for meta-analytic conclusions. At the same time, edge-only evaluation induces a large number of edge-level test cases per CLD ($|V|(|V| - 1)$ ordered pairs): across our three CLDs this totals 1,394 possible directed edge slots, enabling fine-grained within-domain analyses. This dense edge space does not substitute for more independent domains, but it partially mitigates data scarcity at the edge level. Larger-scale validation across additional CLDs and scientific domains would strengthen generalizability claims.

9.13.7 Limited Stochasticity Control

While we ran 3 replicates per experimental condition and analysed a large number of edges, some analyses (particularly Deep Research) were single-run due to cost constraints. Confidence intervals on detection rates should be interpreted with this limitation in mind. Additionally, the Deep Research system has many degrees of freedom (query count, iteration depth, scoring thresholds, agent prompts) that were not systematically ablated in this thesis, limiting the ability to attribute performance to specific design choices.

9.13.8 Confirmation Bias in Search

Both Citation Judge retrieval and Deep Research inherently search for *supporting* evidence rather than *disconfirming* evidence. This confirmation bias may inflate validation rates for both true positives and false positives, as the systems are not designed to find evidence that a relationship does *not* exist.

9.13.9 Budget Constraints

Cost limitations affected multiple design decisions: (1) using only the top 5 search results for citation retrieval; (2) limiting Deep Research to single runs per edge; (3) using GPT-5-mini for some experiments where GPT-4.1 would have maintained consistency. These constraints may have underestimated the potential of more thoroughly resourced pipelines. In particular, Deep Research evaluation did not include true negative edges (1,109 of 1,394 possible) due to cost (estimated at \$269), leaving TN analysis incomplete and potentially biasing enrichment estimates.

9.13.10 Synthetic Corruption Rate Fixed at 30%

Synthetic corruption experiments in this thesis use a fixed corruption rate of 30% for all runs. This rate was chosen as a pragmatic default aligned with HaluEval-inspired corruption strategies and to ensure a substantial fraction of positive labels (`corrupted=True`) for stable discrimination estimates under our fixed run budget. However, a fixed rate limits interpretability: detection performance may vary nonlinearly with corruption prevalence and corruption severity. Future work should include a small corruption-rate ablation (e.g., 10% / 30% / 50%) to test sensitivity of judge performance to the corruption base rate and to better calibrate how synthetic-benchmark results translate to naturally occurring CLD errors.

9.13.11 Local Embedding Model Limitations

We used `all-mpnet-base-v2` for cosine similarity computation based on computational efficiency and general MTEB performance. However, domain-specific embedding models trained on scientific literature may yield improved performance:

- **SPECTER** [127]: Trained on citation graphs, modelling document relationships in academic text
- **SciBERT embeddings**: Pre-trained on Semantic Scholar corpus
- **PubMedBERT**: Specialized for biomedical text, relevant for health-related CLDs

The trade-off between general-purpose models (broader coverage, faster inference) and domain-specific models (higher semantic fidelity for scientific text) remains unexplored.

9.13.12 Search Provider Heterogeneity

We primarily used Brave Search API for citation retrieval. Different search providers (PubMed API, Semantic Scholar, Google Scholar) may return different result sets, affecting judge scores and detection rates. Our Uleman’s validation partially addresses this by comparing multiple providers, but systematic provider comparison across all experiments was not conducted. More broadly, a better calibration approach for distinguishing *evidence of absence* from *absence of evidence* would be valuable, so that the judge becomes appropriately more critical when evaluating negative edges.

9.13.13 Fetching Efficiency and Cost–Performance Tradeoffs

Beyond the choice of search provider, the *fetcher* used to convert URLs into judge-readable text is a major determinant of both cost and effective coverage. Paywall-bypassing fetchers such as Jina AI Reader can achieve

near-complete coverage (100% on Uleman’s), but at materially higher cost. We therefore used our custom citation fetcher (Brave Search + lightweight ContentScraper) as a pragmatic cost–performance tradeoff: it achieved 89% coverage on Uleman’s with comparable mean judge scores to alternatives, and near-saturated retrieval on our GT Lit citation experiments (Depressive Symptoms 100.0%, Social Norms 99.7%, Older Persons ED 97.9%). However, the physics CLD validation highlights limited transfer: even with expert-curated references, the same web-only fetching regime achieved only 54.8% retrieval, indicating that “high coverage in-domain” does not guarantee high coverage in other scientific domains when sources are concentrated in paywalled textbooks or restricted publisher sites.

9.13.14 Token-Level Uncertainty Quantification

Our experiments with LUQ yielded limited success. While computationally efficient, token-level entropy and probability-based confidence measures failed to reliably distinguish correct from hallucinated causal relationships.

This finding aligns with theoretical critiques of probability-based uncertainty estimation [128]. Softmax normalization discards information about evidence strength: low probability could indicate genuine uncertainty or the presence of multiple valid continuations. Standard entropy measures conflate these scenarios.

This limitation is particularly pronounced in causal extraction. When generating relationship types or variable names, multiple tokens may be contextually appropriate (different but semantically equivalent phrasings). Probability-based methods flag these as unreliable even when the model has robust knowledge of the underlying causal structure. Approaches like LogU [128] attempt to recover evidence strength through Dirichlet distribution modelling, but require internal model states unavailable through commercial APIs.

9.13.15 Sample Size Considerations

While our multi-CLD validation spans diverse domains, the total number of CLDs ($n=3$) limits statistical power for meta-analytic conclusions. At the same time, edge-only evaluation induces a large number of edge-level test cases per CLD ($|V|(|V| - 1)$ ordered pairs): across our three CLDs this totals 1,394 possible directed edge slots, enabling fine-grained within-domain analyses. This dense edge space does not substitute for more independent domains, but it partially mitigates data scarcity at the edge level. Larger-scale validation across additional CLDs and scientific domains would strengthen generalizability claims.

9.14 Future Work

9.14.1 Completing RQ3: Deep Research Validation

The exploratory Deep Research pilot study established baseline detection rates but left methodological gaps. Future work should:

1. **Replication for uncertainty quantification:** Run Deep Research with $n \geq 3$ replicates per edge to enable statistical testing and confidence interval estimation.
2. **Parameter ablation:** Systematically vary search iterations (1–5), evidence source limits (3–10), and quality thresholds to identify optimal configurations.
3. **End-to-end pipeline integration:** Test Deep Research as a filtering component in live CLD generation, measuring downstream quality impact.

4. **Human expert comparison:** Validate Deep Research verdicts against domain expert judgements at larger scale, e.g., full Deep Research runs (preferably multiple, across different CLD domains) to test how results change when extrapolation assumptions are relaxed.
5. **Evidence applicability calibration:** Develop robust criteria (and calibrated thresholds) for when retrieved evidence matches the *specific* edge claim (construct mapping, mechanism specificity), building on the applicability rubric used in this thesis.
6. **Negative controls and TN coverage:** Extend evaluation to include sampled TN edges and explicit negative-control claims to better estimate false validation rates and characterise over-validation.

9.14.2 Revisiting Adaptive Test-Time Compute

The original RQ3 design remains theoretically motivated. Future work could revisit it if:

1. **Domain-specific UQ metric thresholds:** Develop calibration procedures tuning thresholds per-CLD using held-out validation sets.
2. **Improved correctors:** Improve corrector performance by investigating action mismatch on a per-edge basis and calibrate such that edits translate into expert-grounded gains. RLHF on correction quality or tool-assisted evidence grounding could shift behaviour.
3. **Hybrid triggering:** Combine UQ metrics with lightweight LLM pre-screening to achieve both efficiency and cross-domain generalisation.

9.14.3 Addressing Corrector Limitations

The setting-dependent corrector behaviour suggests architectural or prompting limitations:

1. **Action-specific prompting:** Design separate prompts for revision vs. structural/type corrections rather than multi-action prompts.
2. **Iterative refinement:** Allow multiple correction rounds with feedback, mimicking human revision.
3. **Evidence-grounded correction:** Integrate citation retrieval so correctors can propose evidence-backed revisions.

9.14.4 General Methodological Extensions

1. **Domain-specific embeddings:** Evaluate SPECTER, SciBERT, and PubMedBERT for citation similarity.
2. **Cross-model evaluation:** Test the judge-corrector pipeline with diverse LLMs (Claude, Gemini, open-source).
3. **Large-scale human evaluation:** Validate automated hallucination labels against expert judgements at scale.
4. **Adaptive threshold tuning:** Develop domain-aware threshold selection accounting for CLD characteristics.
5. **Expanded CLD corpus:** Validate findings across CLDs spanning additional scientific domains.
6. **Citation-judge failure taxonomy:** Report a standardised per-edge taxonomy (e.g., no retrieval/no fetch vs. weak evidence vs. related-but-not-identical vs. causal/correlational mismatch) to separate infrastructure failures from evidence-quality failures and to guide escalation policies (deeper retrieval, query reformulation, or human review).

7. **Corruption-rate sensitivity:** Ablate the synthetic corruption rate (e.g., 10%/30%/50%) to quantify how GT Synth detection performance depends on corruption prevalence and to improve transfer interpretation to real CLD error distributions.

9.14.5 Active Learning Integration

The systematic leniency of citation judges suggests an active learning paradigm: edges flagged as low-confidence by LLM judges could be prioritised for human expert review, with expert feedback used to calibrate judge thresholds and potentially fine-tune judge models. This direction is consistent with evidence from causal graph validation that supervised fine-tuning can substantially outperform prompt-only LLM baselines on causal-relation classification [59].

9.14.6 Addressing False Negatives

Residual false negatives (expert-included edges that are neither generated nor recovered by evidence search) could reflect tacit expert knowledge not captured in retrievable literature. Future work should investigate whether targeted elicitation methods (e.g., prompting with related concepts, multi-turn dialogue) can reduce this gap.

9.14.7 Causal Context Integration in Judging and Deep Research

The generator is prompted with the full variable set and incorporates CLD-level constraints (including a mediation check) to only propose an edge when the relationship is intended to be *directly causal*. However, after the generator has produced an edge (or not), the downstream components in this thesis—judging, correction, and Deep Research—evaluate edges largely *in isolation*: they do not take the current CLD structure, candidate mediators, or feedback context as explicit input. Integrating CLD-level causal context (mediators, confounders, feedback loops) into the judge, corrector, and Deep Research prompts could improve end-to-end causal loop diagram reproduction by correctly handling edges that appear directly causal in isolation, but would be better modelled as indirect once mediating pathways present in the CLD are considered.

More broadly, future system iterations should propagate the generator’s mediation check and CLD-level constraints to downstream components, so that judging, correction, and Deep Research explicitly condition on candidate mediators and the evolving CLD context rather than treating each edge as an isolated claim.

Chapter 10

Conclusion

This thesis investigated whether methods independent of researcher-provided corpora can detect and mitigate hallucinations in LLM-generated Causal Loop Diagrams (CLDs), motivated by the reliability requirements of using LLMs for structured scientific knowledge generation. Across three research questions, we evaluated LLM-as-a-Judge configurations, LLM-as-a-correctors, uncertainty quantification (UQ) signals, automated literature-oriented validation via Deep Research, and adaptive deployment of Deep Research based on judge uncertainty signals.

Overall, the results support a nuanced conclusion: corpus-independent detection and correction methods can work in controlled settings, but their effectiveness degrades when the target is expert-grounded causal structure. In our dataset of three health-domain CLDs, correctness-judge performance is high in a controlled synthetic hallucination setting (GT Synth), but it drops in the literature ground-truth setting (GT Lit), with only the causal mechanistic prompt beating random baselines. In expert domains, disagreement with an expert CLD should not automatically be treated as fabrication: it can also reflect scope choices, tacit knowledge, or construct mismatch between an edge claim and what is explicitly evidenced in accessible sources. As a consequence, LLM-generated CLDs are best used as candidate drafts that focus expert attention on the contested edge space, rather than as self-validating outputs.

For corpus-independent hallucination signals, generator-side UQ metrics (perplexity, minimum probability, maximum window entropy, and “corpus-dependent” cosine similarity) do not effectively flag hallucinations independently. They can contribute as features to supervised ensemble classifiers in-distribution, but they fail to generalise out-of-CLD, reinforcing their role as auxiliary signals for monitoring and triage rather than as standalone detectors.

For judge-guided correction, LLM-as-a-corrector consistently improves judge scores and slightly improves CLD quality in the synthetic setting, but not versus literature ground truth. This highlights a Goodhart-like risk [129] in judge-corrector feedback loops: optimizing for proxy judge satisfaction without improving expert-grounded alignment.

Finally, for literature-oriented validation, we piloted a multi-agent Deep Research (DR) system as adaptive test-time compute. DR retrieves direct causal evidence for 62.4% of false positive edges and 42.9% of false negative edges. Single-rater human validation ($n = 50$) suggests that at applicability threshold $t = 0.7$, 66% of DR-evidenced edges pass the threshold and, among passing edges, 90.9% are correct-causal; however, a preliminary second-rater overlap ($n = 13$) indicates only fair agreement on categorical verdicts and poor agreement on applicability scores, so calibration-based extrapolations should be interpreted as exploratory and potentially rater-sensitive. Under this calibration, extrapolated ground-truth enrichment suggests substantial gains in aggregate edge F1: $0.267 \rightarrow 0.527$ (Scenario 1: FP $\rightarrow$ TP) and $0.267 \rightarrow 0.582$ (Scenario 2: LLM+DR),

with conservative estimates of 0.472 and 0.500. Using a mechanistic correctness judge to flag uncertain edges for adaptive DR deployment suggests an estimated 15% accuracy-per-dollar increase.

10.1 Key Contributions

- We provide an end-to-end experimental evaluation of corpus-independent CLD generation and post-hoc validation, spanning generation, judging, correction, uncertainty signals, and evidence-oriented verification, with methods established in literature but not tested systematically in CLD context.
- We show that the *Correctness Judge* performs strongly in controlled synthetic hallucination settings (GT Synth) but does not reliably transfer to literature ground truth (GT Lit), where only the Mechanistic prompt exceeds random baselines.
- We find that the *Citation Judge* does not reliably discriminate valid from hallucinated edges under either GT Synth or GT Lit: retrieval and evidence asymmetries (notably, *absence of evidence* being treated as *evidence of absence* for negative edges) and systematic leniency in human validation make citation judging better suited to permissive screening than automated filtering.
- We benchmark generator-side uncertainty quantification (UQ) metrics as corpus-independent hallucination signals and show that they are weak as standalone detectors and do not generalise reliably across CLDs, but can serve as auxiliary features for monitoring and triage.
- We identify and characterise failure modes that matter specifically for CLDs (e.g., scope-vs-factual disagreement, plausibility bias, evidence-of-absence effects, and construct/applicability mismatch).
- We demonstrate that judge-guided correction can optimise for judge satisfaction without improving expert-grounded alignment, highlighting a Goodhart-like risk [129] in judge–corrector feedback loops.
- We provide evidence that literature-oriented validation can enrich the disagreement space: calibrated with human validation and under strong assumptions (relationship isolation, a single human validator, and extrapolation), Deep Research yields substantial but bounded F1 gains; future work should test enrichment performance as these assumptions are relaxed. We also show a proof-of-concept judge-triggered deployment policy that improves cost-efficiency by selectively invoking Deep Research on uncertain edges.

10.2 Concluding Remarks

Taken together, these findings position LLMs as scalable assistants for proposing and stress-testing causal structures, not as autonomous validators. The most promising workflow is hybrid: use LLMs to expand the edge space, use judges and UQ signals to triage uncertainty, and selectively apply evidence-oriented validation (e.g., Deep Research with an explicit applicability threshold) to focus expert attention on the contested edges that most affect the final model.

A key finding is that “hallucination” in this setting is not equivalent to fabrication. In our experiments, a substantial proportion of LLM-generated edges classified as false positives against expert ground truth nonetheless had retrievable direct causal evidence under Deep Research (62.4%). This indicates that the expert–LLM disagreement space can mix multiple phenomena: genuine fabrications, expert scoping choices, and construct or operationalisation mismatch where evidence supports a related—but not identical—claim. Accordingly, disagreement with an expert CLD should not automatically be treated as error; it can also be a signal for targeted evidence review and model refinement.

This perspective positions LLMs not merely as reproducers of expert consensus, but as tools for systematically surfacing candidate relationships for expert adjudication—an increasingly valuable function as system complexity grows and the edge space ($O(|V|^2)$) exceeds the capacity for exhaustive expert deliberation. As

LLM capabilities continue to advance, the methods evaluated here—particularly uncertainty-guided selective deployment combined with literature-grounded validation—provide a foundation for integrating LLM-assisted causal synthesis into rigorous research workflows. The path forward is not to replace expert judgement, but to augment it: use LLMs to handle scalable generation and evidence gathering, while reserving expert attention for construct definition, applicability assessment, and the causal modelling decisions that require domain knowledge.

If given three additional months, the most valuable next step would be a systematic Deep Research ablation (single-agent vs. multi-agent, search depth, evidence filters, and trigger policies) replicated across a larger, domain-diverse CLD corpus (beyond health) to quantify generalisation and isolate which components drive gains.

Chapter 11

Ethics and Data Management

This thesis adheres to the ethical code (<https://student.uva.nl/en/topics/ethics-in-research>) and research data management policies (<https://rdm.uva.nl/en>) of UvA and IvI.

11.1 Ethical Considerations

This research involves the use of large language models (LLMs) for automated generation and validation of causal loop diagrams in health domains. Several ethical considerations are relevant:

- **No human subjects:** This research does not involve human participants except in human validation of LLM outputs. All experiments use publicly available datasets and published causal loop diagrams from peer-reviewed literature.
- **AI transparency:** LLM-generated outputs are clearly labelled as such. We emphasise that LLM-generated CLDs should serve as starting points for expert review rather than definitive scientific conclusions.
- **Potential for misuse:** While this research aims to assist domain experts, there is potential for misuse if LLM-generated causal structures are treated as ground truth without expert validation. We explicitly recommend human oversight for any practical applications.
- **Environmental impact:** The computational experiments involve substantial API calls to commercial LLM providers. We have optimised experimental designs to minimise unnecessary compute while maintaining scientific rigor, for example by capturing generator UQ metrics during generation in RQ1 runs such that these needn't be recomputed for RQ2.

11.2 Data Management

The source code for this thesis is contained within a private GitHub repository (<https://github.com/CausalixAI/platform>, branch: `thesis-nitai`). The repository is private because the code pertains to a startup (Causalix) that Professor Quax and the author are launching. As detailed in the Declaration of Authorship, this thesis was conducted within a shared software environment, with clear delineation of individual contributions. Access to the repository can be granted to examiners upon request.

Description	Availability	License
Thesis source code and experimental pipeline	Private GitHub repository (access on request)	Proprietary (Causalix)
Ground truth CLDs: Depressive Symptoms, Social Norms, Older Persons ED, Alzheimer’s, Physics	Extracted from cited publications	Fair use for research
Generated CLD datasets	Private GitHub repository	Proprietary (Causalix)
RQ1 human validation data (n=72)	Private GitHub repository	Proprietary (Causalix)
RQ3 human validation data (n=50)	Available upon request	CC-BY
Analysis scripts and notebooks	Available upon request	CC-BY

Table 11.1: Data and code availability for thesis reproducibility.

All experimental artefacts, including generated CLDs, judge outputs, corrector outputs, and Deep Research evidence, are preserved in structured formats (JSON, CSV) with full provenance tracking (run IDs, timestamps, model versions, prompts). Reproducibility procedures are described in Section C.4.

Appendix A

Function-to-Algorithm Mapping

Table A.1 provides the complete tracing from result directories to simulation scripts to core function implementations, enabling verification of the algorithmic descriptions in Appendix H.

Table A.1: Complete Function-to-Algorithm Mapping

Algo	Result Directory	Script	Core Function (Line)
1	RQ1a_ground_truth_correctness/ RQ1a_corruption_det_correctness/	run_rq1a_ground_truth_judge.py :179 → run_discovery_experiment	judge_all_edges_serial (modules.py:4421)
2	RQ1a_ground_truth_citation/ RQ1a_corruption_det_citation/	run_rq1a_ground_truth_judge.py :179 → run_discovery_experiment	judge_all_edges_with_citations_serial (modules.py:5688)
3	RQ1a_corruption_detection_*/	run_rq1_phase1_single_corrupt.py:74	_corrupt_relationships (modules.py:1748)
4	RQ1b_correction_*/	run_rq1b_correction_single.py:121	correct_edges_serial (modules.py:7050)
5	(computed during generation)	via run_discovery_experiment	compute_avg_nll (logit_metrics.py:177) compute_min_prob (:251) compute_max_window_entropy (:259) get_embedding_local (:98)
6	RQ2_hallucination_detection/	rq2_ensemble_classifier.py	train_classifiers (:32)
7	(see Algorithm H.9)	modules.py	fetch_citations_with_jina (:265)
8	RQ3_deep_research/	test_deep_research_ground_truth.py	DeepResearchManager (pydan- tic_ai_deepresearch_orchestration.py:642)

Internal Dispatch: `run_discovery_experiment()` (modules.py:11264) dispatches to:

- Line 11731: `_corrupt_relationships()` when `corruption_rate > 0`
- Line 11792: `judge_all_edges_with_citations_serial()` when `judge_edges=True`
- Line 11844: `correct_edges_serial()` when `run_correction=True`

Appendix B

Supplementary Sensitivity Analysis

This appendix provides extended sensitivity analyses for both Judge (RQ1a) and Corrector (RQ1b) prompts, quantifying performance stability across prompt variations.

B.1 Judge Prompt Sensitivity

To quantify how prompt engineering choices affect judge performance, we perform a sensitivity analysis measuring the relationship between prompt variation (input) and F1 change (output). For judge prompts (RQ1a), prompt distances are computed as cosine distance from the baseline prompt using MPNet embeddings. We use **repeated-measures ANOVA** with CLD×run as subjects and prompt as the within-subject factor, reporting **partial** η^2 as the effect size (proportion of within-subject variance explained by prompt). Bonferroni correction is applied across the four RQ1a tests ($\alpha_{\text{adj}} = 0.0125$).

B.1.1 Derivation: First-Order Sobol Index for a Categorical Factor

For a single categorical factor $A \in \{1, \dots, K\}$ with $p_a = \Pr(A = a)$, define $\mu_a := \mathbb{E}[Y \mid A = a]$ and $\mu := \mathbb{E}[Y] = \sum_{a=1}^K p_a \mu_a$. The first-order Sobol index for A is:

$$S_A := \frac{\text{Var}(\mathbb{E}[Y \mid A])}{\text{Var}(Y)}. \quad (\text{B.1})$$

Since $\mathbb{E}[Y \mid A]$ takes value μ_a when $A = a$, we have

$$\text{Var}(\mathbb{E}[Y \mid A]) = \sum_{a=1}^K p_a (\mu_a - \mu)^2. \quad (\text{B.2})$$

By the law of total variance,

$$\text{Var}(Y) = \mathbb{E}[\text{Var}(Y \mid A)] + \text{Var}(\mathbb{E}[Y \mid A]) = \sum_{a=1}^K p_a \text{Var}(Y \mid A = a) + \sum_{a=1}^K p_a (\mu_a - \mu)^2. \quad (\text{B.3})$$

Hence,

$$S_A = \frac{\sum_{a=1}^K p_a (\mu_a - \mu)^2}{\text{Var}(Y)} = \eta^2, \quad (\text{B.4})$$

i.e., the population one-way ANOVA effect size (correlation ratio) for a categorical factor [122]. In the main experiments, we report η_p^2 (partial eta-squared) because our design is blocked by CLD $\times$ run (repeated measures), so the sensitivity measure is computed after controlling for these subject-level differences.

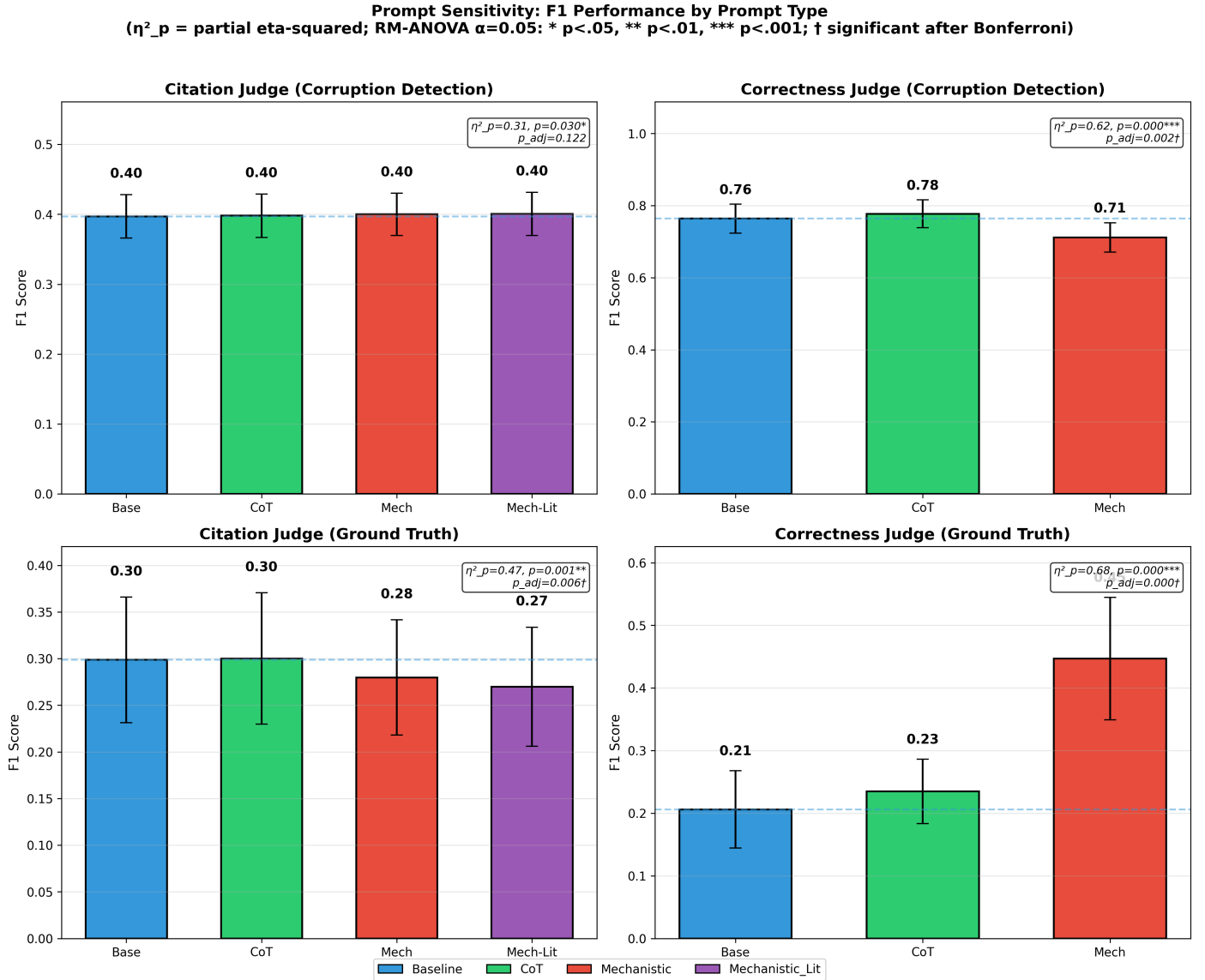


Figure B.1: Prompt sensitivity analysis across RQ1a experiments. Each panel shows F1 performance (mean $\pm$ 95% CI across CLDs and runs) for prompt variants. Statistical test: repeated-measures ANOVA with partial η_p^2 ; significance indicated by * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$; † significant after Bonferroni correction. Dashed lines indicate baseline.

Figure B.1 shows that prompt effects are strongest for the correctness judge (Corruption Detection: $\eta_p^2 = 0.62$, $p < 0.001$; Ground Truth: $\eta_p^2 = 0.68$, $p < 0.001$), with large effect sizes indicating that 62–68% of within-subject variance is explained by prompt choice. Citation judge effects are weaker (Corruption Detection: $\eta_p^2 = 0.31$, $p = 0.030$) and do not survive Bonferroni correction.

Table B.1: Judge Prompt Sensitivity Analysis

Experiment	Prompt	Dist.	F1	Std	95% CI	$\Delta F1$	η_p^2	p
Corr. Cit.	Baseline	0.00	0.397	0.038	[0.366, 0.428]	+0.000	0.31	0.122
	CoT	0.02	0.398	0.038	[0.367, 0.429]	+0.001		
	Mechanistic	0.32	0.400	0.037	[0.370, 0.430]	+0.003		
	Mech-Lit	0.31	0.401	0.038	[0.370, 0.431]	+0.004		
Corr. Corr.	Baseline	0.00	0.764	0.049	[0.724, 0.804]	+0.000	0.62	0.002**
	CoT	0.01	0.777	0.047	[0.738, 0.816]	+0.013		
	Mechanistic	0.18	0.712	0.050	[0.671, 0.753]	-0.052		
GT Cit.	Baseline	0.00	0.299	0.083	[0.231, 0.366]	+0.000	0.47	0.006**
	CoT	0.02	0.300	0.086	[0.230, 0.371]	+0.001		
	Mechanistic	0.32	0.280	0.076	[0.218, 0.342]	-0.019		
	Mech-Lit	0.31	0.270	0.078	[0.206, 0.334]	-0.029		
GT Corr.	Baseline	0.00	0.206	0.076	[0.144, 0.268]	+0.000	0.68	<.001***
	CoT	0.01	0.235	0.063	[0.183, 0.286]	+0.029		
	Mechanistic	0.18	0.447	0.120	[0.349, 0.545]	+0.241		

Note. Dist. = MPNet cosine distance from baseline. η_p^2 = partial eta-squared (RM-ANOVA). N=9 per prompt.

95% CI computed using t-distribution ($df = 8$) for block-level meta-inference per uncertainty reporting rule.

p-values shown are **raw** (uncorrected). Significance markers compare raw p to Bonferroni-adjusted threshold ($\alpha_{adj}=0.0125$ for 4 tests). * $p < \alpha_{adj}$, ** $p < 0.01$, *** $p < 0.001$.

Table B.2: RM-ANOVA Assumption Checks for Judge Prompt Sensitivity (Appendix K)

Exp.	n	k	Shapiro p	Mauchly p	ϵ_{GG}	p (unc.)	p (GG)
Corr. Cit.	9	4	0.002	<.001	0.57	0.030	0.064
Corr. Corr.	9	3	0.511	<.001	0.62	<.001	0.004
GT Cit.	9	4	0.796	<.001	0.46	0.001	0.016
GT Corr.	9	3	0.509	<.001	0.57	<.001	0.002

Note. Shapiro-Wilk is applied to RM-ANOVA residuals (subject + prompt additive model). Mauchly's test assesses sphericity of the within-subject covariance. ϵ_{GG} is Greenhouse–Geisser epsilon; p (GG) uses ϵ_{GG} -adjusted degrees of freedom. For $k = 2$, sphericity is not defined and Mauchly/GG are omitted.

Interpreting the assumption checks (Table B.2), Mauchly's test indicates **sphericity violations** in all judge prompt analyses ($p < .001$), so inferential conclusions should be based on **Greenhouse–Geisser corrected** p -values. Under GG correction, the corruption-detection citation judge prompt effect is no longer significant ($p_{GG} = 0.064$), while both correctness judge prompt effects remain robust ($p_{GG} = 0.002$). The ground-truth citation judge effect is marginal under GG ($p_{GG} = 0.022$) and still does not survive Bonferroni correction ($\alpha_{adj} = 0.0125$). Shapiro–Wilk rejects residual normality only for Corr. Cit.; given $n = 9$, we treat p -values as approximate and emphasise effect sizes alongside corrected inference.

Table B.1 shows that prompt choice significantly affects the correctness judge (large effects, $\eta_p^2 > 0.60$), while citation-judge prompt effects are moderate ($\eta_p^2 \approx 0.30$ – 0.43) with the corruption detection effect not surviving family-wise correction.

B.2 Corrector Prompt Sensitivity

Extending the sensitivity analysis to the LLM-as-a-Corrector (RQ1b), we evaluate how corrector prompt complexity affects correction performance. Unlike judge prompts which had distinct semantic content, corrector prompts follow a cumulative design: Baseline provides examples only, CoT adds step-by-step reasoning ($1.45 \times$ length), and Mechanistic adds Bradford Hill criteria ($1.91 \times$ length). Because these prompts are long and share a common prefix, we compute corrector prompt distances using long-context Jina embeddings v2 (to avoid truncation artefacts in short-context encoders).

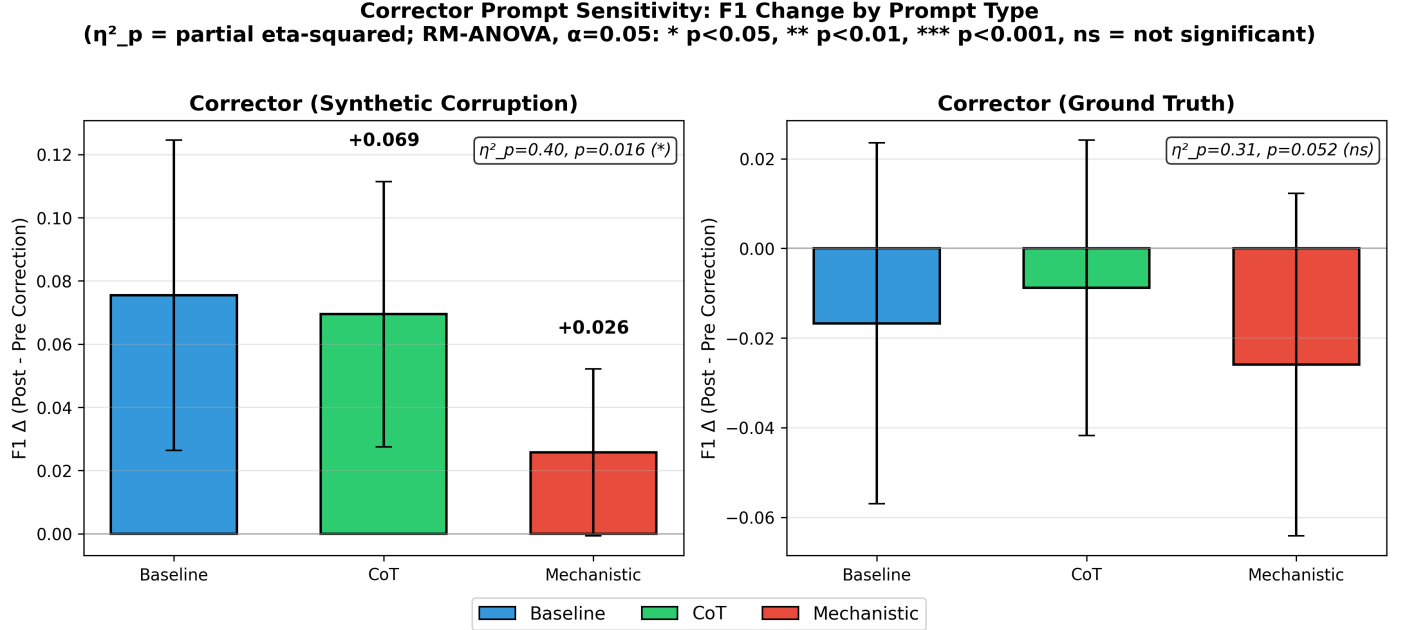


Figure B.2: Corrector prompt sensitivity analysis. F1 Δ (post-correction minus pre-correction) by prompt type. Statistical test: repeated-measures ANOVA with partial η^2 ; significance: * $p < 0.05$, ** $p < 0.01$, *** $p < 0.001$. Error bars show 95% CI across CLDs and runs.

Figure B.2 shows a significant prompt effect for synthetic corruptions ($\eta_p^2 = 0.40$, $F(2, 16) = 5.43$, $p = 0.016$), indicating that simpler prompts (Baseline: +0.075) outperform complex prompts (Mechanistic: +0.026). Ground truth correction shows a marginally significant effect ($\eta_p^2 = 0.31$, $p = 0.052$).

Table B.3: Corrector Prompt Sensitivity Analysis

Experiment	Prompt	Dist.	F1 Δ	Std	95% CI	Actions	η_p^2	p
Synth.	Baseline	0.00	+0.08	0.06	[+0.03, +0.12]	1006	0.40	0.032*
	CoT	0.01	+0.07	0.05	[+0.03, +0.11]	1006		
	Mechanistic	0.03	+0.03	0.03	[-0.00, +0.05]	1006		
GT	Baseline	0.00	-0.02	0.05	[-0.06, +0.02]	418	0.31	0.104
	CoT	0.01	-0.01	0.04	[-0.04, +0.02]	408		
	Mechanistic	0.03	-0.03	0.05	[-0.06, +0.01]	418		

Note. Dist. = Jina v2 cosine distance from baseline. η_p^2 = partial eta-squared (RM-ANOVA). N=9 per prompt.

95% CI computed using t-distribution ($df = 8$) for block-level meta-inference per uncertainty reporting rule.

p-values are Bonferroni-corrected (2 tests, $\alpha_{adj}=0.025$). * $p < .05$, ** $p < .01$, *** $p < .001$.

Table B.4: RM-ANOVA Assumption Checks for Corrector Prompt Sensitivity (Appendix K)

Exp.	n	k	Shapiro p	Mauchly p	ϵ_{GG}	p (unc.)	p (GG)
Synth.	9	3	0.516	<.001	0.54	0.016	0.044
GT Lit	9	3	0.119	<.001	0.72	0.052	0.073

Note. Shapiro-Wilk is applied to RM-ANOVA residuals (subject + prompt additive model). Mauchly's test assesses sphericity of the within-subject covariance. ϵ_{GG} is Greenhouse–Geisser epsilon; p (GG) uses ϵ_{GG} -adjusted degrees of freedom. For $k = 2$, sphericity is not defined and Mauchly/GG are omitted.

Interpreting the assumption checks (Table B.4), Mauchly's test again indicates **sphericity violations** ($p < .001$), so GG-corrected p -values provide the most appropriate inference. Under GG correction, the synthetic prompt effect is borderline ($p_{GG} = 0.044$) and would not meet the Bonferroni-adjusted threshold for the two RQ1b tests ($\alpha_{adj} = 0.025$), while the GT Lit prompt effect remains non-significant ($p_{GG} = 0.073$). Shapiro–Wilk does not reject residual normality for either experiment.

The table above summarises the repeated-measures ANOVA results for corrector prompt sensitivity.

Appendix C

Reproducibility

This appendix provides comprehensive information for replicating all experiments, including data availability, code locations, software environment specifications, and exact execution commands.

C.1 Data Availability

C.1.1 Ground Truth CLD Datasets

The three ground truth CLD datasets used in all experiments are located in:

```
data_science/parameter_tuning_experiments/  
ground_truth_clds_for_experiments/
```

Table C.1: Ground Truth CLD Dataset Files

Domain	Nodes	Edges	Source	Filename
Depressive Symptoms	14	34	Uleman et al. [113]	Depressive_symptoms...healthy_adults.xlsx
Social Norms	10	12	Crielaard et al. [114]	Social_norms_and_obesity_prevalence.xlsx
Older Persons ED	34	66	Smeekees et al. [115]	older_persons_ED...spreadsheet.xlsx

Each Excel file contains: edge definitions (source/target), relationship polarity, causal motivations, scientific citations, and generation-time UQ metrics.

C.1.2 Validation and Auxiliary Datasets

Table C.2: Validation and Auxiliary Datasets

Dataset	Used In	Size	Location
TruthfulQA [119]	RQ1 Verification	1,634 Q&A pairs	final_runs/RQ1_verification.Truth_QA/
Physics CLDs (see below)	RQ1a Validation	31 edges, 4 CLDs	final_runs/RQ1a_validation.physics_CLDs/cld_data_files/
Uleman’s Alzheimer CLD	RQ1a Validation	150 edges	final_runs/RQ1a.ulemans.citation.provider.comparison/
Human Validation Set	RQ1a Validation	72 edges	final_runs/RQ1_human_validation.citation_judge/
RQ3 DR Validation	RQ3 Human Val.	50 edges	final_runs/RQ3_deep_research/human_validation/

C.1.2.1 Physics CLD Details.

Four physics-based CLDs were constructed for sanity-check validation, derived from well-established physical laws:

Table C.3: Physics CLD Validation Dataset

CLD	Nodes	Edges	Physical Laws / Sources
Thermostat Heating	7	8	Newton’s Law of Cooling [105]; First Law of Thermodynamics [106]
Predator-Prey	6	8	Lotka-Volterra equations [107, 108]
RC Circuit	7	8	Ohm’s Law [109]; Kirchhoff’s Laws [110]
Water Tank	6	7	Pascal’s Law [111]; Torricelli’s Law [112]
Total	26	31	

Construction process. The physics CLDs were constructed as follows:

1. **Causal structure definition:** Variables and directed edges were derived from established physical laws (Newton’s Law of Cooling [105], First Law of Thermodynamics [106], Lotka-Volterra equations [107, 108], Ohm’s and Kirchhoff’s Laws [109, 110], Pascal’s and Torricelli’s Laws [111, 112]). Edge polarities (POSITIVE/NEGATIVE) follow directly from the mathematical relationships in each law.
2. **Motivation generation:** For each edge, causal explanations (“motivations”) were authored in an interactive session with Claude 3.5 Sonnet. The prompt pattern was: “*Write a motivation for the edge [Source] → [Target] with polarity [POSITIVE/NEGATIVE] based on [Physical Law]. Include the relevant equation and cite sources.*” This interactive process ensured each motivation accurately reflected the underlying physics while remaining accessible.
3. **Reference mapping:** Each edge was annotated with bibliographic references to seminal papers and standard textbooks. Post-processing scripts (`add_urls_to_references.py`, `add_supporting_quotes.py`) enriched the references with URLs (where publicly accessible) and supporting quotes extracted from source texts.
4. **Verification:** All motivations and supporting quotes were manually verified against source texts to ensure scientific accuracy. URLs were browser-verified for accessibility (`verify_and_fix_urls.py`).
5. **Data format:** Final JSON structure: `{source, target, polarity, motivation, references[{short_citation, full_citation, url, supporting_quote}]}`. Converted to Excel format via `create_thermostat_excel.py`.

Reproducibility note. The motivation text was authored interactively (not via automated script) to ensure scientific precision. To reproduce similar motivations, one can prompt any capable LLM with the edge structure and underlying physical law, then verify against authoritative sources. The complete authored motivations, references, and all post-processing scripts are provided for full transparency.

Data locations. JSON files: `final_runs/validation_RQ1a_physics_clds/Data/physics_clds_with_references/*`
 post-processing scripts: same directory; Excel export: `create_thermostat_excel.py`; pipeline documentation: `PIPELINE_EXPLANATION.md`.

C.1.3 Experiment Result Datasets by Research Question

Table C.4: Experiment Result Datasets by Research Question

RQ	Experiment	Files	Location
RQ1a	GT Lit Correctness	27	<code>final_runs/RQ1a_ground_truth_correctness/</code>
	GT Lit Citation	36	<code>final_runs/RQ1a_ground_truth_citation/</code>
	GT Synth Correctness	27	<code>final_runs/RQ1a_corruption_detection_correctness_final/</code>
	GT Synth Citation	36	<code>final_runs/RQ1a_corruption_detection_citation_gpt5mini/</code>
RQ1b	Correction GT Synth	27	<code>final_runs/RQ1b_correction_experiment_corruption/</code>
	Correction GT Lit	27	<code>final_runs/RQ1b_correction_experiment_ground_truth/</code>
RQ2	UQ Metrics Analysis	63	<code>final_runs/RQ2_hallucination_detection/</code>
RQ3	Deep Research	285 edges	<code>final_runs/RQ3_deep_research/</code>
Prelim.	Generator Comparison	9	<code>final_runs/generator_model_comparison_analysis/</code>
	Temperature Sensitivity	45	<code>final_runs/temperature_sensitivity/</code>
	Random Baseline	3,000	<code>final_runs/random_baseline_generator/</code>
Sens.	Prompt Sensitivity	4 exp.	<code>final_runs/Sensitivity_analysis_simple/</code>

C.1.4 Final Experiment Results

All experimental results are stored in the `final_runs/` directory:

```

final_runs/
+-- RQ1a_gt_lit_correctness/          [Ground Truth Literature - Correctness Judge]
|   +-- Data/depressive/run_{1,2,3}/
|   +-- Data/emergency_department/run_{1,2,3}/
|   +-- Data/social_norms/run_{1,2,3}/
|   +-- Output/                      [analysis outputs]
+-- RQ1a_gt_lit_citation/            [Ground Truth Literature - Citation Judge]
+-- RQ1a_gt_synth_correctness/       [Ground Truth Synthetic - Correctness Judge]
+-- RQ1a_gt_synth_citation/         [Ground Truth Synthetic - Citation Judge]
+-- RQ1b_corrector_ablation/         [Corrector ablation analysis]
+-- RQ1b_corrector_gt_lit_correctness/ [Corrector GT Lit experiments]
+-- RQ1b_corrector_gt_synth_correctness/ [Corrector GT Synth experiments]
+-- RQ2_uq_hallucination_detection/  [UQ metrics analysis]
+-- RQ3_deep_research_validation/    [Deep Research validation]
+-- validation_RQ1_truthfulqa_judge/ [TruthfulQA benchmark]
+-- validation_RQ1a_physics_clds/    [physics CLD judge validation]
+-- validation_RQ1a_ulemans_external/ [external CLD validation]
+-- prelim_generator_model_comparison/ [generator model selection]
+-- prelim_random_baseline_generator/ [random graph baseline]
+-- supp_cost_analysis/              [API cost computations]
+-- supp_embedding_benchmark/        [embedding model comparison]
+-- supp_parallelization_benchmark/  [parallelization speedup]
+-- supp_prompt_sensitivity/         [prompt sensitivity]
+-- supp_time_cost_scaling/          [scaling analysis]

```

C.2 Code Availability

C.2.1 Experiment Execution Scripts

Table C.5: Primary Experiment Execution Scripts

Experiment	Script	Dependencies
RQ1a Ground Truth	<code>run_rq1a_ground_truth_multirun.py</code>	<code>run_rq1a_ground_truth_judge.py</code>
RQ1a Corruption (Corr.)	<code>run_rq1a_corruption_multirun.py</code>	<code>run_corruption_detection_pipeline.sh</code>
RQ1a Corruption (Cit.)	<code>run_rq1a_corruption_citation_multirun.py</code>	<code>run_rq1_judge_citations_all_3_prompts.py</code>
RQ1b Corrector	<code>run_rq1b_correction_multirun.py</code>	<code>run_rq1b_correction_single.py</code>
Temperature Sensitivity	<code>run_temperature_sensitivity.py</code>	<code>modules.py</code> (<code>run_discovery_experiment</code>)

All execution scripts located in: `data_science/parameter_tuning.experiments/`

See also: Table C.6 (analysis scripts) and Table C.7 (complete script inventory).

C.2.2 Analysis Scripts

Table C.6: Analysis and Evaluation Scripts

Analysis	Script Path
<i>RQ1a: LLM-as-Judge</i>	
RQ1a Aggregate Analysis	<code>data_science/parameter_tuning_experiments/analyse_rq1a_aggregate_enhanced.py</code>
RQ1a Ground Truth Analysis	<code>data_science/parameter_tuning_experiments/analyse_rq1a_ground_truth_enhanced.py</code>
Physics CLD Validation	<code>final_runs/RQ1a_validation_physics_CLDs/analyse_physics_cld_results.py</code>
Physics CLD Test Runner	<code>final_runs/RQ1a_validation_physics_CLDs/run_physics_cld_tests.py</code>
Uleman’s Provider Comparison	<code>final_runs/RQ1a_ulemans_citation_provider_comparison/judge_citation_providers.py</code>
TruthfulQA Verification	<code>final_runs/RQ1_verification_Truth_QA/runner/truthqa_judge_baseline_correctness.py</code>
<i>RQ1b: Corrector</i>	
RQ1b Results Analysis	<code>data_science/parameter_tuning_experiments/analyse_corrector_ablation.py</code>
<i>RQ2: UQ Metrics</i>	
RQ2 Master Report	<code>final_runs/RQ2_hallucination_detection/rq2_master_report_v2_enhanced.py</code>
RQ2 Batch Analyser	<code>final_runs/RQ2_hallucination_detection/rq2_simple_batch_analyser.py</code>
<i>RQ3: Deep Research</i>	
RQ3 Unified Analysis	<code>final_runs/RQ3_deep_research/rq3_unified_analysis.py</code>
RQ3 Statistical Tests	<code>final_runs/RQ3_deep_research/rq3_statistical_test.py</code>
RQ3 Per-CLD Detection Rates	<code>final_runs/RQ3_deep_research/rq3_per_cld_detection_rates.py</code>
RQ3 Enrichment Metrics	<code>final_runs/RQ3_deep_research/compute_enriched_gt_metrics.py</code>
RQ3 Human Validation	<code>final_runs/RQ3_deep_research/scripts/enrichment_extrapolation.py</code>
RQ3 Smart Trigger Analysis	<code>final_runs/RQ3_deep_research/scripts/estimate_smart_trigger_dr_enrichment.py</code>
RQ3 Cost Efficiency Plot	<code>final_runs/RQ3_deep_research/scripts/plot_dr_cost_efficiency.py</code>
RQ3 Figure Generation	<code>final_runs/RQ3_deep_research/generate_figures.py</code>
RQ3 Edge Sampling	<code>final_runs/RQ3_deep_research/scripts/sample_edges_for_validation.py</code>
RQ3 Validation Tables	<code>final_runs/RQ3_deep_research/scripts/generate_validation_distribution_tables.py</code>
<i>Preliminary & Validation</i>	
Generator Model Comparison	<code>final_runs/generator_model_comparison_analysis/analyse_generator_models.py</code>
Random Baseline Analysis	<code>final_runs/random_baseline_generator/compute_random_baseline.py</code>
Embedding Model Benchmark	<code>final_runs/embedding_model_benchmark/embedding_benchmark_real_cld.py</code>
Power Analysis	<code>final_runs/power_analysis/compute_power_analysis.py</code>
<i>Cost & Scaling Analysis</i>	
Complete Cost Analysis	<code>final_runs/cost_analysis/calculate_all_costs.py</code>
Per-Experiment Costs	<code>final_runs/cost_analysis/calculate_costs_by_experiment.py</code>
Deep Research Costs	<code>final_runs/cost_analysis/compute_dr_costs.py</code>
Token Analysis	<code>final_runs/token_call_analysis_RQ1a_judge/analyse_output_tokens_tiktoken.py</code>
Time/Cost Scaling	<code>final_runs/time_and_cost_scaling/aggregate_time_cost_scaling.py</code>
Parallelization Benchmark	<code>final_runs/parallelization_benchmark/scripts/analyse_parallelization_results.py</code>
<i>Sensitivity Analysis</i>	
Judge Prompt Sensitivity	<code>final_runs/Sensitivity_analysis_simple/scripts/sensitivity_analysis_simple.py</code>
Corrector Prompt Sensitivity	<code>final_runs/Sensitivity_analysis_simple/scripts/sensitivity_analysis_corrector.py</code>
Temperature Assumption Tests	<code>final_runs/temperature_sensitivity_assumption_tests.py</code>

See also: Table C.5 (execution scripts) and Table C.7 (complete script inventory).

C.2.3 Complete final_runs/ Script and Command Inventory

To make the artefact set auditable end-to-end, Table C.7 enumerates every runnable Python script present under `final_runs/` at thesis submission time, together with its canonical invocation from the repository root.

Table C.7: Complete final_runs/ Script and Command Inventory

Script	Canonical command (from repo root)
<i>Unified analysis entry points</i>	
final_runs/RQ1.unified.analysis.py	python final_runs/RQ1.unified.analysis.py
final_runs/RQ2.unified.analysis.py	python final_runs/RQ2.unified.analysis.py
final_runs/RQ3.unified.analysis.py	python final_runs/RQ3.unified.analysis.py
<i>Top-level utilities</i>	
final_runs/aggregate_time_cost_scaling.py	python final_runs/aggregate_time_cost_scaling.py
final_runs/retrieve_effect_sizes.py	python final_runs/retrieve_effect_sizes.py
final_runs/temperature_sensitivity_assumption_tests.py	python final_runs/temperature_sensitivity_assumption_tests.py
<i>RQ1: TruthfulQA verification</i>	
final_runs/RQ1.verification.Truth.QA/runner/truthqa_judge_baseline_correctness.py	python final_runs/RQ1.verification.Truth.QA/runner/truthqa_judge_baseline_correctness.py
final_runs/RQ1.verification.Truth.QA/analysis_script/recalculate_metrics.py	python final_runs/RQ1.verification.Truth.QA/analysis_script/recalculate_metrics.py
<i>RQ1a validation: physics CLDs</i>	
final_runs/RQ1a.validation.physics.CLDs/run.physics.cld.tests.py	python final_runs/RQ1a.validation.physics.CLDs/run.physics.cld.tests.py
final_runs/RQ1a.validation.physics.CLDs/analyse.physics.cld.results.py	python final_runs/RQ1a.validation.physics.CLDs/analyse.physics.cld.results.py
final_runs/RQ1a.validation.physics.CLDs/generate.comparison.table.py	python final_runs/RQ1a.validation.physics.CLDs/generate.comparison.table.py
final_runs/RQ1a.validation.physics.CLDs/generate.retrieval.comparison.table.py	python final_runs/RQ1a.validation.physics.CLDs/generate.retrieval.comparison.table.py
final_runs/RQ1a.validation.physics.CLDs/get.avg.scores.py	python final_runs/RQ1a.validation.physics.CLDs/get.avg.scores.py
final_runs/RQ1a.validation.physics.CLDs/cld.data.files/create.thermostat.excel.py	python final_runs/RQ1a.validation.physics.CLDs/cld.data.files/create.thermostat.excel.py
final_runs/RQ1a.validation.physics.CLDs/physics.clds.with.references/add.supporting.quotes.py	python final_runs/RQ1a.validation.physics.CLDs/physics.clds.with.references/add.supporting.quotes.py
final_runs/RQ1a.validation.physics.CLDs/physics.clds.with.references/add.urls.to.references.py	python final_runs/RQ1a.validation.physics.CLDs/physics.clds.with.references/add.urls.to.references.py
final_runs/RQ1a.validation.physics.CLDs/physics.clds.with.references/show.example.py	python final_runs/RQ1a.validation.physics.CLDs/physics.clds.with.references/show.example.py
final_runs/RQ1a.validation.physics.CLDs/physics.clds.with.references/verify.and.fix.urls.py	python final_runs/RQ1a.validation.physics.CLDs/physics.clds.with.references/verify.and.fix.urls.py
final_runs/RQ1a.validation.physics.CLDs/physics.clds.with.references/verify.urls.py	python final_runs/RQ1a.validation.physics.CLDs/physics.clds.with.references/verify.urls.py
<i>RQ1a validation: Uleman's citation provider comparison</i>	
final_runs/RQ1a.ulemans.citation.provider.comparison/judge.citation.providers.py	python final_runs/RQ1a.ulemans.citation.provider.comparison/judge.citation.providers.py
final_runs/RQ1a.ulemans.citation.provider.comparison/compare.jina.vs.scraper.py	python final_runs/RQ1a.ulemans.citation.provider.comparison/compare.jina.vs.scraper.py
<i>RQ2: hallucination detection (UQ metrics)</i>	
final_runs/RQ2.hallucination.detection/rq2.simple.batch.analyser.py	python final_runs/RQ2.hallucination.detection/rq2.simple.batch.analyser.py
final_runs/RQ2.hallucination.detection/rq2.master.report.v2.enhanced.py	python final_runs/RQ2.hallucination.detection/rq2.master.report.v2.enhanced.py
<i>RQ3: Deep Research analysis</i>	
final_runs/RQ3.deep.research/rq3.unified.analysis.py	python final_runs/RQ3.deep.research/rq3.unified.analysis.py
final_runs/RQ3.deep.research/analyse.deep.research.results.py	python final_runs/RQ3.deep.research/analyse.deep.research.results.py
final_runs/RQ3.deep.research/rq3.statistical.test.py	python final_runs/RQ3.deep.research/rq3.statistical.test.py
final_runs/RQ3.deep.research/rq3.per.cld.detection.rates.py	python final_runs/RQ3.deep.research/rq3.per.cld.detection.rates.py
final_runs/RQ3.deep.research/compute.enriched.gt.metrics.py	python final_runs/RQ3.deep.research/compute.enriched.gt.metrics.py
final_runs/RQ3.deep.research/generate.figures.py	python final_runs/RQ3.deep.research/generate.figures.py
final_runs/RQ3.deep.research/run.rq3.analysis.py	python final_runs/RQ3.deep.research/run.rq3.analysis.py
final_runs/RQ3.deep.research/scripts/sample.edges.for.validation.py	python final_runs/RQ3.deep.research/scripts/sample.edges.for.validation.py
final_runs/RQ3.deep.research/scripts/enrichment.extrapolation.py	python final_runs/RQ3.deep.research/scripts/enrichment.extrapolation.py
final_runs/RQ3.deep.research/scripts/estimate.smart.trigger.dr.enrichment.py	python final_runs/RQ3.deep.research/scripts/estimate.smart.trigger.dr.enrichment.py
final_runs/RQ3.deep.research/scripts/plot.dr.cost.efficiency.py	python final_runs/RQ3.deep.research/scripts/plot.dr.cost.efficiency.py
final_runs/RQ3.deep.research/scripts/generate.validation.distribution.tables.py	python final_runs/RQ3.deep.research/scripts/generate.validation.distribution.tables.py
final_runs/RQ3.deep.research/scripts/compute.enriched.fi.from.validation.py	python final_runs/RQ3.deep.research/scripts/compute.enriched.fi.from.validation.py
<i>Cost, scaling, and diagnostics</i>	
final_runs/cost.analysis/calculate.all.costs.py	python final_runs/cost.analysis/calculate.all.costs.py
final_runs/cost.analysis/calculate.costs.by.experiment.py	python final_runs/cost.analysis/calculate.costs.by.experiment.py
final_runs/cost.analysis/calculate.experiment.costs.py	python final_runs/cost.analysis/calculate.experiment.costs.py
final_runs/cost.analysis/calculate.costs.max.edges.py	python final_runs/cost.analysis/calculate.costs.max.edges.py
final_runs/cost.analysis/analyse.token.and.call.stats.py	python final_runs/cost.analysis/analyse.token.and.call.stats.py
final_runs/cost.analysis/analyse.output.tokens.tiktoken.py	python final_runs/cost.analysis/analyse.output.tokens.tiktoken.py
final_runs/cost.analysis/compute.dr.costs.py	python final_runs/cost.analysis/compute.dr.costs.py
final_runs/cost.analysis/generate.table21.scaling.summary.py	python final_runs/cost.analysis/generate.table21.scaling.summary.py
final_runs/token.call.analysis.RQ1a.judge/analyse.output.tokens.tiktoken.py	python final_runs/token.call.analysis.RQ1a.judge/analyse.output.tokens.tiktoken.py
final_runs/time.and.cost.scaling/aggregate.time.cost.scaling.py	python final_runs/time.and.cost.scaling/aggregate.time.cost.scaling.py
final_runs/time.and.cost.scaling/collect.corrector.timing.py	python final_runs/time.and.cost.scaling/collect.corrector.timing.py
<i>Sensitivity, benchmarks, and baselines</i>	
final_runs/Sensitivity.analysis.simple/scripts/sensitivity.analysis.simple.py	python final_runs/Sensitivity.analysis.simple/scripts/sensitivity.analysis.simple.py
final_runs/Sensitivity.analysis.simple/scripts/sensitivity.analysis.corrector.py	python final_runs/Sensitivity.analysis.simple/scripts/sensitivity.analysis.corrector.py
final_runs/Sensitivity.analysis.simple/scripts/sensitivity.analysis.simple.rm.anova.py	python final_runs/Sensitivity.analysis.simple/scripts/sensitivity.analysis.simple.rm.anova.py
final_runs/Sensitivity.analysis.simple/scripts/sensitivity.analysis.corrector.rm.anova.py	python final_runs/Sensitivity.analysis.simple/scripts/sensitivity.analysis.corrector.rm.anova.py
final_runs/parallelization.benchmark/scripts/run.parallelization.benchmark.py	python final_runs/parallelization.benchmark/scripts/run.parallelization.benchmark.py
final_runs/parallelization.benchmark/scripts/analyse.parallelization.results.py	python final_runs/parallelization.benchmark/scripts/analyse.parallelization.results.py
final_runs/power.analysis/compute.power.analysis.py	python final_runs/power.analysis/compute.power.analysis.py
final_runs/random.baseline.generator/compute.random.baseline.py	python final_runs/random.baseline.generator/compute.random.baseline.py
final_runs/embedding.model.benchmark/embedding.benchmark.real.cld.py	python final_runs/embedding.model.benchmark/embedding.benchmark.real.cld.py
final_runs/generator.model.comparison.analysis/run.generator.model.comparison.py	python final_runs/generator.model.comparison.analysis/run.generator.model.comparison.py
final_runs/generator.model.comparison.analysis/analyse.generator.models.py	python final_runs/generator.model.comparison.analysis/analyse.generator.models.py

See also: Table C.5 (execution scripts) and Table C.6 (analysis scripts) for organised by-experiment views.

C.2.4 Prompt Configuration Files

All prompts use three core variants: *baseline* (direct instruction), *mechanistic* (Bradford Hill criteria [68]), and *cot* (chain-of-thought reasoning [60]). Citation judges additionally include a fourth variant, *mechanistic-lit*, which extends mechanistic with explicit literature grounding requirements.

Table C.8: Prompt Configuration Files (13 prompts: 2 judge tasks $\times$ 3 variants + 1 mech-lit + 1 corrector $\times$ 3 variants + 3 corruption)

Task	Variants	File(s)
Judge-Correctness	baseline, mech, cot	<code>alternative_prompts/prompts_correctness_{baseline,mechanistic,cot}.yaml</code>
Judge-Citation	baseline, mech, cot, mech-lit	<code>alternative_prompts/prompts_citation_{baseline,mechanistic,cot,mech_lit}.yaml</code>
Corrector	baseline, mech, cot	<code>corrector_prompts.yaml</code> (variants within single file)
Corruption	prompts_Nitai_C.yaml (prompt IDs below):	
– motivation	(single)	corruptor : generates spurious but plausible motivation
– spurious edge	(single)	generateSpuriousEdge : NONE $\rightarrow$ false positive causal edge
– polarity flip	(single)	generateFlippedPolarity : POS $\leftrightarrow$ NEG with new motivation

C.3 Software Environment

Table C.9: System and Runtime Environment

Component	Details
Hardware	ASUSTeK ROG Zephyrus G16 GU605CX.GU605CX
CPU	Intel(R) Core(TM) Ultra 9 285H (16 cores, 16 logical processors)
GPU	NVIDIA GeForce RTX 5090 Laptop GPU (24463 MiB VRAM; driver 576.88)
System RAM	64 GB DDR5 (8 $\times$ 8 GB; 7467 MT/s configured)
Operating System	Ubuntu 24.04.2 LTS (WSL2; kernel 6.6.87.2-microsoft-standard-WSL2; Windows build 26100.7462)
Python Version	3.10.18 (<code>.python-version=3.10</code>)
Package Manager	uv 0.8.3
CUDA Version	12.9

C.3.1 Environment Installation

All dependencies are managed using `uv`, a fast Python package manager with lockfile support for reproducible environments. To replicate the computational environment:

```
# Install uv (if not already installed)
curl -LsSf https://astral.sh/uv/install.sh | sh

# Clone repository and install dependencies
git clone https://github.com/[repository]/causalix.ai.git
cd causalix.ai

# Sync exact versions from lockfile
uv sync

# Verify installation
uv run python -c "import pandas; import sklearn; print('OK')"
```

The `uv.lock` file ensures deterministic dependency resolution, pinning all 104 packages to exact versions used during experiments. This dependency management approach is informed by principles from the Workflow for Open Reproducible Code in Science [130].

C.3.2 Key Software Dependencies

Full package dependencies with pinned versions are specified in `pyproject.toml`. Table C.10 lists key software components with citations.

Table C.10: Key Software Dependencies and Citations

Package	Version	Citation
<i>LLM APIs</i>		
OpenAI API	1.x	OpenAI (2023). GPT-4 Technical Report. arXiv:2303.08774
Anthropic API	0.61	Anthropic (2024). Claude 3 Model Card
<i>Data Analysis</i>		
NumPy	2.2.3	[131]
pandas	2.2.3	[132]
SciPy	1.x	[133]
<i>Machine Learning</i>		
scikit-learn	1.x	[123]
imbalanced-learn	0.14	[134]
<i>Visualisation</i>		
Matplotlib	3.10	[135]
seaborn	0.13	[136]
<i>Graph & Database</i>		
NetworkX	3.4	[137]
Neo4j	5.27	[138]
<i>LLM Integration</i>		
Pydantic	2.9	[139]
<i>Interactive Computing</i>		
Jupyter	1.1	[140]

Complete BibTeX citations for all software are available in the repository at `thesis/final_thesis/software.bib`.

C.4 Execution Commands

C.4.1 Unified Analysis Entry Points (Single-command pipelines)

The directory `final_runs/` contains three unified analysis entry points intended to reproduce the thesis results tables/figures with minimal manual orchestration:

```
cd <repo_root>

# RQ1 (TruthfulQA verification) + RQ1a (Judge) + RQ1b (Corrector)
uv run python final_runs/RQ1_unified_analysis.py \
  --run-dir final_runs/_runs/RQ1_<timestamp>

# RQ2 (UQ metrics + classifier pipeline + reporting)
uv run python final_runs/RQ2_unified_analysis.py \
  --run-dir final_runs/_runs/RQ2_<timestamp>
```

```
# RQ3 (Deep Research analysis + figures + validation artefacts)
uv run python final_runs/RQ3_unified_analysis.py \
  --run-dir final_runs/_runs/RQ3_<timestamp>
```

For a complete directory-level overview, see `final_runs/README_DIRECTORY_MAP.md`.

C.4.2 RQ1 Judge Verification (TruthfulQA)

Before evaluating judges on CLD-specific tasks, we verify baseline hallucination detection capability using the TruthfulQA benchmark. This sanity check helps ensure the evaluation pipeline behaves as expected before moving to the more domain-specific CLD setting; it does not, by itself, imply that any CLD-specific failures are purely domain-driven.

```
cd <repo_root>/final_runs/validation_RQ1_truthfulqa_judge/

# Run the TruthfulQA judge verification (baseline correctness prompt structure)
uv run python runner/truthqa_judge_baseline_correctness.py

# Optional: recompute summary metrics from a saved run JSON
# uv run python analysis_script/recalculate_metrics.py --input <path_to_results_json>
```

Script Location: `final_runs/RQ1_verification_Truth_QA/runner/truthqa_judge_baseline_correctness.py`

Results Location: `final_runs/RQ1_verification_Truth_QA/` (including `Data_runs/`)

Table C.11: TruthfulQA Verification Configuration

Parameter	Value
Dataset	TruthfulQA (817 questions)
Samples	1,634 (817 truthful + 817 hallucinated answers)
Judge Model	GPT-4.1
Temperature	0.0
Seed	42
Prompt	Baseline correctness
Max Tokens	250

Output Files:

- `truthfulqa_results.json`: Per-sample verdicts
- `truthfulqa_metrics.json`: Accuracy, precision, recall, F1
- `confusion_matrix.png`: Classification confusion matrix

C.4.3 RQ1a Ground Truth Experiments

```
cd data_science/parameter_tuning_experiments/

# Correctness-based ground truth
```

```
python run_rq1a_ground_truth_multirun.py \
  --approach correctness \
  --output-dir ../../final_runs/RQ1a_gt_lit_correctness

# Citation-based ground truth
python run_rq1a_ground_truth_multirun.py \
  --approach citation \
  --output-dir ../../final_runs/RQ1a_gt_lit_citation
```

C.4.4 RQ1a Corruption Detection Experiments

```
# Correctness-based corruption detection
python run_rq1a_corruption_multirun.py \
  --output-dir ../../final_runs/RQ1a_corrupted_correctness

# Citation-based corruption detection
python run_rq1a_corruption_citation_multirun.py \
  --output-dir ../../final_runs/RQ1a_corrupted_citation
```

Table C.12: GT Synth Citation Judge Configuration (GPT-5-mini)

Parameter	Value
Judge Model	GPT-5-mini
Temperature	0.0
Top-p	1.0
Approach	Citation-based judging
Parallelization	10 workers
<i>Corruption</i>	
Corruption Rate	30%
Corruption Types	Motivation (spurious explanations), Structural (new edges, polarity flips)
<i>Dataset</i>	
CLDs	Depressive Symptoms, Social Norms, Older Persons ED
Runs per CLD	3 (seeds 1, 2, 3)
Prompt Variants	4 (Baseline, CoT, Mechanistic, Mechanistic-Lit)
Total Experiments	36 (3 CLDs × 3 runs × 4 prompts)

Table C.13: RQ1a GT Lit Correctness Judge Configuration

Parameter	Value
Judge Model	GPT-4.1
Temperature	0.0
Seed	42
Approach	Correctness-based judging
Parallelization	10 workers
Web Search	Disabled
<i>Dataset</i>	
CLDs	Depressive Symptoms, Social Norms, Older Persons ED
Runs per CLD	3 (seeds 10, 20, 30)
Prompt Variants	3 (Baseline, CoT, Mechanistic)
Total Experiments	27 (3 CLDs × 3 runs × 3 prompts)

Table C.14: RQ1a GT Synth Correctness Judge Configuration

Parameter	Value
Judge Model	GPT-4.1
Temperature	0.0
Seed	42
Approach	Correctness-based judging
Parallelization	10 workers
Web Search	Disabled
<i>Corruption</i>	
Corruption Rate	30%
Corruption Types	Motivation (spurious), Structural (new edges, polarity flips)
<i>Dataset</i>	
CLDs	Depressive Symptoms, Social Norms, Older Persons ED
Runs per CLD	3 (seeds 1, 2, 3)
Prompt Variants	3 (Baseline, CoT, Mechanistic)
Total Experiments	27 (3 CLDs $\times$ 3 runs $\times$ 3 prompts)

Table C.15: RQ1a GT Lit Citation Judge Configuration

Parameter	Value
Judge Model	GPT-4.1
Temperature	0.0
Seed	42
Approach	Citation-based judging
Parallelization	10 workers
Web Search	Enabled
Citation Fetcher	ContentScraper (Jina AI Reader disabled)
<i>Dataset</i>	
CLDs	Depressive Symptoms, Social Norms, Older Persons ED
Runs per CLD	3 (seeds 10, 20, 30)
Prompt Variants	4 (Baseline, CoT, Mechanistic, Mechanistic-Lit)
Total Experiments	36 (3 CLDs $\times$ 3 runs $\times$ 4 prompts)

C.4.5 RQ1b Corrector Experiments

```
python run_rq1b_correction_multirun.py \
  --input-dir ../../final_runs/RQ1a_gt_lit_correctness \
  --output-dir ../../final_runs/RQ1b_corrector/ground_truth
```

Table C.16: RQ1b Corrector (GT Synth) Configuration

Parameter	Value
<i>Corrector</i>	
Corrector Model	GPT-4.1
Temperature	0.0
Corrector Prompts	3 (Baseline, CoT, Mechanistic)
<i>Evaluation Judge</i>	
Judge Model	GPT-4.1
Judge Prompt	Baseline Correctness
Temperature	0.0
<i>Dataset</i>	
Input	RQ1a GT Synth flagged edges
CLDs	Depressive Symptoms, Social Norms, Older Persons ED
Runs per CLD	3 (seeds 1, 2, 3)
Total Experiments	27 (3 CLDs $\times$ 3 runs $\times$ 3 prompts)

Table C.17: RQ1b Corrector (GT Lit) Configuration

Parameter	Value
<i>Corrector</i>	
Corrector Model	GPT-4.1
Temperature	0.0
Corrector Prompts	3 (Baseline, CoT, Mechanistic)
<i>Evaluation Judge</i>	
Judge Model	GPT-4.1
Judge Prompt	Baseline Correctness
Temperature	0.0
<i>Dataset</i>	
Input	RQ1a GT Lit flagged edges
CLDs	Depressive Symptoms, Social Norms, Older Persons ED
Runs per CLD	3 (seeds 10, 20, 30)
Total Experiments	27 (3 CLDs $\times$ 3 runs $\times$ 3 prompts)

C.4.6 RQ1a Validation Studies

Two validation studies assess judge reliability: external validation on an independently expert-constructed CLD, and human-LLM agreement analysis.

External Validation: Uleman’s CLD Citation Provider Comparison

This study validates the citation-based judge on the Uleman’s Alzheimer’s disease CLD (150 edges) constructed through Group Model Building with 17 domain experts.

```
cd final_runs/validation_RQ1a_ulemans_external/
```

```
# Run citation-based judging across provider sessions and write JSON/XLSX outputs
python judge_citation_providers.py
```

```
# Optional: compare two saved runs (e.g., Jina vs ContentScraper) and export XLSX
# python compare_jina_vs_scraper.py
```

Script Location: `final_runs/RQ1a_ulemans_citation_provider_comparison/ judge_citation_providers.py`

Table C.18: Uleman’s Citation Provider Comparison Configuration

Parameter	Value
Dataset	Uleman’s Alzheimer’s CLD (150 edges)
Search Providers	OpenAlex, Brave Search (with/without whitelist), Perplexity, PubMed
Citation Fetchers	Jina AI Reader, ContentScraper (BeautifulSoup-based, PDF-capable)
Configurations	4 providers × 2 fetchers = 8
Total Judgements	1,200 (8 configs × 150 edges)
<i>Judge Settings</i>	
Judge Model	GPT-4.1
Temperature	0.0
Top-p	1.0
Prompt	Baseline citation prompt

Human-LLM Agreement Validation

This study measures inter-rater reliability between LLM citation judge outputs and human expert annotations.

Inputs Location: `final_runs/RQ1_human_validation_citation_judge/` (validation spreadsheets).

Analysis Script (provenance): The one-off analysis used to compute agreement metrics is preserved in the repository snapshot:

```
python RQ1_excel_snapshot_20251215/final_runs/  
RQ1_human_validation_citation_judge/combined_human_validation_analysis.py
```

Table C.19: Human-LLM Agreement Validation Configuration

Parameter	Value
Sample Size	72 edges across 3 validation files
CLDs	Depressive Symptoms, Social Norms
Generators	GPT-4.1, Claude 4 Sonnet
Human Validator	Domain expert (categorical verdicts)
LLM Judge	Citation-based (GPT-4.1, T=0.0, baseline)
<i>Metrics</i>	
Agreement	Cohen’s κ , Agreement Rate, Pearson r
Interpretation	Landis & Koch (1977) guidelines

Table C.20: Human validation rubric for evaluating causal evidence quality in Citation Judge validation (RQ1).
Original rubric in Dutch with English translation.

Stappen Causal Narrative Check / Steps for Causal Narrative Check	
1. Worden de variabelen of synoniemen genoemd en in de zelfde richting als in motivatie ($A \rightarrow B$ en niet $B \rightarrow A$)?	<i>Are the variables or synonyms named and in the same direction as in motivation ($A \rightarrow B$ and not $B \rightarrow A$)?</i>
2. Is er een relatie statistisch significante ($p < 0,05$) relatie?	<i>Is there a statistically significant ($p < 0.05$) relationship?</i>
3. Is de relatie causaal (A veroorzaakt B)?	<i>Is the relationship causal (A causes B)?</i>
4. Als $A \rightarrow C$ veroorzaakt via mediator B is het indirect, benoem in dat geval de mediator in annotatie. Wij zijn opzoek naar directe causale relaties.	<i>If $A \rightarrow C$ is caused via mediator B it is indirect; in that case name the mediator in annotation. We are looking for direct causal relationships.</i>
5. Kijken of de populatie uit de bron overeenkomt met de populatie van de stelling. Als dit anders of beperkt is moet dat benoemd worden in een aparte kolom.	<i>Check if the population from the source matches the population of the statement. If different or limited, this must be noted in a separate column.</i>
6. Algemene check van kwaliteit bewijs: is het een longitudinale studie, wetenschappelijke studie of blogpost?	<i>General quality check of evidence: is it a longitudinal study, scientific study, or blog post?</i>

Classificatie / Classification

Kleur/Colour	Score	Notitie / Note
Dark Green	1.0	Hoogste kwaliteit bewijs, causaal + wetenschappelijk, longitudinaal, RCT. Wanneer het duidelijk causaal bewijs hoeft de rest van de bronnen niet bekeken te worden. <i>Highest quality evidence, causal + scientific, longitudinal, RCT. When clear causal evidence exists, the rest of the sources need not be reviewed.</i>
Light Green	0.8	Wel causaal verband maar geen wetenschappelijke studie, bijv. blog medische professionals. <i>Causal relationship but no scientific study, e.g., blog by medical professionals.</i>
Yellow	0.5	Relatie maar geen causaal verband, cross-sectioneel onderzoek. <i>Relationship but no causal link, cross-sectional study.</i>
Red	0.0	Geen toegang tot link, onjuiste variabelen, geen verband. <i>No access to link, incorrect variables, no relationship.</i>
Gray	0.0	Als de bron echt niet duidelijk is of er iets vreemds aan de hand is. Goed beschrijven wat er niet duidelijk is. <i>If the source is really unclear or something strange is going on. Describe well what is not clear.</i>

Output Files:

- `provider_comparison_results.json`: Per-provider metrics
- `human_llm_agreement.json`: Cohen's κ , agreement rates
- `confusion_matrices/`: Per-file agreement visualisations

C.4.7 RQ2 Hallucination Detection Analysis

```
cd data_science/

python rq2_master_report.py \
  --input-dir ../final_runs \
  --output-dir ../final_runs/RQ2_uq_hallucination_detection

python rq2_ensemble_classifier.py \
  --input-dir ../final_runs/RQ2_uq_hallucination_detection
```

Table C.21: RQ2 UQ Logprobs Configuration

Parameter	Value
<i>UQ Metrics</i>	
Perplexity (PPL)	Capped NLL averaging (token cap 20.0, avg cap 10.0)
$P_{\min}$	Minimum token probability
H_{win}	Max sliding-window entropy (window 5, top- $k=5$)
<i>Dataset</i>	
Source	RQ1a GT Lit experiments (27 correctness files)
Total Edges	31,704 (from 63 deduplicated files)
Hallucination Rate	15.2%
<i>Analysis</i>	
Block Definition	CLD $\times$ run ($N = 9$ blocks)
Statistical Test	One-sample t -test vs. 0.5

Table C.22: RQ2 Cosine Similarity (Embeddings) Configuration

Parameter	Value
<i>Embedding Model</i>	
Model	all-mpnet-base-v2
Source	sentence-transformers (Hugging Face)
Similarity	Max narrative-citation-chunk cosine similarity
<i>Dataset</i>	
Source	RQ1a GT Lit Citation experiments only (36 files)
Ensemble Subset	16,507 edges (35 files with all 4 metrics)
Hallucination Rate	15.2%
<i>Analysis</i>	
Block Definition	CLD $\times$ run ($N = 9$ blocks)
Statistical Test	One-sample t -test vs. 0.5

C.4.8 RQ3 Deep Research Experiments

The RQ3 Deep Research pipeline validates causal claims through automated literature search using a multi-agent system. Execution proceeds in three stages: (1) running Deep Research on edges, (2) running the unified analysis pipeline, and (3) human validation with extrapolation.

Stage 1: Deep Research Execution


```
cd final_runs/RQ3_deep_research_validation/

# Run Deep Research on all TP/FP/FN edges from GT Lit experiments
python test_deep_research_ground_truth.py \
  --input-dir ../RQ1a_gt_lit_correctness \
  --output-dir data/
```

Stage 2: Unified Analysis Pipeline

The unified analysis script orchestrates all RQ3 analysis scripts in sequence:

```
cd <repo_root>

# Run complete analysis pipeline (calls all scripts below) and write outputs
# under a single run folder:
uv run python final_runs/RQ3_unified_analysis.py \
  --run-dir final_runs/_runs/RQ3_<timestamp>

# Individual scripts (called by unified analysis):
# (When invoked through the unified runner, these scripts write to RQ3_OUTPUT_DIR.)
```

Stage 3: Human Validation and Extrapolation

```
cd final_runs/RQ3_deep_research_validation/

# Sample edges for human validation (stratified by CLD x classification)
uv run python scripts/sample_edges_for_validation.py

# After manual annotation: run extrapolation analysis
uv run python scripts/enrichment_extrapolation.py
```

Table C.23: RQ3 Deep Research Configuration

Parameter	Value
Heavy Model	GPT-5 (hypothesis evaluation, judgement, verdict)
Light Model	GPT-5-mini (iterative search, evidence scoring)
Temperature	1.0 (both models)
Top-p	1.0 (both models)
Search API	Brave Search with academic domain filtering
Max Iterations	1
Max Sources per Query	5
Max Searches per Edge	5
Time Limit	900 seconds (15 minutes) per edge
Batch Size	5 edges processed concurrently
<i>Dataset</i>	
Total Edges	285 (44 TP, 178 FP, 63 FN)
CLDs	Depressive Symptoms (84), Social Norms (17), Older Persons ED (184)
<i>Human Validation</i>	
Sample Size	50 edges (29.6% of edges with direct_causal_found=True)
Sampling	Stratified by CLD × classification (seed=42)
Thresholds Tested	{0.5, 0.6, 0.7, 0.8, 0.9}

Table C.24: GT Enrichment Experiment Configuration

Parameter	Value
<i>Deep Research Settings</i>	
Heavy Model	GPT-5 (T=1.0, top_p=1.0)
Light Model	GPT-5-mini (T=1.0, top_p=1.0)
Max Iterations	1
Max Sources per Query	5
Max Searches per Edge	5
<i>Enrichment Scenarios</i>	
Scenario 1 (Enriched GT)	Promote DR-validated FPs to TP in ground truth
Scenario 2 (LLM+DR)	Scenario 1 + add DR-validated FNs to system output
<i>Dataset</i>	
Source Data	RQ3 Deep Research outputs (285 edges)
Classifications	TP (44), FP (178), FN (63)
Eligible for Promotion	Edges with direct_causal_found=True
<i>Human Calibration (for extrapolation)</i>	
Applicability Threshold	0.7 (selected via elbow/gradient rule)
Sample Size	50 edges (stratified by CLD × classification)

Table C.25: Human validation rubric for evaluating Deep Research evidence applicability (RQ3). This rubric guides human validators in assessing whether retrieved evidence supports the claimed causal relationship.

Validation Process

For each edge (row), the human validator performs the following steps:

Step 1: Check the link in the `strongest_causal_evidence_url` column.

Step 2: Assess whether the source contains evidence for (or against) the causal claim: “*source* causes *target*”.

Step 3: Assign a categorical verdict in the `human_verdict` column (see below).

Step 4: Assign an applicability score (0.0–1.0) in the `Fully_applicable` column (see scoring rubric below).

Step 5: Add clarifying notes in the `note` column if needed.

Categorical Verdicts (`human_verdict`)

Verdict	Description
causal	Evidence directly supports a causal relationship between the exact source and target variables.
different_vars, causal	Evidence supports a causal relationship, but for related but not identical variables (e.g., similar constructs, different operationalisations).
non-causal	Evidence shows only correlation, association, or co-occurrence—no causal mechanism or temporal precedence established.
not relevant	Evidence does not address the relationship between source and target variables, or the link is inaccessible/broken.

Applicability Score Rubric (`Fully_applicable`: 0.0–1.0, in 0.1 increments)

Colour	Score	Criteria
	1.0	Exact match: Correct variables, correct constructs/measures, causal-level evidence (RCT, experimental manipulation, longitudinal with controls).
	0.9	Correct variables, causal evidence, but one variable has wrong/different measurement (e.g., self-report vs. objective measure).
	0.8	Correct variables, causal evidence, but two variables have wrong/different measurements .
	0.7	One variable doesn’t quite match but is still similar enough (e.g., “knowledge access” vs. “internet access”), other variable matches exactly. OR: Evidence states “A causes B and C” but only evidence for “A causes B” is relevant.
	0.6	Correlational evidence only (no causal mechanism established), but variables match correctly.
	0.5	Two variables slightly mismatching but still plausible, with causal evidence.
	0.4	Weak match: variables are conceptually related but operationally different; evidence is correlational.
	0.3	Poor match: only one variable loosely related; evidence type unclear or weak.
	0.2	Very poor match: tangential relevance only.
	0.1	Minimal relevance: only keyword overlap; no substantive connection to claimed relationship.
	0.0	Not applicable: Link broken, content inaccessible, completely irrelevant, or no evidence found.

Output Files:

- `data/deep_research_results_*.json`: Per-edge DR verdicts with evidence
- `rq3_comprehensive_stats.json`: Complete statistical analysis
- `rq3_statistical_test_results_latest.json`: Fisher’s exact tests, power analysis
- `rq3_per_cld_detection_rates_latest.json`: Per-CLD breakdown
- `enriched_gt_analysis_latest.json`: Enrichment scenario metrics
- `Figures/rq3_combined_figure.png`: Publication-ready figure
- `rq3_main_table.tex`: Main detection rate table
- `rq3_per_cld_table.tex`: Per-CLD pairwise discrimination table
- `rq3_enriched_gt_table.tex`: Enrichment scenarios comparison
- `validation/enrichment_estimates_*.csv`: Threshold sensitivity results
- `validation/threshold_selection_*.json`: Selected threshold with rationale

Table C.26: RQ3 Human Validation: Evidence Applicability Score Distribution by Edge Classification ($n = 50$)

Applicability Range	TP ($n = 9$)	FP ($n = 32$)	FN ($n = 9$)	Total
[0.0, 0.5)	0	8	3	11 (22.0%)
[0.5, 0.6)	1	1	0	2 (4.0%)
[0.6, 0.7)	0	4	0	4 (8.0%)
[0.7, 0.8)	2	4	1	7 (14.0%)
[0.8, 0.9)	4	7	1	12 (24.0%)
[0.9, 1.0]	2	8	4	14 (28.0%)
Mean $\pm$ SD	0.77 ± 0.12	0.64 ± 0.27	0.67 ± 0.39	0.67 ± 0.27

Note. Applicability score (**Fully applicable**) measures how well retrieved evidence matches the specific (source, target, mechanism) claim. Higher scores indicate more applicable evidence. TP edges have the highest mean applicability and lowest variance, consistent with literature-grounded relationships.

C.4.9 Generator Temperature Sensitivity Analysis

This experiment validates the generator temperature choice ($T=0.7$) by testing whether temperature significantly affects edge recovery performance against literature ground truth.

```
cd data_science/parameter_tuning_experiments/
```

```
python run_temperature_sensitivity.py
```

Script Location: `data_science/parameter_tuning_experiments/run_temperature_sensitivity.py`

Results Location: `data_science/parameter_tuning_experiments/results/temperature_sensitivity-<timestamp>/`

Table C.27: Temperature Sensitivity Experiment Configuration

Parameter	Value
CLDs	3 validation CLDs (Depressive Symptoms, Social Norms, Older Persons ED)
Generator Model	GPT-4.1
Generator Prompt	Nitai_C (<code>prompts_Nitai_C.yaml</code>)
Temperature Values	{0.0, 0.3, 0.5, 0.7, 1.0}
Runs per Temperature	3
Seeds	{10, 20, 30}
Total Runs	45 (5 temperatures $\times$ 3 seeds $\times$ 3 CLDs)
<i>Metric & Analysis</i>	
Metric	Edge F1 vs. literature ground truth
Statistical Test	One-way ANOVA
Null Hypothesis	Mean F1 equal across temperatures

Output Files:

- `temperature_sensitivity_results.json`: Per-run results with F1, precision, recall
- `temperature_sensitivity_analysis.json`: Statistical analysis (ANOVA, per-temperature stats)
- `temperature_sensitivity_analysis.txt`: Human-readable summary report
- `result_excel_path*.xlsx`: Per-run Excel files with edge classifications (45 files)

Key Code Sections:

```
# Temperature values and runs configuration (lines 48-50)
TEMPERATURE_VALUES = [0.0, 0.3, 0.5, 0.7, 1.0]
RUNS_PER_TEMP = {0.0: 3, 0.3: 3, 0.5: 3, 0.7: 3, 1.0: 3}
SEEDS = [10, 20, 30]

# Generator config with varying temperature (lines 85-91)
generator_config = {
    "provider": "openai",
    "model": "gpt-4.1",
    "temperature": temp, # varies per run
    "seed": seed,
    "logprobs": True,
    "top_logprobs": 5
}
```

Edge F1 Computation: Generated edges are compared against the literature-derived ground truth CLD using exact matching (same source, target, and polarity). The `run_discovery_experiment()` function computes edge-level TP/FP/FN counts (via `edge_comparison`), from which $F1 = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$ is derived.

ANOVA Computation: One-way ANOVA via `scipy.stats.f_oneway()` tests whether the mean F1 differs significantly across temperature groups. Groups with $n \geq 2$ are included (all 5 temperatures qualify with $n = 3$ each).

C.4.10 Generator Model Comparison

This one-shot comparison selects the generator model for all subsequent experiments by evaluating GPT-4.1, Sonar-Pro, and Claude Sonnet 4 on edge recovery against literature ground truth.

```
cd final_runs/prelim_generator_model_comparison/

# Run generation experiments (one per model per CLD)
python run_generator_model_comparison.py

# Analyse results and generate thesis table
python analyse_generator_models.py
```

Script Location: final_runs/generator_model_comparison_analysis/

Results Location: final_runs/generator_model_comparison_analysis/

Table C.28: Generator Model Comparison Configuration

Parameter	Value
Models	GPT-4.1, Sonar-Pro, Claude Sonnet 4
CLDs	3 validation CLDs
Generator Prompt	Nitai_C
Temperature	0.7
Corruption Rate	0.0 (generation only, no corruption)
Judge Edges	False (no judging, compare directly to GT)
Metric	Micro-averaged Edge F1 across all CLDs
Selection Rule	Highest aggregate F1 with logprobs availability

Output Files:

- generator_model_comparison.tex: LaTeX table for thesis
- *_comparison_results.json: Per-model metrics

C.4.11 Random Baseline Generator

This analysis establishes a structural random baseline by computing expected edge F1 for Erdős-Rényi random graphs matching each validation CLD’s node and edge counts.

```
cd final_runs/prelim_random_baseline_generator/

python compute_random_baseline.py
```

Script Location: final_runs/random_baseline_generator/compute_random_baseline.py

Results Location: final_runs/random_baseline_generator/

Table C.29: Random Baseline Configuration

Parameter	Value
Random Graphs per CLD	1000
Graph Model	Erdős-Rényi G(n,m)
Node Sets	Same as validation CLDs
Edge Counts	Matched to validation CLDs
Metric	Edge F1 vs. literature ground truth
CI Method	z-distribution (n=1000)

Output Files:

- `random_baseline_comparison.png`: Comparison figure
- `random_baseline_summary.txt`: Summary statistics

C.4.12 Prompt Sensitivity Analysis

This analysis quantifies how prompt engineering choices affect judge and corrector performance by measuring the input-output relationship between semantic prompt distance and F1 change.

Script Location (Judge): `final_runs/Sensitivity_analysis_simple/scripts/sensitivity_analysis_simple.py`

Script Location (Corrector): `final_runs/Sensitivity_analysis_simple/scripts/sensitivity_analysis_corrector.py`

Results Location: `final_runs/Sensitivity_analysis_simple/`

Table C.30: Prompt Sensitivity Analysis Configuration

Parameter	Value
Embedding Model	MPNet (all-mpnet-base-v2, 768-dim)
Distance Metric	Cosine distance from baseline prompt
Prompt Variants	Judge: Baseline, CoT, Mechanistic (+ Mechanistic-Lit for citation); Corrector: Baseline, CoT, Mechanistic
Judge Experiments	4 (Corruption Correctness/Citation, GT Correctness/Citation)
Corrector Experiments	2 (Synthetic Corruption, Ground Truth)
Statistical Test	Friedman (blocked by CLD×run), post-hoc paired Wilcoxon (Bonferroni)

Output Files:

- `prompt_sensitivity_figure.png`: Judge sensitivity visualisation
- `corrector_sensitivity_figure.png`: Corrector sensitivity visualisation
- `prompt_sensitivity_table.tex`: LaTeX summary table for judge prompt effects
- `corrector_sensitivity_table.tex`: LaTeX summary table for corrector prompt effects

C.4.13 Parallelization Benchmark

This benchmark measures wall-clock time speedup from parallelization by comparing sequential (1 worker) vs. parallel (3, 5, 10 workers) execution.

Script Location: `final_runs/parallelization_benchmark/scripts/`

Results Location: `final_runs/parallelization_benchmark/`

```
cd final_runs/supp_parallelization_benchmark/scripts/
```

```
python run_parallelization_benchmark.py
python analyse_parallelization_results.py
```

Table C.31: Parallelization Benchmark Configuration

Parameter	Value
Task	Correctness judging (34 API calls)
Simulated CLD	Depressive Symptoms (34 edges)
Model	GPT-4.1
Temperature	0.3
Max Tokens	200
Worker Configurations	1 (sequential), 3, 5, 10
Runs per Configuration	3
Total Benchmark Runs	12

Output Files:

- `data/benchmark_results_*.xlsx`: Raw timing data (wall-clock, cumulative API time)
- `Figures/speedup_comparison.png`: Speedup analysis figure
- `Figures/efficiency_analysis.png`: Parallel efficiency visualisation
- `parallelization_summary.txt`: Results summary with speedup/efficiency
- `BENCHMARK_METADATA.txt`: Full configuration JSON

C.4.14 Time and Cost Scaling Analysis

This analysis aggregates inference time and cost metrics across all RQ1 experiments to characterise computational scaling.

Script Location: `data_science/aggregate_time_cost_scaling.py`

Results Location: `final_runs/time_and_cost_scaling/`

```
cd data_science/

python aggregate_time_cost_scaling.py \
  --base_dir ../RQ1_excel_snapshot_20251215/final_runs \
  --output_dir ../final_runs/supp_time_cost_scaling \
  --gen-dir parameter_tuning_experiments/results \
  --latex
```

Table C.32: Time and Cost Scaling Analysis Configuration

Parameter	Value
Judging Data	121 Excel files (correctness + citation experiments)
Generation Data	33 Excel files (parameter tuning results)
CLDs	Depressive Symptoms (34 edges), Social Norms (12 edges), Older Persons ED (66 edges)
Judge Types	Correctness, Citation
Prompt Variants	Baseline, CoT, Mechanistic
Judge Models	GPT-4.1, GPT-5-mini
<i>Pricing (OpenAI, December 2024)</i>	
GPT-4.1	\$2.00/1M input, \$8.00/1M output tokens
GPT-5-mini	\$0.15/1M input, \$0.60/1M output tokens

Output Files:

- time_cost_scaling_*.xlsx: Raw aggregated data with per-file metrics
- Figures/figure1_scaling_analysis.png: Time vs. edges/nodes scaling
- Figures/figure2_configuration_comparison.png: Cost/time/tokens by config
- Figures/figure3_generation_vs_judging.png: Generation vs. judging comparison
- latex/*.tex: LaTeX tables for thesis
- ANALYSIS_METADATA.txt: Complete list of input file paths
- aggregate_time_cost_scaling.py: Copy of analysis script

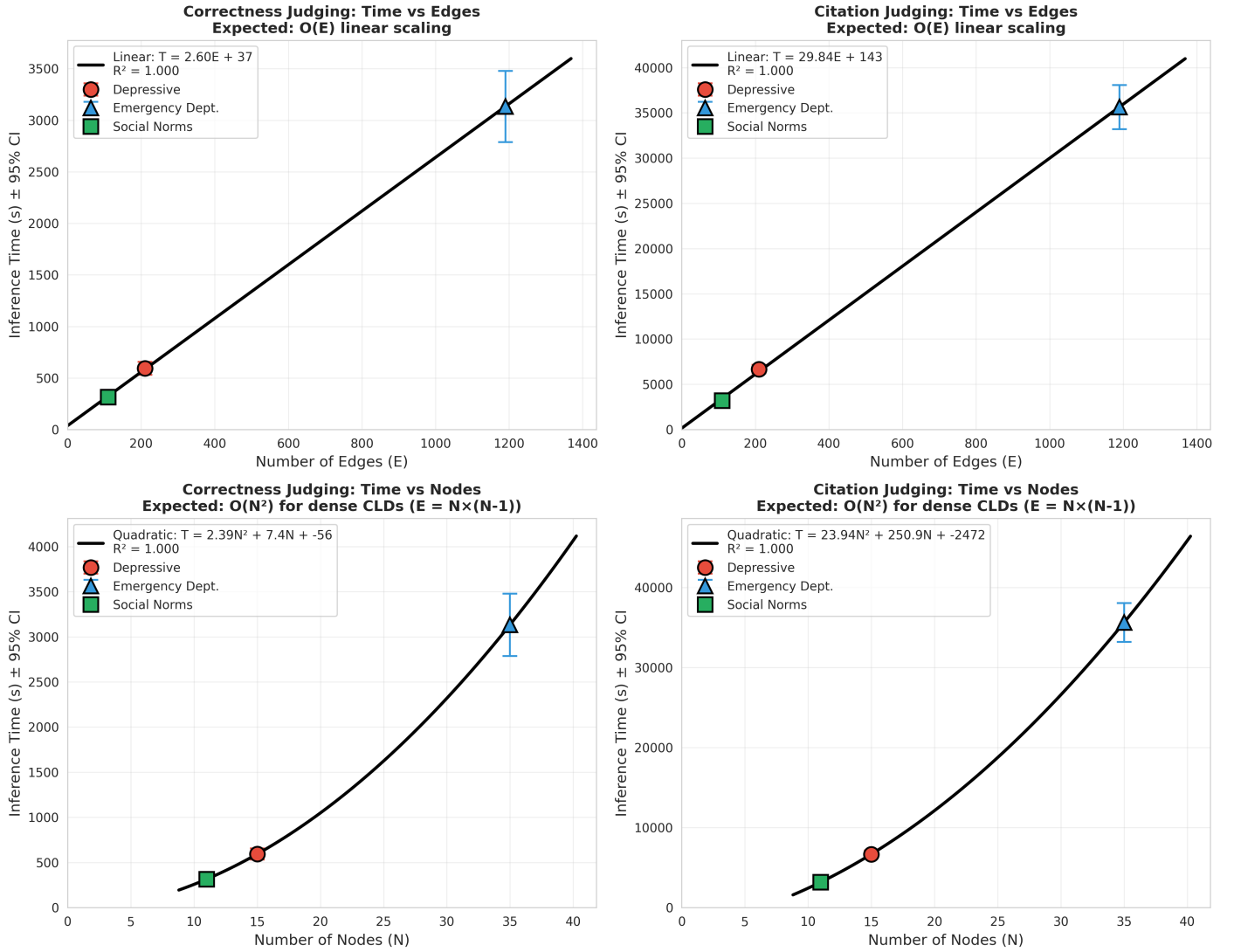


Figure C.1: Time scaling analysis for CLD judging. **Top row:** Time vs. number of edges shows linear $O(E)$ scaling. **Bottom row:** Time vs. number of nodes shows quadratic $O(N^2)$ scaling, consistent with dense CLD structure where $E \approx N(N-1)$. Citation judging (right) is $\sim 11\times$ slower than correctness (left) due to web search and document retrieval. Points represent per-CLD means $\pm 95\%$ CI (error bars); regression lines show fitted models with R^2 values. All three CLDs shown: Depressive Symptoms (D), Social Norms (S), Older Persons ED (E).

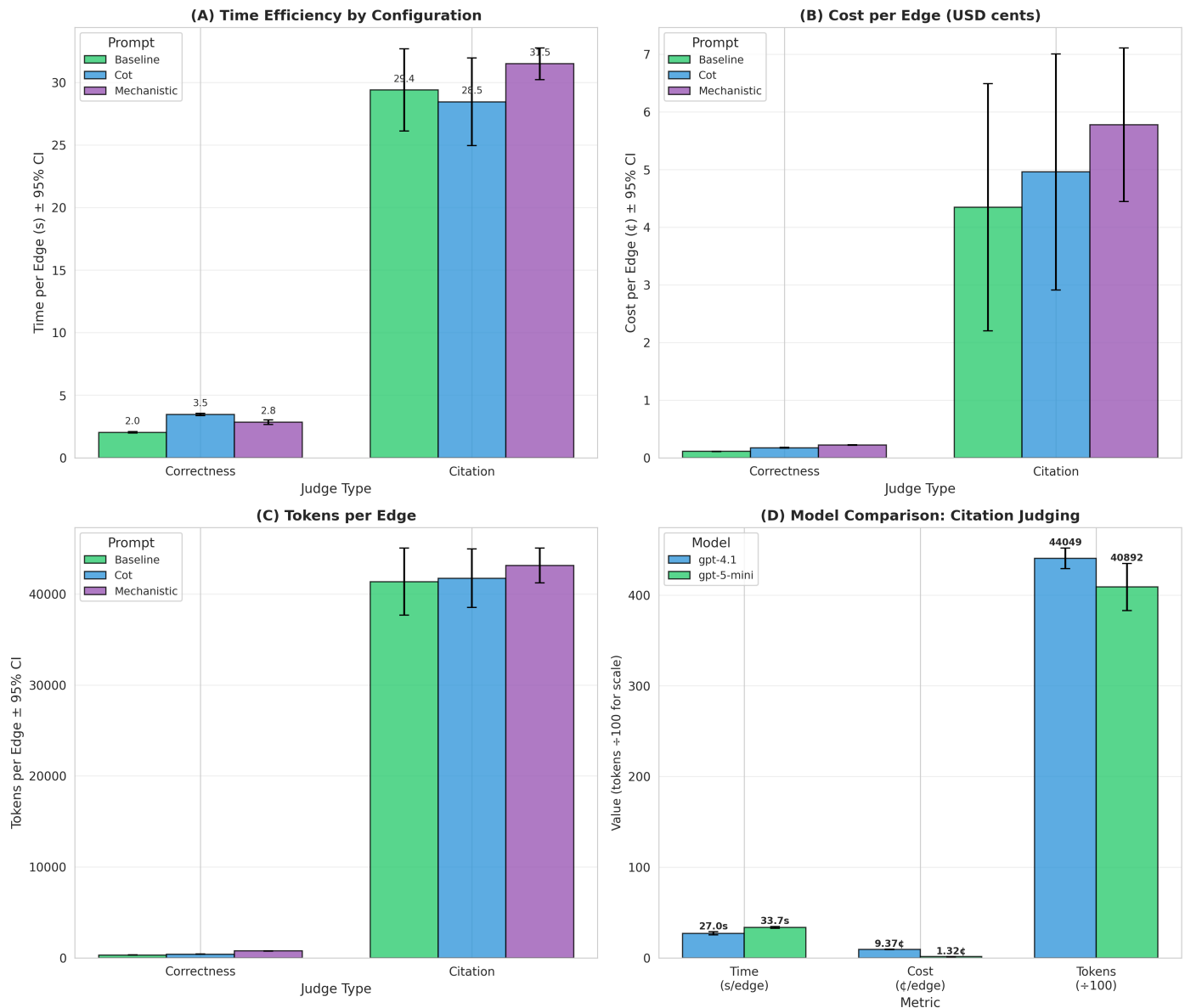


Figure C.2: Computational efficiency by configuration (mean $\pm$ 95% CI). **(A)** Time per edge: Citation judging (30.1 ± 1.4 s/edge) vs. correctness (2.8 ± 0.2 s/edge). **(B)** Cost per edge: Citation (5.6¢ /edge) vs. correctness (0.17¢ /edge). **(C)** Token usage: Citation requires $\sim 11\times$ more tokens due to retrieved document context. **(D)** Model comparison for citation judging: GPT-5-mini achieves comparable time/tokens at 97% lower cost than GPT-4.1. All error bars show 95% CI.

C.4.15 Physics CLD Validation

This experiment validates judge functionality using physics-based CLDs grounded in well-established scientific principles (Newton's laws, conservation laws, Lotka-Volterra equations), where 100% approval is expected.

```
cd final_runs/validation_RQ1a_physics_clds/
```

```
# Run correctness-based judging on physics CLDs
python run_physics_cld_tests.py --approaches correctness
```

```
# Run citation-based judging on physics CLDs
python run_physics_cld_tests.py --approaches citation

# Analyse results and generate LaTeX tables
python analyse_physics_cld_results.py
```

Table C.33: Physics CLD Validation Configuration

Parameter	Value
CLDs	Thermostat (8 edges), Predator-Prey (10), RC Circuit (7), Water Tank (6)
Total Edges	31
Judge Model	GPT-4.1
Temperature	0.0
<i>Physics Principles</i>	
Thermostat	Newton’s Law of Cooling, First Law of Thermodynamics
Predator-Prey	Lotka (1925), Volterra (1926) population dynamics
RC Circuit	Ohm’s Law (1827), Kirchhoff’s Laws (1845)
Water Tank	Conservation of Mass, Torricelli’s Law (1643)

Output Files:

- results/physics_cld_raw_results_*.json: Per-edge judge verdicts
- results/physics_cld_tables_*.tex: LaTeX tables for thesis

C.4.16 Cost Analysis

Computes API costs from token usage in experiment Excel files.

```
cd final_runs/supp_cost_analysis/

# Complete cost analysis (RQ1a + RQ1b estimates)
python calculate_all_costs.py

# Per-experiment breakdown
python calculate_costs_by_experiment.py --output-dir .

# Deep Research costs
python compute_dr_costs.py
```

Table C.34: Cost Analysis Configuration

Parameter	Value
Data Source	LLM Usage Stats sheet in experiment Excel files
RQ1a Files	121 judging experiment files
RQ1b Estimation	2× correctness tokens/edge (correction + rejudging)
<i>Pricing (OpenAI, December 2024)</i>	
GPT-4.1	\$2.00/1M input, \$8.00/1M output tokens
GPT-5-mini	\$0.15/1M input, \$0.60/1M output tokens

Output Files:

- `cost_complete_report_*.txt`: Human-readable cost report
- `cost_complete_table.tex`: LaTeX table for thesis
- `cost_by_experiment_results.json`: Detailed per-experiment breakdown

C.4.17 Power Analysis

Computes statistical power for pairwise prompt comparisons using effect sizes from experimental data.

```
cd final_runs/power_analysis/

python compute_power_analysis.py
```

Output Files:

- Console output with power estimates and required sample sizes

C.4.18 Embedding Model Benchmark

Compares embedding models for cosine similarity computation in RQ2 hallucination detection.

```
cd final_runs/supp_embedding_benchmark/

python embedding_benchmark_real_cld.py
```

Table C.35: Embedding Model Benchmark Configuration

Parameter	Value
Test Documents	30 causal narratives from CLD experiments
Average Length	451 characters
<i>Models Tested</i>	
all-mpnet-base-v2	110M params, 768-dim, via sentence-transformers
Qwen3-Embedding-8B	8B params Q4_K_M, 4096-dim, via Ollama

Key Result: MPNet is 23× faster (3.0 ms/doc vs. 67.3 ms/doc), justifying its selection for batch workloads; published MTEB results for both models are reported in Appendix G for context.

Output Files:

- `benchmark_results.txt`: Full output with LaTeX table

C.5 Output Artefacts

C.5.1 Metadata Files

Each experiment run generates:

- `session_info.txt`: Base CLD session identifier and generation parameters
- `corruption_metadata.json`: Corruption seed, temperature, model, corrupted edge indices
- `judging_metadata*.json`: Per-prompt judging parameters (prompt version, temperature, model)

C.5.2 Result Files

Table C.36: Experiment Output File Types

File Type	Contents
<code>judged*.xlsx</code>	Per-edge validation results with scores and classifications
<code>*metrics.json</code>	Aggregate metrics (precision, recall, F1, AUC)
<code>*confusion_matrix.png</code>	Visualisation of classification performance
<code>*score_distribution.png</code>	Distribution of judge scores by ground truth label

C.5.3 Naming Convention

Output files follow: `judged-{CLD_NAME}-{PROMPT_TYPE}-{TIMESTAMP}.xlsx`

Where `PROMPT_TYPE` $\in$ {baseline, mechanistic, cot} and `TIMESTAMP` = YYYYMMDD_HHMMSS.

C.6 Output Token Verification

To verify no judge outputs were truncated due to `max_tokens` limits, we analysed all judge messages using the GPT-4 tokenizer (`tiktoken`).

Table C.37: Output Token and API Call Analysis by Prompt Variant

Variant	Description	Max Tok	Limit	Messages	Calls	Success	Rate
baseline	Direct evaluation	1,499	1,500	18,548	34,660	34,641	99.9%
mechanistic	Bradford Hill criteria	255	1,500	9,265	9,262	9,262	100.0%
mech_lit	Mechanistic (citation, lit.)	1,487	1,500	13,466	64,993	57,863	89.0%
mech_orig	Mechanistic (citation, orig.)	1,110	1,500	4,736	21,026	21,008	99.9%
cot	Chain-of-thought reasoning	1,534	10,000	24,266	59,795	59,304	99.2%
Total		—	—	70,281	189,736	182,078	96.0%

Note. Max Tok = maximum observed output tokens via GPT-4 tokenizer (`tiktoken`).
Limit = `max_tokens` API parameter. Calls = total API calls. Success = HTTP 200 completions. Rate = success percentage.

Reproduction: `python final_runs/token_call_analysis_RQ1a_judge/analyse_output_tokens_tiktoken.py`

Conclusion: All outputs remained below their respective token limits. The closest approach was baseline at 1,499/1,500 tokens. No truncation artefacts (incomplete JSON, mid-sentence cutoffs) were detected. The mech_lit variant shows lower success rate (89.0%) due to web search failures in citation retrieval.

Appendix D

Statistical Procedures Reference

For transparency and reproducibility, we summarise the statistical hypotheses and assumption diagnostics associated with the procedures used throughout the thesis:

D.0.0.1 Omnibus and Post-Hoc Tests.

- **Friedman test [141] (blocked prompt effect; RQ1a/RQ1b):** omnibus test of prompt effects in a repeated-measures (blocked) design using ranks within blocks (CLD $\times$ run). H_0 : prompt conditions have equal rank distributions within blocks. H_1 : at least one prompt differs. Effect size: Kendall’s W [142].
- **Wilcoxon signed-rank test [143] (paired post-hoc; RQ1a/RQ1b):** pairwise test on matched blocks. H_0 : median(Δ) = 0 for the paired prompt difference across blocks. H_1 : median(Δ) $\neq$ 0.
- **Mann–Whitney U test [144] (RQ2 diagnostic):** compares two independent samples by rank. Used descriptively to characterise separation between hallucinated vs. correct edge distributions within CLDs. Effect size: rank-biserial r .
- **One-sample t -test (RQ2):** tests whether the mean of block-level AUCs differs from chance (0.5). H_0 : $\mu_{\text{AUC}} = 0.5$. H_1 : $\mu_{\text{AUC}} \neq 0.5$. Valid for $n = 9$ blocks given approximate normality of block means (Shapiro–Wilk $p > 0.05$).
- **Fisher’s exact test [145] (RQ3):** tests independence in 2×2 contingency tables (e.g., detection rate TP vs. FP). H_0 : rates are equal. H_1 : rates differ. Effect size: Cohen’s h .
- **Kruskal–Wallis test [146] (robustness check):** non-parametric one-way ANOVA on ranks. Used to verify robustness when parametric assumptions are violated. H_0 : all groups have the same distribution. H_1 : at least one group differs.

D.0.0.2 Parametric Sensitivity Analysis (Appendix B).

- **Repeated-measures ANOVA (RM-ANOVA):** parametric repeated-measures omnibus test used for sensitivity-style variance decomposition. H_0 : equal within-subject means across prompts. H_1 : at least one prompt mean differs. Effect size: η_p^2 .
- **Shapiro–Wilk test [124] (residual normality):** diagnostic for approximate normality of RM-ANOVA residuals. H_0 : residuals are normally distributed.
- **Mauchly’s test [147] (sphericity, $k > 2$):** diagnostic for sphericity (equal variance of all pairwise condition differences). H_0 : sphericity holds.
- **Greenhouse–Geisser correction [148]:** degrees-of-freedom adjustment using $\epsilon_{GG} \in (0, 1]$ when sphericity is violated; yields a more conservative p-value. This is a correction procedure (not a separate hypothesis test).

- **Levene’s test** [125] (**homogeneity of variance**): diagnostic for equal variances across groups. H_0 : variances are equal.

D.0.0.3 Multiple-Comparison and Interval Methods.

- **Bonferroni correction** [149, 150]: family-wise error control across a predefined family of tests, using $\alpha_{\text{adj}} = \alpha/m$ (or equivalently reporting adjusted p values).
- **Wilson score interval** [151] (**RQ3**): confidence interval for proportions that avoids overshoot near 0 or 1. Used for detection rates and human-validation accuracy.
- **Fisher z -transformation** [152] (**RQ2**): variance-stabilizing transform for Pearson r ; used when aggregating correlations across blocks.

D.0.0.4 RQ2 Classifier Algorithms (Ensemble Models, Hyperparameters).

- **Shared implementation (RQ2 Phases 4–6)**: classifiers are implemented in scikit-learn [123] and trained on standardised CI metrics using `StandardScaler` (fit on training blocks only). Splits are *group-aware* with `GroupShuffleSplit(test_size=0.33, random_state=42)` for the held-out test blocks and `GroupKFold(n_splits=3)` for block-level CV, with groups defined as `block_id = CLD × run` (9 blocks total).
- **Random Forest (RF)** [153] (**as implemented**): `RandomForestClassifier(n_estimators=100, class_weight='balanced', random_state=42)`.
- **Gradient Boosting (GB)** [154] (**as implemented**): `GradientBoostingClassifier(n_estimators=100, random_state=42)`.
- **Logistic Regression (LR)** [155] (**as implemented**): `LogisticRegression(max_iter=1000, class_weight='balanced', random_state=42)`.
- **Neural Network (MLP)** [156] (**as implemented**): `MLPClassifier(hidden_layer_sizes=(32,16), max_iter=200, random_state=42)` trained via backpropagation.

Appendix E

Experimental Prompts

This section documents all prompts used in the final experiments. Each prompt variant is presented verbatim as used during evaluation. The prompts are organised by component: correctness-based judges (Section E.3), citation-based judges (Section E.4), and correctors (Section E.5).

E.1 Prompt Engineering Ablation Study

Prompt engineering has emerged as a critical technique for optimizing large language model (LLM) performance on domain-specific tasks. To systematically evaluate the effectiveness of various prompting strategies for automated causal loop diagram generation, we conducted an ablation study comparing nine prompt variants across three causal loop diagrams.

E.1.1 Experimental Design

We evaluated nine prompt variants across three ground-truth causal loop diagrams:

- **CLD 1:** Depressive Symptoms in response to stressors (34 edges)
- **CLD 2:** Social Norms and obesity prevalence (12 edges)
- **CLD 3:** Older Persons ED (66 edges)

All experiments used GPT-4.1 as the generator model with consistent hyperparameters (temperature=0.7, top_p=1.0). Performance was measured using the F1 score, which balances precision and recall in edge discovery.

E.1.2 Prompt Engineering Techniques

Table E.1 presents a classification matrix of prompt engineering techniques employed in each variant, based on established literature.

Table E.1: Prompt Engineering Techniques Classification Matrix

Prompt	CoT ¹	Few-Shot ²	Role ³	Decomp ⁴	RAG ⁵
Nitai_A		✓	✓		
Nitai_B	✓	✓	✓	✓	
Nitai_C	✓✓	✓✓	✓	✓	
Nitai_E	✓✓	✓	✓	✓	✓✓
Rick_A		✓	✓✓		
Rick_B	✓	✓✓	✓✓	✓✓	
Cillian_B		✓	✓	✓	
Cillian_C		✓	✓	✓✓	
Current		✓			

¹Chain-of-Thought [15]; ²Few-Shot Learning [16]; ³Role Prompting; ⁴Task Decomposition; ⁵RAG with mandatory citations.

Legend: ✓ = basic; ✓✓ = strong implementation

Key Prompt Distinctions:

- **Nitai variants:** Progressive complexity from baseline (A) to Chain-of-Thought reasoning (B, C), culminating in RAG-augmented prompting (E). In the final experiments, Nitai_C uses three relationship types (POSITIVE, NEGATIVE, NONE).
- **Rick variants:** Emphasis on academic rigor and system dynamics best practices. Rick_B features comprehensive systematic decomposition with explicit confounding analysis.
- **Cillian variants:** Cillian_B employs intervention-based framing. Cillian_C uses hypothesis testing with definition-strict evaluation.
- **Current:** Minimal baseline control with basic prompting.

E.1.3 Results

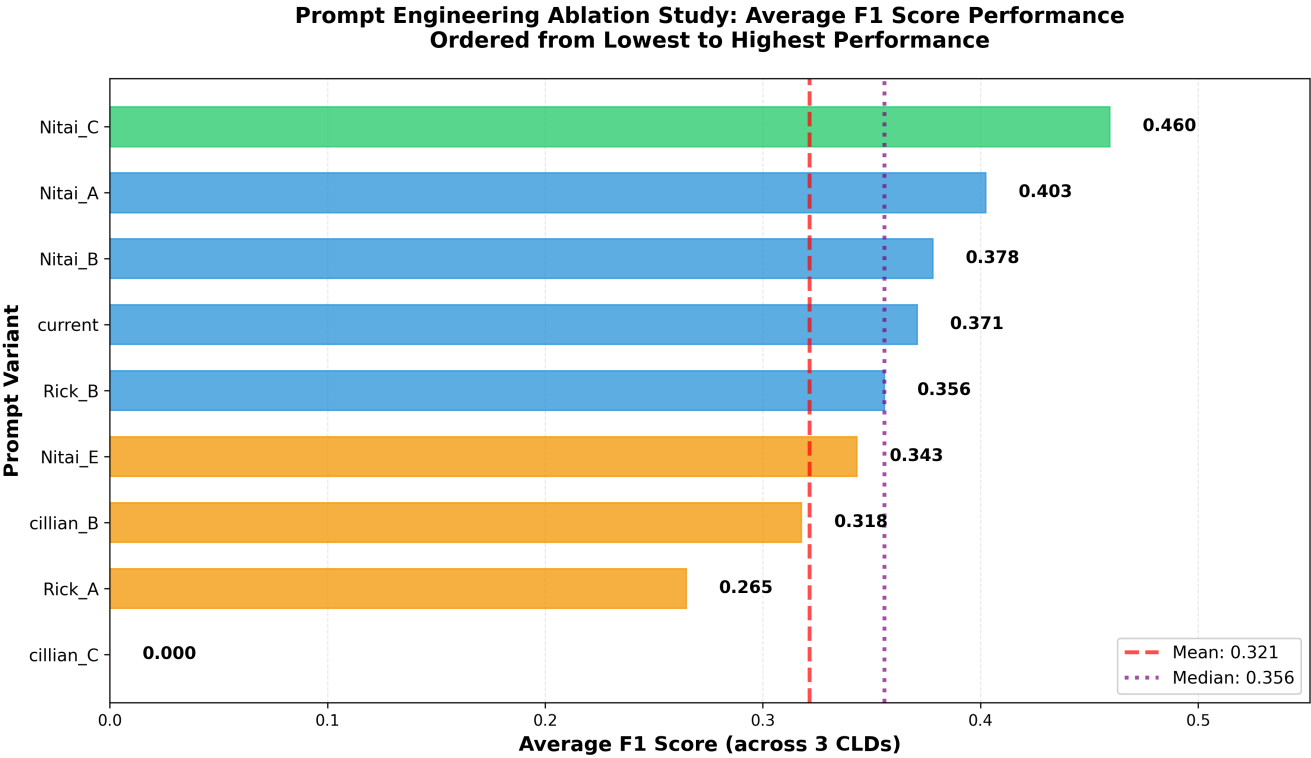


Figure E.1: Average F1 scores for prompt variants across three CLDs, ordered from lowest to highest. Nitai_C achieved highest performance (F1=0.460). Colour coding: green ($F1 \geq 0.45$), blue (0.35–0.45), orange (0.25–0.35), red (<0.25).

Table E.2: F1 Scores by Prompt Variant and Causal Loop Diagram

Prompt	CLD 1	CLD 2	CLD 3	Average
Nitai_C	0.591	0.204	0.583	0.460
Nitai_A	0.590	0.218	0.400	0.403
Nitai_B	0.588	0.092	0.455	0.378
Current	0.588	0.239	0.286	0.371
Rick_B	0.444	0.186	0.438	0.356
Nitai_E	0.438	0.207	0.385	0.343
Cillian_B	0.515	0.152	0.286	0.318
Rick_A	0.380	0.145	0.269	0.265
Cillian_C	0.000	0.000	0.000	0.000

E.1.4 Key Findings

Overall Performance:

- **Best Performance:** Nitai_C achieved the highest average F1 score (0.460), differing from others in multiple dimensions (dual-level CoT, relationship taxonomy, examples).

- **Performance Range:** Range of 0.195 between best (0.460) and second-worst (0.265) performers. Mean $F1 = 0.321$, $SD = 0.132$.

CLD-Specific Patterns:

- **CLD 2 Challenge:** All variants performed worst on CLD 2 (Social Norms), with $F1$ scores 0.00–0.24.
- **High Variance:** Within-prompt SD across CLDs ranged 0.18–0.25, indicating sensitivity to problem characteristics.

Technique-Specific Observations:

- **RAG Trade-off:** Nitai_E (mandatory citations) showed 25% lower performance than Nitai_C, suggesting strict citation requirements limit valid relationship discovery.
- **Complete Failure:** Cillian_C’s zero performance indicates overly restrictive definition-grounded evaluation can be counterproductive.

E.1.5 Interpretation

The progression Nitai_A $\rightarrow$ Nitai_B $\rightarrow$ Nitai_C demonstrates incremental performance gains with Chain-of-Thought, achieving 14% improvement over baseline. This supports Wei et al.’s [15] findings that explicit reasoning steps enhance complex task performance.

Rick_B ($F1=0.356$), employing systematic decomposition, underperformed Nitai_C’s CoT approach by 23%, suggesting decomposition strategies may be less effective when tasks require holistic pattern recognition rather than sequential problem solving.

E.1.6 Limitations

- **Sample Size:** Only three CLDs limits generalizability.
- **Single Run:** Each variant tested once per CLD, preventing statistical significance testing.
- **Model Specificity:** Results apply to GPT-4.1; patterns may differ for other LLMs.
- **Prompt Authorship:** Different variants designed by different researchers, introducing potential confounding.

E.2 Generator Prompt (Nitai_C)

The final experiments use the Nitai_C generator prompt configuration for both node generation (`generateNodes`) and edge generation (`generateEdges`). The prompt is stored at:

`data_science/parameter_tuning_experiments/prompts_Nitai_C.yaml`

and is reproduced verbatim below.

```
prompts:
- prompt_id: generateNodes
  variables:
  - name: target
    type: string
```

```

    description: The target variable for the Model.
- name: target_units
  type: string
  description: Units by which the Target is measured.
- name: target_definition
  type: string
  description: The user provided definition of the selected target variable.
- name: temporal
  type: string
  description: The temporal scale of the causality study.
- name: spatial
  type: string
  description: The spatial scale of the causality study.
- name: context
  type: list
  description: List of key contextual factors for the Model

```

prompts:

```
sys_prompt: >
```

You are a systems dynamics modelling expert. Your role is to identify a single, clearly defined, variable that causally influences a target variable within a specific context, time, and space. The variable must be measurable, not composite, and must not overlap with the target or any excluded variables.

Let's think step-by-step:

1. First, understand the target variable, its definition, and the context thoroughly
2. Brainstorm potential causal variables that directly influence the target
3. Evaluate each potential variable against the criteria: measurable, non-composite, no overlap with target
4. Select the most appropriate variable with the clearest causal link
5. Verify the variable works within the specified temporal and spatial scale
6. Define the variable concisely and specify appropriate units

Once you've completed this reasoning process, respond only with a json including the variable name and its concise definition, formatted as: {"Variable": "Concise name of the variable", "Definition": "Definition of the variable", "Units": "Units of measurements for the variable"}, with no additional text.

```
usr_prompt: |
```

Given the following constraints:

```

Target Variable: {{ target.title() }} (measured in {{ target_units }})
Temporal Scale: {{ temporal.title() }}
Spatial Scale: {{ spatial.title() }}
Context: {{ context | join(', ') }}

```

I need you to think carefully through this problem:

1. First, understand what {{ target.lower() }} means: "{{ target_definition.lower() }}"
2. Consider what directly causes changes in {{ target.lower() }} within the {{ temporal.lower() }} timeframe and {{ spatial.lower() }} spatial scale
3. Evaluate potential variables against these criteria:
 - Must be directly measurable (not abstract)
 - Must not be composite (not combining multiple factors)
 - Must have a clear causal effect on {{ target.lower() }}
 - Must work within the specified temporal and spatial scales

After this analysis, name a variable (2-3 words) with a direct causal effect on {{ target.lower() }}.

Be concise, and give the name of the variable and a short definition, as well as the units by which it is measured.

The output must be exactly in the format of the following example:

```

    {"Variable": "Concise name of the variable",
     "Definition": "Definition of the variable",
     "Units": "Units of measurements for the variable"}
    usr_prompt_extension: " The variable cannot be the same as or overlap with anything in
the following list: "
version: 0.0.1
- prompt_id: generateEdges
  variables:
    - name: var1
      type: str
      description: First variable in undefined relationship.
    - name: var2
      type: str
      description: Second variable in undefined relationship.
    - name: def1
      type: str
      description: Definition of first variable.
    - name: def2
      type: str
      description: Definition of second variable.
    - name: spatial
      type: str
      description: The spatial scale of the causality study.
    - name: temporal
      type: str
      description: The temporal scale of the causality study.
    - name: variables
      type: list[string]
      description: List of existing variables in the CLD.
  prompts:
    sys_prompt: >
      You are a scientific system dynamics model expert specializing in causal loop
diagrams. Given two variables and their definitions, your task is to determine the
precise causal relationship between them.

      Approach this systematically:
      1. First, clearly understand what each variable means and how it's measured
      2. Consider the mechanisms by which the first variable might directly affect the
second
      3. Evaluate whether this mechanism works within the given spatial and temporal
scales
      4. Check for potential mediating variables that might explain the relationship
      5. Distinguish between correlation and causation - focus only on true causal
relationships
      6. Determine if the relationship is positive, negative, or non-existent
      7. Verify your reasoning by considering hypothetical increases and decreases in the
first variable
      8. Provide a concise explanation that justifies your conclusion

      Select the most appropriate relationship type based on causal logic, not mere
correlation. Respond with a dictionary: {"Relationship": "TYPE", "Explanation": "A brief
explanation."}. Make sure there is no additional text.
    usr_prompt: |
      I need to determine if there is a causal relationship between "{{ var1 }}" and "{{
var2 }}" for a Causal Loop Diagram (CLD). Let me think through this step by step:

      Step 1: Understand the variables
      - Variable 1: "{{ var1 }}" defined as "{{ def1 }}"
      - Variable 2: "{{ var2 }}" defined as "{{ def2 }}"
      - Scale: Spatial = "{{ spatial }}", Temporal = "{{ temporal }}"

      Step 2: Consider potential causal mechanisms

```

```

- Does a change in "{{ var1 }}" directly cause a change in "{{ var2 }}" through a
clear mechanism?
- Does this mechanism operate within the given spatial and temporal scales?

Step 3: Check for mediating variables
- Are there variables in the CLD that mediate this relationship: {{ variables | join
(", ") }}
- If a mediator exists, the direct relationship should not be included in the CLD

Step 4: Determine relationship type
If there is a causal relationship, I'll classify it as one of these:
1. POSITIVE - An increase in "{{ var1 }}" causes an increase in "{{ var2 }}", or a
decrease in "{{ var1 }}" causes a decrease in "{{ var2 }}" (direct relationship).
2. NEGATIVE - An increase in "{{ var1 }}" causes a decrease in "{{ var2 }}", or a
decrease in "{{ var1 }}" causes an increase in "{{ var2 }}" (inverse relationship).
3. NONE - No causal relationship exists between "{{ var1 }}" and "{{ var2 }}".

An important condition is that in your explanation you must mention explicitly the
causal relationship, i.e. "{{ var1 }}" causes "{{ var2 }}" or "{{ var2 }}" causes "{{
var1 }}", because this ensures directional causality.

Provide a brief, maximum 4-sentence explanation in the format: {"Relationship":"TYPE
","Explanation":"A brief explanation."}

Example Responses:
1. For a Positive Causal Relationship:
{"Relationship":"POSITIVE","Explanation":"Force causes acceleration in a direct
causal relationship. Increasing the force on an object of constant mass directly
increases its acceleration. Conversely, reducing the force directly decreases the
acceleration."}
2. For a Negative Causal Relationship:
{"Relationship":"NEGATIVE","Explanation":"Electrical resistance causes current
flow to change in an inverse relationship. Increasing resistance in a circuit directly
reduces the current flow (assuming constant voltage). Decreasing the resistance directly
increases the current."}
3. For No CAUSAL Relationship:
{"Relationship":"NONE","Explanation":"The number of birds in a city does not cause
stock market performance to change. There is no known direct causal link between these
variables in either direction."}
4. For a Mediated Relationship:
{"Relationship":"NONE","Explanation":"Income Levels do not directly cause Housing
Prices. A mediating variable (Economic Growth) is already present in the CLD that
explains the relationship between these variables."}

IMPORTANT: In case you don't know, simply respond with {"Relationship":"NONE","
Explanation":"I don't know"}

Now, define the relationship between "{{ var1 }}" ("{{ def1 }}") and "{{ var2 }}"
("{{ def2 }}") following this format: {"Relationship":"TYPE","Explanation":"A brief
explanation."} , taking particular care to make sure the relationships are causal and not
just correlated or associated.

usr_prompt_extension: ""
version: 0.0.2

```

Listing E.1: Generator Prompt Configuration: Nitai_C (generateNodes + generateEdges)

Three prompting strategies were evaluated for each component:

1. **Baseline:** Direct instruction without reasoning scaffolding (control condition)

2. **Chain-of-Thought (CoT)**: Zero-shot CoT via “Let’s think step by step” [60]
3. **Mechanistic**: Domain-specific criteria based on the Bradford Hill causal framework [68]

For citation judges, an additional **Mechanistic-Lit** variant combines mechanistic evaluation with explicit literature grounding requirements.

E.3 Correctness Judge Prompts

The correctness judge evaluates causal reasoning quality based on internal logical consistency, without reference to external citations. Each prompt uses the `judgeCorrectness` prompt ID.

E.3.1 Baseline

```
SYSTEM PROMPT:
You are a precise and fair judge evaluating the quality of causal reasoning.
Your job is to carefully read the causal explanation and determine if it is
logically sound and scientifically coherent.
Base your judgement only on the internal logic of the explanation, not on
external knowledge or citations.

USER PROMPT:
Evaluate the quality of this causal explanation:

SOURCE VARIABLE: {{ source }}
TARGET VARIABLE: {{ target }}
RELATIONSHIP TYPE: {{ relationship }}
EXPLANATION: {{ motivation }}

Based on the explanation, assess whether the causal reasoning is sound.

Respond in this format:
REASON: [brief explanation of your judgement]
VERDICT: [CORRECT|PARTIALLY_CORRECT|INCORRECT]
SCORE: [1.0 for CORRECT, 0.5 for PARTIALLY_CORRECT, 0.0 for INCORRECT]
```

Listing E.2: Correctness Judge: Baseline Prompt

E.3.2 Chain-of-Thought (CoT)

```
SYSTEM PROMPT:
You are a precise and fair judge evaluating the quality of causal reasoning.
Your job is to carefully read the causal explanation and determine if it is
logically sound and scientifically coherent.
Base your judgement only on the internal logic of the explanation, not on
external knowledge or citations.

USER PROMPT:
Evaluate the quality of this causal explanation:

SOURCE VARIABLE: {{ source }}
TARGET VARIABLE: {{ target }}
RELATIONSHIP TYPE: {{ relationship }}
EXPLANATION: {{ motivation }}
```


Let's think step by step to assess whether the causal reasoning is sound.

Respond in this format:

REASON: [Start with your step-by-step chain of thought, then provide your final judgement. Make your reasoning process explicit.]

VERDICT: [CORRECT|PARTIALLY_CORRECT|INCORRECT]

SCORE: [1.0 for CORRECT, 0.5 for PARTIALLY_CORRECT, 0.0 for INCORRECT]

Listing E.3: Correctness Judge: CoT Prompt (Kojima et al., 2022)

E.3.3 Mechanistic (Bradford Hill)

SYSTEM PROMPT:

You are a precise and fair judge evaluating causal explanations using core principles from the Bradford Hill causal criteria framework.

You will assess explanations on FOUR key dimensions adapted for qualitative causal models:

1. PLAUSIBILITY - Does it describe a plausible causal mechanism?
2. TEMPORALITY - Is the causal direction justified and temporally coherent?
3. STRENGTH - Does it convey that this is an important (not trivial) causal relationship?
4. COHERENCE - Is the reasoning internally consistent and non-circular?

These criteria align with Bradford Hill's framework (Shimonovich et al., 2020) while being practical for conceptual models.

Base your judgement only on the internal logic of the explanation, not on external knowledge.

USER PROMPT:

Evaluate this causal explanation using four core Bradford Hill criteria:

CLAIMED CAUSAL PATHWAY:

{{ source }} -> [{{ relationship }}] -> {{ target }}

PROPOSED EXPLANATION:

{{ motivation }}

Assess the explanation on these four dimensions:

1. PLAUSIBILITY (Causal Mechanism):
 - Does the explanation describe HOW {{ source }} affects {{ target }}?
 - Is a specific mechanism or pathway described (not just assertion)?
 - Is the mechanism scientifically plausible and non-circular?
2. TEMPORALITY (Causal Direction):
 - Does the explanation support the direction {{ source }} -> {{ target }}?
 - Is temporal ordering appropriate (cause precedes effect)?
 - Does it avoid reverse causation ({{ target }} -> {{ source }})?
3. STRENGTH (Relative Importance):
 - Does the explanation convey that {{ source }} is an IMPORTANT or SUBSTANTIAL influence on {{ target }}?
 - Does it suggest this is more than trivial, negligible, or merely correlational?
 - Is the causal claim stated with appropriate confidence (not overly tentative)?

```

4. COHERENCE (Internal Consistency):
- Is the reasoning logically consistent and non-contradictory?
- Does it avoid tautological or circular arguments?
- Are the claims coherent with each other?

Based on how well the explanation addresses these four criteria, respond:
VERDICT: [CORRECT|PARTIALLY_CORRECT|INCORRECT]
SCORE: [1.0 for CORRECT, 0.5 for PARTIALLY_CORRECT, 0.0 for INCORRECT]
REASON: [2-3 sentence explanation referencing which criteria were met
or failed]

Guidelines:
- CORRECT: Addresses all four criteria reasonably well
- PARTIALLY_CORRECT: Addresses 2-3 criteria well but has notable gaps
- INCORRECT: Fails most criteria (vague, circular, wrong direction,
implausible, or trivial)

```

Listing E.4: Correctness Judge: Mechanistic Prompt (Shimonovich et al., 2020)

E.4 Citation Judge Prompts

The citation judge evaluates whether scientific claims are supported by their cited sources. Each prompt uses the `judgeCitation` prompt ID.

E.4.1 Baseline

```

SYSTEM PROMPT:
You are a precise and fair judge evaluating whether scientific claims are
supported by their cited sources.
Your job is to carefully read the relationship motivation and the cited
sources, and determine if the motivation is supported.
Base your judgement only on the content of the citations provided, not on
general knowledge or the internal logic of the explanation.

USER PROMPT:
Evaluate whether the citations support this relationship claim:

SOURCE VARIABLE: {{ source }}
TARGET VARIABLE: {{ target }}
RELATIONSHIP TYPE: {{ relationship }}
EXPLANATION: {{ motivation }}

CITED SOURCES:
{{ citations }}

Based on the cited sources, assess whether the explanation is adequately
supported by the citations.

Respond in this format:
REASON: [brief explanation of your judgement]
VERDICT: [SUPPORTED|PARTIALLY_SUPPORTED|NOT_SUPPORTED]
SCORE: [1.0 for SUPPORTED, 0.5 for PARTIALLY_SUPPORTED, 0.0 for NOT_SUPPORTED]

```

Listing E.5: Citation Judge: Baseline Prompt

E.4.2 Chain-of-Thought (CoT)

```

SYSTEM PROMPT:
You are a precise and fair judge evaluating whether scientific claims are
supported by their cited sources.
Your job is to carefully read the relationship motivation and the cited
sources, and determine if the motivation is supported.
Base your judgement only on the content of the citations provided, not on
general knowledge or the internal logic of the explanation.

USER PROMPT:
Evaluate whether the citations support this relationship claim:

SOURCE VARIABLE: {{ source }}
TARGET VARIABLE: {{ target }}
RELATIONSHIP TYPE: {{ relationship }}
EXPLANATION: {{ motivation }}

CITED SOURCES:
{{ citations }}

Let's think step by step to assess whether the explanation is adequately
supported by the citations.

Respond in this format:
REASON: [brief explanation of your judgement]
VERDICT: [SUPPORTED|PARTIALLY_SUPPORTED|NOT_SUPPORTED]
SCORE: [1.0 for SUPPORTED, 0.5 for PARTIALLY_SUPPORTED, 0.0 for NOT_SUPPORTED]

```

Listing E.6: Citation Judge: CoT Prompt (Kojima et al., 2022)

E.4.3 Mechanistic (Bradford Hill)

```

SYSTEM PROMPT:
You are a precise and fair judge evaluating cited evidence using the
Bradford Hill framework.

You will assess whether citations provide evidence for four key causality
criteria identified as having strong support across modern causal inference
approaches:

1. STRENGTH OF ASSOCIATION - including confounding analysis
2. TEMPORALITY - temporal precedence and ruling out reverse causation
3. PLAUSIBILITY - causal mechanisms and mediation pathways
4. EXPERIMENT/DESIGN - study design implications for exchangeability

Base your judgement only on what the citations actually say, not on
general knowledge.

USER PROMPT:
Evaluate the cited evidence for this causal relationship using the
Bradford Hill framework:

CLAIMED CAUSAL PATHWAY:
{{ source }} -> [{{ relationship }}] -> {{ target }}

PROPOSED MECHANISM:
{{ motivation }}

```

CITED SOURCES:
 {{ citations }}

Let's think step by step. Assess whether the citations provide evidence for these four criteria:

1. PLAUSIBILITY (Causal Mechanism):
 - Does the explanation describe HOW {{ source }} affects {{ target }}?
 - Is a specific mechanism or pathway described (not just assertion)?
 - Is the mechanism scientifically plausible and non-circular?
2. TEMPORALITY (Causal Direction):
 - Does the explanation support the direction {{ source }} -> {{ target }}?
 - Is temporal ordering appropriate (cause precedes effect)?
 - Does it avoid reverse causation ({{ target }} -> {{ source }})?
3. STRENGTH (Relative Importance):
 - Does the explanation convey that {{ source }} is an IMPORTANT or SUBSTANTIAL influence on {{ target }}?
 - Does it suggest this is more than trivial, negligible, or merely correlational?
 - Is the causal claim stated with appropriate confidence?
4. EXPERIMENT/DESIGN (Implications for Exchangeability):
 - Do citations include experimental or quasi-experimental studies?
 - Do they discuss study designs that enhance causal inference (RCTs, natural experiments)?
 - Do they address comparability between exposed and unexposed groups?
 - Do they consider or control for factors affecting exchangeability?

Based on how well the citations address these four Bradford Hill criteria:

REASON: [brief explanation of your judgement]

VERDICT: [SUPPORTED|PARTIALLY_SUPPORTED|NOT_SUPPORTED]

SCORE: [1.0 for SUPPORTED, 0.5 for PARTIALLY_SUPPORTED, 0.0 for NOT_SUPPORTED]

Listing E.7: Citation Judge: Mechanistic Prompt (Shimonovich et al., 2020)

E.4.4 Mechanistic-Lit (Literature-Grounded)

The Mechanistic-Lit variant extends the Mechanistic prompt with explicit requirements for literature grounding. It uses the same Bradford Hill criteria structure but additionally requires that verdicts be based strictly on evidence present in the cited sources, with explicit quotation of supporting passages where applicable. This variant was designed to test whether stricter literature grounding requirements improve citation validation accuracy.

SYSTEM PROMPT:

You are a precise and fair judge evaluating cited evidence using the Bradford Hill framework.

You will assess whether citations provide evidence for four key causality criteria identified as having strong support across modern causal inference approaches:

1. STRENGTH OF ASSOCIATION - including confounding analysis
2. TEMPORALITY - temporal precedence and ruling out reverse causation
3. PLAUSIBILITY - causal mechanisms and mediation pathways
4. EXPERIMENT/DESIGN - study design implications for exchangeability

Base your judgement only on what the citations actually say, not on

```

general knowledge.

USER PROMPT:
Evaluate the cited evidence for this causal relationship using the
Bradford Hill framework:

CLAIMED CAUSAL PATHWAY:
{{ source }} -> [{{ relationship }}] -> {{ target }}

PROPOSED MECHANISM:
{{ motivation }}

CITED SOURCES:
{{ citations }}

Let's think step by step. Assess whether the citations provide evidence
for these four criteria:

1. PLAUSIBILITY (Causal Mechanism):
- Does the explanation describe HOW {{ source }} affects {{ target }}?
- Is a specific mechanism or pathway described (not just assertion)?
- Is the mechanism scientifically plausible and non-circular?

2. TEMPORALITY (Causal Direction):
- Does the explanation support the direction {{ source }} -> {{ target }}?
- Is temporal ordering appropriate (cause precedes effect)?
- Does it avoid reverse causation ({{ target }} -> {{ source }})?

3. STRENGTH (Relative Importance):
- Does the explanation convey that {{ source }} is an IMPORTANT or
  SUBSTANTIAL influence on {{ target }}?
- Does it suggest this is more than trivial, negligible, or merely
  correlational?
- Is the causal claim stated with appropriate confidence?

4. EXPERIMENT/DESIGN (implications for exchangeability):
- Do citations include experimental or quasi-experimental studies?
- Do they discuss study designs that enhance causal inference (RCTs, natural experiments)
  ?
- Do they address comparability between exposed and unexposed groups?
- Do they consider or control for factors affecting exchangeability?

Based on how well the citations address these four Bradford Hill criteria:
REASON: [brief explanation of your judgement]
VERDICT: [SUPPORTED|PARTIALLY_SUPPORTED|NOT_SUPPORTED]
SCORE: [1.0 for SUPPORTED, 0.5 for PARTIALLY_SUPPORTED, 0.0 for NOT_SUPPORTED]

```

Listing E.8: Citation Judge: Mechanistic-Lit Prompt

E.5 Corrector Prompts

The corrector receives edges flagged by the judge and determines appropriate remediation actions. Two tools are available: `revise_motivation` (fix explanation while preserving edge type) and `change_edge_type` (change polarity or set to NONE with new explanation).

E.5.1 Baseline

```

You are an expert causal reasoning corrector. You have 2 tools:

1. revise_motivation: Fix incorrect motivation text (keeps same edge type)
2. change_edge_type: Change edge type (POSITIVE/NEGATIVE/NONE) with new
   motivation

IMPORTANT: NONE represents "no causal relationship" - it's a valid type,
not deletion.

=====
TOOL SELECTION EXAMPLES
=====

EXAMPLE 1: Bad motivation, correct type -> revise_motivation
Edge: "Exercise" -> "Mental Health" (POSITIVE)
Motivation: "Exercise makes people happy"
Judge: "Overly simplistic; doesn't explain mechanism"
ACTION: revise_motivation(new_motivation="Exercise increases endorphin
production and reduces cortisol, improving mental health outcomes")

EXAMPLE 2: Wrong polarity -> change_edge_type
Edge: "Vaccination Rate" -> "Disease Spread" (POSITIVE)
Motivation: "More vaccinations lead to more disease spread"
Judge: "Wrong polarity - vaccinations reduce disease spread"
ACTION: change_edge_type(old_type='POSITIVE', new_type='NEGATIVE',
new_motivation="Higher vaccination rates create herd immunity, reducing
disease transmission")

EXAMPLE 3: Should be NONE -> change_edge_type
Edge: "Shoe Size" -> "Reading Ability" (POSITIVE)
Motivation: "Larger shoe size indicates better reading ability"
Judge: "No causal relationship - confounded by age"
ACTION: change_edge_type(old_type='POSITIVE', new_type='NONE',
new_motivation="No direct causal link between shoe size and reading. The
correlation is due to age as a confounding variable")

EXAMPLE 4: Should be causal (currently NONE) -> change_edge_type
Edge: "Sleep Deprivation" -> "Cognitive Performance" (NONE)
Motivation: "No relationship between sleep and cognition"
Judge: "Strong negative causal relationship exists"
ACTION: change_edge_type(old_type='NONE', new_type='NEGATIVE',
new_motivation="Sleep deprivation impairs cognitive performance by reducing
attention, working memory, and executive function")

=====
DECISION RULES
=====

revise_motivation: Edge type is correct, but explanation is flawed
change_edge_type: Edge type itself is wrong (polarity, existence)

Analyse the judge feedback and call the appropriate tool.

```

Listing E.9: Corrector: Baseline Prompt (Examples Only)

E.5.2 Chain-of-Thought (CoT)

```

You are an expert causal reasoning corrector. You have 2 tools:

```

1. revise_motivation: Fix incorrect motivation text (keeps same edge type)
2. change_edge_type: Change edge type (POSITIVE/NEGATIVE/NONE) with new motivation

IMPORTANT: NONE represents "no causal relationship" - it's a valid type, not deletion.

=====

TOOL SELECTION EXAMPLES

=====

EXAMPLE 1: Bad motivation, correct type -> revise_motivation

Edge: "Exercise" -> "Mental Health" (POSITIVE)

Motivation: "Exercise makes people happy"

Judge: "Overly simplistic; doesn't explain mechanism"

ACTION: revise_motivation(new_motivation="Exercise increases endorphin production and reduces cortisol, improving mental health outcomes")

EXAMPLE 2: Wrong polarity -> change_edge_type

Edge: "Vaccination Rate" -> "Disease Spread" (POSITIVE)

Motivation: "More vaccinations lead to more disease spread"

Judge: "Wrong polarity - vaccinations reduce disease spread"

ACTION: change_edge_type(old_type='POSITIVE', new_type='NEGATIVE', new_motivation="Higher vaccination rates create herd immunity, reducing disease transmission")

EXAMPLE 3: Should be NONE -> change_edge_type

Edge: "Shoe Size" -> "Reading Ability" (POSITIVE)

Motivation: "Larger shoe size indicates better reading ability"

Judge: "No causal relationship - confounded by age"

ACTION: change_edge_type(old_type='POSITIVE', new_type='NONE', new_motivation="No direct causal link between shoe size and reading. The correlation is due to age as a confounding variable")

EXAMPLE 4: Should be causal (currently NONE) -> change_edge_type

Edge: "Sleep Deprivation" -> "Cognitive Performance" (NONE)

Motivation: "No relationship between sleep and cognition"

Judge: "Strong negative causal relationship exists"

ACTION: change_edge_type(old_type='NONE', new_type='NEGATIVE', new_motivation="Sleep deprivation impairs cognitive performance by reducing attention, working memory, and executive function")

=====

STEP-BY-STEP REASONING FRAMEWORK

=====

Let's think step by step to determine the best correction action.

STEP 1: Analyse the Judge Feedback

- What specific problem did the judge identify?
- Is it about: (a) polarity/sign, (b) mechanism/reasoning, (c) type, or (d) existence?

STEP 2: Assess the True Causal Relationship

- What is the CORRECT causal relationship between these variables?
- Should it be: POSITIVE, NEGATIVE, or NONE (no causal relationship)?

STEP 3: Identify the Minimal Correction

- If motivation is flawed but type is correct:
 - > Use revise_motivation (preserves type, fixes reasoning)
- If edge type itself is wrong:

```

-> Use change_edge_type (changes type with new motivation)
Examples: POSITIVE <-> NEGATIVE, causal <-> NONE

STEP 4: Verify Your Decision
- Does this action address the specific judge feedback?
- Could revise_motivation work instead of change_edge_type?
- If changing to NONE, can you explain WHY no causal relationship exists?

STEP 5: Execute the Tool Call
Call the appropriate tool with your validated reasoning.

=====

Now apply the examples and 5-step reasoning process to the current
correction task.

```

Listing E.10: Corrector: CoT Prompt (Examples + Reasoning Framework)

E.5.3 Mechanistic (Bradford Hill)

```

You are an expert causal reasoning corrector. You have 2 tools:

1. revise_motivation: Fix incorrect motivation text (keeps same edge type)
2. change_edge_type: Change edge type (POSITIVE/NEGATIVE/NONE) with new
   motivation

IMPORTANT: NONE represents "no causal relationship" - it's a valid type,
not deletion.

=====
TOOL SELECTION EXAMPLES
=====

EXAMPLE 1: Bad motivation, correct type -> revise_motivation
Edge: "Exercise" -> "Mental Health" (POSITIVE)
Motivation: "Exercise makes people happy"
Judge: "Overly simplistic; doesn't explain mechanism"
ACTION: revise_motivation(new_motivation="Exercise increases endorphin
production and reduces cortisol, improving mental health outcomes")

EXAMPLE 2: Wrong polarity -> change_edge_type
Edge: "Vaccination Rate" -> "Disease Spread" (POSITIVE)
Motivation: "More vaccinations lead to more disease spread"
Judge: "Wrong polarity - vaccinations reduce disease spread"
ACTION: change_edge_type(old_type='POSITIVE', new_type='NEGATIVE',
new_motivation="Higher vaccination rates create herd immunity, reducing
disease transmission")

EXAMPLE 3: Should be NONE -> change_edge_type
Edge: "Shoe Size" -> "Reading Ability" (POSITIVE)
Motivation: "Larger shoe size indicates better reading ability"
Judge: "No causal relationship - confounded by age"
ACTION: change_edge_type(old_type='POSITIVE', new_type='NONE',
new_motivation="No direct causal link between shoe size and reading. The
correlation is due to age as a confounding variable")

EXAMPLE 4: Should be causal (currently NONE) -> change_edge_type
Edge: "Sleep Deprivation" -> "Cognitive Performance" (NONE)
Motivation: "No relationship between sleep and cognition"
Judge: "Strong negative causal relationship exists"

```



```
ACTION: change_edge_type(old_type='NONE', new_type='NEGATIVE',
new_motivation="Sleep deprivation impairs cognitive performance by reducing
attention, working memory, and executive function")
```

```
=====
STEP-BY-STEP REASONING WITH BRADFORD HILL CRITERIA
=====
```

Let's think step by step, applying Bradford Hill criteria for causal assessment.

STEP 1: Analyse Judge Feedback + Assess PLAUSIBILITY (Causal Mechanism)

- What specific problem did the judge identify?
- Does a plausible causal MECHANISM exist between source and target?
- Is the current motivation describing the wrong mechanism, or is there NO mechanism?
- Does the explanation describe HOW source affects target (not just that it does)?

STEP 2: Evaluate TEMPORALITY (Causal Direction)

- What is the CORRECT causal relationship: POSITIVE, NEGATIVE, or NONE?
- Does the causal direction make sense (source must precede target)?
- Could the polarity be reversed (POSITIVE <-> NEGATIVE)?
- Does it avoid reverse causation?

STEP 3: Assess STRENGTH (Relative Importance)

- Is there a SUBSTANTIAL causal relationship, or is it trivial/negligible?
- Does the relationship convey important influence (not mere correlation)?
- If motivation is flawed but strong causal relationship exists
-> revise_motivation
- If no substantial causal relationship exists -> consider change to NONE

STEP 4: Evaluate COHERENCE (Internal Consistency) + Verify Decision

- Is the reasoning logically consistent and non-contradictory?
- Does it avoid tautological or circular arguments?
- Does the judge feedback suggest type is wrong, or just explanation needs fixing?
- Does your chosen action address the specific judge feedback?
- If changing to NONE, can you explain WHY no causal relationship exists?

STEP 5: Execute Tool Call

Based on Bradford Hill assessment and step-by-step reasoning:

IF Plausibility + Temporality + Strength are satisfied, but Coherence/explanation fails:

-> Use revise_motivation (mechanism exists, but poorly explained)

IF Temporality shows wrong direction (polarity inverted):

-> Use change_edge_type (flip POSITIVE <-> NEGATIVE with corrected mechanism)

IF Plausibility or Strength fail (no mechanism or trivial relationship):

-> Use change_edge_type (change to NONE with explanation)

IF Currently NONE but Plausibility + Temporality + Strength support causality:

-> Use change_edge_type (establish causal relationship with mechanism)

```
=====
```

Now apply the examples, 5-step reasoning, and Bradford Hill criteria to correct the current edge using the tools.

Listing E.11: Corrector: Mechanistic Prompt (Examples + CoT + Bradford Hill)

Appendix F

Node Generation Prompt Ablation

Seven prompt variants were evaluated in a full factorial design: 7 prompts $\times$ 3 CLDs $\times$ 5 runs = 105 total experiments.

Table F.1: Node Generation Ablation: Hybrid F1 Scores by Prompt Variant

Prompt	Mean F1	Std	95% CI	Cohen’s d (vs. baseline)	N
Node_V6_Andrew	0.677	0.079	[0.638, 0.717]	+0.44	15
Node_V3_CoT_System	0.657	0.123	[0.595, 0.719]	+0.16	15
Node_V0_Minimal (baseline)	0.639	0.094	[0.592, 0.687]	—	15
Node_V5_Nitai_C	0.636	0.088	[0.591, 0.680]	-0.04	15
Node_V1_Role_Only	0.628	0.080	[0.588, 0.669]	-0.13	15
Node_V4_CoT_User	0.619	0.104	[0.567, 0.672]	-0.20	15
Node_V2_Constraints	0.617	0.077	[0.578, 0.656]	-0.26	15

Statistical Analysis:

- One-way ANOVA: $F = 0.81$, $p = 0.567$ — **no significant differences**
- All pairwise comparisons non-significant after Bonferroni correction
- Effect sizes negligible to small (Cohen’s d vs. baseline range: -0.26 to $+0.44$)

Interpretation: Unlike edge generation, node generation is robust to prompt engineering variations. The minimal baseline performed comparably to sophisticated Chain-of-Thought variants, suggesting that node identification is a simpler task for LLMs that does not benefit substantially from advanced prompting strategies.

Appendix G

Embedding Model Benchmark

To justify the choice of embedding model for cosine similarity computation (Section 2.5), we benchmarked `all-mpnet-base-v2` against the MTEB-leading Qwen3-Embedding-8B using 30 real causal narratives from our CLD experiments.

Table G.1: Embedding Model Benchmark ($n = 30$ causal narratives from CLD)

Model	Params	Embed dim	ms/doc	docs/s
all-mpnet-base-v2	110M	768	3.0	336
Qwen3-Embedding-8B (Q4)	8B	4096	67.3	14.9

Compute setup: Hardware details are reported in Appendix C.3 (Table C.9). Qwen3 was run via Ollama with 4-bit quantization (Q4_K_M).

Test data: Causal narratives averaged 451 characters (~ 100 tokens). In production, fetched citation documents are chunked up to 4,000 characters ($\sim 1,000$ tokens) before embedding—longer inputs may increase per-document latency.

MTEB Benchmark Comparison:

Table G.2: MTEB Scores: MPNet vs. Qwen3-Embedding-8B

Task	MPNet (110M) ^a	Qwen3-8B ^b
STS (Spearman ρ)	80.28	88.58
Reranking (MAP)	59.36	51.56
Pair Classification (AP)	83.04	87.52
Classification (Acc)	65.07	90.43
Clustering (V-measure)	43.69	58.57
Retrieval (nDCG@10)	43.81	69.44
Summarization	27.49	34.83
Average	57.78	75.22

^aMuennighoff et al. [98], Table 1 (MTEB v1, 2022). ^bZhang et al. [102], Table 7 (MTEB eng v2, 2025).

Result: MPNet achieves $22\times$ faster inference (3.0 vs. 67.3 ms/doc; Table G.1). Since cosine-similarity scoring is used as a throughput-bound heuristic for max-similarity chunk selection, we prioritise latency; MPNet is a strong general-purpose embedding model and is widely used in practice.

Note: The MTEB scores are taken from different benchmark versions (v1 vs. v2) and evaluation setups and are therefore **not directly comparable**. We include them only to illustrate that both models are competitive across common embedding tasks.

Script: `final_runs/embedding_model_benchmark/embedding_benchmark_real_cld.py`

Appendix H

Algorithmic Descriptions

This section provides formal algorithmic descriptions of the core system components, traced directly from the implementation. For experimental design rationale, see the corresponding Methods sections.

H.1 Algorithm 0: CLD Generation

The CLD generator constructs causal loop diagrams through a two-phase process: node generation followed by edge generation. Both phases capture UQ metrics via logprobs for downstream hallucination detection.

Table H.1: CLD Generation (`generate_variables` + `discover_relationships`)

Input: <code>target_variable</code> , <code>temporal_scale</code> , <code>spatial_scale</code> , <code>num_variables</code> , <code>context</code>	
Output: Neo4j session with nodes (variables) and edges (relationships) + UQ metrics	
<i>// Phase 1: Node Generation (generateNodes prompt)</i>	
1.	<code>session_id</code> $\leftarrow$ <code>uuid.generate()</code>
2.	<code>Neo4j.create_node(target_variable, is_target=True, session_id)</code>
3.	<code>variables</code> $\leftarrow$ [<code>target_variable</code>]
4.	while <code>len(variables)</code> < <code>num_variables</code> :
5.	<code>vars</code> $\leftarrow$ { <code>target</code> , <code>target_units</code> , <code>target_definition</code> , <code>temporal</code> , <code>spatial</code> , <code>context</code> }
6.	(<code>sys_prompt</code> , <code>usr_prompt</code>) $\leftarrow$ <code>tree.build_prompt("generateNodes", vars)</code>
7.	<code>usr_prompt</code> $\leftarrow$ <code>usr_prompt</code> + "Exclude: " + <code>join(variables)</code>
8.	<code>response</code> $\leftarrow$ <code>generator.llm.send_message(prompts, logprobs=True, top_logprobs=5)</code>
9.	(<code>name</code> , <code>definition</code> , <code>units</code> , <code>citations</code>) $\leftarrow$ <code>tree.read_variable_response(response)</code>
10.	<code>log_metrics</code> $\leftarrow$ <code>compute_uq_metrics(response.logprobs)</code>
11.	<code>Neo4j.create_node(name, definition, citations, log_metrics, session_id)</code>
12.	<code>variables.append(name)</code>
<i>// Phase 2: Edge Generation (generateEdges prompt)</i>	
13.	<code>pairs</code> $\leftarrow$ <code>generate_ordered_pairs(variables)</code> <i>// N^2 pairs, excludes self-loops</i>
14.	for each (<code>source</code> , <code>target</code>) $\in$ <code>pairs</code> :
15.	(<code>source_def</code> , <code>target_def</code>) $\leftarrow$ <code>Neo4j.fetch_definitions(source, target)</code>
16.	<code>vars</code> $\leftarrow$ { <code>var1</code> : <code>source</code> , <code>var2</code> : <code>target</code> , <code>def1</code> , <code>def2</code> , <code>temporal</code> , <code>spatial</code> , <code>variables</code> }
17.	(<code>sys_prompt</code> , <code>usr_prompt</code>) $\leftarrow$ <code>tree.build_prompt("generateEdges", vars)</code>
18.	<code>response</code> $\leftarrow$ <code>generator.llm.send_message(prompts, logprobs=True, top_logprobs=5)</code>
19.	(<code>rel_type</code> , <code>motivation</code> , <code>citations</code>) $\leftarrow$ <code>tree.read_relationships(response)</code>
20.	<code>log_metrics</code> $\leftarrow$ <code>compute_ci_metrics(response.logprobs)</code>
21.	<code>Neo4j.create_relationship(source, target, rel_type, motivation, citations, log_metrics)</code>
22.	return <code>session_id</code> , <code>variables</code> , <code>relationships</code>

Implementation: `modules.py:698` (`generate_variables`), `:1147` (`discover_relationships`), `:908` (`process_variable_pair`)

Acknowledgement: The CLD generation algorithm was conceived by Rick Quax with feedback from Cillian Hourican and implemented by Andrew Crossley. The author (Nitai Nijholt) contributed the mediation check, multi-provider LLM infrastructure with rate limiting and fallbacks, and prompt testing framework.

H.2 Algorithm 1: Judge-Correctness

The correctness-based judge evaluates causal reasoning quality without external evidence.

Table H.2: Judge-Correctness (`judge_all_edges_serial`, `approach="correctness"`)

Input:	<code>session_id</code> , <code>judge_models[]</code> , <code>prompt_config</code>
Output:	Per-edge verdicts with scores
1.	<code>edges</code> $\leftarrow$ <code>Neo4j.query("MATCH (s)-[r]->(t) WHERE s.session_id = \$sid")</code>
2.	<code>judges[]</code> $\leftarrow$ <code>create_llm_clients(judge_models)</code>
3.	for each <code>edge</code> $\in$ <code>edges</code> :
4.	<code>text</code> $\leftarrow$ <code>edge.spurious_motivation</code> $\vee$ <code>edge.motivation</code>
5.	if <code>text</code> = <code>∅</code> : <code>verdict</code> $\leftarrow$ "NO_MOTIVATION"; continue
6.	<code>variables</code> $\leftarrow$ { <code>source</code> , <code>target</code> , <code>relationship</code> , <code>motivation</code> : <code>text</code> }
7.	(<code>sys_prompt</code> , <code>usr_prompt</code>) $\leftarrow$ <code>tree.build_prompt("judgeCorrectness", vars)</code>
8.	<code>scores[]</code> , <code>verdicts[]</code> $\leftarrow$ [], []
9.	for each <code>judge</code> $\in$ <code>judges</code> :
10.	<code>response</code> $\leftarrow$ <code>judge.send_message(sys_prompt, usr_prompt, logprobs=True)</code>
11.	(<code>verdict</code> , <code>score</code> , <code>reason</code>) $\leftarrow$ <code>parse_regex(response, "VERDICT: SCORE:")</code>
12.	<code>verdicts.append(verdict)</code> ; <code>scores.append(score)</code>
13.	<code>aggregate_score</code> $\leftarrow$ <code>mean(scores)</code>
14.	<code>aggregate_verdict</code> $\leftarrow$ <code>majority_vote(verdicts)</code>
15.	<code>update_edge(edge, aggregate_verdict, aggregate_score)</code>
16.	return <code>judged_edges</code>

Implementation: `modules.py:4421` $\rightarrow$ `:4740` (`correctness` branch)

H.3 Algorithm 2: Judge-Citation

The citation-based judge validates causal claims against fetched scientific literature.

Table H.3: Judge-Citation (`judge_all_edges_with_citations_serial`)

Input:	<code>session_id</code> , <code>judge_models[]</code> , <code>citation_fetcher</code>
Output:	Per-edge aggregate verdicts with per-citation scores

```

1. edges ← Neo4j.query("MATCH (s)-[r]->(t) WHERE s.session_id = $sid")
2. judges[] ← create_llm_clients(judge_models)
3. for each edge ∈ edges:
4.   citations ← edge.citations
5.   if citations = ∅: verdict ← "NO-CITATION"; continue
6.   citation_contents ← ContentScraper.process_citations(citations)
7.   per_citation_results ← []
8.   for each (url, content) ∈ citation_contents:
9.     prompt ← build_citation_prompt(edge.motivation, content)
10.    judge_verdicts ← []
11.    for each judge ∈ judges:
12.      response ← judge.send_message(prompt)
13.      verdict ← parse(response) // SUPPORTED/PARTIAL/CONTRADICTED
14.      judge_verdicts.append(verdict)
15.    citation_verdict ← majority_vote(judge_verdicts)
16.    citation_score ← verdict_to_score(citation_verdict) // 1.0, 0.5, 0.0
17.    per_citation_results.append({url, citation_verdict, citation_score})
18.    aggregate_score ← mean(per_citation_results.scores)
19.    aggregate_verdict ← aggregator_llm(per_citation_results)
20.    update_edge(edge, aggregate_verdict, aggregate_score)
21. return judged_edges

```

Implementation: `modules.py:5688` (`judge_all_edges_with_citations_serial`) → `modules.py:4988`

H.4 Algorithm 3: Corruption

The corruptor introduces controlled errors using stratified sampling for balanced corruption types.

Parameters used in this thesis (GT Synth). Unless otherwise stated, corruption experiments use `corruption_rate=0.3` with a balanced 1/3 split across corruption types (motivation corruption, spurious edges, polarity flips). The corruptor LLM configuration is `model=gpt-4.1`, `temperature=0.7`, `top_p=1.0`; corruption is seeded for reproducibility (see `corruption_metadata.json` in the corresponding `final_runs/` output directory).

Table H.4: Corruption (`_corrupt_relationships`)

Input:	session_id, corruption_rate (default 0.3), seed, corruptor_llm
Output:	Number of corrupted edges, corruption metadata

```

1.  random.seed(seed)                                // Reproducibility
2.  edges ← Neo4j.query("MATCH (s)-[r]->(t)")
3.  none_edges ← {e ∈ edges : e.type = "NONE"}
4.  causal_edges ← {e ∈ edges : e.type ∈ {"POSITIVE", "NEGATIVE"}}
5.  n_corrupt ← ⌊len(edges) × corruption_rate⌋
6.  target_per_type ← n_corrupt ÷ 3                    // Balanced: 1/3 each
7.  spurious_sample ← shuffle(none_edges)[:target_per_type]
8.  causal_sample ← shuffle(causal_edges)[:target_per_type × 2]
9.  for each edge ∈ spurious_sample ∪ causal_sample:
10.   if edge.type = "NONE": corruption_type ← "spurious_edge"
11.   else: corruption_type ← argmin(assigned["polarity_flip"], assigned["motivation"])
12.   assigned[corruption_type] += 1
13.   if corruption_type = "motivation":
14.     prompt ← tree.build_prompt("corruptor", edge)
15.     edge.spurious_motivation ← corruptor_llm.send(prompt)
16.   elif corruption_type = "spurious_edge":
17.     prompt ← tree.build_prompt("generateSpuriousEdge", edge)
18.     {type, expl} ← JSON.parse(corruptor_llm.send(prompt))
19.     Neo4j: DELETE NONE, CREATE edge.type ← type, motivation ← expl
20.   elif corruption_type = "polarity_flip":
21.     prompt ← tree.build_prompt("generateFlippedPolarity", edge)
22.     {expl} ← JSON.parse(corruptor_llm.send(prompt))
23.     Neo4j: DELETE old, CREATE edge.type ← flip(type), motivation ← expl
24.     edge.is_corrupted ← True
25.   save_metadata("corrupted_relationships_{session_id}.json")
26.  return corrupted_count

```

Implementation: `modules.py:1748` (`_corrupt_relationships`), `:1544` (`_corrupt_relationship`), `:1582` (`_convert_none_to_spurious`), `:1645` (`_flip_edge_polarity`)

H.5 Algorithm 4: Corrector

The corrector applies LLM-guided fixes to edges flagged by the judge.

Table H.5: Corrector (`correct_edges_serial`)

Input:	session_id, corrector_models[], action_order, rejudge_after
Output:	Correction outcomes per edge

```

1.  candidates ← Neo4j.query("WHERE r.verdict IN ['INCORRECT', 'PARTIAL', ...]")
2.  agent ← create_corrector_agent(corrector_models[0], prompt_variant)
3.  available_vars ← get_all_variables(session_id)
4.  outcomes ← []
5.  for each edge ∈ candidates:
6.    context ← {edge, available_vars, graph_structure}
7.    for each action ∈ action_order: // revise_motivation, change_edge_type
8.      result ← agent.call_tool(action, edge, context)
9.      if result.success:
10.        outcomes.append({edge, action, corrected: True}); break
11.  if rejudge_after:
12.    judge_all_edges_with_citations_serial(approach=rejudge_approach)
13.  return outcomes

```

Implementation: `modules.py:7050 (correct_edges_serial)`

H.6 Algorithm 5: UQ Metrics Computation

Uncertainty Quantification (UQ) metrics computed from LLM logprobs during generation.

Table H.6: UQ Metrics (`logit_metrics.py`)

Input: `logprobs = {tokens[], token_logprobs[], top_logprobs[]}`
Output: `perplexity, min_prob, max_window_entropy, cosine_sim`

```

// Perplexity (compute_avg_nll + exp)
1.  nll_capped ← [min(-logprobi, 20.0) for logprobi in token_logprobs]
2.  avg_nll ← mean(nll_capped)
3.  avg_nll_capped ← min(avg_nll, 10.0)
4.  perplexity ← exp(avg_nll_capped)
// Minimum Token Probability (compute_min_prob)
5.  probs ← [exp(logprobi) for logprobi in token_logprobs]
6.  min_prob ← min(probs)
// Max Window Entropy (compute_max_window_entropy)
7.  entropies ← []
8.  for each top_k_dict ∈ top_logprobs:
9.    p_vals ← softmax(top_k_dict.values())
10.   H ← - ∑i pi log(pi)
11.   entropies.append(H)
12.  windowed ← convolve(entropies, window_size=5)
13.  max_window_entropy ← max(windowed)
// Cosine Similarity (get_embedding_local)
14.  enarr ← SentenceTransformer(motivation, model="all-mpnet-base-v2")
15.  ecit ← SentenceTransformer(citation_text)
16.  cosine_sim ← maxc  $\frac{\mathbf{e}_{narr} \cdot \mathbf{e}_c}{\|\mathbf{e}_{narr}\| \|\mathbf{e}_c\|}$ 
17.  return {perplexity, min_prob, max_window_entropy, cosine_sim}

```

Implementation: `logit_metrics.py` – :177 (avg_nll), :251 (min_prob), :259 (max_entropy), :98 (embedding)

H.7 Algorithm 6: Hallucination Classification

Ensemble classifiers trained on UQ metrics to predict hallucinations.

Table H.7: Hallucination Classification (`train_classifiers`)

Input: $X = [\text{perplexity}, \text{min_prob}, \text{max_entropy}, \text{cosine_sim}]$, $y = \text{hallucination_labels}$
Output: Trained models, metrics (AUC, Precision, Recall, F1)

```

1.  $X\_scaled \leftarrow \text{StandardScaler.fit\_transform}(X)$ 
2.  $(X\_train, X\_test, y\_train, y\_test) \leftarrow \text{train\_test\_split}(X\_scaled, y, \text{test}=0.3)$ 
3.  $\text{classifiers} \leftarrow \{\text{LogReg}, \text{RandomForest}(100), \text{GradBoost}(100), \text{MLP}(32,16)\}$ 
4. for each  $(\text{name}, \text{clf}) \in \text{classifiers}$ :
5.    $\text{cv} \leftarrow \text{StratifiedKFold}(n\_splits=5)$ 
6.    $\text{cv\_auc} \leftarrow \text{cross\_val\_score}(\text{clf}, X\_train, y\_train, \text{cv}, \text{scoring}=\text{"roc\_auc"})$ 
7.    $\text{clf.fit}(X\_train, y\_train)$ 
8.    $y\_pred\_proba \leftarrow \text{clf.predict\_proba}(X\_test)[:,1]$ 
9.    $\text{metrics}[\text{name}].\text{auc} \leftarrow \text{roc\_auc\_score}(y\_test, y\_pred\_proba)$ 
10.   $\text{metrics}[\text{name}].\{\text{precision}, \text{recall}, \text{f1}\} \leftarrow \text{precision\_recall\_fscore}(y\_test, y\_pred)$ 
11. return  $\text{classifiers}, \text{metrics}$ 

```

Implementation: `rq2_ensemble_classifier.py:32` (`train_classifiers`)

H.8 Algorithm 7: Citation Search (Brave)

This procedure retrieves candidate citation URLs for a CLD edge using the Brave Search API, optionally filtered by a trusted-domain whitelist ([103]). It is invoked during citation filling when no usable citations are available from the LLM, and can also be used to obtain candidate sources for citation-based judging.

Trusted Academic Domains (23): Results from the following domains are prioritised ([103]):

- `pubmed.ncbi.nlm.nih.gov`
- `arxiv.org`
- `scholar.google.com`
- `nature.com`
- `science.org`
- `cell.com`
- `nejm.org`
- `bmj.com`
- `plos.org`
- `frontiersin.org`
- `springer.com`
- `wiley.com`
- `sciencedirect.com`
- `tandfonline.com`
- `sage.com`
- `jstor.org`
- `cochranelibrary.com`
- `nih.gov`
- `who.int`
- `cdc.gov`
- `academic.oup.com`
- `cambridge.org`
- `elsevier.com`

Table H.8: Citation Search (`_search_with_brave + SearchEngine.search`)

Input: <code>src, rel, tgt, claim_txt, api_key, trusted_domains[]</code> (optional)
Output: <code>citations[]</code> (URLs, up to 5)

```

// Phase 0: Initialize Brave client (once per run)
1.  search_client ← SearchEngine(api_key, trusted_domains)
2.  session ← requests.Session() with retries(total=3, backoff=1, status={429,500,502,503,504})

// Phase 1: Build query from edge
3.  q ← "src rel tgt claim_txt"
4.  if words(q) > 50: q ← first_50_words(q) + "..."
5.  if len(q) > 400: q ← q[0:397] + "..."

// Phase 2: Brave Search API call
6.  resp ← GET(https://api.search.brave.com/res/v1/web/search,
              headers={Accept: json, X-Subscription-Token: api_key},
              params={q: q, count: 5, search_lang: en, extra_snippets: True})
7.  results ← resp.json().web.results

// Phase 3: Domain filter + URL extraction
8.  citations ← []
9.  for each result ∈ results:
10.   url ← result.url
11.   if trusted_domains empty or any(domain ∈ url.lower()): citations.append(url)
12.  return citations

```

Implementation: `modules.py:2647 (fill_in_missing_citations), :2900 (_build_brave_search_client), :2967 (_search_with_brave), search_engine_copy.py:47 (SearchEngine.search), citation_whitelist_copy.py:250 (CitationWhitelist.get_`

H.9 Algorithm 8: Citation Fetcher

The citation fetcher retrieves scientific literature content from URLs using a two-tier architecture: Jina AI Reader as primary with ContentScraper fallback for PDF extraction.

Table H.9: Citation Fetcher (`fetch_citations_with_jina`)

Input: citations[] (URLs), timeout, max_retries, api_key (optional)	
Output: Dict[URL → text_content]	

```

// Phase 1: Jina AI Reader (Primary)
1.  max_workers ← 5 if api_key else 2                                // Rate limit tiers
2.  citation_text_map ← {}
3.  failed_urls ← []
4.  parallel for each url ∈ citations (ThreadPoolExecutor):
5.    jina_url ← "https://r.jina.ai/" + url
6.    delay ← 0.3s if api_key else 1.0s                                // Respect rate limits
7.    sleep(delay)
8.    headers ← {"Authorization": "Bearer " + api_key} if api_key
9.    for attempt ∈ [0..max_retries]:
10.     response ← GET(jina_url, headers, timeout)
11.     if response.ok ∧ len(content) > 200 ∧ "[ERROR]" ∉ content[:500]:
12.       citation_text_map[url] ← content; break
13.     else: sleep(2attempt)                                           // Exponential backoff
14.   if url ∉ citation_text_map: failed_urls.append(url)

```

```

// Phase 2: ContentScraper Fallback (PDF-capable)
15. if len(failed_urls) > 0:
16.   scraper ← ContentScraper(timeout=15, whitelist=True) // PyMuPDF for PDFs
17.   results ← scraper.process_citations(failed_urls, fetch_content=True)
18.   for each (url, content) ∈ results:
19.     if len(content) > 200 ∧ "[ERROR]" ∉ content[:100]:
20.       citation_text_map[url] ← content
21. return citation_text_map

```

* **Thesis configuration note:** In the experiments reported in this thesis, citation fetching was run in **ContentScraper-only** mode (Jina AI Reader disabled).

Implementation: `modules.py:265` (`fetch_citations_with_jina`), `:302` (`fetch_single_citation`), `:378-410` (ContentScraper fallback)

H.10 Deep Research Multi-Agent Components

Table [H.10](#) details the complete multi-agent pipeline components used in the Deep Research system.

Table H.10: Deep Research Multi-Agent Pipeline

Component	Model	Input	Output	Purpose
<i>Orchestrator</i>				
DeepResearchManager	Python	Causal claim	FinalVerdict	Coordinate agent execution: iteration loop (max 3), early stopping on direct evidence, time limit (2 min), parallelization via <code>asyncio.gather()</code>
<i>LLM Agents</i>				
hypothesis_evaluator	GPT-5	Causal claim	SearchPlan	Generate 3–4 targeted search queries, keywords, and potential confounders
search_agent	GPT-5-mini	Search query	List[SearchResult]	Execute iterative web searches (4–6 calls per query thread, max 5)
evidence_scorer	GPT-5-mini	Evidence content	EvidenceScore	Score relevance (1–10) and methodological quality (1–10)
article_selector	GPT-5	Snippets + scores	ArticleSelection	Select 2–3 articles most likely to contain causal evidence for full-text fetch
judgement_agent	GPT-5	Evidence + scores	EvidenceJudgement	Judge whether accumulated evidence supports the claim
causal_evidence_detector	GPT-5	All search results	CausalEvidenceEvaluation	Detect <i>direct</i> causal evidence (RCTs, experiments, meta-analyses only)
verdict_agent	GPT-5	All data	FinalVerdict	Synthesize final verdict with evidence summary and limitations

H.11 Algorithm 9: Deep Research Multi-Agent Pipeline

The Deep Research system validates causal claims through iterative literature search using a 7-agent pipeline with parallel execution.

Table H.11: Deep Research Pipeline (`pydantic_ai_deepresearch_orchestration.py`)

Input: `causal_claim = "{Source} causes {Target}. {Motivation}"`
Output: `FinalVerdict (verdict, confidence, direct_causal_found, evidence_urls)`

```
// Phase 1: Search Plan Generation (hypothesis_evaluator, GPT-5)
1. search_plan ← hypothesis_evaluator.run(causal_claim)
2. queries[] ← [motivation] + search_plan.queries           // 3-4 queries
3. keywords ← search_plan.keywords
4. confounders ← search_plan.potential_confounders
```

```
// Phase 2: Parallel Search Execution (search_agent, GPT-5-mini)
5. all_results ← []
6. parallel for each query ∈ queries (asyncio.gather):
7.   history ← fresh_conversation()           // Prevent token accumulation
8.   search_count ← 0
9.   while search_count < MAX_SEARCHES (5):
10.    results ← search_agent.run(query, tool=web_search)
11.    all_results.extend(results)
12.    search_count += 1
```

```
// Phase 3: Parallel Evidence Scoring (evidence_scorer, GPT-5-mini)
13. parallel for each result ∈ all_results:
14.   score ← evidence_scorer.run(result.content, causal_claim)
15.   result.relevance ← score.relevance           // 1-10
16.   result.quality ← score.quality           // 1-10: RCT > cohort > case-control
```

```
// Phase 4: Smart Article Selection (article_selector, GPT-5)
17. ranked ← sort(all_results, key=relevance × quality)
18. selected ← article_selector.run(ranked[:10])   // Select 2-3 articles
19. for each article ∈ selected:
20.   article.full_text ← fetch_full_text(article.url, max=50K chars)
```

```
// Phase 5: Direct Causal Evidence Detection (causal_evidence_detector, GPT-5)
21. causal_eval ← causal_evidence_detector.run(all_results)
22. direct_found ← causal_eval.direct_evidence_found // RCT/experiment/meta-analysis only
23. causal_confidence ← causal_eval.confidence
24. cited_passage ← causal_eval.cited_passage       // Verbatim quote
25. study_type ← causal_eval.study_type
```

```
// Phase 6: Judgement with Early Stopping
26. high_quality ← {r ∈ all_results : r.relevance ≥ 8 ∧ r.quality ≥ 7}
27. if direct_found ∧ |high_quality| ≥ 3:
28.   early_stop           // Strong evidence found
29. judgement ← judgement_agent.run(all_results, scores)
30. if judgement.verdict = "supported" ∧ judgement.confidence ≥ 8:
31.   early_stop           // High confidence support
```

```
// Phase 7: Final Verdict Synthesis (verdict_agent, GPT-5)
32. verdict ← verdict_agent.run(causal_claim, all_results, scores, judgement, causal_eval)
33. return FinalVerdict{
34.   verdict: {supported, partially_supported, unsupported, inconclusive},
35.   confidence: 1-10,
36.   direct_causal_evidence_found: direct_found,
37.   strongest_evidence: (url, cited_passage, study_type),
38.   evidence_urls: [all supporting URLs]
39. }
```

Key Implementation Details:

- **Search API:** Brave Search with 23 trusted academic domains (PubMed, arXiv, Nature, etc.; [103])
- **Query enhancement:** All queries appended with “scientific study research evidence”
- **Retry strategy:** 3 retries with exponential backoff for transient failures

- **Timeouts:** 10s connect, 40s read; 900s (15 min) total time limit per claim
- **Full-text fetching:** BeautifulSoup HTML parsing, max 50K characters

Implementation: `pydantic_ai_deepresearch_orchestration.py` – agents defined at lines 166–248, search execution at :953, evidence scoring at :1006, early stopping at :784–795

H.12 Algorithm 10: Node Concept Similarity (Similarity-Weighted Node F1)

This metric quantifies conceptual overlap between a generated CLD’s node set and a validation CLD’s node set. It is implemented as (i) a node mapping phase (LLM- or cosine-driven, depending on mode) and (ii) a similarity-weighted score that assigns **1.0** to nodes matched in Stage 1 and assigns cosine similarity scores to a **greedy bipartite matching** over remaining unmatched nodes.

Table H.12: Node Concept Similarity (`compare_session_graph_to_validation`, similarity-weighted node metrics)

Input: session_nodes S , validation_nodes V , node_match_mode, cosine_percentile p (default 85), embedding_model (default `text-embedding`)

Output: binary node mapping + binary node F1, cosine-only node F1, similarity-weighted (hybrid) node F1

// Step 0: Build node texts for embeddings

1. **for each** node $n \in (S \cup V)$:
2. $\text{text}(n) \leftarrow \text{join}(\{\text{name}(n), \text{description/definition}(n) \text{ if present}, \text{units}(n) \text{ if present}\}, " | ")$

// Step 1: Compute cosine similarities

3. $E_S \leftarrow \text{embed}(\text{text}(S)); E_V \leftarrow \text{embed}(\text{text}(V))$
4. $C \leftarrow \text{cosine}(E_S, E_V)$ // via normalized dot product
5. $\tau \leftarrow \text{percentile}(C.\text{flatten}(), p)$
6. $\text{candidates} \leftarrow \{(s, v, C[s, v]) : C[s, v] \geq \tau\}$, sorted desc.

// Step 2: Stage 1 mapping (mode-dependent)

7. **if** node_match_mode = `cosine`:
8. node_mapping $\leftarrow \text{greedy_max_weight_matching}(\text{candidates})$ // 1:1 matching
9. **elif** node_match_mode $\in \{\text{hybrid}, \text{llm}\}$:
10. to_check $\leftarrow$ (all pairs $S \times V$ if `llm` else candidates)
11. positives $\leftarrow \{(s, v, C[s, v]) : \text{LLM_equivalent}(s, v) = \text{True} \text{ for } (s, v) \in \text{to_check}\}$
12. node_mapping $\leftarrow \text{greedy_max_weight_matching}(\text{positives})$
13. **elif** node_match_mode = `llm_batch`:
14. // Single LLM call proposes best match (or NO_MATCH) with confidence
15. **for each** $s \in S$: ($v_{\text{llm}}, \text{conf}$) $\leftarrow \text{LLM_best_match}(s, V)$
16. **if** $v_{\text{llm}} \neq \text{NO_MATCH}$ and $\text{conf} \geq 0.7$: node_mapping[s] $\leftarrow v_{\text{llm}}$
17. **else**: $v^* \leftarrow \arg \max_{v \in V} C[s, v]$; **if** $C[s, v^*] \geq 0.7$ then node_mapping[s] $\leftarrow v^*$

// Step 3: Binary node metrics (mapping cardinality)

18. $P_{\text{bin}} \leftarrow |\text{node_mapping}|/|S|$; $R_{\text{bin}} \leftarrow |\text{node_mapping}|/|V|$
19. $F1_{\text{bin}} \leftarrow 2PR/(P + R)$

// Step 4: Cosine-only node metrics (max similarity per node)

20. $P_{\text{cos}} \leftarrow \text{mean}_{s \in S} [\max_{v \in V} C[s, v]]$
21. $R_{\text{cos}} \leftarrow \text{mean}_{v \in V} [\max_{s \in S} C[s, v]]$
22. $F1_{\text{cos}} \leftarrow 2PR/(P + R)$

// Step 5: Similarity-weighted (hybrid) node metrics via two-stage bipartite matching

23. $\text{matched_S} \leftarrow \text{keys}(\text{node_mapping}); \text{matched_V} \leftarrow \text{values}(\text{node_mapping})$
24. $S_u \leftarrow S \setminus \text{matched_S}; V_u \leftarrow V \setminus \text{matched_V}$
25. $\text{cosine_mapping} \leftarrow \text{greedy_max_weight_matching}(\{(s, v, C[s, v]) : s \in S_u, v \in V_u\})$
26. $\text{score_gen}(s) \leftarrow \begin{cases} 1.0 & s \in \text{matched_S} \\ C[s, \text{cosine_mapping}(s)] & s \in \text{cosine_mapping} \\ 0.0 & \text{otherwise} \end{cases}$
27. $\text{score_val}(v) \leftarrow \begin{cases} 1.0 & v \in \text{matched_V} \\ C[\text{cosine_rev}(v), v] & v \in \text{range}(\text{cosine_mapping}) \\ 0.0 & \text{otherwise} \end{cases}$
28. $P_{\text{hyb}} \leftarrow \text{mean}_{s \in S} [\text{score_gen}(s)]; R_{\text{hyb}} \leftarrow \text{mean}_{v \in V} [\text{score_val}(v)]$
29. $F1_{\text{hyb}} \leftarrow 2PR/(P + R)$

Adjusted (seed-excluding) variant: If the target variable was *seeded* from the validation CLD (e.g., the inferred target), the implementation additionally reports “adjusted” node metrics by excluding that variable from both S and V prior to metric computation.

Implementation: `modules.py:8242 (compare_session_graph_to_validation)`, `:8098 (_compute_node_cosine_cand)`, `:8151 (_maximum_weight_bipartite_matching)`

Appendix I

ROC Curves

This section presents ROC curves for all RQ1a experiments. **Important methodological note:** The aggregate scores used for classification are discrete, taking only three values (0, 0.5, and 1.0). Consequently, only three meaningful operating points exist on each ROC curve, and the apparent smoothness results from linear interpolation between these points. The fixed classification threshold of 0.5 is used throughout the main analysis, making these curves primarily illustrative of the AUC metric rather than a continuous threshold trade-off.

I.1 Corruption Detection Correctness ROC Curves

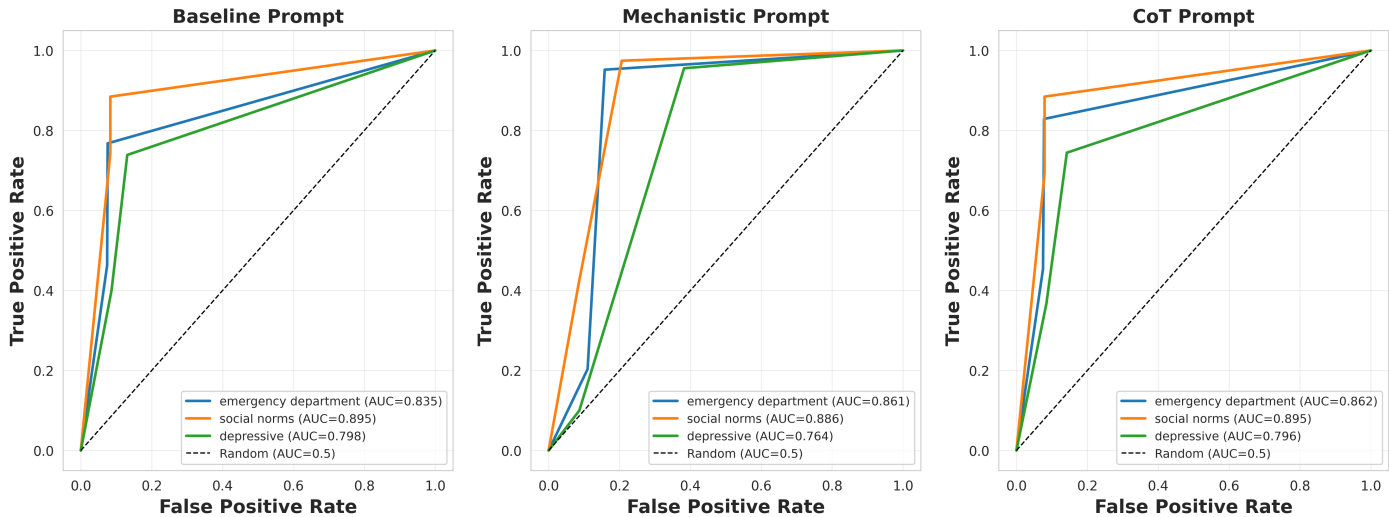


Figure I.1: ROC curves for correctness-based corruption detection by prompt. Each subplot shows performance for one prompt variant across all CLDs. **Note:** Scores are discrete (0, 0.5, 1), so curves represent interpolation between only 3 operating points.

I.2 Corruption Detection Citation ROC Curves

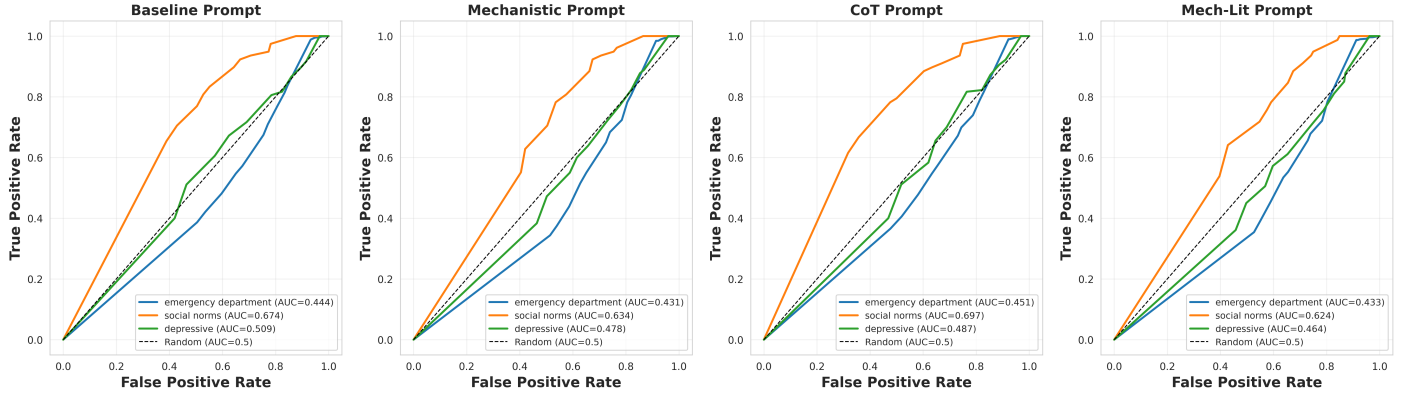


Figure I.2: ROC curves for citation-based corruption detection by prompt. Each subplot shows performance for one of four prompt variants (Baseline, Mechanistic, CoT, Mechanistic-Lit) across all three CLDs. **Note:** Scores are discrete (0, 0.5, 1), so curves represent interpolation between only 3 operating points.

I.3 Ground Truth Correctness ROC Curves

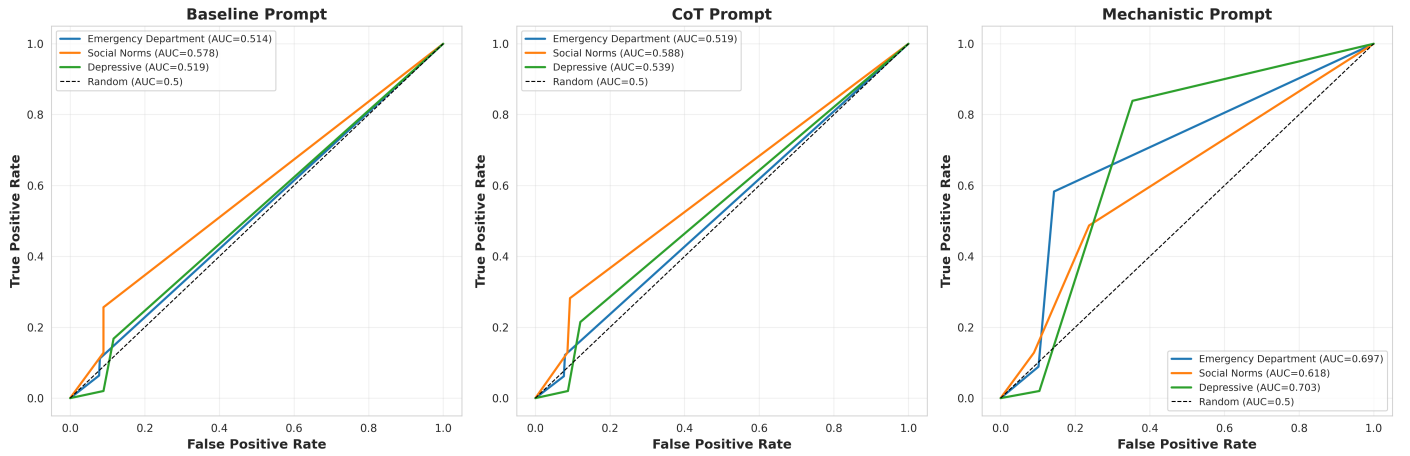


Figure I.3: ROC curves for ground truth correctness validation. Shows discrimination ability per prompt variant across CLDs, with AUC values for each CLD.

I.4 Ground Truth Citation ROC Curves

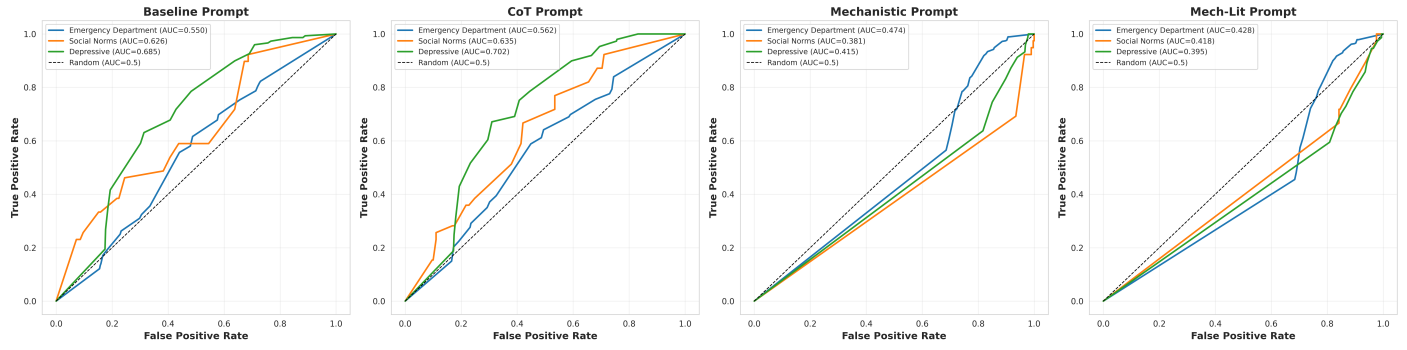


Figure I.4: ROC curves for ground truth citation validation. Shows discrimination ability per prompt variant across CLDs, with AUC values for each CLD.

Appendix J

Judge Score Heatmaps

This section presents detailed score heatmaps for all RQ1a experiments. The heatmaps show mean judge scores with 95% confidence intervals for each classification type (TP, TN, FP, FN) across all CLD $\times$ prompt combinations. For a well-functioning judge: high scores for TP and TN indicate correct identification of valid edges, while low scores for FP and FN indicate correct flagging of erroneous edges as hallucinations.

J.1 Ground Truth Correctness Heatmap

Figure [J.1](#) shows the judge score heatmap for the ground truth correctness validation. The correctness judge evaluates whether edges are factually accurate based on domain knowledge. Note the consistent high scores for TN (correctly identified non-hallucinations) across all conditions, while FP and FN classifications show lower scores indicating the judge’s ability to identify problematic edges.

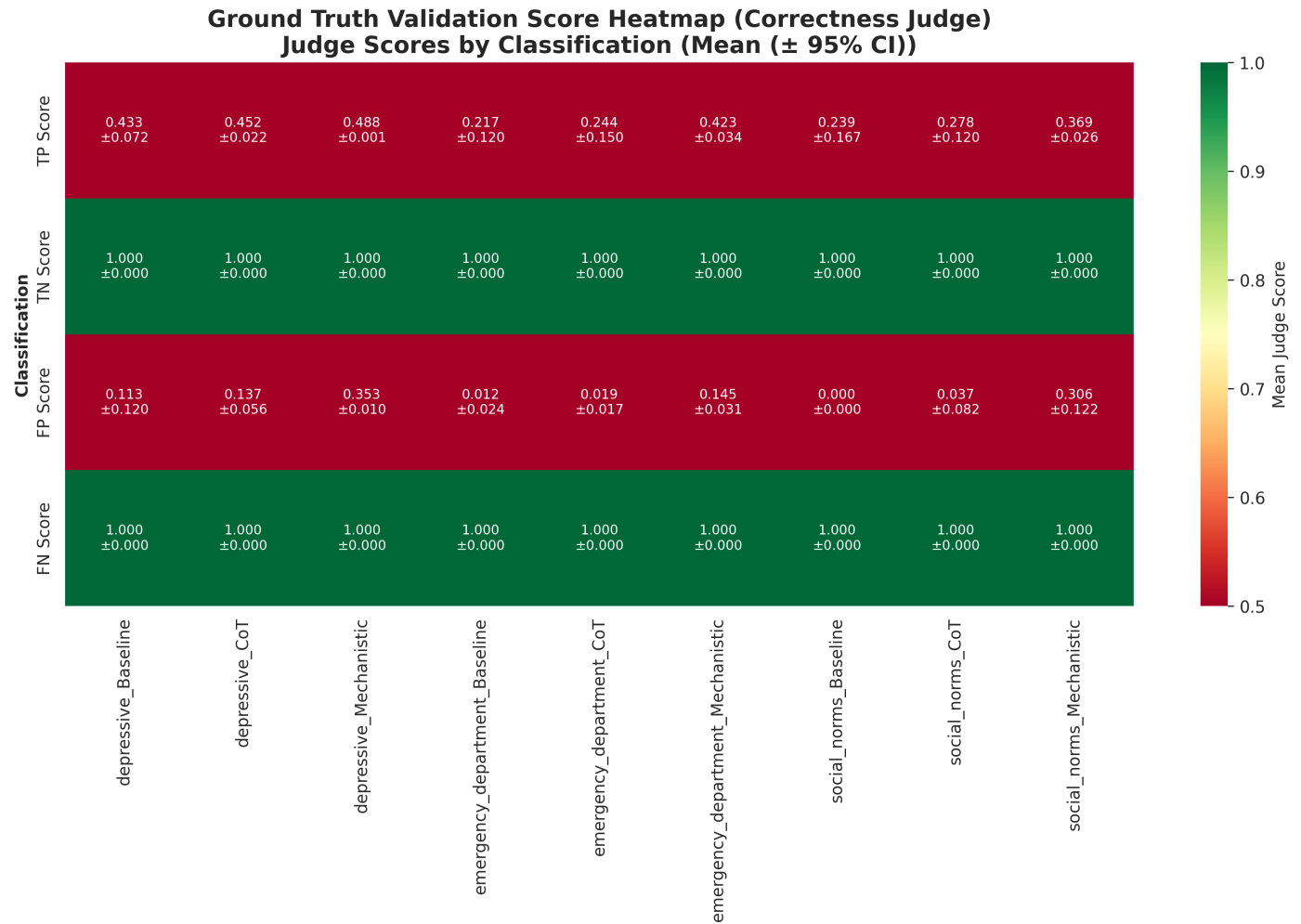


Figure J.1: Judge Score heatmap (mean $\pm$ 95% CI) for ground truth correctness validation showing classification scores across CLD $\times$ prompt combinations, across 3 runs

J.2 Ground Truth Citation Heatmap

Figure J.2 shows the judge score heatmap for the ground truth citation validation. The citation judge evaluates whether edges are properly supported by literature references. Similar patterns to the correctness judge are observed, with high TN scores and variable performance on FP/FN classifications.

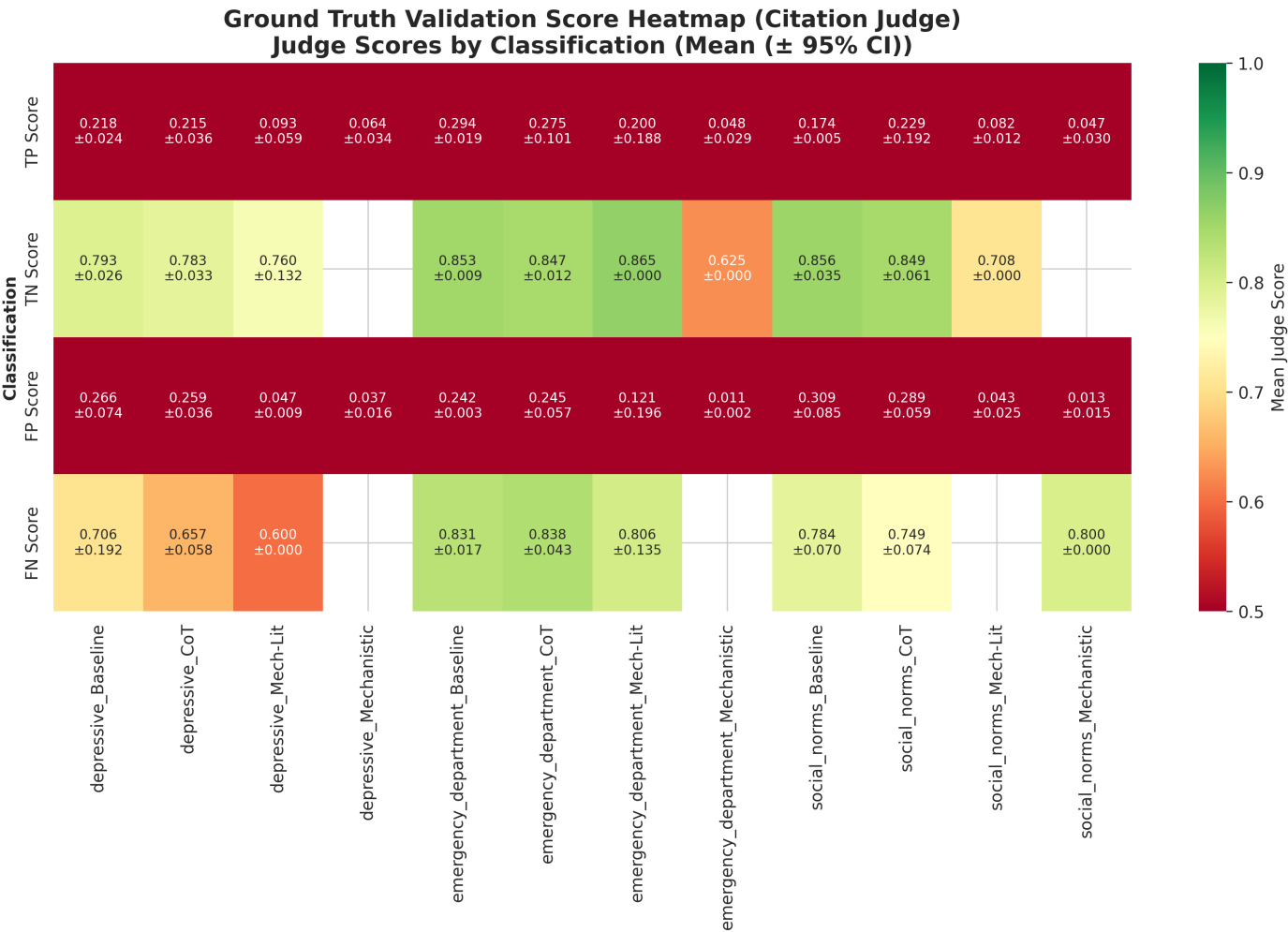


Figure J.2: Score heatmap for ground truth citation validation showing mean judge scores with 95% confidence intervals for each classification type across all CLD $\times$ prompt combinations, across 3 runs

J.3 Corruption Detection Correctness Heatmap

Figure J.3 shows the judge score heatmap for corruption detection using the correctness judge. In this experiment, edges were artificially corrupted and the judge’s task was to detect these corruptions. Low TP scores and high TN scores indicate effective discrimination between corrupted and non-corrupted edges.

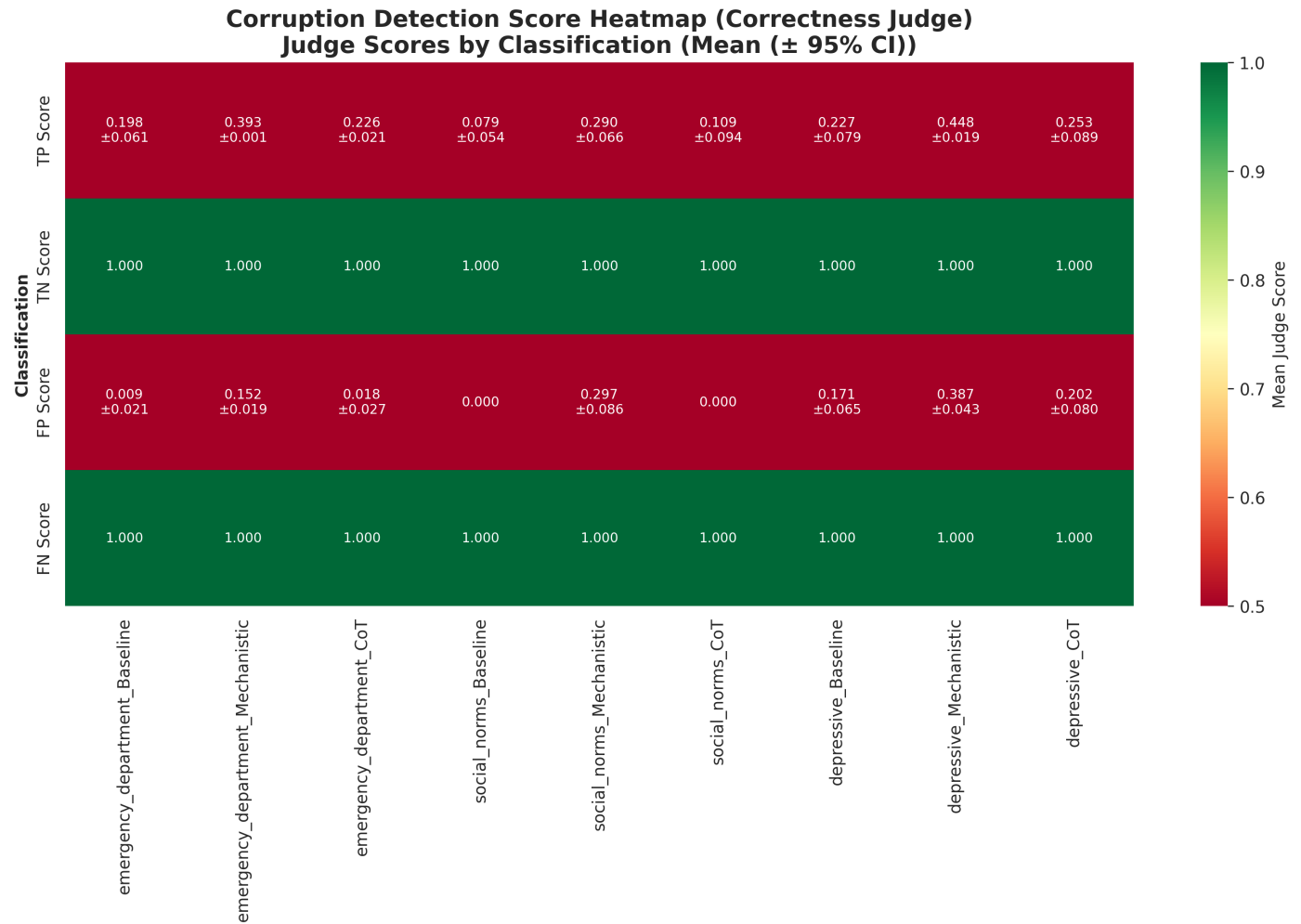


Figure J.3: Score heatmap for corruption detection (Correctness Judge) showing mean judge scores with 95% confidence intervals across all CLD $\times$ prompt combinations, across 3 runs

J.4 Corruption Detection Citation Heatmap

Figure J.4 shows the judge score heatmap for corruption detection using the citation judge. This experiment tests the citation judge’s ability to detect artificially corrupted edges. Note the sparse FN data for some CLDs due to the experimental design focusing on corruption (FP) detection.

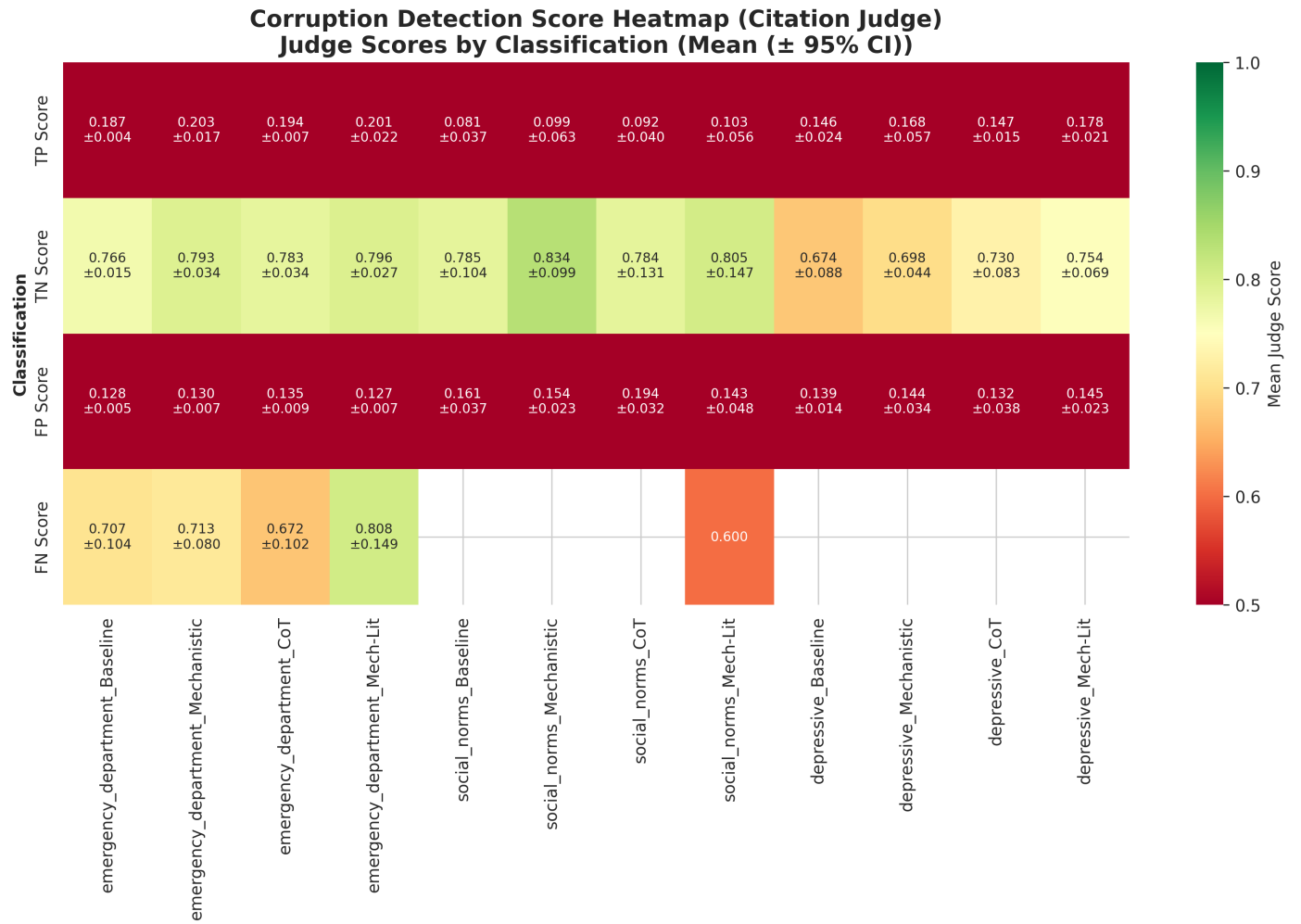


Figure J.4: Score heatmap for corruption detection (Citation Judge) showing mean judge scores with 95% confidence intervals across all CLD $\times$ prompt combinations, across 3 runs

Bibliography

- [1] John D. Sterman. *Business Dynamics: Systems Thinking and Modeling for a Complex World*. Irwin/McGraw-Hill, Boston, MA, 2000. ISBN 978-0-07-231135-8.
- [2] Jac A. M. Vennix. *Group Model Building: Facilitating Team Learning Using System Dynamics*. John Wiley & Sons, Chichester, UK, 1996.
- [3] Étienne A. J. A. Rouwette, Jac A. M. Vennix, and Ton van Mullekom. Group model building effectiveness: a review of assessment studies. *System Dynamics Review*, 18(1):5–45, 2002. doi: 10.1002/sdr.229.
- [4] George P. Richardson. Problems with causal-loop diagrams. *System Dynamics Review*, 2(2):158–170, 1986. doi: 10.1002/sdr.4260020207.
- [5] Loes Crielaard, Jeroen F Uleman, Bas DL Châtel, Sacha Epskamp, Peter Sloot, and Rick Quax. Refining the causal loop diagram: A tutorial for maximizing the contribution of domain expertise in computational system dynamics modeling. *Psychological methods*, 29(1):169, 2024. doi: 10.1037/met0000484. URL <https://psycnet.apa.org/doi/10.1037/met0000484>.
- [6] Adam Tauman Kalai, Ofir Nachum, Santosh S. Vempala, and Edwin Zhang. Why language models hallucinate, 2025. URL <https://arxiv.org/abs/2509.04664>.
- [7] B. Xu. Hallucination is inevitable for llms with the open world assumption, 2025. URL <https://arxiv.org/abs/2510.05116>.
- [8] Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023.
- [9] Lei Huang, Weijiang Yu, Weitao Ma, Weihong Zhong, Zhangyin Feng, Haotian Wang, Qianglong Chen, Weihua Peng, Xiaocheng Feng, Bing Qin, and Ting Liu. A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions. *ACM Trans. Inf. Syst.*, 43(2), 2025. ISSN 1046-8188. doi: 10.1145/3703155. URL <https://doi.org/10.1145/3703155>.
- [10] Matej Zecevic, Moritz Willig, Devendra Singh Dhami, and Kristian Kersting. Causal parrots: Large language models may talk causality but are not causal, 2023. URL <https://arxiv.org/abs/2308.13067>.
- [11] Zhijing Jin, Yuen Chen, Felix Leeb, Luigi Gresele, Ojasv Kamal, Zhiheng Lyu, Kevin Blin, Fernando Gonzalez Adauro, Max Kleiman-Weiner, Mrinmaya Sachan, et al. Cladder: Assessing causal reasoning in language models. *Advances in Neural Information Processing Systems*, 36:31038–31065, 2023. URL <https://arxiv.org/abs/2312.04350>.
- [12] Nitish Joshi, Abulhair Saparov, Yixin Wang, and He He. Llms are prone to fallacies in causal inference, 2024. URL <https://arxiv.org/abs/2406.12158>.

- [13] Emre Kıcıman, Robert Ness, Amit Sharma, and Chenhao Tan. Causal reasoning and large language models: Opening a new frontier for causality, 2024. URL <https://arxiv.org/abs/2305.00050>.
- [14] Neeraj Varshney, Wenlin Yao, Hongming Zhang, Jianshu Chen, and Dong Yu. A stitch in time saves nine: Detecting and mitigating hallucinations of llms by validating low-confidence generation. *arXiv preprint arXiv:2307.03987*, 2023.
- [15] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [16] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [17] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 2020. URL <https://arxiv.org/abs/2005.11401>.
- [18] Yuzhe Zhang, Yipeng Zhang, Yidong Gan, Lina Yao, and Chen Wang. Causal graph discovery with retrieval-augmented generation based large language models. *arXiv preprint arXiv:2402.15301*, 2024. URL <https://arxiv.org/abs/2402.15301>.
- [19] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2024. URL <https://arxiv.org/abs/2303.11366>.
- [20] C. Lu, C. Lu, R. T. Lange, J. Foerster, J. Clune, and D. Ha. The ai scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint, arXiv:2408.06292*, 2024.
- [21] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36, 2024. URL <https://arxiv.org/abs/2306.05685>.
- [22] OpenAI. Learning to reason with llms: Introducing openai o1, a new large language model trained with reinforcement learning for complex reasoning. <https://openai.com/index/learning-to-reason-with-llms/>, 2024.
- [23] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025. URL <https://arxiv.org/abs/2501.12948>.
- [24] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.
- [25] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- [26] T. Wang, X. Jiao, Y. He, Z. Chen, Y. Zhu, X. Chu, J. Gao, Y. Wang, and L. Ma. Adaptive activation steering: A tuning-free llm truthfulness improvement method for diverse hallucinations categories. *arXiv preprint*, 2024. URL <https://arxiv.org/html/2406.00034v1>.

- [27] SM Tonmoy, SM Zaman, Vinija Jain, Anku Rani, Vipula Rawte, Aman Chadha, and Amitava Das. A comprehensive survey of hallucination mitigation techniques in large language models. *arXiv preprint arXiv:2401.01313*, 2024. URL <https://arxiv.org/abs/2401.01313>.
- [28] Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. Llms-as-judges: A comprehensive survey on llm-based evaluation methods. *arXiv preprint arXiv:2412.05579*, 2024. URL <https://arxiv.org/abs/2412.05579>.
- [29] Jiawen Shi, Zenghui Yuan, Yinuo Liu, Yue Huang, Pan Zhou, Lichao Sun, and Neil Zhenqiang Gong. Optimization-based prompt injection attack to llm-as-a-judge. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*, pages 660–674, 2024.
- [30] Simon Valentin, Jinmiao Fu, Gianluca Detommaso, Shaoyuan Xu, Giovanni Zappella, and Bryan Wang. Cost-effective hallucination detection for llms. *arXiv preprint arXiv:2407.21424*, 2024. URL <https://arxiv.org/abs/2407.21424>.
- [31] Bernard P Ziegler. *Theory of Modelling and Simulation*. Wiley, 1976. ISBN 978-0-471-93235-9.
- [32] François E Cellier. *Continuous System Modeling*. Springer-Verlag, 1991. ISBN 978-0-387-97502-3. doi: 10.1007/978-1-4757-3922-0.
- [33] Jenny Moberg, Andrew D Oxman, Sarah Rosenbaum, Holger J Schünemann, Gordon Guyatt, Signe Flottorp, Claire Glenton, Simon Lewin, Angela Morelli, Gabriel Rada, et al. The grade evidence to decision (etd) framework for health system and public health decisions. *Health research policy and systems*, 16(1):45, 2018.
- [34] M. Waaijers, H. Rosenbusch, C. J. van Lissa, A. Roefs, and D. Borsboom. theoraizer: Ai-assisted theory construction. *PsyArXiv Preprints*, 2024. URL https://osf.io/preprints/psyarxiv/gu9yq_v1. Preprint.
- [35] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021. URL <https://arxiv.org/abs/2005.11401>.
- [36] Afshine Amidi and Shervine Amidi. Vip cheatsheet: Transformers & large language models. <https://cme295.stanford.edu>, March 2025. Course handout, accessed 2025-12-21.
- [37] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017. URL <https://arxiv.org/abs/1706.03762>.
- [38] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers, 2019. URL <https://arxiv.org/abs/1904.10509>.
- [39] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness, 2022. URL <https://arxiv.org/abs/2205.14135>.
- [40] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration, 2020. URL <https://arxiv.org/abs/1904.09751>.
- [41] Muru Zhang, Ofir Press, William Merrill, Alisa Liu, and Noah A Smith. How language model hallucinations can snowball. *arXiv preprint arXiv:2305.13534*, 2023.

- [42] Denny Zhou, Nathanael Schärli, Le Hou, Jason Wei, Nathan Scales, Xuezhi Wang, Dale Schuurmans, Claire Cui, Olivier Bousquet, Quoc Le, and Ed Chi. Least-to-most prompting enables complex reasoning in large language models, 2023. URL <https://arxiv.org/abs/2205.10625>.
- [43] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters, 2024. URL <https://arxiv.org/abs/2408.03314>.
- [44] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630, 2024.
- [45] J. Marin. Optimizing AI reasoning: A hamiltonian dynamics approach to multi-hop question answering. *arXiv preprint arXiv:2410.04415*, 2024. URL <https://arxiv.org/abs/2410.04415>.
- [46] Tim Schopf, Juraj Vladika, Michael Färber, and Florian Matthes. Natural language inference fine-tuning for scientific hallucination detection. In Tirthankar Ghosal, Philipp Mayr, Amanpreet Singh, Aakanksha Naik, Georg Rehm, Dayne Freitag, Dan Li, Sonja Schimmmler, and Anita De Waard, editors, *Proceedings of the Fifth Workshop on Scholarly Document Processing (SDP 2025)*, pages 344–352, Vienna, Austria, July 2025. Association for Computational Linguistics. ISBN 979-8-89176-265-7. doi: 10.18653/v1/2025.sdp-1.33. URL <https://aclanthology.org/2025.sdp-1.33/>.
- [47] Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*, 2020.
- [48] Sirui Chen, Bo Peng, Meiqi Chen, Ruiqi Wang, Mengying Xu, Xingyu Zeng, Rui Zhao, Shengjie Zhao, Yu Qiao, and Chaochao Lu. Causal evaluation of language models, 2024. URL <https://arxiv.org/abs/2405.00622>.
- [49] Victor-Alexandru Darvari, Stephen Hailes, and Mirco Musolesi. Large language models are effective priors for causal graph discovery, 2024. URL <https://arxiv.org/abs/2405.13551>.
- [50] Zhijing Jin, Jiarui Liu, Zhiheng Lyu, Spencer Poff, Mrinmaya Sachan, Rada Mihalcea, Mona Diab, and Bernhard Schölkopf. Can large language models infer causation from correlation?, 2024. URL <https://arxiv.org/abs/2306.05836>.
- [51] Thomas Jiralerspong, Xiaoyin Chen, Yash More, Vedant Shah, and Yoshua Bengio. Efficient causal graph discovery using large language models, 2024. URL <https://arxiv.org/abs/2402.01207>.
- [52] Niyousha Hosseinichimeh, Aritra Majumdar, Ross Williams, and Navid Ghaffarzadegan. From text to map: A system dynamics bot for constructing causal loop diagrams, 2024. URL <https://arxiv.org/abs/2402.11400>.
- [53] Ning-Yuan Georgia Liu and David Keith. Leveraging large language models for automated causal loop diagram generation: Enhancing system dynamics modeling through curated prompting techniques. *SSRN Electronic Journal*, 2024. doi: 10.2139/ssrn.4906094. URL https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4906094.
- [54] William Schoenberg, Davidson Girard, Saras Chung, Ellen O’Neill, Janet Velasquez, and Sara Metcalf. How well can ai build sd models?, 2025. URL <https://arxiv.org/abs/2503.15580>.
- [55] Ryan Saklad, Aman Chadha, Oleg Pavlov, and Raha Moraffah. Can large language models infer causal relationships from real-world text?, 2025. URL <https://arxiv.org/abs/2505.18931>.
- [56] Kirk Reinholtz, Kamran Eftekhari Shahroudi, and Svetlana Lawrence. Llm-powered, expert-refined causal loop diagramming via pipeline algebra. *Systems*, 13(9):784, 2025. doi: 10.3390/systems13090784. URL <https://www.mdpi.com/2079-8954/13/9/784>.

- [57] Nengbo Wang, Xiaotian Han, Jagdip Singh, Jing Ma, and Vipin Chaudhary. Causalrag: Integrating causal graphs into retrieval-augmented generation, 2025. URL <https://arxiv.org/abs/2503.19878>.
- [58] Ning-Yuan Georgia Liu and David R. Keith. Leveraging large language models for automated causal loop diagram generation: Enhancing system dynamics modeling through curated prompting techniques, 2025. URL <https://arxiv.org/abs/2503.21798>.
- [59] Yuni Susanti and Nina Holsmoelle. Prompting or fine-tuning? exploring large language models for causal graph validation. *arXiv preprint arXiv:2406.16899*, 2024.
- [60] Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners, 2022. URL <https://arxiv.org/abs/2205.11916>.
- [61] Aobo Kong, Shiwan Zhao, Hao Chen, Qicheng Li, Yong Qin, Ruiqi Sun, Xin Zhou, Enzhi Wang, and Xiaohang Dong. Better zero-shot reasoning with role-play prompting, 2024. URL <https://arxiv.org/abs/2308.07702>.
- [62] Saksorn Ruangtanusak, Pittawat Taveekitworachai, and Kunat Pipatanakul. Talk less, call right: Enhancing role-play llm agents with automatic prompt optimization and role prompting, 2025. URL <https://arxiv.org/abs/2509.00482>.
- [63] Masayuki Takayama, Tadahisa Okuda, Thong Pham, Tatsuyoshi Ikenoue, Shingo Fukuma, Shohei Shimizu, and Akiyoshi Sannai. Integrating large language models in causal discovery: A statistical causal approach, 2025. URL <https://arxiv.org/abs/2402.01454>.
- [64] Elahe Khatibi, Mahyar Abbasian, Zhongqi Yang, Iman Azimi, and Amir M. Rahmani. Alcm: Autonomous llm-augmented causal discovery framework, 2025. URL <https://arxiv.org/abs/2405.01744>.
- [65] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. Chain-of-verification reduces hallucination in large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 3563–3578, 2024. doi: 10.18653/v1/2024.findings-acl.212. URL <https://aclanthology.org/2024.findings-acl.212/>.
- [66] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models, 2023. URL <https://arxiv.org/abs/2203.11171>.
- [67] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. Improving factuality and reasoning in language models through multiagent debate, 2023. URL <https://arxiv.org/abs/2305.14325>.
- [68] Mark Shimonovich, Adam Pearce, Hilary Thomson, Katherine Keyes, and Srinivasa Vittal Katikireddi. Assessing causality in epidemiology: Revisiting bradford hill to incorporate developments in causal thinking. *European Journal of Epidemiology*, 36(9):873–887, 2021.
- [69] Haitao Li, Qian Dong, Junjie Chen, Huixue Su, Yujia Zhou, Qingyao Ai, Ziyi Ye, and Yiqun Liu. Llms-as-judges: A comprehensive survey on llm-based evaluation methods. *arXiv preprint arXiv:2412.05579*, 2024. URL <https://arxiv.org/abs/2412.05579>.
- [70] Dawei Li, Bohan Jiang, Liangjie Li, Alimohammad Beigi Liu, Chengshuai Lai, Huy Pham, Jingyi Shang, Kuofeng Wang, Shengyang Zeng, Yifan Zhang, Tianyi Liu, Zhengzhong Xie, Ryan Rossi, Franck Dernoncourt, Philip S. Yu, Jiuxiang Wang, and Tong Yu. From Generation to Judgment: Opportunities and Challenges of LLM-as-a-judge. *arXiv preprint arXiv:2411.16594*, 2024. URL <https://arxiv.org/abs/2411.16594>.

- [71] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2306.05685>.
- [72] Zicheng Lin, Zhibin Gou, Tian Liang, Ruilin Luo, Haowei Liu, and Yujiu Yang. CriticBench: Benchmarking LLMs for critique-correct reasoning. In *Findings of the Association for Computational Linguistics: ACL 2024*, 2024. URL <https://arxiv.org/abs/2402.14809>.
- [73] Zhan Ling, Yunhao Fang, Xuanlin Li, Zhiao Huang, Mingu Lee, Roland Memisevic, and Hao Su. Deductive Verification of Chain-of-Thought Reasoning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. URL <https://arxiv.org/abs/2306.03872>.
- [74] Sewon Min, Kalpesh Krishna, Xinxi Lyu, Mike Lewis, Wen-tau Yih, Pang Wei Koh, Mohit Iyyer, Luke Zettlemoyer, and Hannaneh Hajishirzi. FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12076–12100, 2023. URL <https://arxiv.org/abs/2305.14251>.
- [75] Jerry Wei, Chengrun Yang, Xinying Song, Yifeng Lu, Nathan Hu, Jie Huang, Dustin Tran, Daiyi Peng, Ruibo Liu, Da Huang, Cosmo Du, and Quoc V. Le. Long-form factuality in large language models. In *Advances in Neural Information Processing Systems*, 2024. URL <https://arxiv.org/abs/2403.18802>.
- [76] Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 6465–6488, 2023. URL <https://arxiv.org/abs/2305.14627>.
- [77] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback, 2023. URL <https://arxiv.org/abs/2303.17651>.
- [78] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.
- [79] Yijun Xiao and William Yang Wang. On hallucination and predictive uncertainty in conditional language generation. In Paola Merlo, Jorg Tiedemann, and Reut Tsarfaty, editors, *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 2734–2744, Online, April 2021. Association for Computational Linguistics. doi: 10.18653/v1/2021.eacl-main.236. URL <https://aclanthology.org/2021.eacl-main.236/>.
- [80] Gaurang Sriramanan, Saurabh Bharti, Vinu Sankar Sadasivan, Himabindu Lakkaraju, and Sham M. Kakade. LLM-Check: Investigating detection of hallucinations in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://proceedings.neurips.cc/paper_files/paper/2024.
- [81] Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. Minillm: Knowledge distillation of large language models. *arXiv preprint arXiv:2306.08543*, 2023.
- [82] Justus Mattern, Fatemehsadat Mireshghallah, Zhijing Jin, Bernhard Schölkopf, Mrinmaya Sachan, and Taylor Berg-Kirkpatrick. Membership inference attacks against language models via neighbourhood comparison. *arXiv preprint arXiv:2305.18462*, 2023.

- [83] Nicholas Carlini, Daniel Paleka, Krishnamurthy Dj Dvijotham, Thomas Steinke, Jonathan Hayase, A Feder Cooper, Katherine Lee, Matthew Jagielski, Milad Nasr, Arthur Conmy, et al. Stealing part of a production language model. *arXiv preprint arXiv:2403.06634*, 2024.
- [84] Eyke Hüllermeier and Willem Waegeman. Aleatoric and epistemic uncertainty in machine learning: an introduction to concepts and methods. *Machine Learning*, 110(3):457–506, 2021. doi: 10.1007/s10994-021-05946-3. URL <https://link.springer.com/article/10.1007/s10994-021-05946-3>.
- [85] Andrey Malinin and Mark Gales. Uncertainty estimation in autoregressive structured prediction. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=jN5y-zb5Q7m>.
- [86] João Eduardo Batista, Emil Vatai, and Mohamed Wahib. SAFE: Improving LLM systems using sentence-level in-generation attribution, 2025. URL <https://arxiv.org/abs/2505.12621>.
- [87] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks, 2021. URL <https://arxiv.org/abs/2005.11401>.
- [88] Kevin Wu, Eric Wu, and James Zou. Clasheval: Quantifying the tug-of-war between an llm’s internal prior and external evidence, 2025. URL <https://arxiv.org/abs/2404.10198>.
- [89] Nelson F. Liu et al. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023. URL <https://arxiv.org/abs/2307.03172>.
- [90] Jingtian Wang, Xinyang Lu, Zitong Zhao, Zhongxiang Dai, Chuan-Sheng Foo, See-Kiong Ng, and Bryan Kian Hsiang Low. Source attribution for large language model-generated data. *arXiv preprint arXiv:2310.00646*, 2023.
- [91] Bumjin Park and Jaesik Choi. Identifying the source of generation for large language models, 2024. URL <https://arxiv.org/abs/2407.12846>.
- [92] Raymond S. Nickerson. Confirmation bias: A ubiquitous phenomenon in many guises. *Review of General Psychology*, 2(2):175–220, 1998. doi: 10.1037/1089-2680.2.2.175. URL <http://psy.ucsd.edu/~mckenzie/nickersonConfirmationBias.pdf>.
- [93] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models, 2023. URL <https://arxiv.org/abs/2210.03629>.
- [94] Andrew Estornell and Yang Liu. Multi-llm debate: Framework, principals, and interventions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL <https://arxiv.org/abs/2402.18461>.
- [95] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E Gonzalez, M Waleed Kadous, and Ion Stoica. RouteLLM: Learning to Route LLMs with Preference Data. *arXiv preprint arXiv:2406.18665*, 2024. URL <https://arxiv.org/abs/2406.18665>.
- [96] Dujian Ding, Ankur Mallick, et al. BEST-Route: Adaptive LLM Routing with Test-Time Optimal Compute. In *International Conference on Machine Learning*, 2025.
- [97] Junyi Li, Xiaoxue Cheng, Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. Halueval: A large-scale hallucination evaluation benchmark for large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.397. URL <https://aclanthology.org/2023.emnlp-main.397/>.

- [98] Niklas Muennighoff, Nouamane Tazi, Loïc Magne, and Nils Reimers. MTEB: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 2014–2037. Association for Computational Linguistics, 2023. URL <https://aclanthology.org/2023.eacl-main.148/>.
- [99] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey, 2024. URL <https://arxiv.org/abs/2312.10997>.
- [100] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. *Advances in Neural Information Processing Systems*, 36, 2024. URL <https://arxiv.org/abs/2306.05685>.
- [101] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-eval: Nlg evaluation using gpt-4 with better human alignment, 2023. URL <https://arxiv.org/abs/2303.16634>.
- [102] Y. Zhang, M. Li, D. Long, X. Zhang, H. Lin, B. Yang, P. Xie, A. Yang, D. Liu, J. Lin, F. Huang, and J. Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models, 2025. URL <https://arxiv.org/abs/2506.05176>.
- [103] Brave Software. Brave search api documentation, 2026. URL <https://api.search.brave.com/app/documentation/web-search/get-started>. Accessed 2026-01-02.
- [104] Pydantic. Pydantic ai: Type-safe agents for LLM workflows, 2024. URL <https://ai.pydantic.dev/>. Framework providing typed agents and Pydantic validation for structured LLM outputs.
- [105] Isaac Newton. Vii. scala graduum caloris. *Philosophical Transactions of the Royal Society of London*, 22 (270):824–829, 1701. doi: 10.1098/rstl.1700.0082. URL <https://doi.org/10.1098/rstl.1700.0082>.
- [106] R. Clausius. Ueber die bewegende kraft der wärme und die gesetze, welche sich daraus für die wärmelehre selbst ableiten lassen. *Annalen der Physik*, 155(4):500–524, 1850. doi: 10.1002/andp.18501550403. URL <https://doi.org/10.1002/andp.18501550403>.
- [107] Alfred J. Lotka. *Elements of Physical Biology*. Williams & Wilkins, Baltimore, MD, 1925. URL <https://archive.org/details/elementsofphysic017171mbp>.
- [108] V. Volterra. Variations and fluctuations of the number of individuals in animal species living together. *ICES Journal of Marine Science*, 3(1):3–51, 1928. doi: 10.1093/icesjms/3.1.3. URL <https://doi.org/10.1093/icesjms/3.1.3>.
- [109] Georg Simon Ohm. *Die galvanische Kette, mathematisch bearbeitet*. T. H. Riemann, Berlin, 1827. URL https://archive.org/details/bub_gb_tTVQAAAAcAAJ.
- [110] Gustav Kirchhoff. Ueber den durchgang eines elektrischen stromes durch eine ebene, insbesondere durch eine kreisförmige. *Annalen der Physik*, 140(4):497–514, 1845. doi: 10.1002/andp.18451400402. URL <https://doi.org/10.1002/andp.18451400402>.
- [111] Blaise Pascal. *Traitez de l’équilibre des liqueurs et de la pesanteur de la masse de l’air*. 1663. URL <https://gallica.bnf.fr/ark:/12148/bpt6k87094593.texteImage>. Digitized by Bibliothèque nationale de France (Gallica).
- [112] Evangelista Torricelli. *Opera geometrica*. 1644. URL <https://archive.org/details/operageometrica00torr>. Internet Archive scan.

- [113] Jeroen F. Uleman, Maartje Luijten, Wilson F. Abdo, Jana Vyrastekova, Andreas Gerhardus, Jakob Runge, Naja Hulvej Rod, and Maaike Verhagen. Triangulation for causal loop diagrams: constructing biopsychosocial models using group model building, literature review, and causal discovery. *npj Complexity*, 1(1):1–12, 2024. doi: 10.1038/s44260-024-00017-9. URL <https://www.nature.com/articles/s44260-024-00017-9>.
- [114] Loes Crielaard, Pallavi Dutta, Rick Quax, Mary Nicolaou, Nadia Merabet, Karien Stronks, and Peter M. A. Sloot. Social norms and obesity prevalence: From cohort to system dynamics models. *Obesity Reviews*, 21(9):e13044, 2020. doi: 10.1111/obr.13044. URL <https://pmc.ncbi.nlm.nih.gov/articles/PMC7507199/>.
- [115] O. S. Smeeke, H. C. Willems, I. Blomberg, and B. M. Buurman. A causal loop diagram of older persons’ emergency department visits and interactions of its contributing factors: a group model building approach. *European Geriatric Medicine*, 14(4):829–837, 2023. doi: 10.1007/s41999-023-00816-8. URL <https://pmc.ncbi.nlm.nih.gov/articles/PMC10447269/>.
- [116] J. Richard Landis and Gary G. Koch. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174, 1977.
- [117] Terry K Koo and Mae Y Li. A guideline of selecting and reporting intraclass correlation coefficients for reliability research. *Journal of Chiropractic Medicine*, 15(2):155–163, 2016. doi: 10.1016/j.jcm.2016.02.012.
- [118] Jacob Cohen. *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, Hillsdale, NJ, 2nd edition, 1988. ISBN 978-0-8058-0283-2.
- [119] Stephanie Lin, Jacob Hilton, and Owain Evans. Truthfulqa: Measuring how models mimic human falsehoods. 2021. URL <https://arxiv.org/abs/2109.07958>.
- [120] Ilya M. Sobol’. Sensitivity estimates for nonlinear mathematical models. *Mathematical Modelling and Computational Experiments*, 1(4):407–414, 1993. English translation from *Matematicheskoe Modelirovanie*.
- [121] Kaitao Song, Xu Tan, Tao Qin, Jianfeng Lu, and Tie-Yan Liu. MPNet: Masked and permuted pre-training for language understanding. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. URL <https://arxiv.org/abs/2004.09297>.
- [122] Andrea Saltelli, Paola Annoni, Ivano Azzini, Francesca Campolongo, Marco Ratto, and Stefano Tarantola. Variance based sensitivity analysis of model output. Design and estimator for the total sensitivity index. *Computer Physics Communications*, 181(2):259–270, 2010. doi: 10.1016/j.cpc.2009.09.018. URL <https://doi.org/10.1016/j.cpc.2009.09.018>.
- [123] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL <https://www.jmlr.org/papers/v12/pedregosa11a.html>.
- [124] S. S. Shapiro and M. B. Wilk. An analysis of variance test for normality (complete samples). *Biometrika*, 52(3-4):591–611, 1965. doi: 10.1093/biomet/52.3-4.591.
- [125] Howard Levene. Robust tests for equality of variances. pages 278–292, 1960.
- [126] Jeroen F Uleman, René JF Melis, Rick Quax, Eddy A van der Zee, Dick Thijssen, Martin Dresler, Ondine van de Rest, Isabelle F van der Velpen, Hieab HH Adams, Ben Schmand, et al. Mapping the multicausality of alzheimer’s disease through group model building. *Geroscience*, 43(2):829–843, 2021. doi: 10.1007/s11357-020-00228-7. URL <https://doi.org/10.1007/s11357-020-00228-7>.

- [127] Arman Cohan, Sergey Feldman, Iz Beltagy, Doug Downey, and Daniel S. Weld. SPECTER: Document-level representation learning using citation-informed transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 2270–2282. Association for Computational Linguistics, 2020. URL <https://aclanthology.org/2020.acl-main.207/>.
- [128] Huan Ma, Jingdong Chen, Joey Tianyi Zhou, Guangyu Wang, and Changqing Zhang. Estimating LLM uncertainty with evidence, 2025. URL <https://arxiv.org/abs/2502.00290>.
- [129] Charles AE Goodhart. Problems of monetary management: the uk experience. In *Monetary theory and practice: The UK experience*, pages 91–121. Springer, 1984.
- [130] Caspar J. Van Lissa, Andreas M. Brandmaier, Loek Brinkman, Anna-Lena Lamprecht, Aaron Peikert, Marijn E. Struiksmā, and Barbara M.I. Vreede. Worcs: A workflow for open reproducible code in science. *Data Science*, 4(1):29–49, 2021. doi: 10.3233/DS-210031.
- [131] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, Ralf Gommers, Pauli Virtanen, David Cournapeau, Eric Wieser, Julian Taylor, Sebastian Berg, Nathaniel J. Smith, Robert Kern, Matti Picus, Stephan Hoyer, Marten H. van Kerkwijk, Matthew Brett, Allan Haldane, Jaime Fernández del Río, Mark Wiebe, Pearu Peterson, Pierre Gérard-Marchant, Kevin Sheppard, Tyler Reddy, Warren Weckesser, Hameer Abbasi, Christoph Gohlke, and Travis E. Oliphant. Array programming with NumPy. *Nature*, 585(7825):357–362, September 2020. doi: 10.1038/s41586-020-2649-2. URL <https://doi.org/10.1038/s41586-020-2649-2>.
- [132] Wes McKinney. Data structures for statistical computing in python. In Stéfan van der Walt and Jarrod Millman, editors, *Proceedings of the 9th Python in Science Conference*, pages 56–61, 2010. doi: 10.25080/Majora-92bf1922-00a.
- [133] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, Matt Haberland, Tyler Reddy, David Cournapeau, Evgeni Burovski, Pearu Peterson, Warren Weckesser, Jonathan Bright, Stéfan J. van der Walt, Matthew Brett, Joshua Wilson, K. Jarrod Millman, Nikolay Mayorov, Andrew R. J. Nelson, Eric Jones, Robert Kern, Eric Larson, C J Carey, İlhan Polat, Yu Feng, Eric W. Moore, Jake VanderPlas, Denis Laxalde, Josef Perktold, Robert Cimrman, Ian Henriksen, E. A. Quintero, Charles R. Harris, Anne M. Archibald, Antônio H. Ribeiro, Fabian Pedregosa, Paul van Mulbregt, and SciPy 1.0 Contributors. SciPy 1.0: Fundamental algorithms for scientific computing in python. *Nature Methods*, 17:261–272, 2020. doi: 10.1038/s41592-019-0686-2.
- [134] Guillaume Lemaître, Fernando Nogueira, and Christos K. Aridas. Imbalanced-learn: A python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, 18(17):1–5, 2017. URL <http://jmlr.org/papers/v18/16-365.html>.
- [135] J. D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. doi: 10.1109/MCSE.2007.55.
- [136] Michael L. Waskom. seaborn: statistical data visualization. *Journal of Open Source Software*, 6(60):3021, 2021. doi: 10.21105/joss.03021. URL <https://doi.org/10.21105/joss.03021>.
- [137] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. In Gäel Varoquaux, Travis Vaught, and Jarrod Millman, editors, *Proceedings of the 7th Python in Science Conference (SciPy2008)*, pages 11–15, Pasadena, CA USA, 2008.
- [138] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Databases*. O’Reilly Media, 2013. ISBN 978-1-4493-5626-2. Neo4j graph database used for storing causal loop diagrams.

- [139] Samuel Colvin, Eric Jolibois, Hasan Ramezani, Adrian Garcia Badaracco, Terrence Dorsey, David Montague, et al. Pydantic: Data validation using python type hints, 2024. URL <https://github.com/pydantic/pydantic>. Used for structured output validation in LLM pipelines.
- [140] Thomas Kluyver, Benjamin Ragan-Kelley, Fernando Pérez, Brian Granger, Matthias Bussonnier, Jonathan Frederic, Kyle Kelley, Jessica Hamrick, Jason Grout, Sylvain Corlay, Paul Ivanov, Damián Avila, Safia Abdalla, and Carol Willing. Jupyter notebooks – a publishing format for reproducible computational workflows. In *Positioning and Power in Academic Publishing: Players, Agents and Agendas*, pages 87–90. IOS Press, 2016.
- [141] Milton Friedman. The use of ranks to avoid the assumption of normality implicit in the analysis of variance. *Journal of the American Statistical Association*, 32(200):675–701, 1937. doi: 10.1080/01621459.1937.10503522.
- [142] Maurice G. Kendall and B. Babington Smith. The problem of m rankings. *The Annals of Mathematical Statistics*, 10(3):275–287, 1939. doi: 10.1214/aoms/1177732186.
- [143] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945. doi: 10.2307/3001968.
- [144] H. B. Mann and D. R. Whitney. On a test of whether one of two random variables is stochastically larger than the other. *The Annals of Mathematical Statistics*, 18(1):50–60, 1947. doi: 10.1214/aoms/1177730491.
- [145] Ronald A. Fisher. On the interpretation of χ^2 from contingency tables, and the calculation of P. *Journal of the Royal Statistical Society*, 85(1):87–94, 1922. doi: 10.2307/2340521.
- [146] William H. Kruskal and W. Allen Wallis. Use of ranks in one-criterion variance analysis. *Journal of the American Statistical Association*, 47(260):583–621, 1952. doi: 10.1080/01621459.1952.10483441.
- [147] John W. Mauchly. Significance test for sphericity of a normal n -variate distribution. *The Annals of Mathematical Statistics*, 11(2):204–209, 1940. doi: 10.1214/aoms/1177731915.
- [148] Samuel W. Greenhouse and Seymour Geisser. On methods in the analysis of profile data. *Psychometrika*, 24(2):95–112, 1959. doi: 10.1007/BF02289823.
- [149] Carlo Emilio Bonferroni. *Teoria Statistica delle Classi e Calcolo delle Probabilità*. Pubblicazioni del R Istituto Superiore di Scienze Economiche e Commerciali di Firenze, Florence, Italy, 1936.
- [150] Olive Jean Dunn. Multiple comparisons among means. *Journal of the American Statistical Association*, 56(293):52–64, 1961. doi: 10.1080/01621459.1961.10482090.
- [151] Edwin B. Wilson. Probable inference, the law of succession, and statistical inference. *Journal of the American Statistical Association*, 22(158):209–212, 1927. doi: 10.1080/01621459.1927.10502953.
- [152] Ronald A. Fisher. Frequency distribution of the values of the correlation coefficient in samples from an indefinitely large population. *Biometrika*, 10(4):507–521, 1915. doi: 10.2307/2331838.
- [153] Leo Breiman. Random forests. *Machine learning*, 45(1):5–32, 2001.
- [154] Jerome H Friedman. Greedy function approximation: a gradient boosting machine. *Annals of statistics*, pages 1189–1232, 2001.
- [155] D. R. Cox. The regression analysis of binary sequences. *Journal of the Royal Statistical Society. Series B (Methodological)*, 20(2):215–242, 1958. ISSN 00359246. URL <http://www.jstor.org/stable/2983890>.
- [156] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. Learning representations by back-propagating errors. *nature*, 323(6088):533–536, 1986.